Artificial Intelligence 2
Summer Semester 2026

– Lecture Notes –

Prof. Dr. Michael Kohlhase

Professur für Wissensrepräsentation und -verarbeitung
Informatik, FAU Erlangen-Nürnberg
Michael.Kohlhase@FAU.de

2026-10-18

18.1 Preface

18.1.1 Course Concept

Objective: The course aims at giving students a solid (and often somewhat theoretically oriented) foundation of the basic concepts and practices of artificial intelligence. The course will predominantly cover symbolic AI – also sometimes called "good old-fashioned AI (GofAI)" – in the first semester and offers the very foundations of statistical approaches in the second. Indeed, a full account sub symbolic, machine learning based AI deserves its own specialization courses and needs much more mathematical prerequisites than we can assume in this course.
Context: The course "Artificial Intelligence" (AI 1 & 2) at FAU Erlangen is a two-semester course in the "Wahlpflichtbereich" (specialization phase) in semester 5/6 of the bachelor program "Computer Science" at FAU Erlangen. It is also available as a (somewhat remedial) course in the "Vertiefungsmodul Künstliche Intelligenz" in the Computer Science Master's program.
Prerequisites: AI-1 & 2 builds on the mandatory courses in the FAU bachelor's program, in particular the course "Grundlagen der Logik in der Informatik" [Glo], which already covers a lot of the materials usually presented in the "knowledge and reasoning" part of an introductory AI course. The AI 1& 2 course also minimizes overlap with the course.

The course is relatively elementary, we expect that any student who attended the mandatory CS course at FAU Erlangen can follow it.
Open to external students: Other bachelor programs are increasingly co-opting the course as specialization option. There is no inherent restriction to computer science students in this course. Students with other study biographies – e.g. students from other bachelor programs our external Master's students should be able to pick up the prerequisites when needed.

18.1.2 Course Contents

Goal: To give students a solid foundation of the basic concepts and practices of the field of artificial intelligence. The course will be based on Russell/Norvig's book "*Artificial Intelligence; A modern Approach*" [RN21]
Artificial Intelligence I (the first semester): introduces AI as an area of study, discusses "rational agents" as a unifying conceptual paradigm for AI and covers problem solving, search, constraint propagation, logic, knowledge representation, and planning.
Artificial Intelligence II (the second semester): is more oriented towards exposing students to the basics of statistically based AI: We start out with reasoning under uncertainty, setting the foundation with Bayesian Networks and extending this to rational decision theory. Building on this we cover the basics of machine learning.

18.1.3 This Document

Presentation: The document mixes the slides presented in class with comments of the instructor to give students a more complete background reference. **Caveat:** This document is primarily made available for the students of the AI-2 course only. After multiple iterations of this course it is reasonably feature-complete, but will evolve and be polished in coming academic years.
Licensing: This document is licensed under a Creative Commons license that requires attribution, allows commercial use, and allows derivative works as long as these are licensed under the same license. **Knowledge Representation Experiment:** This document is also an experiment in knowledge representation. Under the hood, it uses the sTEX package [Koh08; sTeX], a TEX/LATEX extension for semantic markup, which allows to export the contents into active documents that adapt to the reader and can be instrumented with services based on the explicitly represented meaning of the documents.

18.1.4 Acknowledgments

Materials: Most of the materials in this course is based on Russel/Norvik's book "Artificial Intelligence — A Modern Approach" (AIMA [RN95]). Even the slides are based on a LaTeX-based slide set, but heavily edited. The section on search algorithms is originallly based on materials obtained from Bernhard Beckert (then Uni Koblenz), which is in turn based on AIMA. Some extensions have been inspired by an AI course by Jörg Hoffmann and Wolfgang Wahlster at Saarland University in 2016. Finally Dennis Müller suggested and supplied some extensions on AGI.

In Summer 2024 Dennis Müller gave the AI-2 lecture and improved the presentation considerably.

Last but not least, Florian Rabe, Max Rapp and Katja Berčič have carefully re-read the text and pointed out problems.

All course materials have been restructured and semantically annotated in the sTeX format, so that we can base additional semantic services on them.

AI Students: The following students have submitted corrections and suggestions to this and earlier versions of the notes: Rares Ambrus, Ioan Sucan, Yashodan Nevatia, Dennis Müller, Simon Rainer, Demian Vöhringer, Lorenz Gorse, Philipp Reger, Benedikt Lorch, Maximilian Lösch, Luca Reeb, Marius Frinken, Peter Eichinger, Oskar Herrmann, Daniel Höfer, Stephan Mattejat, Matthias Sonntag, Jan Urfei, Tanja Würsching, Adrian Kretschmer, Tobias Schmidt, Maxim Onciul, Armin Roth, Liam Corona, Tobias Völk, Lena Voigt, Yinan Shao, Michael Girstl, Matthias Vietz, Anatoliy Cherepantsev, Stefan Musevski, Matthias Lobenhofer, Philipp Kaludercic, Diwarkara Reddy, Martin Helmke, Stefan Müller, Dominik Mehlich, Paul Martini, Vishwang Dave, Arthur Miehlich, Christian Schabesberger, Vishaal Saravanan, Simon Heilig, Michelle Fribrance, Wenwen Wang, Xinyuan Tu, Lobna Eldeeb.

Recorded Syllabus The recorded syllabus – a record the progress of the course in the 2 – is in the course page in the ALeA system at `https://alea.education/FAU/ai-2/SS26`. The table of contents in the AI-2 lecture notes in ALeA indicates the material covered to date in yellow.

Contents

18.1 Preface i
18.1.1 Course Concept i
18.1.2 Course Contents i
18.1.3 This Document i
18.1.4 Acknowledgments ii

19 Preliminaries **1**
19.1 Administrative Ground Rules 2
19.2 Getting Most out of AI-2 6
19.3 Learning Resources for AI-2 9

20 Overview over AI and Topics of AI-II **13**
20.1 Ways to Attack the AI Problem 13
20.2 AI in the KWARC Group 15
20.3 Agents and Environments in AI2 16
20.3.1 Recap: Rational Agents as a Conceptual Framework 17
20.3.2 Sources of Uncertainty 22
20.3.3 Agent Architectures based on Belief States 23

I Reasoning with Uncertain Knowledge **27**

21 Quantifying Uncertainty **31**
21.1 Probability Theory 31
21.1.1 Prior and Posterior Probabilities 31
21.1.2 Independence 37
21.1.3 Conclusion 40
21.2 Probabilistic Reasoning Techniques 41
21.2.1 Probability Distributions 41
21.2.2 Naive Bayes 44
21.2.3 Inference by Enumeration 49
21.2.4 Example – The Wumpus is Back 49

22 Probabilistic Reasoning: Bayesian Networks **53**
22.1 Introduction 53
22.2 Constructing Bayesian Networks 56
22.3 Inference in Bayesian Networks 62
22.4 Conclusion 66

23 Making Simple Decisions Rationally **69**
23.1 Introduction 69
23.2 Decision Networks 73
23.3 Preferences and Utilities 74

23.4 Utilities in Practice . . . 77
23.5 Multi-Attribute Utility . . . 79
23.6 The Value of Information . . . 82

24 Temporal Probability Models **87**
24.1 Modeling Time and Uncertainty with Markov Processes . . . 87
24.2 Inference: Filtering, Prediction, Smoothing, and Explanation . . . 92
24.2.1 Filtering . . . 92
24.2.2 Prediction . . . 94
24.2.3 Smoothing/Hindsight . . . 95
24.2.4 Most Likely Explanation . . . 97
24.3 Hidden Markov Models – Extended Example . . . 99
24.4 Dynamic Bayesian Networks . . . 101

25 Making Complex Decisions **105**
25.1 Sequential Decision Problems . . . 105
25.2 Utilities over Time . . . 108
25.3 Value/Policy Iteration . . . 111
25.4 Partially Observable MDPs . . . 115
25.5 Reducing POMDPs to belief-state-level MDPs . . . 116
25.6 Online Agents with POMDPs . . . 121

II Machine Learning **125**

26 Learning from Observations **129**
26.1 Forms of Learning . . . 129
26.2 Inductive Learning Problems . . . 131
26.3 Learning Decision Trees . . . 133
26.4 Using Information Theory . . . 137
26.5 Evaluating and Choosing the Best Hypothesis . . . 139
26.6 Computational Learning Theory . . . 146
26.7 Regression and Classification with Linear Models . . . 151
26.7.1 Univariate Linear Regression . . . 151
26.7.2 Multivariate Linear Regression . . . 153
26.7.3 Linear Regression . . . 155
26.7.4 Logistic Regression . . . 156
26.8 Support Vector Machines . . . 158
26.9 Artificial Neural Networks . . . 161

27 Statistical Learning **173**
27.1 Full Bayesian Learning . . . 173
27.2 Approximations of Bayesian Learning . . . 176
27.3 Parameter Learning for Bayesian Networks . . . 177

28 Reinforcement Learning **183**
28.1 Reinforcement Learning: Introduction & Motivation . . . 183
28.2 Passive Learning . . . 184
28.3 Active Reinforcement Learning . . . 188

29 Knowledge in Learning **191**
29.1 Logical Formulations of Learning . . . 191
29.2 Inductive Logic Programming . . . 194
29.2.1 An Example . . . 195
29.2.2 Top-Down Inductive Learning: FOIL . . . 197
29.2.3 Inverse Resolution . . . 200

III Natural Language **205**

30 Natural Language Processing **209**
30.1 Introduction to NLP . . . 209
30.2 Natural Language and its Meaning . . . 210
30.3 Looking at Natural Language . . . 213
30.4 Language Models . . . 217
30.5 Part of Speech Tagging . . . 220
30.6 Text Classification . . . 222
30.7 Information Retrieval . . . 224
30.8 Information Extraction . . . 228
30.9 Grammar . . . 229

31 Deep Learning for NLP **237**
31.1 Word Embeddings . . . 237
31.2 Recurrent Neural Networks . . . 242
31.3 Sequence-to-Sequence Models . . . 245
31.4 The Transformer Architecture . . . 248
31.5 Large Language Models . . . 250

32 What did we learn in AI 1/2? **255**

IV Excursions **261**

A ALeA – AI-Supported Learning **265**

Chapter 19

Preliminaries

In this chapter, we want to get all the organizational matters out of the way, so that we can get course contents unencumbered. We will talk about the necessary administrative details, go into how students can get most out of the course, talk about where the various resources provided with the course can be found, and finally introduce the ALEA system, an experimental – using AI methods – learning support system for the AI-2 course.

What you should learn here . . .

- **What you should learn in AI-1/2:**
  - In the broadest sense: *A bunch of tools for your tool chest*, (i.e. various (quasi-mathematical) models, first and foremost)
  - the underlying *principles* of these models, (assumptions, limitations, the math behind them . . .)
  - the ability to describe real-world problems in terms of these models, **where adequate**, (. . . and knowing **when** they are adequate!), and
  - the ideas behind effective *algorithms* that solve these problems. (and to understand them well enough to implement them)
- **Note:** You will likely never get payed to implement an algorithm that e.g. solves Bayesian networks. (they already exist)
  - *But* you might get payed to *recognize* that some given problem *can be* represented as a Bayesian network!
  - **Or:** you can recognize that it is *similar to* a Bayesian network, and reuse the underlying principles to develop new specialized tools.

SOME RIGHTS RESERVED

In other words: Many things you learn here are *means to an end* (e.g. understanding the underlying *ideas* behind algorithms), not the end itself. But the best way to understand these means is to first treat them as an end in themselves.

Compare Two Employees

- **"We have the following problem and we need a solution: . . . "**

▷ **Employee E0 - Let's ask ChatGPT:** "*Let me relay that question to ChatGPT, I will relay the answer to you . . .*"

▷ **Employee E1 – Deep Learning can do everything:** "*I just need ≈1.5 million labeled examples of potentially sensitive data, a GPU cluster for training, and a few weeks to train, tweak and fine-tune the model.*"

"*But* then *I can solve the problem . . . with a confidence of 95%, within* 40 *seconds of inference per input. Oh, as long as the input isn't longer than 15unit, or I will need to retrain on a bigger input layer . . .*"

▷ **Employee E2 – AI-1/2 Alumna:** "*. . . while you were talking, I quickly built a custom UI for an off-the-shelf <problem> solver that runs on a medium-sized potato and returns a* provably correct *result in a few milliseconds. For inputs longer than 1000 unit, you need a slightly bigger potato . . .*"

▷ **Effect:**

- ▹ **E0** was let go — he did not add value for his salary! (the Boss had already consulted ChatGPT)
- ▹ **E2** was promoted, since she proposed a cost-effective solution.
- ▹ **E1** now works in the team led by **E2**. (so he can learn from her)

▷ **Moral of the story:** Know your *tools* well enough to select the right one for the job.

SOME RIGHTS RESERVED

Obviously, that is not to say that machine learning is not a useful tool! (It is!)

If your job is to e.g. filter customer support requests, or to recognize cats in pictures, trying to write a prolog program from scratch is probably the wrong approach: Just use a language model / image model and fine-tune it on a classification head.

But it is also not the only tool, and it is not always the right tool for the job – despite what some people might tell you. And even in scenarios where machine learning *can* yield decent results, it is not always the *best* tool. (Some people care about efficiency, explainability, etc ;))

In an ideal world . . . We would spend weeks on each topic, give you lots of interesting problems to solve, give you individual feedback and tutoring.

As an exam, you would have to solve a few real-world problems by choosing the right tools, model the problem accordingly, customize the algorithms to the specifics, implement them.

⇝ You would each write a 10 page essay in 4 hours, we would spend the next 6 months grading them, and then 95% of you would probably fail: Really understanding this stuff takes time and *lots of practice!*

Instead: we will teach you all the important stuff, give you practice problems to do on your own, and then test you on the basics in a manner that is actually gradable in a reasonable time frame, and doable.

Hopefully, in five years, when you encounter a problem, you will remember enough of the broad strokes to recognize the "kind of problem" you have, and are able to look up the rest easily.

19.1 Administrative Ground Rules

We will now go through the ground rules for the course. This is a kind of a social contract between the instructor and the students. Both have to keep their side of the deal to make learning as efficient and painless as possible.

Prerequisites

▷ **Remember:** **AI-1** dealt with situations with "complete information" and strictly computable, "perfect" solutions to problems. (i.e. tree search, logical inference, planning, etc.)

▷ **AI-2** will focus on *probabilistic* scenarios by introducing uncertain situations, and *approximate* solutions to problems. (Bayesian networks, Markov models, machine learning, etc.)

▷ **Weak Prerequisites for AI-2:** (if you do not have them, study up as needed)

- ▷ AI-1 (in particular: PEAS, propositional logic/first-order logic (mostly the syntax), some logic programming)
- ▷ (very) elementary complexity theory. (big Oh and friends)
- ▷ rudimentary probability theory (e.g. from stochastics)
- ▷ basic linear algebra (vectors, matrices,...)
- ▷ basic real analysis (aka. calculus) (primarily: (partial) derivatives)

▷ **Meaning:** I will *assume* you know these things, but some of them we will recap, and what you don't know will make things slightly harder for you, but by no means prohibitively difficult.

SOME RIGHTS RESERVED

"Strict" Prerequisites

▷ **Most crucially – Mathematical Literacy:** Mathematics is the language that computer scientists express their ideas in! ("*A search problem is a tuple* $(N, S, G, ...)$ *such that...*")

▷ **Note:** This is a skill that can be *learned*, and more importantly, *practiced!* Not having/honing this skill *will* make things more difficult for you. Be aware of this and, if necessary, work on it – it will pay off, not only in this course.

▷ **Suggestion:** Study up with SMAI (Symbolic Methods for AI) on ALEA.
(`https://alea.education/FAU/smai/WS25-26`)

▷ **But also:** Motivation, interest, curiosity, hard work. (AI-2 is non-trivial)

▷ **Note:** Grades correlate significantly with invested effort; including, but not limited to:

- ▷ time spent on exercises, (learning is 80% perspiration, only 20% inspiration)
- ▷ being here in presence, (humans are social animals $\rightsquigarrow$ mirror neurons)
- ▷ asking questions, (Q/A dialogues activate brains)
- ▷ talking to your peers, (pool your insights, share your triumphs/frustrations)...

All of these we try to support with the ALEA system.(which also gives us the data to prove this)

SOME RIGHTS RESERVED

Now we come to a topic that is always interesting to the students: the grading scheme: and how we run the exams.

Assessment, Grades

- **Overall (Module) Grade:**
  - Grade via the exam (Klausur) $\rightsquigarrow$ 100% of the grade.
  - Up to 10% bonus on-top for an exam with $\geq 50\%$ points. ($< 50\% \rightsquigarrow$ no bonus)
  - Bonus points $\widehat{=}$ percentage sum of the best 10 prepquizzes divided by 100.
  - Bonus points apply to the exam in the same semester and the retake in the semester after. (be quick)
- **Exam:** 90 minutes exam conducted in presence on paper! ($\sim$ October 1. 2026)
- **Retake Exam:** 90 minutes exam six months later. ($\sim$ April 1. 2027)
- **Allowed in Exams:** Pen, well-trained brain, drink/snack, mascot, collected cheat sheet. (more later)
- **Forbidden in Exams:** All electronics, communication media, all other documents.
- You have to register for exams in `https://campo.fau.de` (see Campo for deadline)
- **Note:** You can de-register from an exam on `https://campo.fau.de` up to three working days before exam. (do not miss that if you are not prepared)

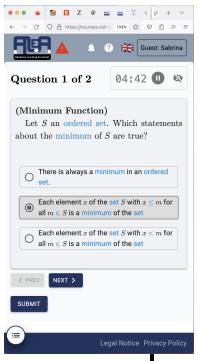
SOME RIGHTS RESERVED

The next topic is about measure we take in AI-2 to keep students work continuously, and give them feedback how well they understand the (edge cases) of the topics and concepts covered in the lectures.

Preparedness Quizzes

- **PrepQuizzes:** Before every Tuesday lecture we offer a 10 min online quiz – the PrepQuiz – about the material from the previous week. (16:15-16:25; starts in week 2)
- **Motivations:** We do this to
  - keep you prepared and working continuously. (primary)
  - bonus points if the exam has $\geq 50\%$ points (bonus points transferred to the (first) retake)
  - update the ALeA learner model. (fringe benefit)
- The prepquizes will be given in the ALeA system

- ▷ `https://alea.education/FAU/ai-2/SS26/quiz-dash`
- ▷ You have to be logged into ALEA! (via FAU IDM)
- ▷ You can take the prepquiz on your laptop or phone, ...
- ▷ ... in the lecture or at home ...
- ▷ ... via WLAN or 4G Network. (do not overload)
- ▷ Prepquizzes will only be available 16:15-16:25!



SOME RIGHTS RESERVED

Now we can become specific about the collected cheat sheets we hinted at before.

Collected Cheat Sheet for the AI-2 Exam

- ▷ We allow you a cheat sheet for the exam. (but only a very special one)
- ▷ **Definition 19.1.1.** A collected cheat sheet is a personal, hand-written document collected by the following process:
  - ▷ We collect a weekly cheat sheetlet every week on Tuesday before the prepquiz. (useful there as well)
  - ▷ A cheat sheetlet is a handwritten (A5) document:
    1. Print the generated template from `https://alea.education/FAU/ai-2/SS26/cheatsheet` (different for each week/student)
    2. Write into the box in the lower half (anything outside will be discarded)
    3. Scan or take a picture and upload there. (you are responsible for quality/legibility)
    4. Later submits supersede earlier ones until the deadline (as often as you want)

  We will assemble the cheat sheetlets into a collected cheat sheet, print it in black/white, and give it back to you in the exam. (in the first retake as well)
- ▷ **Alternative:** you print it or collect original and we verify equality before the exam.(e.g. for color)
- ▷ **Intention:** To entice you to think about AI-2 content continually. (very positively correlated with learning success)
- ▷ **Upshot:** Your success is very much in your hands! (where it should be)

SOME RIGHTS RESERVED

Actually, collected cheat sheets are a new feature in the AI-2 course. We will have to see how this plays out; it can be that we have to change the rules accordingly to reach the goal of helping you learn.

Generated Cheatsheetlet Template



SOME RIGHTS RESERVED

Due to the current AI hype, the course Artificial Intelligence is very popular and thus many degree programs at FAU have adopted it for their curricula. Sometimes the course setup that fits for the CS program does not fit the other's very well, therefore there are some special conditions. I want to state here.

⚠ Special Admin Conditions ⚠

- ▷ Some degree programs do not "import" the course Artificial Intelligence 1, and thus you may not be able to register for the exam via `https://campo.fau.de`.
  - ▷ Just send me an e-mail and come to the exam, (we do the necessary admin)
  - ▷ Tell your program coordinator about AI-1/2 so that they remedy this situation
- ▷ In "Wirtschafts-Informatik" you can only take AI-1 and AI-2 together in the "Wahlpflichtbereich".
  - ▷ ECTS credits need to be divisible by five $\rightsquigarrow 7.5 + 7.5 = 15$.

SOME RIGHTS RESERVED

I can only warn of what I am aware, so if your degree program lets you jump through extra hoops, please tell me and then I can mention them here.

19.2 Getting Most out of AI-2

In this section we will discuss a couple of measures that students may want to consider to get most out of the AI-2 course.

None of the things discussed in this section – homeworks, tutorials, study groups, and attendance – are mandatory (we cannot force you to do them; we offer them to you as learning opportunities), but most of them are very clearly correlated with success (i.e. passing the exam and getting a good grade), so taking advantage of them may be in your own interest.

AI-2 Homework Assignments

- **Goal:** Homework assignments reinforce what was taught in lectures.
- **Homework Assignments:** Small individual problem/programming/proof task
  - but take time to solve (at least read them directly ⇝ questions)
- **Didactic Intuition:** Homework assignments give you material to test your understanding and show you how to apply it.
- Homeworks give no points, but without trying them you are less likely to pass the exam.
- **Our Experience:** Doing your homework is probably even *more* important (and predictive of exam success) than attending the lecture in person!
- Homeworks will be mainly peer-graded in the ALeA system.
- **Didactic Motivation:** Through peer grading, students are able to see mistakes in their thinking and can correct any problems in future assignments. By grading assignments, students may learn how to complete assignments more accurately and how to improve their future results. (not just instructors being lazy)

SOME RIGHTS RESERVED

It is very well-established experience that without doing the homework assignments (or something similar) on your own, you will not master the concepts, you will not even be able to ask sensible questions, and take very little home from the course. Just sitting in the course and nodding is not enough!

AI-2 Homework Assignments – Howto

- **Homework Workflow:** in ALeA (see below)
  - Homework assignments will be published on thursdays: see `https://alea.education/hw/ai-2`
  - Submission of solutions via the ALeA system in the week after.
  - Peer grading/feedback (and master solutions) via answer classes.
- **Quality Control:** TAs and instructors will monitor and supervise peer grading.
- **Experiment:** Can we motivate enough of you to make peer assessment self-sustaining?
  - I am appealing to your sense of community responsibility here . . .
  - You should only expect others to grade your submission if you grade their's. (cf. Kant's "Moral Imperative")
  - **Make no mistake**: The grader usually learns at least as much as the gradee.
  - **Added Value**: The graders get to see answer classes. (⇝ intuition of our expectation)
- **Homework/Tutorial Discipline:**
  - Start early! (many assignments need more than one evening's work)
  - Don't start by sitting at a blank screen (talking & study groups help)

▷ Humans will be trying to understand the text/code/math when grading it.

▷ Go to the tutorials, discuss with your TA! (they are there for you!)

SOME RIGHTS RESERVED

If you have questions please make sure you discuss them with the instructor, the teaching assistants, or your fellow students. There are three sensible venues for such discussions: online in the lectures, in the tutorials, which we discuss now, or in the course forum – see below. Finally, it is always a very good idea to form study groups with your friends.

Tutorials for Artificial Intelligence 1

▷ **Approach:** Weekly tutorials and homework assignments. (first one in week two)

▷ **Goal 1:** Reinforce what was taught in the lectures. (you need practice)

▷ **Goal 2:** Allow you to ask any question you have in a protected environment.

▷ **Tutorials** organized by Prof. Dr. Florian Rabe

▷ Room: 11.137 @ Händler building, `florian.rabe@fau.de`

▷ **Tutorials** will start in week 3. (before, there is nothing to do)

Details (rooms, times, etc) will be announced in time (i.e. not now) on the matrix channel. (Also on `https://alea.education/FAU/ai-2/SS26`)

▷ **Life-saving Advice:** Go to your tutorial, and prepare for it by having looked at the slides and the homework assignments!

SOME RIGHTS RESERVED

Collaboration – in particular Study Groups

▷ **Definition 19.2.1.** Collaboration (or cooperation) is the process of groups of agents acting together for common, mutual benefit, as opposed to acting in competition for selfish benefit. In a collaboration, every agent contributes to the common goal and benefits from the contributions of others.

▷ In learning situations, the benefit is "better learning".

▷ **Observation:** In collaborative learning, the overall result can be significantly better than in competitive learning.

▷ **Good Practice:** Form study groups. (long- or short-term)

1. ⚠ Those learners who work/help most, learn most!
2. ⚠ Freeloaders – individuals who only watch – learn very little!

▷ It is OK to collaborate on homework assignments in AI-2! (no bonus points)

▷ Choose your study group well! (ALeA helps via the study buddy feature)

As we said above, almost all of the components of the AI-2 course are optional. That even applies to attendance. But make no mistake, attendance is important to most of you. Let me explain, ...

Do I need to attend the AI-2 Lectures

- Attendance is not mandatory for the AI-2 course. (official version)
- **Note:** There are two ways of learning: (both are OK, your mileage may vary)
  - Approach B: Read a book/papers (here: lecture notes)
  - Approach I: come to the lectures, be involved, interrupt the instructor whenever you have a question.

  The only advantage of I over B is that books/papers do not answer questions
- Approach S: come to the lectures and sleep does not work!
- The closer you get to research, the more we need to discuss!

Do use the opportunity to discuss the AI-2 topics with others. After all, one of the non-trivial skills you want to learn in the course is how to talk about artificial intelligence topics. And that takes practice, practice, and practice.

19.3 Learning Resources for AI-2

We will now discuss the learning resources we provide for helping AI-2 students learn. Of course there are other resources, students can profit from, but these are probably the most important ones.

Textbooks and supplementary Literature

- **Textbook:** *Russel/Norvig: Artificial Intelligence, A modern Approach* [RN21].
  - basically "broad but somewhat shallow"
  - great to get intuitions on the basics of AI. (broadly conceived)

  Make sure that you read the edition ≥ 3 $\rightsquigarrow$ vastly improved over ≤ 2.

The textbook above is great for background and supplemental reading, and while the AI-2 course covers a subset of the contents, [RN21] is not tailored to our course. We use local resources for that: ALeA for learner-adaptive course materials, the matrix for communication and announcements, and the FAU video portal for video streams and recordings.

Course Notes/Slides/Videos, Matrix

- **Lecture notes and slides** will be posted on ALeA at `https://alea.education/FAU/`

ai-2/SS26 as HTML and PDF

- ▷ via the NOTES and SLIDES icons. (slides view synchronized to lecture video)
- ▷ We mostly prepare/update them as we go along. (semantically preloaded ⇝ research resource)
- ▷ Please report any errors/shortcomings you notice. (improve for the group/successors)
  The best way to do this is via ALeA: select the fragment with the error, click on the yellow icon, give a good description of the error.

▷ **Matrix Channel:** https://matrix.to/#/#ai-12:fau.de for questions, discussion with instructors and among your fellow students. (your channel, use it!)
Login via FAU IDM ⇝ instructions

▷ **Course Videos** are https://www.fau.tv/series/artificial-intelligence-ii-ss26-s26.

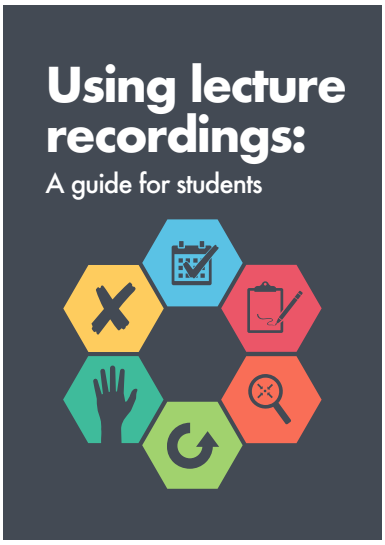
▷ **Do not let the videos mislead you:** Coming to class is highly correlated with passing the exam!

SOME RIGHTS RESERVED

FAU has issued a very insightful guide on using lecture videos. It is a good idea to heed these recommendations, even if they seem annoying at first.

Practical recommendations on Lecture Videos

▷ **Excellent Guide:** [Nor+18a] (German version at [Nor+18b])



- Attend lectures.
- Take notes.
- Be specific.
- Catch up.
- Ask for help.
- Don't cut corners.

SOME RIGHTS RESERVED

Now that we have discussed the resources for the AI-2 course, let us discuss something that is often misconstrued as a resource for learning: generative AI tools like LLM.

NOT a Resource for AI-2: LLMs – GenAI-based tools like ChatGPT

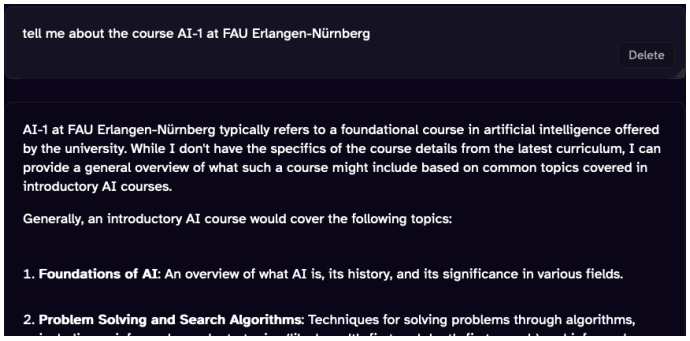
▷ **Definition 19.3.1.** A large language model (LLM) is a computational model capable of

language generation or other natural language processing tasks.

▷ **Example 19.3.2.** OpenAI's GPT, Mistral, or Google's Gemini. (there are many more)

▷ **Definition 19.3.3.** A chatbot is a software application or web interface that is designed to mimic human conversation through text or voice interactions. Modern chatbots are usually based on LLMs.

▷ **Example 19.3.4 (ChatGPT talks about AI-2).** (but remains vague)



▷ **Note:** LLM-based chatbots invent *every word*! (surprisingly often correct)

▷ **Example 19.3.5 (In the AI-2 exam).** ChatGPT scores ca. 50% of the points.

- ▷ ChatGPT can almost pass the exam ... (We could award it a Master's degree)
- ▷ But can you? (the AI-2 exams will be in person on paper)

You will only pass the exam, if you can do AI-2 yourself!

▷ **Intuition:** AI tools like ChatGPT, CoPilot, etc. (see also [She24])

- ▷ can help you solve problems, (valuable tools in production situations)
- ▷ hinders learning if used for homeworks/quizzes, etc. (like driving instead of jogging)

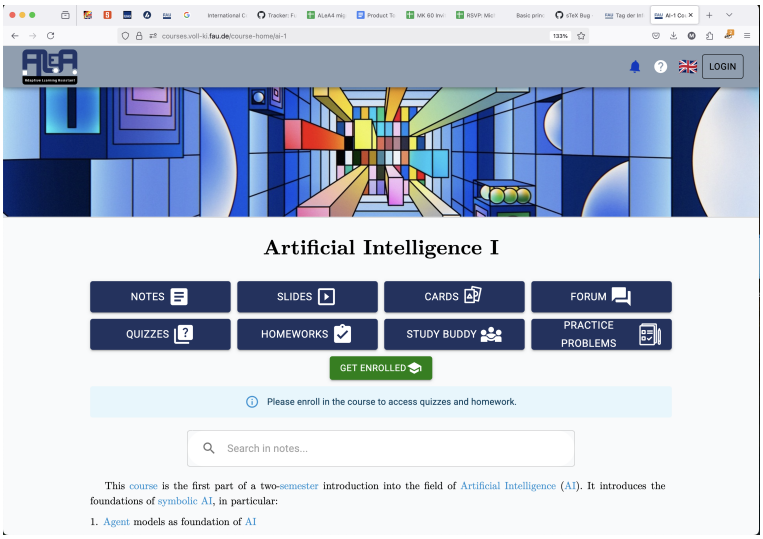
▷ **What (not) to do:** (to get most of the brave new AI-supported world)

- ▷ try out these tools to get a first-hand intuition what they can/cannot do
- ▷ challenge yourself while learning so that you can also do it (mind over matter!)

SOME RIGHTS RESERVED

ALEA in AI-2

▷ We assume that you already know the ALEA system from last semester



▷ Use it for

- ▷ lecture notes (notes- vs slides-oriented)
- ▷ flashcards (drill yourself on the AI-2 jargon/concepts)
- ▷ course forum (questions, discussions and error reporting)
- ▷ solving and peer-grading homework assignments
- ▷ finding study groups (you need not endure AI-2 alone)
- ▷ practicing with targeted problems (e.g. from old exams)
- ▷ doing the prepquizzes (before each lecture)

SOME RIGHTS RESERVED

Excursion: We will recap an introduction to ALEA system in theappendix.

Chapter 20

Overview over AI and Topics of AI-II

We restart the new semester by reminding ourselves of (the problems, methods, and issues of) artificial intelligence, and what has been achived so far.

20.1 Ways to Attack the AI Problem

There are currently three main avenues of attack to the problem of building artificially intelligent systems. The (historically) first is based on the symbolic representation of knowledge about the world and uses inference-based methods to derive new knowledge on which to base action decisions. The second uses statistical methods to deal with uncertainty about the world state and learning methods to derive new (uncertain) world assumptions to act on.

Four Main Approaches to Artificial Intelligence

▷ **Definition 20.1.1.** A software system is called symbolic, iff the manipulation of symbols, combining them into meaning-carrying structures (expressions) and manipulating them (using processes) to produce new expressions is the principal mode of operation.

Definition 20.1.2. Symbolic AI is a subfield of AI based on the assumption that many aspects of intelligence can be achieved symbolic software systems.

▷ **Definition 20.1.3.** Statistical AI remedies the two shortcomings of symbolic AI approaches: that all concepts represented by symbols are crisply defined, and that all aspects of the world are knowable/representable in principle. Statistical AI adopts sophisticated mathematical models of uncertainty and uses them to create more accurate world models and reason about them.

▷ **Definition 20.1.4.** Subsymbolic AI (also called connectionism or neural AI) is a subfield of AI that posits that intelligence is inherently tied to brains, where information is represented by a simple sequence pulses that are processed in parallel via simple calculations realized by neurons, and thus concentrates on neural computing.

▷ **Definition 20.1.5.** Embodied AI posits that intelligence cannot be achieved by reasoning about the state of the world (symbolically, statistically, or connectivist), but must be embodied i.e. situated in the world, equipped with a "body" that can interact with it via sensors and actuators. Here, the main method for realizing intelligent behavior is by learning from the world.

 SOME RIGHTS RESERVED

As a consequence, the field of artificial intelligence (AI) is an engineering field at the intersection of computer science (logic, programming, applied statistics), Cognitive Science (psychology, neuroscience), philosophy (can machines think, what does that mean?), linguistics (natural language understanding), and mechatronics (robot hardware, sensors).
Subsymbolic AI and in particular machine learning is currently hyped to such an extent, that many people take it to be synonymous with "Artificial Intelligence". It is one of the goals of this course to show students that this is a very impoverished view.

Two ways of reaching Artificial Intelligence?

▷ We can classify the AI approaches by their coverage and the analysis depth. (they are complementary)

Analysis ↑ vs. Coverage →	Narrow	Wide
Deep	symbolic AI-1	not there yet cooperation?
Shallow	no-one wants this	statistical/sub symbolic AI-2

▷ **This semester** we will cover foundational aspects of symbolic AI. (deep/narrow processing)

▷ **next semester** concentrate on statistical/subsymbolic AI. (shallow/wide-coverage)

 SOME RIGHTS RESERVED

We combine the topics in this way in this course, not only because this reproduces the historical development but also as the methods of statistical and subsymbolic AI share a common basis.

It is important to notice that all approaches to AI have their application domains and strong points. We will now see that exactly the two areas, where symbolic AI and statistical/subsymbolic AI have their respective fortes correspond to natural application areas.

Environmental Niches for both Approaches to AI

▷ **Observation:** There are two kinds of applications/tasks in AI

- ▷ Consumer tasks: consumer grade applications have tasks that must be fully generic and wide coverage. (e.g. machine translation like Google Translate)
- ▷ Producer tasks: producer grade applications must be high-precision, but can be domain-specific (e.g. multilingual documentation, machinery-control, program verification, medical technology)

Precision		
100%	Producer Tasks	
50%	Consumer Tasks	
	$10^{3\pm1}$ Concepts	$10^{6\pm1}$ Concepts Coverage

after Aarne Ranta [Ran17].

▷ **General Rule:** Subsymbolic AI is well suited for consumer tasks, while symbolic AI is better suited for producer tasks.

▷ A domain of producer tasks I am interested in: mathematical/technical documents.

SOME RIGHTS RESERVED

An example of a producer task – indeed this is where the name comes from – is the case of a machine tool manufacturer T, which produces digitally programmed machine tools worth multiple million Euro and sells them into dozens of countries. Thus T must also provide comprehensive machine operation manuals, a non-trivial undertaking, since no two machines are identical and they must be translated into many languages, leading to hundreds of documents. As those manual share a lot of semantic content, their management should be supported by AI techniques. It is critical that these methods maintain a high precision, operation errors can easily lead to very costly machine damage and loss of production. On the other hand, the domain of these manuals is quite restricted. A machine tool has a couple of hundred components only that can be described by a couple of thousand attributes only.

Indeed companies like T employ high-precision AI techniques like the ones we will cover in this course successfully; they are just not so much in the public eye as the consumer tasks.

20.2 AI in the KWARC Group

The KWARC Research Group

▷ **Observation:** The ability to represent knowledge about the world and to draw logical inferences is one of the central components of intelligent behavior.

▷ **Thus** reasoning components of some form are at the heart of many AI systems.

▷ **KWARC Angle:** Scaling up (web-coverage) without dumbing down (too much)

- ▷ Content markup instead of full formalization (too tedious)
- ▷ User support and quality control instead of "The Truth" (elusive anyway)
- ▷ use Mathematics as a test tube (⚠ Mathematics ≙ Anything Formal ⚠)
- ▷ care more about applications than about philosophy (we cannot help getting this right anyway as logicians)

▷ The KWARC group was established at Jacobs Univ. in 2004, moved to FAU Erlangen in 2016

▷ See `http://kwarc.info` for projects, publications, and links

Overview: KWARC Research and Projects

Applications: eMath 3.0, Active Documents, Active Learning, Semantic Spreadsheets/CAD/CAM, Change Mangagement, Global Digital Math Library, Math Search Systems, SMGloM: Semantic Multilingual Math Glossary, Serious Games, ...

Foundations of Math:	**KM & Interaction**:	**Semantization**:
▷ MathML, *OpenMath*	▷ Semantic Interpretation (aka. Framing)	▷ RusTeX: LaTeX ⇝ XML
▷ advanced Type Theories	▷ math-literate interaction	▷ sTeX: Semantic LaTeX
▷ Mmt: Meta Meta Theory	▷ MathHub: math archives & active docs	▷ invasive editors
▷ Logic Morphisms/Atlas	▷ Active documents: embedded semantic services	▷ Context-Aware IDEs
▷ Theorem Prover/CAS Interoperability	▷ Model-based Education	▷ Mathematical Corpora
▷ Mathematical Models/Simulation		▷ Linguistics of Math

Foundations: Computational Logic, Web Technologies, *OMDoc*/Mmt

Research Topics in the KWARC Group

- ▷ We are always looking for bright, motivated KWARCies.
- ▷ We have topics in for all levels! (Enthusiast, Bachelor, Master, Ph.D.)
- ▷ List of current topics: `https://gl.kwarc.info/kwarc/thesis-projects/`
  - ▷ Automated Reasoning: Maths Representation in the Large
  - ▷ Logics development, $(\text{Meta})^n$-Frameworks
  - ▷ Math Corpus Linguistics: Semantics Extraction
  - ▷ Serious Games, Cognitive Engineering, Math Information Retrieval, Legal Reasoning, ...
  - ▷ ... last but not least: KWARC is the home of ALeA!
- ▷ We always try to find a topic at the intersection of your and our interests.
- ▷ We also sometimes have positions!. (HiWi, Ph.D.: $\frac{1}{2}$ E-13, PostDoc: full E-13)

20.3 Agents and Environments in AI2

This part of the lecture notes addresses inference and agent decision making in partially observable environments, i.e. where we only know probabilities instead of certainties whether propositions are true/false. We cover basic probability theory and – based on that – Bayesian

Networks and simple decision making in such environments. Finally we extend this to probabilistic temporal models and their decision theory.

20.3.1 Recap: Rational Agents as a Conceptual Framework

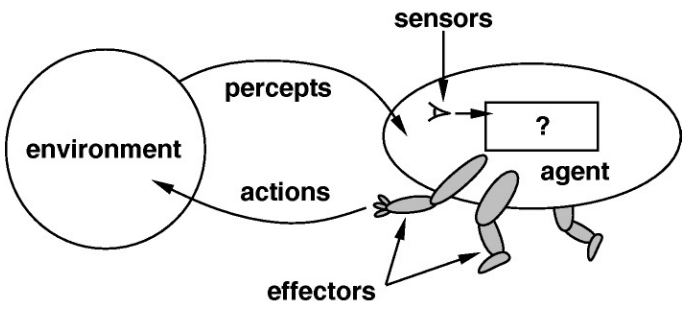
Agents and Environments

▷ **Definition 20.3.1.** An agent is anything that

- ▷ perceives its environment via sensors (a means of sensing the environment)
- ▷ acts on it with actuators (a means of changing the environment).

Any recognizable, coherent employment of the actuators of an agent is called an action. The abstract principle that agents, are capable of acting by themselves is called agency.

Given an agent A, we call the totality of other objects and factors that affect A its environment. Some aspects of the environment (i.e. objects, their properties, and relationships) are immutable, and some can be changed by A's actions or change independently of them. Any configuration of the mutable aspects of the environment is called a state of the environment.



▷ **Example 20.3.2.** Agents include humans, robots, softbots, thermostats, etc.

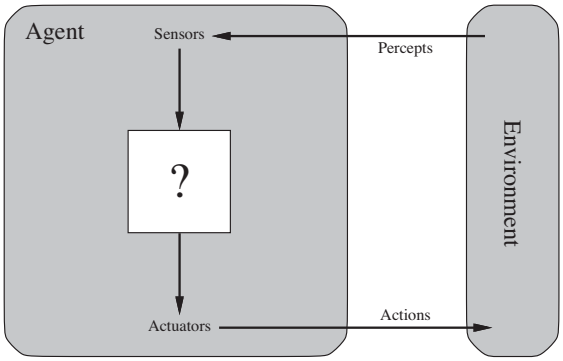
▷ **Remark:** The notion of an agent and its environment is intentionally designed to be inclusive. We will classify and discuss subclasses of both later.

SOME RIGHTS RESERVED

One possible objection to this is that the agent and the environment are conceptualized as separate entities; in particular, that the image suggests that the agent itself is not part of the environment. Indeed that is intended, since it makes thinking about agents and environments easier and is of little consequence in practice. In particular, the offending separation is relatively easily fixed if needed.

Agent Schema: Visualizing the Internal Agent Structure

▷ **Agent Schema:** We will use the following kind of agent schema to visualize the internal structure of an agent:



Different agents differ on the contents of the white box in the center.

SOME RIGHTS RESERVED

Rationality

- ▷ **Idea:** Try to design agents that are successful! (aka. "do the right thing")
- ▷ **Problem:** What do we mean by "successful", how do we measure "success"?
- ▷ **Definition 20.3.3.** A performance measure is a function that evaluates a sequence of environments.
- ▷ **Example 20.3.4.** A performance measure for a vacuum cleaner could
  - ▷ award one point per "square" cleaned up in time T?
  - ▷ award one point per clean "square" per time step, minus one per move?
  - ▷ penalize for $> k$ dirty squares?
- ▷ **Definition 20.3.5.** An agent is called rational, if it chooses whichever action maximizes the expected value of the performance measure given the percept sequence to date.
- ▷ **Observation:** Rationality is an "external property" of an agent:

  The agent (only) acts on the (external) environment, the performance measure evaluates how it changes.
- ▷ **Critical Observation:** We only need to maximize the expected value, not the actual value of the performance measure!
- ▷ **Question:** Why is rationality a good quality to aim for?

SOME RIGHTS RESERVED

Let us see how the observation that we only need to maximize the expected value, not the actual value of the performance measure affects the consequences.

Consequences of Rationality: Exploration, Learning, Autonomy

- ▷ **Note:** A rational agent need not be perfect:

- ▷ It only needs to maximize expected value (rational ≠ omniscient)
  - ▷ need not predict e.g. very unlikely but catastrophic events in the future.
- ▷ Percepts may not supply all relevant information. (rational ≠ clairvoyant)
  - ▷ if we cannot perceive things we do not need to react to them.
  - ▷ but we may need to try to find out about hidden dangers. (exploration)
- ▷ Action outcomes may not be as expected. (rational ≠ successful)
  - ▷ but we may need to take action to ensure that they do (more often). (learning)

▷ **Note:** Rationality may entail exploration, learning, autonomy. (depending on the environment / task)

▷ **Definition 20.3.6.** An agent is called autonomous, if it does not rely on the designer's prior knowledge about the environment.

▷ Autonomy avoids fixed behaviors that can become unsuccessful in a changing environment. (anything else would be irrational)

▷ The agent may have to learn all relevant traits, invariants, properties of the environment and actions.

SOME RIGHTS RESERVED

For the design of agent for a specific task – i.e. for choosing an agent architecture and designing an agent program, we have to take into account the performance measure, the environment, and the characteristics of the agent itself; in particular its actions and sensors.

PEAS: Describing the Task Environment

▷ **Observation:** To design a rational agent for a task, we must specify the

- ▷ task environment and performance measure as well as (external to the agent)
- ▷ the agent's actuators and sensors. (agent-specific)

Together these are called the PEAS components.

▷ **Example 20.3.7.** When designing an automated taxi:

- ▷ **Performance measure:** safety, destination, profits, legality, comfort, ...
- ▷ **Environment:** US streets/freeways, traffic, pedestrians, weather, ...
- ▷ **Actuators:** steering, accelerator, brake, horn, speaker/display, ...
- ▷ **Sensors:** Video, accelerometers, gauges, engine sensors, keyboard, GPS, ...

▷ **Example 20.3.8 (Internet Shopping Agent).** The task environment:

- ▷ Performance measure: price, quality, appropriateness, efficiency
- ▷ Environment: current and future WWW sites, vendors, shippers
- ▷ Actuators: display to user, follow URL, fill in form
- ▷ Sensors: HTML pages (text, graphics, scripts)

SOME RIGHTS RESERVED

The PEAS criteria are essentially a laundry list of what an agent design task description should

include.

Environment types

▷ **Observation 20.3.9.** *Agent design is largely determined by the type of environment it is intended for.*

▷ **Problem:** There is a vast number of possible kinds of environments in AI.

▷ **Solution:** Classify along a few "dimensions". (independent characteristics)

▷ **Definition 20.3.10.** For an agent a we classify the environment e of a by its type, which is acombination of the following. We call e

1. fully observable, iff the a's sensors give it access to the complete state of the environment at any point in time, else partially observable.
2. deterministic, iff the next state of the environment is completely determined by the current state and a's action, else stochastic.
3. episodic, iff a's experience is divided into atomic episodes, where it perceives and then performs a single action. Crucially, the next episode does not depend on previous ones. Non-episodic environments are called sequential.
4. dynamic, iff the environment can change without an action performed by a, else static.
5. discrete, iff the sets of e's state and a's actions are countable, else continuous.
6. single-agent, iff only a acts on e; else multi-agent

▷ **Note again:** We are not modeling environments directly, the environment type should influence the agent design! (the agent designer has to know, not the agent)

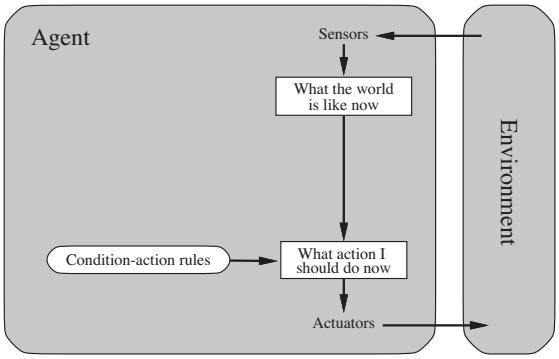
SOME RIGHTS RESERVED

The first kind of agent a reflex agent, we will look into is very simple – it is really only a strawman concept, so that we can build more realistic agent types on the concepts first introduced for the reflex agents later.

Reflex Agents

▷ **Definition 20.3.11.** An agent $\langle \mathcal{P}, \mathcal{A}, f \rangle$ is called a reflex agent, iff it only takes the last percept into account when choosing an action, i .e. $f(p_1, \ldots, p_k) = f(p_k)$ for all $p_1, \ldots, p_k \in \mathcal{P}$.

▷ **Agent Schema:** We choose the following agent architecture to implement reflex agents:



The crucial ingredient is a set of condition-action rules, which tell the reflex agent how to react to the incoming percepts.

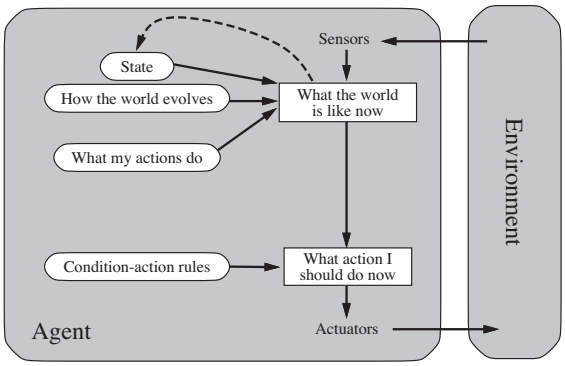
SOME RIGHTS RESERVED

Model-based Reflex Agents: Idea

▷ **Problem:** Reflex agents have not way of keeping track of the state of the environment.

▷ **Idea:** Equip agents with a world model $\widehat{=}$ data structure that represents the state of the environment.

▷ **Agent Schema:** Instead of acting directly on the percepts, the model-based agent maintains a world model and triggers its actions on that. (reflexes at a higher level)



▷ **Observation:** The world model is influenced by the agent's percepts and actions. (see below)

SOME RIGHTS RESERVED

Model-based Reflex Agents: Definition

▷ **Definition 20.3.12.** A model-based agent $\langle \mathcal{P}, \mathcal{A}, \mathcal{S}, \mathcal{T}, s_0, S, a \rangle$ is an agent $\langle \mathcal{P}, \mathcal{A}, f \rangle$ whose actions depend on

1. a world model: a set $\mathcal{S}$ of states, and a start state $s_0 \in \mathcal{S}$.
2. a transition model $\mathcal{T}: \mathcal{S} \times \mathcal{A} \to \mathcal{S}$, that predicts a new state $\mathcal{T}(s,a)$ from a state s and an action a.
3. a sensor model $S: \mathcal{S} \times \mathcal{P} \to \mathcal{S}$ that given a state s and a percept p determine a new state $S(s,p)$.
4. an action function $a: \mathcal{S} \to \mathcal{A}$ that given a state selects the next action.

If the world model of a model-based agent A is in state s and A has last taken action a, and now perceives p, then A will transition to state $s' := S(p, \mathcal{T}(s,a))$ and take action $a' := a(s')$.

So, given a sequence $p_1, \ldots, p_n$ of percepts, we recursively define states $s_n = S(\mathcal{T}(s_{n-1}, a(s_{n-1})), p_n)$ with $s_1 = S(s_0, p_1)$. Then the agent function is defined to be $f(p_1, \ldots, p_n) := a(s_n)$.

▷ **Note (crucially):** The world model is different from the environment and its state!

The first (only) models the latter; agent success depends on how well!

▷ **Note:** A different percept sequences lead to different states, so the agent function $f: \mathcal{P}^* \to \mathcal{A}$ no longer only depends on the last percept.

▷ **Example 20.3.13 (Tail Lights Again).** Model-based agents can do the the tail lights example (see ???) if the states include a concept of tail light brightness.

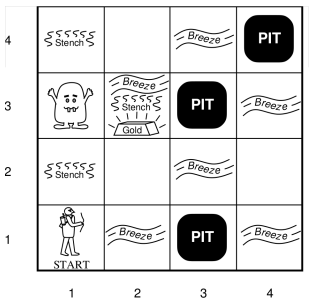
▷ **Note:** Model-based agents only add memory to agents; actions are still determined by "reflex" (condition-action rules).

SOME RIGHTS RESERVED

20.3.2 Sources of Uncertainty

Sources of Uncertainty in Decision-Making

Where's that d... Wumpus?
And where am I, anyway??



▷ **Non-deterministic Actions:**

▷ "When I try to go forward in this dark cave, I might actually go forward-left or forward-right."

▷ **Partial observability with unreliable sensors:**

▷ "Did I feel a breeze right now?";

▷ "I think I might smell a Wumpus here, but I got a cold and my nose is blocked."

▷ "According to the heat scanner, the Wumpus is probably in cell [2,3]."

▷ **Uncertainty about the domain behavior:**

▷ "Are you *sure* the Wumpus never moves?"

SOME RIGHTS RESERVED

Unreliable Sensors

▷ **Robot Localization:** Suppose we want to support localization using landmarks to narrow down the area.

▷ **Example 20.3.14.** "*If you see the Eiffel tower, then you're in Paris.*"

▷ **Difficulty:** Sensors can be imprecise.

▷ Even if a landmark is perceived, we cannot conclude with certainty that the robot is at that location.

▷ "*This is the half-scale Las Vegas copy, you dummy.*"

▷ Even if a landmark is *not* perceived, we cannot conclude with certainty that the robot is *not* at that location.

▷ "*Top of Eiffel tower hidden in the clouds.*"

▷ Only the probability of being at a location increases or decreases.

SOME RIGHTS RESERVED

20.3.3 Agent Architectures based on Belief States

We are now ready to proceed to environments which can only partially observed and where actions are non deterministic. Both sources of uncertainty conspire to allow us only partial knowledge about the world, so that we can only optimize "expected utility" instead of "actual utility" of our actions.

World Models for Uncertainty

▷ **Problem:** We do not know with certainty what state the world is in!

▷ **Idea:** Just keep track of all the possible states it could be in.

▷ **Definition 20.3.15.** A model-based agent has a world model consisting of

▷ a belief state that has information about the possible states the world may be in,

▷ a sensor model that updates the belief state based on sensor information, and

▷ a transition model that updates the belief state based on actions.

▷ **Idea:** The agent environment determines what the world model can be.

▷ In a fully observable, deterministic environment,

▷ we can observe the initial state and subsequent states are given by the actions alone.

▷ Thus the belief state is a singleton (we call its sole member the world state) and the transition model is a function from states and actions to states: a transition function.

SOME RIGHTS RESERVED

That is exactly what we have been doing until now: we have been studying methods that build on descriptions of the "actual" world, and have been concentrating on the progression from atomic to factored and ultimately structured representations. Tellingly, we spoke of "world states" instead of "belief states"; we have now justified this practice in the brave new belief-based world models by the (re-) definition of "world states" above. To fortify our intuitions, let us recap from a belief-state-model perspective.

World Models by Agent Type in AI-1

▷ **Search-based Agents:** In a fully observable, deterministic environment

- ▷ goal-based agent with world state $\widehat{=}$ "current state"
- ▷ no inference. (goal $\widehat{=}$ goal state from search problem)

▷ **CSP-based Agents:** In a fully observable, deterministic environment

- ▷ goal-based agent withworld state $\widehat{=}$ constraint network,
- ▷ inference $\widehat{=}$ constraint propagation. (goal $\widehat{=}$ satisfying assignment)

▷ **Logic-based Agents:** In a fully observable, deterministic environment

- ▷ model-based agent with world state $\widehat{=}$ logical formula
- ▷ inference $\widehat{=}$ e.g. DPLL or resolution.

▷ **Planning Agents:** In a fully observable, deterministic, environment

- ▷ goal-based agent with world state $\widehat{=}$ PL0, transition model $\widehat{=}$ STRIPS,
- ▷ inference $\widehat{=}$ state/plan space search. (goal: complete plan/execution)

SOME RIGHTS RESERVED

Let us now see what happens when we lift the restrictions of total observability and determinism.

World Models for Complex Environments

▷ In a fully observable, but stochastic environment,

- ▷ the belief state must deal with a set of possible states.
- ▷ $\rightsquigarrow$ generalize the transition function to a transition relation.

▷ **Note:** This even applies to online problem solving, where we can just perceive the state. (e.g. when we want to optimize utility)

▷ In a deterministic, but partially observable environment,

- ▷ the belief state must deal with a set of possible states.

- ▷ we can use transition functions.
- ▷ We need a sensor model, which predicts the influence of percepts on the belief state – during update.

▷ In a stochastic, partially observable environment,

- ▷ mix the ideas from the last two. (sensor model + transition relation)

SOME RIGHTS RESERVED

Preview: New World Models (Belief) ⇝ new Agent Types

▷ **Probabilistic Agents:** In a partially observable environment

- ▷ belief state $\widehat{=}$ Bayesian networks,
- ▷ inference $\widehat{=}$ probabilistic inference.

▷ **Decision-Theoretic Agents:** In a partially observable, stochastic environment

- ▷ belief state + transition model $\widehat{=}$ decision networks,
- ▷ inference $\widehat{=}$ maximizing expected utility.

▷ We will study them in detail this semester.

SOME RIGHTS RESERVED

Overview: AI2

▷ Basics of probability theory (probability spaces, random variables, conditional probabilities, independence,...)

▷ Probabilistic reasoning: Computing the *a posteriori* probabilities of events given evidence, causal reasoning (Representing distributions efficiently, Bayesian networks,...)

▷ Probabilistic reasoning over time (Markov chains, Hidden Markov models,...)

⇝ We can update our world model episodically based on observations (i.e. sensor data)

▷ Decision theory: Making decisions under uncertainty (Preferences, Utilities, Decision networks, Markov Decision Processes,...)

⇝ We can choose the right action based on our world model and the likely outcomes of our actions

▷ Machine learning: Learning from data (Decision Trees, Classifiers, Neural Networks,...)

SOME RIGHTS RESERVED

Part I

Reasoning with Uncertain Knowledge

This part of the lecture notes addresses inference and agent decision making in partially observable environments, i.e. where we only know probabilities instead of certainties whether propositions are true/false. We cover basic probability theory and – based on that – Bayesian Networks and simple decision making in such environments. Finally we extend this to probabilistic temporal models and their decision theory.

Chapter 21

Quantifying Uncertainty

In this chapter we develop a machinery for dealing with uncertainty: Instead of thinking about what we know to be true, we must think about what is likely to be true.

21.1 Probability Theory

21.1.1 Prior and Posterior Probabilities

Probabilistic Models (Intuitions)

▷ **Definition 21.1.1.** A probability theory is an assertion language for talking about possible worlds and an inference method for quantifying the degree of belief in such assertions.

▷ **Remark:** Like logic, but for non binary belief degree.

▷ The possible worlds are

- ▷ mutually exclusive: different possible worlds cannot both be the case and
- ▷ exhaustive: one possible world must be the case.

▷ This determines the set of possible worlds.

▷ **Example 21.1.2.** If we roll two (distinguishable) dice with six sides, then we have 36 possible worlds: $(1,1)$, $(2,1)$, $\ldots$, $(6,6)$.

▷ We will restrict ourselves to a discrete, countable sample space. (others more complicated, less useful in AI)

SOME RIGHTS RESERVED

Probabilistic Models

▷ **Definition 21.1.3 (Mathematically (slightly simplified)).** A probability space or (probability model) is a pair $\langle \Omega, P \rangle$ such that:

- ▷ Ω is a set of outcomes (called the sample space),
- ▷ P is a function $\mathcal{P}(\Omega) \to [0,1]$, such that:

▷ $P(\Omega) = 1$ and

▷ $P(\bigcup_i A_i) = \sum_i P(A_i)$ for all pairwise disjoint $A_i \in \mathcal{P}(\Omega)$.

P is called a probability measure.

These properties are called the Kolmogorov axioms.

▷ **Intuition:** We run some experiment, the outcome of which is any $\omega \in \Omega$.

▷ For $X \subseteq \Omega$, $P(X)$ is the probability that the result of the experiment is *any one* of the outcomes in X.

▷ Naturally, the probability that *any* outcome occurs is 1 (hence $P(\Omega) = 1$).

▷ The probability of pairwise disjoint sets of outcomes should just be the sum of their probabilities.

▷ **Example 21.1.4 (Dice throws).** Assume we throw a (fair) die two times. Then the sample space Ω is $\{(i,j) \mid 1 \leq i,j \leq 6\}$. We define P by letting $P(\{A\}) = \frac{1}{36}$ for every $A \in \Omega$.

Since the probability of any outcome is the same, we say P is uniformly distributed.

▷ **Connection to Graded Belief States:** The notion of a probability space fits very well:

▷ The sample space Ω can be seen as a belief state (a set of possible worlds), and

▷ the probability measure P tells us how likely the respective worlds (a.k.a. outcomes) are the actual worlds.

SOME RIGHTS RESERVED

The definition is simplified in two places: Firstly, we assume that P is defined on the full power set. This is not always possible, especially if Ω is uncountable. In that case we need an additional set of "events" instead, and lots of mathematical machinery to make sure that we can safely take unions, intersections, complements etc. of these events.

Secondly, we would technically only demand that P is additive on countably many disjoint sets.

In this course we will assume that our sample space is at most countable anyway; usually even finite.

Random Variables

▷ In practice, we are rarely interested in the *specific* outcome of an experiment, but rather in some *property* of the outcome. This is especially true in the very common situation where we don't even *know* the precise probabilities of the individual outcomes.

▷ **Example 21.1.5.** The probability that the *sum* of our two dice throws is 7 is $P(\{(i,j) \in \Omega \mid i+j=7\}) = P(\{(6,1),(1,6),(5,2),(2,5),(4,3),(3,4)\}) = \frac{6}{36} = \frac{1}{6}$.

▷ **Definition 21.1.6 (Again, slightly simplified).** Let D be a set. A random variable is a function $X : \Omega \to D$. We call D (somewhat confusingly) the domain of X, denoted $\mathrm{dom}(X)$.

For $x \in D$, we define the probability of x as $P(X = x) := P(\{\omega \in \Omega \mid X(\omega) = x\})$.

▷ **Definition 21.1.7.** We say that a random variable X is finite domain, iff its domain $\mathrm{dom}(X)$ is finite and Boolean, iff $\mathrm{dom}(X) = \{\mathsf{T}, \mathsf{F}\}$.

For a Boolean random variable, we will simply write $P(X)$ for $P(X = \mathsf{T})$ and $P(\neg X)$ for $P(X = \mathsf{F})$.

Note that a random variable, according to the formal definition, is *neither* random *nor* a variable: It is a function with clearly defined domain and codomain – and what we call the domain of the "variable" is actually its codomain... are you confused yet? ☺

This confusion is a side effect of the *mathematical* formalism. In practice, a random variable is some indeterminate value that results from some statistical experiment – i.e. it is *random*, because the result is not predetermined, and it is a variable, because it can take on different values.

It just so happens that if we want to model this scenario *mathematically*, a function is the most natural way to do so.

Some Examples

▷ **Example 21.1.8.** Summing up our two dice throws is a random variable $S\colon \Omega \to [2,12]$ with $S((i,j)) = i+j$. The probability that they sum up to 7 is written as $P(S=7) = \frac{1}{6}$.

▷ **Example 21.1.9.** The first and second of our two dice throws are random variables $\mathrm{First}, \mathrm{Second}\colon \Omega \to [1,6]$ with $\mathrm{First}((i,j)) = i$ and $\mathrm{Second}((i,j)) = j$.

▷ *Remark 21.1.10.* Note, that the *identity* $\Omega \to \Omega$ is a random variable as well.

▷ **Example 21.1.11.** We can model toothache, cavity and gingivitis as Boolean random variables, with the underlying probability space being...?? ¯_(ツ)_/¯

▷ **Example 21.1.12.** We can model tomorrow's weather as a random variable with domain $\{\texttt{sunny}, \texttt{rainy}, \texttt{foggy}, \texttt{warm}, \texttt{cloudy}, \texttt{humid}, \ldots\}$, with the underlying probability space being...?? ¯_(ツ)_/¯

⇝ This is why *probabilistic reasoning* is necessary: We can rarely reduce probabilistic scenarios down to clearly defined, fully known probability spaces and derive all the interesting things from there.

But: The definitions here allow us to *reason* about probabilities and random variables in a *mathematically* rigorous way, e.g. to make our intuitions and assumptions precise, and prove our methods to be *sound*.

Propositions

▷ This is nice and all, but in practice we are interested in "compound" probabilities like:

"What is the probability that the sum of our two dice throws is 7, but neither of the two dice is a 3?"

▷ **Idea:** Reuse the syntax of propositional logic and define the logical connectives for random variables!

▷ **Example 21.1.13.** We can express the above as: $P(\neg(\mathrm{First}=3) \wedge \neg(\mathrm{Second}=3) \wedge (S=7))$

▷ **Definition 21.1.14.** Let X_1, X_2 be random variables, $x_1 \in \mathrm{dom}(X_1)$ and $x_2 \in \mathrm{dom}(X_2)$. We define:

1. $P(X_1 \neq x_1) := P(\neg(X_1 = x_1)) := P(\{\omega \in \Omega \mid X_1(\omega) \neq x_1\}) = 1 - P(X_1 = x_1)$.

2. $P((X_1 = x_1) \wedge (X_2 = x_2)) := P(\{\omega \in \Omega \mid (X_1(\omega) = x_1) \wedge (X_2(\omega) = x_2)\}) = P(\{\omega \in \Omega \mid X_1(\omega) = x_1\} \cap \{\omega \in \Omega \mid X_2(\omega) = x_2\})$.
3. $P((X_1 = x_1) \vee (X_2 = x_2)) := P(\{\omega \in \Omega \mid (X_1(\omega) = x_1) \vee (X_2(\omega) = x_2)\}) = P(\{\omega \in \Omega \mid X_1(\omega) = x_1\} \cup \{\omega \in \Omega \mid X_2(\omega) = x_2\})$.

It is also common to write $P(A, B)$ for $P(A \wedge B)$

▷ **Example 21.1.15.** $P((\text{First} \neq 3) \wedge (\text{Second} \neq 3) \wedge (S = 7)) = P(\{(1,6),(6,1),(2,5),(5,2)\}) = \frac{1}{9}$

SOME RIGHTS RESERVED

Events

▷ **Definition 21.1.16 (Again slightly simplified).** Let $\langle \Omega, P \rangle$ be a probability space. An event is a subset of Ω.

▷ **Definition 21.1.17 (Convention).** We call an event (by extension) anything that *represents* a subset of Ω: any statement formed from the logical connectives and values of random variables, on which $P(\cdot)$ is defined.

▷ **Problem 1.1 (Remember)**

We can define $A \vee B := \neg(\neg A \wedge \neg B)$, $\mathsf{T} := A \vee \neg A$ and $\mathsf{F} := \neg \mathsf{T}$ — is this compatible with the definition of probabilities on propositional formulae? And why is $P(X_1 \neq x_1) = 1 - P(X_1 = x_1)$?

▷ **Problem 1.2 (Inclusion-Exclusion-Principle)**

Prove or refute that $P(A \vee B) = P(A) + P(B) - P(A \wedge B)$.

▷ **Problem 1.3**

Prove or refute that $P(A) = P(A \wedge B) + P(A \wedge \neg B)$

SOME RIGHTS RESERVED

Conditional Probabilities

▷ **Observation:** As we gather new information, our beliefs (*should*) change, and thus our probabilities!

▷ **Example 21.1.18.** Your "probability of missing the connecting train" increases when you are informed that your current train has 30 minutes delay.

▷ **Example 21.1.19.** The "probability of cavity" increases when the doctor is informed that the patient has a toothache.

▷ **Example 21.1.20.** The probability that $S = 3$ is clearly higher if I know that $\text{First} = 1$ than otherwise – or if I know that $\text{First} = 6$!

▷ **Definition 21.1.21.** Let A and B be events where $P(B) \neq 0$. The conditional probability of A given B is defined as:

$$P(A \mid B) := \frac{P(A \wedge B)}{P(B)}$$

We also call $P(A)$ the prior probability of A, and $P(A \mid B)$ the posterior probability.

▷ **Intuition:** If we *assume* B to hold, then we are only interested in the "part" of Ω where A is true *relative to* B.

▷ **Alternatively:** We restrict our sample space Ω to the subset of outcomes where B holds. We then define a new probability space on this subset by scaling the probability measure so that it sums to 1 – which we do by dividing by $P(B)$. (We "*update our beliefs* based on new evidence")

SOME RIGHTS RESERVED

Examples

▷ **Example 21.1.22.** If we assume $\text{First} = 1$, then $P(S = 3 \mid (\text{First} = 1))$ should be precisely $P(\text{Second} = 2) = \frac{1}{6}$. We check:

$$P(S = 3 \mid (\text{First} = 1)) = \frac{P((S = 3) \wedge (\text{First} = 1))}{P(\text{First} = 1)} = \frac{1/36}{1/6} = \frac{1}{6}$$

▷ **Example 21.1.23.** Assume the prior probability $P(\text{cavity})$ is 0.122. The probability that a patient has both a cavity and a toothache is $P(\text{cavity} \wedge \text{toothache}) = 0.067$. The probability that a patient has a toothache is $P(\text{toothache}) = 0.15$.

If the patient complains about a toothache, we can update our estimation by calculate the posterior probability:

$$P(\text{cavity} \mid \text{toothache}) = \frac{P(\text{cavity} \wedge \text{toothache})}{P(\text{toothache})} = \frac{0.067}{0.15} = 0.45.$$

▷ **Note:** We just calculated the probability of some underlying *disease* based on the presence of a *symptom*!

▷ **More Generally:** We calculated the probability of a *cause* from observing its *effect*.

SOME RIGHTS RESERVED

Some Rules

▷ Equations on unconditional probabilities have direct analogues for conditional probabilities.

▷ **Problem 1.4**

Convince yourself of the following:

- $P(A \mid C) = 1 - P(\neg A \mid C)$.
- $P(A \mid C) = P(A \wedge B \mid C) + P(A \wedge \neg B \mid C)$.

▷ $P(A \vee B \mid C) = P(A \mid C) + P(B \mid C) - P(A \wedge B \mid C)$.

▷ But **not on the right hand side!**

▷ **Problem 1.5**

Find counterexamples for the following (**false**) claims:

▷ $P(A \mid C) = 1 - P(A \mid \neg C)$

▷ $P(A \mid C) = P(A \mid (B \wedge C)) + P(A \mid (B \wedge \neg C))$.

▷ $P(A \mid (B \vee C)) = P(A \mid B) + P(A \mid C) - P(A \mid (B \wedge C))$.

SOME RIGHTS RESERVED

x0

Bayes' Rule

▷ **Note:** By definition, $P(A \mid B) = \frac{P(A \wedge B)}{P(B)}$. In practice, we often know the conditional probability already, and use it to calculate the probability of the conjunction instead: $P(A \wedge B) = P(A \mid B) \cdot P(B) = P(B \mid A) \cdot P(A)$.

▷ **Theorem 21.1.24 (Bayes' Theorem).** *Given propositions A and B where $P(A) \neq 0$ and $P(B) \neq 0$, we have:*

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

▷ *Proof:*

1. $P(A \mid B) = \frac{P(A \wedge B)}{P(B)} = \frac{P(B \mid A) \cdot P(A)}{P(B)}$

□

...okay, that was straightforward... what's the big deal?

▷ **(Somewhat Dubious) Claim:** Bayes' rule is the entire scientific method condensed into a single equation!

▷ This is an extreme overstatement, but there is a grain of truth in it.

SOME RIGHTS RESERVED

Bayes' Theorem - Why the Hype?

▷ Say we have a *hypothesis* H about the world. (e.g. "The universe had a beginning")

▷ We have *some prior belief* $P(H)$.

▷ We gather *evidence* E. (e.g. "We observe a cosmic microwave background at 2.7K everywhere")

▷ Bayes' rule tells us how to *update our belief* in H based on H's ability to *predict* E (the *likelihood* $P(E \mid H)$) – *and*, importantly, the ability of *competing hypotheses* to predict the *same* evidence. (This is actually how scientific hypotheses should be evaluated)

$$\underbrace{P(H \mid E)}_{\text{posterior}} = \frac{P(E \mid H) \cdot P(H)}{P(E)} = \frac{\overbrace{P(E \mid H)}^{\text{likelihood}} \cdot \overbrace{P(H)}^{\text{prior}}}{\underbrace{P(E \mid H)}_{\text{likelihood}} \underbrace{P(H)}_{\text{prior}} + \underbrace{P(E \mid \neg H) P(\neg H)}_{\text{competition}}}$$

. . . if I keep gathering evidence and update, ultimately the impact of the prior belief will diminish.

"You're entitled to your own priors, but not your own likelihoods"

SOME RIGHTS RESERVED

21.1.2 Independence

Independence

▷ **Question:** What is the probability that $S = 7$ *and* the patient has a toothache?

Or less contrived: What is the probability that the patient has a gingivitis *and* a cavity?

▷ **Definition 21.1.25.** Two events A and B are called independent, iff $P(A \wedge B) = P(A) \cdot P(B)$.

Two random variables X_1, X_2 are called independent, iff for all $x_1 \in \mathrm{dom}(X_1)$ and $x_2 \in \mathrm{dom}(X_2)$, the events $X_1 = x_1$ and $X_2 = x_2$ are independent. We write $A \perp B$ or $X_1 \perp X_2$, respectively.

▷ **Theorem 21.1.26.** *Equivalently: Given events A and B with $P(B) \neq 0$, then A and B are independent iff $P(A \mid B) = P(A)$ (equivalently: $P(B \mid A) = P(B)$).*

▷ *Proof:*

1. $\Rightarrow$
   By definition, $P(A \mid B) = \frac{P(A \wedge B)}{P(B)} = \frac{P(A) \cdot P(B)}{P(B)} = P(A)$,
2. $\Leftarrow$
   Assume $P(A \mid B) = P(A)$.
   Then $P(A \wedge B) = P(A \mid B) \cdot P(B) = P(A) \cdot P(B)$.

□

▷ **Note:** Independence asserts that two events are *"not related"* – the probability of one does not depend on the other.

Mathematically, we can *determine* independence by checking whether $P(A \wedge B) = P(A) \cdot P(B)$.

In practice, this is impossible to check. Instead, we *assume* independence based on *domain knowledge*, and then *exploit* this to compute $P(A \wedge B)$.

Independence (Examples)

▷ **Example 21.1.27.**

- ▷ $\text{First} = 2$ and $\text{Second} = 3$ are independent – more generally, First and Second are independent (The outcome of the first die throw does not affect the outcome of the second die)
  Quick check: $P((\text{First} = a) \wedge (\text{Second} = b)) = \frac{1}{36} = P(\text{First} = a) \cdot P(\text{Second} = b)$ ✓
- ▷ First and S are **not** independent. (The outcome of the first die affects the sum of the two dice.) Counterexample: $P((\text{First} = 1) \wedge (S = 4)) = \frac{1}{36} \neq P(\text{First} = 1) \cdot P(S = 4) = \frac{1}{6} \cdot \frac{1}{2} = \frac{1}{72}$
- ▷ **But:** $P((\text{First} = a) \wedge (S = 7)) = \frac{1}{36} = \frac{1}{6} \cdot \frac{1}{6} = P(\text{First} = a) \cdot P(S = 7)$ – so the events $\text{First} = a$ and $S = 7$ *are* independent. (Why?)

▷ **Example 21.1.28.**

- ▷ Are cavity and toothache independent?
  . . . since cavities can cause a toothache, that would probably be a bad design decision . . .
- ▷ Are cavity and gingivitis independent? Cavities do not cause gingivitis, and gingivitis does not cause cavities, so... yes... right? (...as far as I know. I'm not a dentist.)
- ▷ **Probably not!** A patient who has cavities has probably worse dental hygiene than those who don't, and is thus more likely to have gingivitis as well.
- ▷ $\rightsquigarrow$ cavity may be *evidence* that raises the probability of gingivitis, even if they are not directly causally related.

Conditional Independence – Motivation

▷ A dentist can diagnose a cavity by using a *probe*, which may (or may not) *catch* in a cavity.

▷ Say we know from clinical studies that $P(\text{cavity}) = 0.2$, $P(\text{toothache} \mid \text{cavity}) = 0.6$, $P(\text{toothache} \mid \neg\text{cavity}) = 0.1$, $P(\text{catch} \mid \text{cavity}) = 0.9$, and $P(\text{catch} \mid \neg\text{cavity}) = 0.2$.

▷ Assume the patient complains about a toothache, and our probe indeed catches in the aching tooth. What is the likelihood of having a cavity $P(\text{cavity} \mid (\text{toothache} \wedge \text{catch}))$?

▷ **Idea:** Use Bayes' rule:

$$P(\text{cavity} \mid (\text{toothache} \wedge \text{catch})) = \frac{P(\text{toothache} \wedge \text{catch} \mid \text{cavity}) \cdot P(\text{cavity})}{P(\text{toothache} \wedge \text{catch})}$$

▷ **Note:** $P(\text{toothache} \wedge \text{catch}) = P(\text{toothache} \wedge \text{catch} \mid \text{cavity}) \cdot P(\text{cavity}) + P(\text{toothache} \wedge \text{catch} \mid \neg\text{cavity}) \cdot P(\neg\text{cavity})$

▷ **Problem:** Now we're only missing $P(\text{toothache} \wedge \text{catch} \mid (\text{cavity} = b))$ for $b \in \{\mathsf{T}, \mathsf{F}\}$.
. . . Now what?

▷ Are toothache and catch independent, maybe? **No**: Both have a common (possible) cause, cavity.

Also, there's this pesky $P(\cdot \mid \text{cavity})$ in the way.....wait a minute...

SOME RIGHTS RESERVED

Conditional Independence – Definition

▷ *Assuming* the patient has (or does not have) a cavity, the events toothache and catch are independent: Both are caused by a cavity, but they don't influence each other otherwise.

i.e. cavity "contains all the information" that links toothache and catch in the first place.

▷ **Definition 21.1.29.** Given events A, B, C with $P(C) \neq 0$, then A and B are called conditionally independent given C, iff $P(A \wedge B \mid C) = P(A \mid C) \cdot P(B \mid C)$.

Equivalently: iff $P(A \mid (B \wedge C)) = P(A \mid C)$, or $P(B \mid (A \wedge C)) = P(B \mid C)$.

Let Y be a random variable. We call two random variables X_1, X_2 conditionally independent given Y, iff for all $x_1 \in \mathbf{dom}(X_1)$, $x_2 \in \mathbf{dom}(X_2)$ and $y \in \mathbf{dom}(Y)$, the events $X_1 = x_1$ and $X_2 = x_2$ are conditionally independent given $Y = y$.

▷ **Example 21.1.30.** Let's assume toothache and catch are conditionally independent given cavity/$\neg$cavity. Then we can finally calculate:

$$P(\text{cavity} \mid (\text{toothache} \wedge \text{catch})) = \frac{P(\text{toothache} \wedge \text{catch} \mid \text{cavity}) \cdot P(\text{cavity})}{P(\text{toothache} \wedge \text{catch})}$$

$$= \frac{P(\text{toothache} \mid \text{cavity}) \cdot P(\text{catch} \mid \text{cavity}) \cdot P(\text{cavity})}{P(\text{toothache} \mid \text{cavity}) \cdot P(\text{catch} \mid \text{cavity}) \cdot P(\text{cavity}) + P(\text{toothache} \mid \neg\text{cavity}) \cdot P(\text{catch} \mid \neg\text{cavity}) \cdot P(\neg\text{cavity})} = \frac{0.6 \cdot 0.9 \cdot 0.2}{0.6 \cdot 0.9 \cdot 0.2 + 0.1 \cdot v 0.2 \cdot 0.8} = 0.8$$

SOME RIGHTS RESERVED

Conditional Independence

▷ **Lemma 21.1.31.** *If A and B are conditionally independent given C, then $P(A \mid (B \wedge C)) = P(A \mid C)$*

Proof:

$$P(A \mid (B \wedge C)) = \frac{P(A \wedge B \wedge C)}{P(B \wedge C)} = \frac{P(A \wedge B \mid C) \cdot P(C)}{P(B \wedge C)} = \frac{P(A \mid C) \cdot P(B \mid C) \cdot P(C)}{P(B \wedge C)} = \frac{P(A \mid C) \cdot P(B \wedge C)}{P(B \wedge C)} = P(A \mid C)$$

□

▷ **Question:** If A and B are conditionally independent given C, does this imply that A and B are independent? **No.** See previous slides for a counterexample.

▷ **Question:** If A and B are independent, does this imply that A and B are also conditionally independent given C? **No.** For example: First and Second are independent, but not conditionally independent given $S = 4$.

▷ **Question:** Okay, so what if A, B and C are *all* pairwise independent? Are A and B conditionally independent given C *now*? **Still no.** Remember: First $= a$, Second $= b$ and

$S = 7$ are all independent, but First and Second are not conditionally independent given $S = 7$.

▷ **Question:** When can we infer conditional independence from a "more general" notion of independence?

We need *mutual independence*. Roughly: A set of events is called *mutually independent*, if every event is independent from *any conjunction of the others*. (Not really relevant for this course though)

SOME RIGHTS RESERVED

21.1.3 Conclusion

Summary

▷ Probability spaces serve as a mathematical model (and hence justification) for everything related to probabilities.

▷ The "atoms" of any statement of probability are the random variables. (Important special cases: Boolean and finite domain)

▷ We can define probabilities on compound (propositional logical) statements, with (outcomes of) random variables as "propositional variables".

▷ Conditional probabilities represent *posterior probabilities* given some observed outcomes.

▷ Independence and conditional independence are strong assumptions that allow us to simplify calculations of probabilities

▷ Bayes' Theorem (can be used between "causal" and "diagnostic" conditional probabilities)

SOME RIGHTS RESERVED

So much about the math...

▷ We now have a mathematical setup for probabilities.

▷ **But:** The math does not tell us what probabilities *are*:

▷ Assume we can mathematically derive this to be the case: *the probability of rain tomorrow is* 0.3. What does this even *mean*?

▷ **Frequentist Answer:** The probability of an event is the limit of its relative frequency in a large number of trials.

In other words: "In 30% of the cases where we have similar weather conditions, it rained the next day."

▷ **Objection:** Okay, but what about *unique* events? "The probability of me passing the exam is 80%" – does this mean anything, if I only take the exam once? Am I comparable to "similar students"? What counts as sufficiently "similar"?

▷ **Bayesian Answer:** Probabilities are *degrees of belief*. It means you **should** be 30% confident that it will rain tomorrow.

▷ **Objection:** And why *should* I? Is this not purely *subjective* then?

SOME RIGHTS RESERVED

Pragmatics

▷ **Pragmatically** both interpretations amount to the same thing: I should *act as if* I'm 30% confident that it will rain tomorrow. (Whether by fiat, or because in 30% of comparable cases, it rained.)

▷ **Objection:** Still: why should I? And why should my beliefs follow the seemingly arbitrary Kolmogorov axioms?

▷ [DF31]: If an agent has a belief that violates the Kolmogorov axioms, then there exists a combination of "bets" on propositions so that the agent *always* loses money.

▷ **In other words:** If your beliefs are not consistent with the mathematics, and you *act in accordance with your beliefs*, there is a way to exploit this inconsistency to your disadvantage.

▷ . . . and, more importantly, the AI agents you design! ☺

▷ **I (and my agents) do not bet:** That is not true, in a partially observable or non-deterministic environment, every action choice is a necessarily bet: The outcome is not sure.

SOME RIGHTS RESERVED

21.2 Probabilistic Reasoning Techniques

Okay, now how do I implement this?

▷ This is a computer science course. We need to implement this stuff.

▷ Do we... implement random variables as functions? Is a probability space a... class maybe?

▷ **No:** As mentioned, we rarely know the probability space entirely. Instead we will use probability distributions, which are just arrays (of arrays of...) of probabilities.

▷ And then we represent *those* as sparsely as possible, by exploiting independence, conditional independence, . . .

SOME RIGHTS RESERVED

21.2.1 Probability Distributions

Probability Distributions

▷ **Definition 21.2.1.** The probability distribution for a random variable X, written $\mathbb{P}(X)$, is the vector of probabilities for the (ordered) domain of X.

▷ **Note:** The values in a probability distribution are all positive and sum to 1. (Why?)

▷ **Example 21.2.2.** $\mathbb{P}(\text{First}) = \mathbb{P}(\text{Second}) = \langle \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6} \rangle$. (Both First and Second are uniformly distributed)

▷ **Example 21.2.3.** The probability distribution $\mathbb{P}(S)$ is $\langle \frac{1}{36}, \frac{1}{18}, \frac{1}{12}, \frac{1}{9}, \frac{5}{36}, \frac{1}{6}, \frac{5}{36}, \frac{1}{9}, \frac{1}{12}, \frac{1}{18}, \frac{1}{36} \rangle$. Note the symmetry, with a "peak" at 7 – the random variable is (*approximately*, because our domain is discrete rather than continuous) normally distributed (or gaussian distributed, or follows a bell-curve,...).

▷ **Example 21.2.4.** Probability distributions for Boolean random variables are naturally *pairs* (probabilities for T and F), e.g.:

$$\mathbb{P}(\text{toothache}) = \langle 0.15, 0.85 \rangle$$
$$\mathbb{P}(\text{cavity}) = \langle 0.122, 0.878 \rangle$$

▷ More generally:

▷ **Definition 21.2.5.** A probability distribution is a vector $\mathbf{v}$ of values $\mathbf{v}_i \in [0,1]$ such that $\sum_i \mathbf{v}_i = 1$.

SOME RIGHTS RESERVED

The Full Joint Probability Distribution

▷ **Definition 21.2.6.** Given random variables $X_1, \ldots, X_n$ with domains $D_1, \ldots, D_n$, the full joint probability distribution, denoted $\mathbb{P}(X_1, \ldots, X_n)$, is the n-dimensional array of size $|D_1 \times \ldots \times D_n|$ that lists the probabilities of all conjunctions of values of the random variables.

▷ **Example 21.2.7.** $\mathbb{P}(\text{cavity}, \text{toothache}, \text{gingivitis})$ could look something like this:

	toothache		¬toothache	
	gingivitis	¬gingivitis	gingivitis	¬gingivitis
cavity	0.007	0.06	0.005	0.05
¬cavity	0.08	0.003	0.045	0.75

▷ **Example 21.2.8.** $\mathbb{P}(\text{First}, S)$

First \ S	2	3	4	5	6	7	8	9	10	11	12
1	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	0	0	0	0	0
2	0	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	0	0	0	0
3	0	0	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	0	0	0
4	0	0	0	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	0	0
5	0	0	0	0	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	0
6	0	0	0	0	0	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$

Note that if we know the value of First, the value of S is completely determined by the value of Second.

Conditional Probability Distributions

▷ **Definition 21.2.9.** Given random variables X and Y, the conditional probability distribution of X given Y, written $\mathbb{P}(X|Y)$ is the table of all conditional probabilities of values of X given values of Y.

▷ For sets of variables analogously: $\mathbb{P}(X_1, \ldots, X_n|Y_1, \ldots, Y_m)$.

▷ **Example 21.2.10.** $\mathbb{P}(\text{cavity}|\text{toothache})$:

	toothache	¬toothache
cavity	$P(\text{cavity} \mid \text{toothache}) = 0.45$	$P(\text{cavity} \mid \neg\text{toothache}) = 0.065$
¬cavity	$P(\neg\text{cavity} \mid \text{toothache}) = 0.55$	$P(\neg\text{cavity} \mid \neg\text{toothache}) = 0.935$

▷ **Example 21.2.11.** $\mathbb{P}(\text{First}|S)$

First \ S	2	3	4	5	6	7	8	9	10	11	12
1	1	$\frac{1}{2}$	$\frac{1}{3}$	$\frac{1}{4}$	$\frac{1}{5}$	$\frac{1}{6}$	0	0	0	0	0
2	0	$\frac{1}{2}$	$\frac{1}{3}$	$\frac{1}{4}$	$\frac{1}{5}$	$\frac{1}{6}$	$\frac{1}{5}$	0	0	0	0
3	0	0	$\frac{1}{3}$	$\frac{1}{4}$	$\frac{1}{5}$	$\frac{1}{6}$	$\frac{1}{5}$	$\frac{1}{4}$	0	0	0
4	0	0	0	$\frac{1}{4}$	$\frac{1}{5}$	$\frac{1}{6}$	$\frac{1}{5}$	$\frac{1}{4}$	$\frac{1}{3}$	0	0
5	0	0	0	0	$\frac{1}{5}$	$\frac{1}{6}$	$\frac{1}{5}$	$\frac{1}{4}$	$\frac{1}{3}$	$\frac{1}{2}$	0
6	0	0	0	0	0	$\frac{1}{6}$	$\frac{1}{5}$	$\frac{1}{4}$	$\frac{1}{3}$	$\frac{1}{2}$	1

▷ **Note:** Every "column" of a conditional probability distribution is itself a probability distribution.
(Why?)

Convention

▷ We now "lift" multiplication and division to the level of whole probability distributions:

▷ **Definition 21.2.12.** Whenever we use $\mathbb{P}$ in an equation, we take this to mean a *system of equations*, for each value in the domains of the random variables involved.

Example 21.2.13.

▷ $\mathbb{P}(X, Y) = \mathbb{P}(X|Y) \cdot \mathbb{P}(Y)$ represents the system of equations $P(X = x \wedge Y = y) = P(X = x \mid Y = y) \cdot P(Y = y)$ for all x, y in the respective domains.

▷ $\mathbb{P}(X|Y) := \frac{\mathbb{P}(X,Y)}{\mathbb{P}(Y)}$ represents the system of equations $P(X = x \mid (Y = y)) := \frac{P((X=x)\wedge(Y=y))}{P(Y=y)}$

▷ Bayes' Theorem: $\mathbb{P}(X|Y) = \frac{\mathbb{P}(Y|X)\cdot\mathbb{P}(X)}{\mathbb{P}(Y)}$ represents the system of equations $P(X = x \mid (Y = y)) = \frac{P(Y=y \mid (X=x))\cdot P(X=x)}{P(Y=y)}$

So, what's the point?

▷ Obviously, the probability distribution contains all the information about a specific random variable we need.

▷ **Observation:** The full joint probability distribution of variables $X_1, \ldots, X_n$ contains *all* the information about the random variables *and their conjunctions* we need.

▷ **Example 21.2.14.** We can read off the probability $P(\text{toothache})$ from the full joint probability distribution as $0.007+0.06+0.08+0.003=0.15$, and the probability $P(\text{toothache} \wedge \text{cavity})$ as $0.007 + 0.06 = 0.067$

▷ We can actually implement this! (They're just (nested) arrays)

▷ **But:** just as we often don't have a fully specified probability space to work in, we often don't have a full joint probability distribution for our random variables either.

▷ **Also:** Given random variables $X_1, \ldots, X_n$, the full joint probability distribution has $\prod_{i=1}^{n} |\text{dom}(X_i)|$ entries! ($\mathbb{P}(\text{First}, S)$ already has 60 entries!)

▷ **So:** The rest of this section deals with keeping things small, by *computing* probabilities instead of *storing* them all.

SOME RIGHTS RESERVED

Probabilistic Reasoning

▷ **Probabilistic reasoning** refers to inferring probabilities of events from the probabilities of other events

as opposed to determining the probabilities e.g. *empirically*, by gathering (sufficient amounts of *representative*) data and counting.

▷ **Note:** In practice, we are *primarily* interested in, and have access to, conditional probabilities rather than the unconditional probabilities of conjunctions of events:

- ▷ We don't reason in a vacuum: Usually, we have some evidence and want to infer the posterior probability of some related event. (e.g. infer a plausible *cause* given some *symptom*)

  ⇝ we are interested in the conditional probability $P(\text{hypothesis} \mid \text{observation})$.
- ▷ *"80% of patients with a cavity complain about a toothache"* (i.e. $P(\text{toothache} \mid \text{cavity})$) is more the kind of data people actually collect and publish than *"1.2% of the general population have both a cavity and a toothache"* (i.e. $P(\text{cavity} \wedge \text{toothache})$).
- ▷ Consider the probe catching in a cavity. The probe is a diagnostic tool, which is usually evaluated in terms of its *sensitivity* $P(\text{catch} \mid \text{cavity})$ and *specificity* $P(\neg\text{catch} \mid \neg\text{cavity})$. (You have probably heard these words a lot since 2020...)

SOME RIGHTS RESERVED

21.2.2 Naive Bayes

Naive Bayes Models

▷ Consider again the dentistry example with random variables cavity, toothache, and catch. We assume cavity **causes** both toothache and catch, and that toothache and catch are conditionally independent given cavity:



▷ We likely know the *sensitivity* $P(\text{catch} \mid \text{cavity})$ and *specificity* $P(\neg\text{catch} \mid \neg\text{cavity})$, which jointly give us $\mathbb{P}(\text{catch}|\text{cavity})$, and from medical studies, we should be able to determine $P(\text{cavity})$ (the *prevalence* of cavities in the population) and $\mathbb{P}(\text{toothache}|\text{cavity})$.

▷ This kind of situation is surprisingly common, and therefore deserves a name.

SOME RIGHTS RESERVED

Naive Bayes Models



▷ **Definition 21.2.15.** A naive Bayes model (or, less accurately, Bayesian classifier, or, derogatorily, idiot Bayes model) consists of:

1. random variables $C, E_1, \ldots, E_n$ such that all the $E_1, \ldots, E_n$ are conditionally independent given C,
2. the probability distribution $\mathbb{P}(C)$, and
3. the conditional probability distributions $\mathbb{P}(E_i|C)$.

We call C the cause and the $E_1, \ldots, E_n$ the effects of the model.

▷ **Convention:** Whenever we draw a graph of random variables, we take the arrows to connect *causes* to their direct *effects*, and assert that unconnected nodes are conditionally independent given all their ancestors. We will make this more precise later.

▷ Can we compute the full joint probability distribution $\mathbb{P}(\text{cavity}, \text{toothache}, \text{catch})$ from this information?

SOME RIGHTS RESERVED

Recovering the Full Joint Probability Distribution

▷ **Lemma 21.2.16 (Product rule).** $\mathbb{P}(X, Y) = \mathbb{P}(X|Y) \cdot \mathbb{P}(Y)$.

▷ We can generalize this to more than two variables, by repeatedly applying the product rule:

▷ **Lemma 21.2.17 (Chain rule).** *For any sequence of random variables* $X_1, \ldots, X_n$*:*

$$\mathbb{P}(X_1, \ldots, X_n) = \mathbb{P}(X_1|X_2, \ldots, X_n) \cdot \mathbb{P}(X_2|X_3, \ldots X_n) \cdot \ldots \cdot \mathbb{P}(X_{n-1}|X_n) \cdot \mathbb{P}(X_n)$$

Hence:

▷ **Theorem 21.2.18.** *Given a naive Bayes model with effects* $E_1, \ldots, E_n$ *and cause* C*, we have*

$$\mathbb{P}(C, E_1, \ldots, E_n) = \mathbb{P}(C) \cdot (\prod_{i=1}^{n} \mathbb{P}(E_i|C)).$$

▷ *Proof:* Using the chain rule:

1. $\mathbb{P}(E_1, \ldots, E_n, C) = \mathbb{P}(E_1|E_2, \ldots, E_n, C) \cdot \ldots \cdot \mathbb{P}(E_n|C) \cdot \mathbb{P}(C)$
2. Since all the E_i are conditionally independent, we can drop them on the right hand sides of the $\mathbb{P}(E_j|..., C)$

□

SOME RIGHTS RESERVED

Marginalization

▷ Great, so now we can compute $\mathbb{P}(C|E_1, \ldots, E_n) = \frac{\mathbb{P}(C, E_1, \ldots, E_n)}{\mathbb{P}(E_1, \ldots, E_n)}$...

...except that we don't know $\mathbb{P}(E_1, \ldots, E_n)$:-/

...except that we can compute the full joint probability distribution, so we can recover it:

▷ **Lemma 21.2.19 (Marginalization).** *Given random variables* $X_1, \ldots, X_n$ *and* $Y_1, \ldots, Y_m$*, we have* $\mathbb{P}(X_1, \ldots, X_n) = \sum_{y_1 \in \mathrm{dom}(Y_1), \ldots, y_m \in \mathrm{dom}(Y_m)} \mathbb{P}(X_1, \ldots, X_n, Y_1 = y_1, \ldots, Y_m = y_m)$.

(This is just a fancy way of saying "we can add the relevant entries of the full joint probability distribution")

▷ **Example 21.2.20.** Say we observed $\mathrm{toothache} = \mathrm{T}$ and $\mathrm{catch} = \mathrm{T}$. Using marginalization, we can compute

$$\begin{aligned} P(\text{cavity} \mid (\text{toothache} \wedge \text{catch})) &= \frac{P(\text{cavity} \wedge \text{toothache} \wedge \text{catch})}{P(\text{toothache} \wedge \text{catch})} \\ &= \frac{P(\text{cavity} \wedge \text{toothache} \wedge \text{catch})}{\sum_{c \in \{\text{cavity}, \neg\text{cavity}\}} P(c \wedge \text{toothache} \wedge \text{catch})} \\ &= \frac{P(\text{cavity}) \cdot P(\text{toothache} \mid \text{cavity}) \cdot P(\text{catch} \mid \text{cavity})}{\sum_{c \in \{\text{cavity}, \neg\text{cavity}\}} P(c) \cdot P(\text{toothache} \mid c) \cdot P(\text{catch} \mid c)} \end{aligned}$$

SOME RIGHTS RESERVED

Unknowns

▷ What if we don't know catch? (I'm not a dentist, I don't have a probe...)

▷ We split our effects into $\{E_1, \ldots, E_n\} = \{O_1, \ldots, O_{n_O}\} \cup \{U_1, \ldots, U_{n_U}\}$ – the *observed*

and *unknown* random variables.

▷ Let $D_U := \mathrm{dom}(U_1) \times \ldots \times \mathrm{dom}(U_{n_u})$. Then

$$\begin{aligned}\mathbb{P}(C|O_1,\ldots,O_{n_O}) &= \frac{\mathbb{P}(C,O_1,\ldots,O_{n_O})}{\mathbb{P}(O_1,\ldots,O_{n_O})}\\ &= \frac{\sum_{u\in D_U}\mathbb{P}(C,O_1,\ldots,O_{n_O},U_1=u_1,\ldots,U_{n_u}=u_{n_u})}{\sum_{c\in\mathrm{dom}(C)}\sum_{u\in D_U}\mathbb{P}(O_1,\ldots,O_{n_O},C=c,U_1=u_1,\ldots,U_{n_u}=u_{n_u})}\\ &= \frac{\sum_{u\in D_U}\mathbb{P}(C)\cdot(\prod_{i=1}^{n_O}\mathbb{P}(O_i|C))\cdot(\prod_{j=1}^{n_U}\mathbb{P}(U_j=u_j|C))}{\sum_{c\in\mathrm{dom}(C)}\sum_{u\in D_U}P(C=c)\cdot(\prod_{i=1}^{n_O}\mathbb{P}(O_i|C=c))\cdot(\prod_{j=1}^{n_U}P(U_j=u_j\mid(C=c)))}\\ &= \frac{\mathbb{P}(C)\cdot(\prod_{i=1}^{n_O}\mathbb{P}(O_i|C))\cdot(\sum_{u\in D_U}\prod_{j=1}^{n_U}\mathbb{P}(U_j=u_j|C))}{\sum_{c\in\mathrm{dom}(C)}P(C=c)\cdot(\prod_{i=1}^{n_O}\mathbb{P}(O_i|C=c))\cdot(\sum_{u\in D_U}\prod_{j=1}^{n_U}P(U_j=u_j\mid(C=c)))}\end{aligned}$$

...oof...

Unknowns

▷ Continuing from above:

$$\mathbb{P}(C|O_1,\ldots,O_{n_O}) = \frac{\mathbb{P}(C)\cdot(\prod_{i=1}^{n_O}\mathbb{P}(O_i|C))\cdot(\sum_{u\in D_U}\prod_{j=1}^{n_U}\mathbb{P}(U_j=u_j|C))}{\sum_{c\in\mathrm{dom}(C)}P(C=c)\cdot(\prod_{i=1}^{n_O}\mathbb{P}(O_i|C=c))\cdot(\sum_{u\in D_U}\prod_{j=1}^{n_U}P(U_j=u_j\mid(C=c)))}$$

▷ First, note that $\sum_{u\in D_U}\prod_{j=1}^{n_U}P(U_j=u_j\mid(C=c))=1$ (We're summing over all possible events on the (conditionally independent) $U_1,\ldots,U_{n_U}$ given $C=c$)

▷

$$\mathbb{P}(C|O_1,\ldots,O_{n_O}) = \frac{\mathbb{P}(C)\cdot(\prod_{i=1}^{n_O}\mathbb{P}(O_i|C))}{\sum_{c\in\mathrm{dom}(C)}P(C=c)\cdot(\prod_{i=1}^{n_O}\mathbb{P}(O_i|C=c))}$$

▷ Secondly, note that the *denominator* is

1. the same for any given observations $O_1,\ldots,O_{n_O}$, independent of the value of C, and
2. the *sum* over all the *numerators* in the full distribution.

That is: The denominator only serves to *scale* what is *almost* already the distribution $\mathbb{P}(C|O_1,\ldots,O_{n_O})$ to sum up to 1.

Normalization

▷ **Definition 21.2.21 (Normalization).** Given a vector $w := \langle w_1,\ldots,w_k\rangle$ of numbers in $[0,1]$ where $\sum_{i=1}^k w_i \leq 1$.

Then the normalized vector $\alpha(w)$ is defined (component-wise) as

$$(\alpha(w))_i := \frac{w_i}{\sum_{j=1}^k w_j}.$$

Note that $\sum_{i=1}^{k} \alpha(w)_i = 1$, i.e. $\alpha(w)$ is a probability distribution.

▷ This finally gives us:

Theorem 21.2.22 (Inference in a Naive Bayes model). *Let $C, E_1, \ldots, E_n$ a naive Bayes model and $E_1, \ldots, E_n = O_1, \ldots, O_{n_O}, U_1, \ldots, U_{n_U}$.*

Then

$$\mathbb{P}(C|O_1 = o_1, \ldots, O_{n_O} = o_{n_O}) = \alpha(\mathbb{P}(C) \cdot (\prod_{i=1}^{n_O} \mathbb{P}(O_i = o_i|C)))$$

▷ Note, that this is entirely independent of the *unknown* random variables $U_1, \ldots, U_{n_U}$!

▷ Also, note that this is just a fancy way of saying "first, compute all the numerators, then divide all of them by their sums".

SOME RIGHTS RESERVED

Dentistry Example

▷ Putting things together, we get:

$$\begin{aligned}\mathbb{P}(\text{cavity}|\text{toothache} = \mathsf{T}) &= \alpha(\mathbb{P}(\text{cavity}) \cdot \mathbb{P}(\text{toothache} = \mathsf{T}|\text{cavity})) \\ &= \alpha(\langle P(\text{cavity}) \cdot P(\text{toothache} \mid \text{cavity}), P(\neg\text{cavity}) \cdot P(\text{toothache} \mid \neg\text{cavity})\rangle)\end{aligned}$$

▷ Say we have $P(\text{cavity}) = 0.1$, $P(\text{toothache} \mid \text{cavity}) = 0.8$, and $P(\text{toothache} \mid \neg\text{cavity}) = 0.05$. Then

$$\mathbb{P}(\text{cavity}|\text{toothache} = \mathsf{T}) = \alpha(\langle 0.1 \cdot 0.8, 0.9 \cdot 0.05\rangle) = \alpha(\langle 0.08, 0.045\rangle)$$

$0.08 + 0.045 = 0.125$, hence

$$\mathbb{P}(\text{cavity}|\text{toothache} = \mathsf{T}) = \langle \frac{0.08}{0.125}, \frac{0.045}{0.125}\rangle = \langle 0.64, 0.36\rangle$$

SOME RIGHTS RESERVED

Naive Bayes Classification

▷ **Observation:** We can use a naive Bayes model as a very simple classifier.

▷ Assume we want to classify newspaper articles as one of the categories *politics*, *sports*, *business*, *fluff*, etc. based on the words they contain.

▷ Given a large set of articles, we can determine the relevant probabilities by counting the occurrences of the categories $\mathbb{P}(\text{category})$, and of words per category – i.e. $\mathbb{P}(\text{word}_i|\text{category})$ for some (huge) list of words $(\text{word}_i)_{i=1}^{n}$.

▷ We assume that the occurrence of each word is conditionally independent of the occurrence of any other word given the category of the document. (This assumption is clearly wrong, but it makes the model simple and often works well in practice.) ($\rightsquigarrow$ "Idiot Bayes model")

▷ Given a new article, we just count the occurrences k_i of the words in it and compute

$$\mathbb{P}(\text{category}|\text{word}_1 = k_1, \ldots, \text{word}_n = k_n) = \alpha(\mathbb{P}(\text{category}) \cdot (\prod_{i=1}^{n} \mathbb{P}(\text{word}_i = k_i|\text{category})))$$

▷ We then choose the category with the highest probability.

SOME RIGHTS RESERVED

21.2.3 Inference by Enumeration

Inference by Enumeration

▷ The rules we established for naive Bayes models, i.e. Bayes's theorem, the product rule and chain rule, marginalization and normalization, are *general* techniques for probabilistic reasoning, and their usefulness is not limited to the naive Bayes models.

▷ More generally:

▷ **Theorem 21.2.23 (Inference by Enumeration).** *Let $Q, E_1, \ldots, E_{n_E}, U_1, \ldots, U_{n_U}$ be random variables and $D := \text{dom}(U_1) \times \ldots \times \text{dom}(U_{n_U})$. Then*

$$\mathbb{P}(Q|E_1 = e_1, \ldots, E_{n_E} = e_{n_e}) = \alpha(\sum_{u \in D} \mathbb{P}(Q, E_1 = e_1, \ldots, E_{n_E} = e_{n_e}, U_1 = u_1, \ldots, U_{n_U} = u_{n_U}))$$

We call Q the query variable, $E_1, \ldots, E_{n_E}$ the evidence, and $U_1, \ldots, U_{n_U}$ the hidden variables, and computing a conditional probability this way enumeration.

▷ Note that this is just a "mathy" way of saying we

1. sum over all relevant entries of the full joint probability distribution of the variables, and
2. normalize the result to yield a probability distribution.

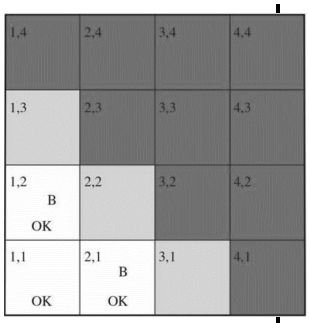
SOME RIGHTS RESERVED

21.2.4 Example – The Wumpus is Back

We will fortify our intuition about naive Bayes models with a variant of the Wumpus world we looked at above to understand whether logic was up to the job of guiding an agent in the Wumpus cave.

Example: The Wumpus is Back

▷ We have a maze where



- ▷ Every cell except $[1,1]$ possibly contains a *pit*, with 20% probability.
- ▷ pits cause a *breeze* in neighboring cells (we forget the Wumpus and the gold for now)

▷ Where should the agent go, if there is a breeze at $[1,2]$ and $[2,1]$?

▷ Pure logical inference can conclude nothing about which cell is *most likely* to be safe!

▷ We can model this using the Boolean random variables:

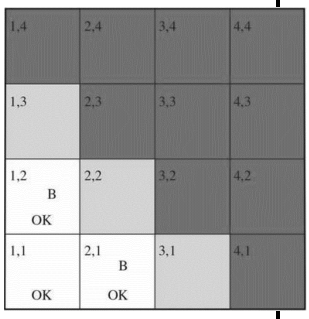
- ▷ $P_{i,j}$ for $i,j \in \{1,2,3,4\}$, stating there is a pit at cell $[i,j]$, and
- ▷ $B_{i,j}$ for $(i,j) \in \{(1,1),(1,2),(2,1)\}$, stating there is a breeze at cell $[i,j]$

⇝ let's apply our machinery!

SOME RIGHTS RESERVED

Wumpus: Probabilistic Model

▷ **First:** Let's try to compute the full joint probability distribution $\mathbb{P}(P_{1,1},\ldots,P_{4,4},B_{1,1},B_{1,2},B_{2,1})$.



1. By the product rule, this is equal to $\mathbb{P}(B_{1,1},B_{1,2},B_{2,1}|P_{1,1},\ldots,P_{4,4})\cdot\mathbb{P}(P_{1,1},\ldots,P_{4,4})$.
2. Note that $\mathbb{P}(B_{1,1},B_{1,2},B_{2,1}|P_{1,1},\ldots,P_{4,4})$ is either 1 (if all the $B_{i,j}$ are consistent with the positions of the pits $P_{k,l}$) or 0 (otherwise).
3. Since the pits are spread independently, we have $\mathbb{P}(P_{1,1},\ldots,P_{4,4}) = \prod_{i,j=1,1}^{4,4}\mathbb{P}(P_{i,j})$

▷ ⇝ We know all of these probabilities.

▷ ⇝ We can now use enumeration to compute $\mathbb{P}(P_{i,j}|<known>) = \alpha(\sum_{<unknowns>}\mathbb{P}(P_{i,j},<known>,<unknowns>))$

SOME RIGHTS RESERVED

Wumpus Continued

▷ **Problem:** We only know $P_{i,j}$ for three cells. If we want to compute e.g. $P_{1,3}$ via enumeration, that leaves $2^{4^2-4} = 4096$ sumands!

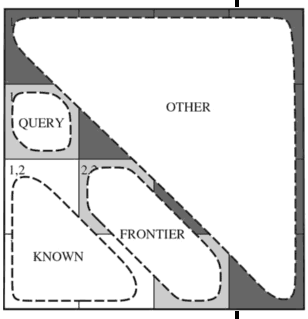
▷ **Let's do better.**

▷ Let $b := \neg B_{1,1} \wedge B_{1,2} \wedge B_{2,1}$ (All the breezes we know about)

▷ Let $p := \neg P_{1,1} \wedge \neg P_{1,2} \wedge \neg P_{2,1}$. (All the pits we know about)

▷ Let $F := \{P_{3,1} \wedge P_{2,2}, \neg P_{3,1} \wedge P_{2,2}, P_{3,1} \wedge \neg P_{2,2}, \neg P_{3,1} \wedge P_{2,2}\}$ (the current *"frontier"*)

▷ Let O be (the set of assignments for) all the other variables $P_{i,j}$. (i.e. except p, F and our query $P_{1,3}$)



Then the observed breezes b are conditionally independent of O given p and F. (Whether there is a pit anywhere else does not influence the breezes we observe.)

▷ $\rightsquigarrow P(b \mid P_{1,3},p,O,F) = P(b \mid P_{1,3},p,F)$. Let's exploit this!

SOME RIGHTS RESERVED

Optimized Wumpus

▷ In particular:

$$\begin{aligned}
\mathbb{P}(P_{1,3}|p,b) &= \alpha(\sum_{o\in O, f\in F} \mathbb{P}(P_{1,3},b,p,f,o)) = \alpha(\sum_{o\in O, f\in F} P(b \mid P_{1,3},p,o,f)\cdot \mathbb{P}(P_{1,3},p,f,o)) \\
&= \alpha(\sum_{f\in F}\sum_{o\in O} P(b \mid P_{1,3},p,f)\cdot \mathbb{P}(P_{1,3},p,f,o)) = \alpha(\sum_{f\in F} P(b \mid P_{1,3},p,f)\cdot(\sum_{o\in O}\mathbb{P}(P_{1,3},p,f,o))) \\
&= \alpha(\sum_{f\in F} P(b \mid P_{1,3},p,f)\cdot(\sum_{o\in O}\mathbb{P}(P_{1,3})\cdot P(p)\cdot P(f)\cdot P(o))) \\
&= \alpha(\mathbb{P}(P_{1,3})\cdot P(p)\cdot(\sum_{f\in F}\underbrace{P(b \mid P_{1,3},p,f)}_{\in\{0,1\}}\cdot P(f)\cdot(\underbrace{\sum_{o\in O} P(o)}_{=1})))
\end{aligned}$$

$\rightsquigarrow$ this is just a sum over the frontier, i.e. 4 summands ☺

▷ So: $\mathbb{P}(P_{1,3}|p,b) = \alpha(\langle 0.2\cdot(0.8)^3\cdot(1\cdot 0.04+1\cdot 0.16+1\cdot 0.16+0), 0.8\cdot(0.8)^3\cdot(1\cdot 0.04+1\cdot 0.16+0+0)\rangle) \approx \langle 0.31, 0.69\rangle$

▷ Analogously: $\mathbb{P}(P_{3,1}|p,b) = \langle 0.31, 0.69\rangle$ and $\mathbb{P}(P_{2,2}|p,b) = \langle 0.86, 0.14\rangle$ ($\rightsquigarrow$ avoid $[2,2]$!)

SOME RIGHTS RESERVED

Cooking Recipe

▷ In general, when you want to reason probabilistically, a good heuristic is:

1. Try to frame the full joint probability distribution in terms of the probabilities you know. Exploit product rule/chain rule, independence, conditional independence, marginalization **and domain knowledge** (as e.g. $\mathbb{P}(b|p,f) \in \{0,1\}$)

   $\rightsquigarrow$ the problem can be solved at all!

2. **Simplify**: Start with the equation for enumeration:

$$\mathbb{P}(Q|E_1,...) = \alpha(\sum_{u\in U} \mathbb{P}(Q, E_1, ..., U_1 = u_1, ...))$$

3. Substitute by the result of 1., and again, exploit all of our machinery
4. Implement the resulting (system of) equation(s)
5. ???
6. Profit

SOME RIGHTS RESERVED

Summary

▷ Probability distributions and conditional probability distributions allow us to represent random variables as convenient datastructures in an implementation (Assuming they are finite domain...)

▷ The full joint probability distribution allows us to compute all probabilities of statements about the random variables contained (But possibly inefficient)

▷ Marginalization and normalization are the specific techniques for extracting the *specific* probabilities we are interested in from the full joint probability distribution.

▷ The product and chain rule, exploiting (conditional) independence, Bayes' Theorem, and of course *domain specific* knowledge allow us to do so much more efficiently.

▷ Naive Bayes models are one example where all these techniques come together.

SOME RIGHTS RESERVED

Chapter 22

Probabilistic Reasoning: Bayesian Networks

22.1 Introduction

John, Mary, and My Brand-New Alarm

▷ **Example 22.1.1 (From Russell/Norvig).**

- ▷ I got very valuable stuff at home. So I bought an alarm. Unfortunately, the alarm just rings at home, doesn't call me on my mobile.
- ▷ I've got two neighbors, Mary and John, who'll call me if they hear the alarm.
- ▷ The problem is that, sometimes, the alarm is caused by an earthquake.
- ▷ Also, John might confuse the alarm with his telephone, and Mary might miss the alarm altogether because she typically listens to loud music.

⇝ Random variables: `Burglary`, `Earthquake`, `Alarm`, `John`, `Mary`.

Given that both John and Mary call me, what is the probability of a burglary?

▷ ⇝ This is *almost* a naive Bayes model, but with multiple causes (`Burglary` and `Earthquake`) for the `Alarm`, which in turn may cause `John` and/or `Mary`.

SOME RIGHTS RESERVED

John, Mary, and My Alarm: Assumptions

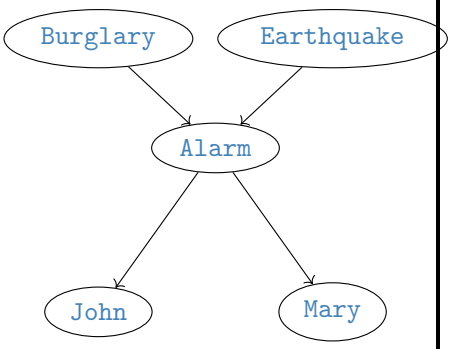
We assume:

▷ We (should) know $\mathbb{P}(\texttt{Alarm}|\texttt{Burglary},\texttt{Earthquake})$, $\mathbb{P}(\texttt{John}|\texttt{Alarm})$, and $\mathbb{P}(\texttt{Mary}|\texttt{Alarm})$.

▷ Burglary and Earthquake are independent.

▷ John and Mary are conditionally independent given Alarm.

▷ Moreover: Both John and Mary are conditionally independent of *any other random variables* in the graph given Alarm. (Only Alarm causes them, and everything else only causes them indirectly *through* Alarm)



▷ **First Step:** Construct the full joint probability distribution,

▷ **Second Step:** Use **enumeration** to compute $\mathbb{P}(\texttt{Burglary}|\texttt{John}=\texttt{T},\texttt{Mary}=\texttt{T})$.

SOME RIGHTS RESERVED

John, Mary, and My Alarm: The Distribution

▷

$$\begin{aligned}&\mathbb{P}(\texttt{John},\texttt{Mary},\texttt{Alarm},\texttt{Burglary},\texttt{Earthquake})\\ =&\mathbb{P}(\texttt{John}|\texttt{Mary},\texttt{Alarm},\texttt{Burglary},\texttt{Earthquake})\cdot\mathbb{P}(\texttt{Mary}|\texttt{Alarm},\texttt{Burglary},\texttt{Earthquake})\\ &\cdot\mathbb{P}(\texttt{Alarm}|\texttt{Burglary},\texttt{Earthquake})\cdot\mathbb{P}(\texttt{Burglary}|\texttt{Earthquake})\cdot\mathbb{P}(\texttt{Earthquake})\\ =&\mathbb{P}(\texttt{John}|\texttt{Alarm})\cdot\mathbb{P}(\texttt{Mary}|\texttt{Alarm})\cdot\mathbb{P}(\texttt{Alarm}|\texttt{Burglary},\texttt{Earthquake})\cdot\mathbb{P}(\texttt{Burglary})\cdot\mathbb{P}(\texttt{Earthquake})\end{aligned}$$

▷ We plug into the equation for enumeration:

$$\mathbb{P}(\texttt{Burglary}|\texttt{John}=\texttt{T},\texttt{Mary}=\texttt{T})=\alpha(\mathbb{P}(\texttt{Burglary})\sum_{a\in\{\texttt{T},\texttt{F}\}}P(\texttt{John}\mid\texttt{Alarm}=a)\cdot P(\texttt{Mary}\mid\texttt{Alarm}=a)$$
$$\cdot\sum_{q\in\{\texttt{T},\texttt{F}\}}\mathbb{P}(\texttt{Alarm}=a|\texttt{Burglary},\texttt{Earthquake}=q)P(\texttt{Earthquake}=q))$$

▷ ⇝ Now let's scale things up to arbitrarily many variables!

SOME RIGHTS RESERVED

Bayesian Networks: Definition

▷ **Definition 22.1.2.** A Bayesian network consists of

1. a directed acyclic graph $\langle\mathcal{X},E\rangle$ of random variables $\mathcal{X}=\{X_1,\ldots,X_n\}$, and
2. a conditional probability distribution $\mathbb{P}(X_i|\text{Parents}(X_i))$ for every $X_i\in\mathcal{X}$ (also called the

CPT for conditional probability table),

such that every X_i is conditionally independent of any conjunctions of non-descendents of X_i given $\mathrm{Parents}(X_i)$.

▷ **Definition 22.1.3.** Let $\langle \mathcal{X}, E \rangle$ be a directed acyclic graph, $X \in \mathcal{X}$, and E^* the reflexive transitive closure of E. The non-descendents of X are the elements of the set $\mathrm{NonDesc}(X) := \{Y \mid (X,Y) \notin E^*\} \setminus \mathrm{Parents}(X)$.

▷ Note that the roots of the graph are conditionally independent given the empty set; i.e. they are independent.

▷ **Theorem 22.1.4.** *The full joint probability distribution of a Bayesian network $\langle \mathcal{X}, E \rangle$ is given by*

$$\mathbb{P}(X_1, \ldots, X_n) = \prod_{X_i \in \mathcal{X}} \mathbb{P}(X_i | \mathrm{Parents}(X_i))$$

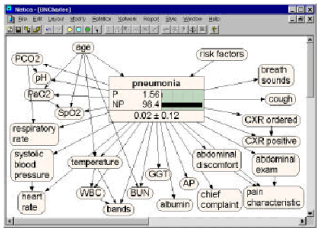
SOME RIGHTS RESERVED

Bayesian networks have applications anywhere we have to do some form of "diagnosis", i.e., we observe some data ("symptoms") and want to infer something about the underlying phenomena ("causes") generating that data! Here are some examples.

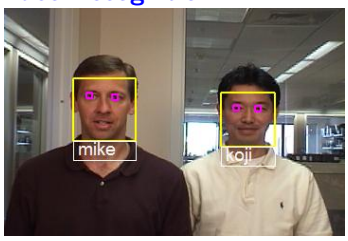
Some Applications

▷ A ubiquitous problem: Observe "symptoms", need to infer "causes".

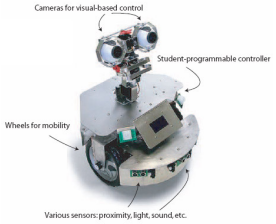
Medical Diagnosis



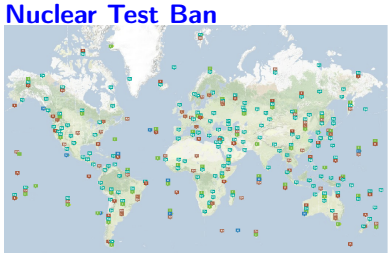
Face Recognition



Self-Localization



Nuclear Test Ban



SOME RIGHTS RESERVED

Some comments on the examples above:

1. **Medical Diagnosis**: We have already seen this in our running example of the previous chapter: We observe the symptoms (toothache) and need to deduce the illness (cavity).

2. **Face Recognition**: Here the "symptoms" are data provided by low-level image processing (edge detection etc), and the "causes" are the persons whose faces it may be.

3. **Self-Localization**: Here the "symptoms" is sensor data (camera, laser rangefinder, infrared, ...) and the "causes" is the position at which we are located.
4. **Nuclear Test Ban**: This is Stuart Russel's application, running on the data the UN has available. Here the "symptoms" is data delivered by 254 seismometers, i.e., monitoring stations all over the planet, and the "causes" are either earthquakes or nuclear tests. There are 10000s of "detections" per day, the data arrives with long time delays and distributedly (no "ID"s attached that would allow us to synchronize which measurements from station X correspond to which measurements from station Y), the challenge is to determine which are natural and which are atomic tests. (US senate refused to ratify the CNTB (comprehensive nuclear test ban treaty) in 1998 saying it's "too hard to monitor")

22.2 Constructing Bayesian Networks

Compactness of Bayesian Networks

▷ **Definition 22.2.1.** Given random variables $X_1, \ldots, X_n$ with finite domains $D_1, \ldots, D_n$, the size of $\mathcal{B} := \langle \{X_1, \ldots, X_n\}, E \rangle$ is defined as

$$\text{size}(\mathcal{B}) := \sum_{i=1}^{n} |D_i| \cdot \Big(\prod_{X_j \in \text{Parents}(X_i)} |D_j| \Big)$$

▷ **Note:** $\text{size}(\mathcal{B}) \mathrel{\widehat{=}}$ The total number of entries in the conditional probability distributions.

▷ **Note:** Smaller BN $\rightsquigarrow$ need to assess less probabilities, more efficient inference.

▷ **Observation 22.2.2.** *Explicit full joint probability distribution has size* $\prod_{i=1}^{n} |D_i|$.

▷ **Observation 22.2.3.** *If* $|\text{Parents}(X_i)| \leq k$ *for every* X_i, *and* $D_{\max}$ *is the largest random variable domain, then* $\text{size}(\mathcal{B}) \leq n|D_{\max}|^{k+1}$.

▷ **Example 22.2.4.** For $|D_{\max}| = 2$, $n = 20$, $k = 4$ we have $2^{20} = 1048576$ probabilities, but a Bayesian network of size $\leq 20 \cdot 2^5 = 640 \ldots !$

▷ In the *worst case*, $\text{size}(\mathcal{B}) = n \cdot (\prod_{i=1}^{n} |D_i|)$, namely if every variable depends on all its predecessors in the chosen variable ordering.

▷ **Intuition:** BNs are compact – i.e. of small size – if each variable is directly influenced only by few of its predecessor variables.

SOME RIGHTS RESERVED

Keeping Networks Small

▷ To keep our Bayesian networks small, we can:

1. **Reduce the number of edges:** $\rightsquigarrow$ Order the variables to allow for exploiting conditional independence (causes before effects), or
2. **represent the conditional probability distributions efficiently:**
   (a) For Boolean random variables X, we only need to store $\mathbb{P}(X = \text{T}|\text{Parents}(X))$

($\mathbb{P}(X = \mathsf{F}|\mathrm{Parents}(X)) = 1 - \mathbb{P}(X = \mathsf{T}|\mathrm{Parents}(X))$) (Cuts the number of entries in half!)

(b) Introduce different **kinds** of nodes exploiting domain knowledge; e.g. deterministic and noisy disjunction nodes.

Reducing Edges: Variable Order Matters

▷ Given a set of random variables $X_1, \ldots, X_n$, consider the following (impractical, but illustrative) pseudo-algorithm for constructing a Bayesian network:

▷ **Definition 22.2.5 (BN construction algorithm).**

1. Initialize $BN := \langle \{X_1, \ldots, X_n\}, E \rangle$ where $E = \emptyset$.
2. Fix any variable ordering, $X_1, \ldots, X_n$.
3. **for** $i := 1, \ldots, n$ **do**
   a. Choose a minimal set $\mathrm{Parents}(X_i) \subseteq \{X_1, \ldots, X_{i-1}\}$ such that
   $$\mathbb{P}(X_i|X_{i-1}, \ldots, X_1) = \mathbb{P}(X_i|\mathrm{Parents}(X_i))$$
   b. For each $X_j \in \mathrm{Parents}(X_i)$, insert (X_j, X_i) into E.
   c. Associate X_i with $\mathbb{P}(X_i|\mathrm{Parents}(X_i))$.

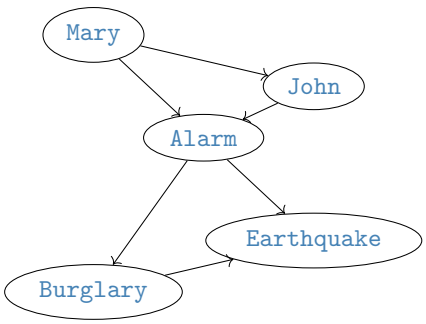
▷ **Attention:** *Which* variables we need to include into $\mathrm{Parents}(X_i)$ depends on what "$\{X_1, \ldots, X_{i-1}\}$" is . . . !

▷ **Thus:** The size of the resulting BN depends on the chosen variable ordering $X_1, \ldots, X_n$.

▷ **In Particular:** The size of a Bayesian network is *not* a fixed property of the domain. It depends on the skill of the designer.

John and Mary Depend on the Variable Order!

▷ **Example 22.2.6.** `Mary`, `John`, `Alarm`, `Burglary`, `Earthquake`.

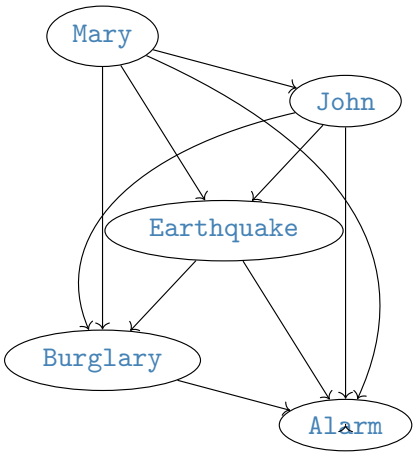


Note: For ??? we try to determine whether – given different value assignments to potential parents – the probability of X_i being true differs? If yes, we include these parents. In the particular case:

1. M to J yes because the common cause may be the alarm.
2. M, J to A yes because they may have heard alarm.
3. A to B yes because if A then higher chance of B.
4. However, M/J to B no because M/J only react to the alarm so if we have the value of A then values of M/J don't provide more information about B.
5. A to E yes because if A then higher chance of E.
6. B to E yes because, if A and not B then chances of E are higher than if A and B.

John and Mary Depend on the Variable Order! Ctd.

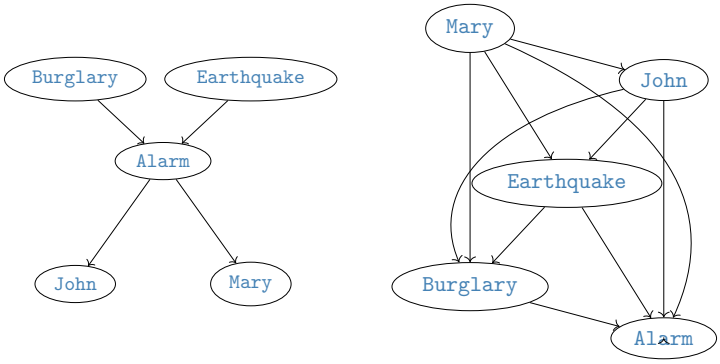
▷ **Example 22.2.7.** Mary, John, Earthquake, Burglary, Alarm.



Again: Given different value assignments to potential parents, does the probability of X_i being true differ? If yes, include these parents.

1. M to J as before.
2. M, J to E as probability of E is higher if M/J is true.
3. Same for B; E to B because, given M and J are true, if E is true as well then prob of B is lower than if E is false.
4. $M/J/B/E$ to A because if $M/J/B/E$ is true (even when changing the value of just one of these) then probability of A is higher.

John and Mary, What Went Wrong?



▷ **Intuition:** These BNs link from *effects* to their *causes*!

⇝ Even though Mary and John are conditionally independent given Alarm, this is not exploited, since Alarm is not ordered before Mary and John!

⇝ **Rule of Thumb:** We should order causes before symptoms.

SOME RIGHTS RESERVED

Representing Conditional Distributions: Deterministic Nodes

▷ **Problem:** Even if $max(\text{Parents})$ is small, the conditional probability table has 2^k entries. (worst-case)

▷ **Idea:** Usually conditional probability tables follow standard patterns called canonical distributions.

▷ only need to determine pattern and some values.

▷ **Definition 22.2.8.** A node X in a Bayesian network is called deterministic, if its value is completely determined by the values of $\text{Parents}(X)$.

▷ **Example 22.2.9.** The sum of two dice throws S is entirely determined by the values of the two dice $First$ and $Second$.

▷ **Example 22.2.10.** In the Wumpus example, the breezes are entirely determined by the pits

▷ ⇝ Deterministic nodes model direct, causal relationships.

▷ ⇝ If X is deterministic, then $P(X \mid \text{Parents}(X)) \in \{0, 1\}$

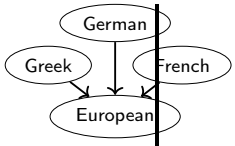
▷ ⇝ we can replace the conditional probability distribution $\mathbb{P}(X|\text{Parents}(X))$ by a Boolean function.

SOME RIGHTS RESERVED

Deterministic Nodes: More Examples

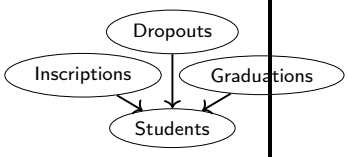
▷ **Example 22.2.11 (Logical Dependencies).**

In the network on the right, the node $European$ is deterministic, the conditional probability table corresponds to a logical disjunction, i.e. $P(\texttt{EU}) = P(\texttt{Greek} \vee \texttt{German} \vee \texttt{French})$.



▷ **Example 22.2.12 (Numerical Dependencies).**

In the network on the right, the node $Students$ is deterministic, the conditional probability table corresponds to a sum, i.e. $P(S = i - d - g) = P(I = i) + P(D = d) + P(G = g)$.



▷ **Intuition:** Deterministic nodes model direct, causal relationships.

SOME RIGHTS RESERVED

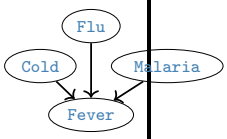
Representing Conditional Distributions: Noisy Nodes

▷ **Problem:** Sometimes, values of nodes are only "almost deterministic". (uncertain, but mostly logical)

▷ **Idea:** Use "noisy" logical relationships. (generalize logical ones softly to $[0,1]$)

▷ **Example 22.2.13 (Inhibited Causal Dependencies).**

In the network on the right, deterministic disjunction for the node Fever is incorrect, since the diseases sometimes fail to develop fever. The causal relation between parent and child is inhibited.



▷ **Assumptions:** We make the following assumptions for modeling Example 22.2.13:

1. Cold, Flu, and Malaria is a complete list of fever causes (add a leak node for the others otherwise).
2. Inhibitions of the parents are independent.

Thus we can model the inhibitions by individual inhibition factors q_d.

▷ **Definition 22.2.14.** The CPT of a noisy disjunction node X in a Bayesian network is given by $P(X \mid \text{Parents}(X)) = 1 - (\prod_{\{j \mid X_j = \mathsf{T}\}} q_j)$, where the q_i are the inhibition factors of $X_i \in \text{Parents}(X)$ and $q_i := P(X = \mathsf{F} \mid X_1 = \mathsf{F}, \ldots, X_{i-1} = \mathsf{F}, X_i = \mathsf{T}, X_{i+1} = \mathsf{F}, \ldots, X_n = \mathsf{F})$

SOME RIGHTS RESERVED

Representing Conditional Distributions: Noisy Nodes

▷ **Example 22.2.15.** Assume the following inhibition factors for Example 22.2.13:

$$\begin{aligned} q_{\texttt{Cold}} &= P(\neg\texttt{Fever} \mid \texttt{Cold},\neg\texttt{Flu},\neg\texttt{Malaria}) = 0.6 \\ q_{\texttt{Flu}} &= P(\neg\texttt{Fever} \mid \neg\texttt{Cold},\texttt{Flu},\neg\texttt{Malaria}) = 0.2 \\ q_{\texttt{Malaria}} &= P(\neg\texttt{Fever} \mid \neg\texttt{Cold},\neg\texttt{Flu},\texttt{Malaria}) = 0.1 \end{aligned}$$

If we model Fever as a noisy disjunction node, then the general rule $P(X_i \mid \mathrm{Parents}(X_i)) = \prod_{\{j \mid X_j=\mathsf{T}\}} q_j$ for the conditional probability table gives the following table:

Cold	Flu	Malaria	$P(\texttt{Fever})$	$P(\neg\texttt{Fever})$
F	F	F	0.0	1.0
F	F	T	0.9	**0.1**
F	T	F	0.8	**0.2**
F	T	T	0.98	$0.02 = 0.2 \cdot 0.1$
T	F	F	0.4	**0.6**
T	F	T	0.94	$0.06 = 0.6 \cdot 0.1$
T	T	F	0.88	$0.12 = 0.6 \cdot 0.2$
T	T	T	0.988	$0.012 = 0.6 \cdot 0.2 \cdot 0.1$

SOME RIGHTS RESERVED

Representing Conditional Distributions: Summary

▷ Note that deterministic nodes and noisy disjunction nodes are just two examples of "specialized" kinds of nodes in a Bayesian network.

▷ **Observation 22.2.16.** *In general, noisy logical relationships in which a variable depends on k parents can be described by $\mathcal{O}(k)$ parameters instead of $\mathcal{O}(2^k)$ for the full conditional probability table. This can make assessment (and learning) tractable.*

▷ **Example 22.2.17.** The CPCS network [Pra+94] uses noisy-OR and noisy-MAX distributions to model relationships among diseases and symptoms in internal medicine. With 448 nodes and 906 links, it requires only 8,254 values instead of 133,931,430 for a network with full conditional probability tables.

SOME RIGHTS RESERVED

Constructing Bayesian Networks

▷ **Question:** What is the Bayesian network we get by constructing according to the ordering

1. $X_1 = \mathrm{LoudNoise}, X_2 = \mathrm{Animal}, X_3 = \mathrm{LikesChappi}$?
2. $X_1 = \mathrm{LoudNoise}, X_2 = \mathrm{LikesChappi}, X_3 = \mathrm{Animal}$?

▷ **Answer:** reserved for the plenary sessions ⇝ be there!

SOME RIGHTS RESERVED

22.3 Inference in Bayesian Networks

Probabilistic Inference Tasks in Bayesian Networks

▷ **Definition 22.3.1 (Probabilistic Inference Task).** Let $X_1, \ldots, X_n$ be a set of random variables, a probabilistic inference task consists of

- ▷ a set $X \subseteq \{X_1, \ldots, X_n\}$ of query variables,
- ▷ a set $E \subseteq \{X_1, \ldots, X_n\}$ of evidence variables, and
- ▷ an event e that assigns values to E.

We wish to compute the conditional probability distribution $\mathbf{P}(X|e)$.

Variables in $Y := \{X_1, \ldots, X_n\} \setminus (X \cup E)$ are called hidden variables.

▷ **Notes:**

- ▷ We assume that a Bayesian network $\mathcal{B}$ for $X_1, \ldots, X_n$ is given.
- ▷ In the remainder, for simplicity, $X = \{X\}$ is a singleton.

▷ **Example 22.3.2.** In $\mathbf{P}(\texttt{Burglary}|\texttt{John}, \texttt{Mary})$ (your turn)

$X = \texttt{Burglary}$, $e = \texttt{John}, \texttt{Mary}$, and $Y = \{\texttt{Alarm}, \texttt{Earthquake}\}$.

SOME RIGHTS RESERVED

Enumeration: The Alarm-Example

▷ Remember our example: $\mathbb{P}(b|j, m)$

(hidden variables: a, e)

$$
\begin{aligned}
&= \alpha\Big(\textstyle\sum_{b_a, b_e \in \{\mathsf{T},\mathsf{F}\}} P(j, m, a = b_a, e = b_e, b)\Big) \\
&= \alpha\Big(\textstyle\sum_{b_a, b_e \in \{\mathsf{T},\mathsf{F}\}} P(j \mid a = b_a) \cdot P(m \mid a = b_a) \\
&\quad \cdot \mathbb{P}(a = b_a | e = b_e, b) \cdot P(e = b_e) \cdot \mathbb{P}(b)\Big)
\end{aligned}
$$

▷ $\rightsquigarrow$ These are 5 factors in 4 summands ($b_a, b_e \in \{\mathsf{T}, \mathsf{F}\}$) over two cases ($b \in \{\mathsf{T}, \mathsf{F}\}$),

▷ $\rightsquigarrow$ 38 arithmetic operations ($+3$ for α)

▷ **General worst case:** $\mathcal{O}(n2^n)$

▷ **Let's do better!**

SOME RIGHTS RESERVED

Enumeration: First Improvement

▷ **Some abbreviations:** $j := \texttt{John}, m := \texttt{Mary}, a := \texttt{Alarm}, e := \texttt{Earthquake}, b := \texttt{Burglary}$,

▷

$$\mathbb{P}(b|j,m) = \alpha(\sum_{b_a,b_e\in\{\mathsf{T},\mathsf{F}\}} P(j \mid a=b_a)\cdot P(m \mid a=b_a)\cdot \mathbb{P}(a=b_a|e=b_e,b)\cdot P(e=b_e)\cdot \mathbb{P}(b))$$

▷ Let's "optimize":

$$\mathbb{P}(b|j,m) = \alpha(\mathbb{P}(b)\cdot(\sum_{b_e\in\{\mathsf{T},\mathsf{F}\}} P(e=b_e)\cdot(\sum_{b_a\in\{\mathsf{T},\mathsf{F}\}} \mathbb{P}(a=b_a|e=b_e,b)\cdot P(j \mid a=b_a)\cdot P(m \mid a=b_a))))$$

⇝ 3 factors in 2 summand + 2 factors in 2 summands + two factors in the outer product, over two cases = 28 arithmetic operations (+3 for α)

SOME RIGHTS RESERVED

Second Improvement: Variable Elimination 1

▷ Consider $\mathbb{P}(j|b=\mathsf{T})$.

▷ Using enumeration:

$$=\alpha(P(b)\cdot(\sum_{b_e\in\{\mathsf{T},\mathsf{F}\}} P(e=b_e)\cdot(\sum_{a_e\in\{\mathsf{T},\mathsf{F}\}} P(a=a_e \mid e=b_e,b)\cdot\mathbb{P}(j|a=a_e)\cdot\underbrace{(\sum_{a_m\in\{\mathsf{T},\mathsf{F}\}} P(m=a_m \mid a=a_e))}_{=1})))$$

⇝ $\mathbb{P}(\texttt{John}|\texttt{Burglary}=\mathsf{T})$ does not depend on `Mary` (duh...)

▷ **More generally**:

▷ **Lemma 22.3.3.** *Given a query $\mathbb{P}(Q_1,\ldots,Q_{n_Q}|E_1=e_1,\ldots,E_{n_E}=e_{n_E})$, we can ignore (and remove) all hidden leaves of the Bayesian network.*

▷ ...doing so yields new leaves, which we can then ignore again, etc., until:

▷ **Lemma 22.3.4.** *Given a query $\mathbb{P}(Q_1,\ldots,Q_{n_Q}|E_1=e_1,\ldots,E_{n_E}=e_{n_E})$, we can ignore (and remove) all hidden variables that are not ancestors of any of the $Q_1,\ldots,Q_{n_Q}$ or $E_1,\ldots,E_{n_E}$.*

SOME RIGHTS RESERVED

Enumeration: First Algorithm

▷ **Definition 22.3.5.** Assume the $X_1,\ldots,X_n$ are topologically sorted (causes before effects)

function ENUMERATE-QUERY(Q,$\langle E_1=e_1,\ldots,E_{n_E}=e_{n_E}\rangle$)
- $P := \langle\rangle$ /* $=\mathbb{P}(Q|E_i=e_i)$ */
- $X_1,\ldots,X_n$:= variables filtered according to ???, topologically sorted
- **for all** $q\in\mathrm{dom}(Q)$ **do**
  - P_i:=ENUMALL($\langle X_1,\ldots,X_n\rangle$,$\langle E_1=e_1,\ldots,E_{n_E}=e_{n_E},Q=q\rangle$)
- **return** $\alpha(P)$

function ENUMALL($\langle Y_1,\ldots,Y_{n_Y}\rangle$,$\langle A_1=a_1,\ldots,A_{n_A}=a_{n_A}\rangle$)
/* *By construction,* $\mathrm{Parents}(Y_1)\subset\{A_1,\ldots,A_{n_A}\}$ */
- **if** $n_y=0$ **then return** 1.0

else if $Y_1 = A_j$ **then return** $P(A_j = \lfloor a_j \mid \text{Parents}(A_j)) \cdot \textsc{EnumAll}(\langle Y_2, \ldots, Y_{n_Y} \rangle, \langle A_1 = a_1, \ldots, A_{n_A} = a_{n_A} \rangle)$
else return $\sum_{y \in \text{dom}(Y_1)} P(Y_1 = y \mid \text{Parents}(Y_1)) \cdot \textsc{EnumAll}(\langle Y_2, \ldots, Y_{n_Y} \rangle, \langle A_1 = \lfloor a_1, \ldots, A_{n_A} = a_{n_A}, Y_1 = y \rangle)$

▷ **General worst case Complexity:** $\mathcal{O}(2^n)$ – better, but still not great

SOME RIGHTS RESERVED

Enumeration: Example

▷ Variable order: b, e, a, j, m

▹ $P_0 := P(b) \cdot \left[+ \begin{array}{l} P(e) \cdot \left[+ \begin{array}{l} P(a \mid b,e) \cdot P(j \mid a) \cdot P(m \mid a) \cdot 1.0 \\ P(\neg a \mid b,e) \cdot P(j \mid \neg a) \cdot P(m \mid \neg a) \cdot 1.0 \end{array} \right] \\ P(\neg e) \cdot \left[+ \begin{array}{l} P(a \mid b,\neg e) \cdot P(j \mid a) \cdot P(m \mid a) \cdot 1.0 \\ P(\neg a \mid b,\neg e) \cdot P(j \mid \neg a) \cdot P(m \mid \neg a) \cdot 1.0 \end{array} \right] \end{array} \right]$

▹ $P_1 := P(\neg b) \cdot \left[+ \begin{array}{l} P(e) \cdot \left[+ \begin{array}{l} P(a \mid \neg b,e) \cdot P(j \mid a) \cdot P(m \mid a) \cdot 1.0 \\ P(\neg a \mid \neg b,e) \cdot P(j \mid \neg a) \cdot P(m \mid \neg a) \cdot 1.0 \end{array} \right] \\ P(\neg e) \cdot \left[+ \begin{array}{l} P(a \mid \neg b,\neg e) \cdot P(j \mid a) \cdot P(m \mid a) \cdot 1.0 \\ P(\neg a \mid \neg b,\neg e) \cdot P(j \mid \neg a) \cdot P(m \mid \neg a) \cdot 1.0 \end{array} \right] \end{array} \right]$

$\Leftarrow \langle \frac{P_0}{P_0+P_1}, \frac{P_1}{P_0+P_1} \rangle$

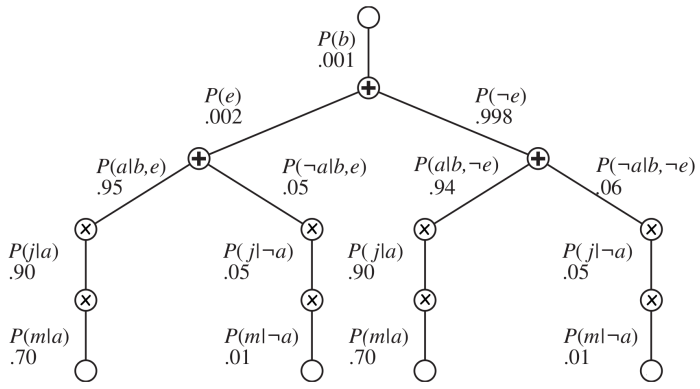
$$\mathbb{P}(b|j=\mathsf{T}, m=\mathsf{T}) = \alpha(\mathbb{P}(b) \cdot (\sum_{b_e \in \{\mathsf{T},\mathsf{F}\}} P(e=b_e) \cdot (\sum_{b_a \in \{\mathsf{T},\mathsf{F}\}} \mathbb{P}(a=b_a|e=b_e,b) \cdot P(j \mid a=b_a) \cdot P(m \mid a=b_a))))$$

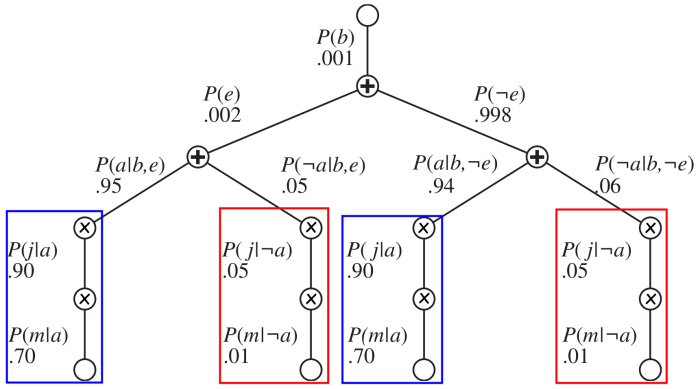
SOME RIGHTS RESERVED

The Evaluation of $P(b \mid j,m)$ as a "Search Tree"

$$\mathbb{P}(b|j,m) = \alpha(\mathbb{P}(b) \cdot (\sum_{b_e \in \{\mathsf{T},\mathsf{F}\}} P(e=b_e) \cdot (\sum_{b_a \in \{\mathsf{T},\mathsf{F}\}} \mathbb{P}(a=b_a|e=b_e,b) \cdot P(j \mid a=b_a) \cdot P(m \mid a=b_a))))$$

Note: Enumerate-Query corresponds to depth-first traversal of an arithmetic expression-tree:





Variable Elimination 2

▷

$$\mathbb{P}(b|j,m) = \alpha(\mathbb{P}(b)\cdot(\sum_{b_e\in\{\mathsf{T},\mathsf{F}\}} P(e=b_e)\cdot(\sum_{b_a\in\{\mathsf{T},\mathsf{F}\}} \mathbb{P}(a=b_a|e=b_e,b)\cdot P(j\mid a=b_a)\cdot P(m\mid a=b_a))))$$

The last two factors $P(j \mid a = b_a), P(m \mid a = b_a)$ only depend on a, but are "trapped" behind the summation over e, hence computed twice in two distinct recursive calls to EnumAll

▷ **Idea:** Instead of left-to-right (top-down depth-first search), operate right-to-left (bottom-up) and store intermediate "factors" along with their "dependencies":

$$\alpha(\underbrace{\mathbb{P}(b)}_{\mathbf{f}_7(b)}\cdot(\underbrace{\sum_{b_e\in\{\mathsf{T},\mathsf{F}\}}\underbrace{P(e=b_e)}_{\mathbf{f}_5(e)}\cdot(\underbrace{\sum_{b_a\in\{\mathsf{T},\mathsf{F}\}}\underbrace{\mathbb{P}(a=b_a|e=b_e,b)}_{\mathbf{f}_3(a,b,e)}\cdot\underbrace{P(j\mid a=b_a)}_{\mathbf{f}_2(a)}\cdot\underbrace{P(m\mid a=b_a)}_{\mathbf{f}_1(a)}}_{\mathbf{f}_4(b,e)})}_{\mathbf{f}_6(b)}))$$

Variable Elimination: Example

▷ We only show variable elimination by example: (implementation details get tricky, but the idea is simple)

$$\mathbb{P}(b)\cdot(\textstyle\sum_{b_e\in\{\mathsf{T},\mathsf{F}\}} P(e=b_e)\cdot(\sum_{b_a\in\{\mathsf{T},\mathsf{F}\}} \mathbb{P}(a=b_a|e=b_e,b)\cdot P(j\mid a=b_a)\cdot P(m\mid a=b_a)))$$

▷ Assume reverse topological order of variables: m, j, a, e, b

- ▹ m is an evidence variable with value T and dependency a, which is a hidden variable. We introduce a new "factor" $\mathbf{f}(a) := \mathbf{f}_1(a) := \langle P(m \mid a), P(m \mid \neg a)\rangle$.
- ▹ j works analogously, $\mathbf{f}_2(a) := \langle P(j \mid a), P(j \mid \neg a)\rangle$. We "multiply" with the existing factor, yielding $\mathbf{f}(a) := \langle \mathbf{f}_1(a) \cdot \mathbf{f}_2(a), \mathbf{f}_1(\neg a) \cdot \mathbf{f}_2(\neg a)\rangle = \langle P(m \mid a) \cdot P(j \mid a), P(m \mid \neg a) \cdot P(j \mid \neg a)\rangle$
- ▹ a is a hidden variable with dependencies e (hidden) and b (query).
  1. We introduce a new "factor" $\mathbf{f}_3(a, e, b)$, a $2 \times 2 \times 2$ table with the relevant conditional probabilities $\mathbb{P}(a|e, b)$.
  2. We multiply each entry of $\mathbf{f}_3$ with the relevant entries of the existing factor $\mathbf{f}$, yielding $\mathbf{f}(a, e, b)$.
  3. We "sum out" the resulting factor over a, yielding a new factor $\mathbf{f}(e, b) = \mathbf{f}(a, e, b) + \mathbf{f}(\neg a, e, b)$.
- ▹ ...

▷ $\rightsquigarrow$ can speed things up by a factor of 1000! (or more, depending on the order of variables!)

SOME RIGHTS RESERVED

The Complexity of Exact Inference

▷ **Definition 22.3.6.** A graph G is called singly connected, or a polytree (otherwise multiply connected), if there is at most one undirected path between any two nodes in G.

▷ **Theorem 22.3.7 (Good News).** *On singly connected Bayesian networks, variable elimination runs in polynomial time.*

▷ Is our BN for Mary & John a polytree? (Yes.)

▷ **Theorem 22.3.8 (Bad News).** *For multiply connected Bayesian networks, probabilistic inference is $\#\mathbf{P}$-hard.* *($\#\mathbf{P}$ is harder than $\mathbf{NP}$, i.e. $\mathbf{NP} \subseteq \#\mathbf{P}$)*

▷ **So?:** Life goes on ... In the hard cases, if need be we can throw exactitude to the winds and approximate.

▷ **Example 22.3.9.** Sampling techniques as in MCTS.

SOME RIGHTS RESERVED

22.4 Conclusion

Summary

▷ Bayesian networks (BN) are a wide-spread tool to model uncertainty, and to reason about it. A BN represents conditional independence relationships between random variables. It consists of a graph encoding the variable dependencies, and of conditional probability tables (conditional probability tables).

▷ Given a variable ordering, the BN is small if every variable depends on only a few of its

predecessors.

▷ Probabilistic inference requires to compute the probability distribution of a set of query variables, given a set of evidence variables whose values we know. The remaining variables are hidden.

▷ Inference by enumeration takes a BN as input, then applies normalization+marginalization, the chain rule, and exploits conditional independence. This can be viewed as a tree search that branches over all values of the hidden variables.

▷ Variable elimination avoids unnecessary computation. It runs in polynomial time for polytree BNs. In general, exact probabilistic inference is $\#\mathbf{P}$-hard. Approximate probabilistic inference methods exist.

SOME RIGHTS RESERVED

Topics We Didn't Cover Here

▷ **Inference by sampling**: A whole zoo of methods for doing this exists.

▷ **Clustering**: Pre-combining subsets of variables to reduce the running time of inference.

▷ **Compilation to SAT**: More precisely, to "weighted model counting" in CNF formulae. Model counting extends DPLL with the ability to determine the number of satisfying variable assignments. Weighted model counting allows to define a mass for each such variable assignment (= the probability of an atomic event).

▷ **Dynamic BN**: BN with one slice of variables at each "time step", encoding probabilistic behavior over time.

▷ **Relational BN**: BN with predicates and object variables.

▷ **First-order BN**: Relational BN with quantification, i.e. probabilistic logic. E.g., the BLOG language developed by Stuart Russel and co-workers.

SOME RIGHTS RESERVED

Reading:

- *Chapter 14: Probabilistic Reasoning* of [RN21].
  - Section 14.1 roughly corresponds to my "What is a Bayesian Network?".
  - Section 14.2 roughly corresponds to my "What is the Meaning of a Bayesian Network?" and "Constructing Bayesian Networks". The main change I made here is to *define* the semantics of the BN in terms of the conditional independence relationships, which I find clearer than RN's definition that uses the reconstructed full joint probability distribution instead.
  - Section 14.4 roughly corresponds to my "Inference in Bayesian Networks". RN give full details on variable elimination, which makes for nice ongoing reading.
  - Section 14.3 discusses how conditional probability tables are specified in practice.
  - Section 14.5 covers approximate sampling-based inference.
  - Section 14.6 briefly discusses relational and first-order BNs.
  - Section 14.7 briefly discusses other approaches to reasoning about uncertainty.

All of this is nice as additional background reading.

Chapter 23

Making Simple Decisions Rationally

We now understand how an agent can model the world in a partially observable and stochastic environment, we complete the development of probabilistic agents with a theory – called decision theory – of how agent can use the world models to reach decisions. We first concentrate on the simple case: episodic environments, where we do not need an explicit representation of time.

23.1 Introduction

Before we go into the details and mathematical development, let us recap what we already know and fix/preview the underlying concepts.

Decision Theory

▷ **Definition 23.1.1.** Decision theory investigates decision problems, i.e. how a utility-based agent a deals with choosing among actions based on the desirability of their outcomes given by a real-valued utility function U on states $s \in S$: i.e. $U : S \to \mathbb{R}$.

Definition 23.1.2. Episodic decision theory deals with choosing among actions based on the desirability of their *immediate* outcomes it is simpler than sequential decision theory, which takes all outcomes into account and therefore needs a representation of time.

▷ **Wait:** Isn't that what we did in Section 5.1 (Problem Solving) in the AI lecture notes?

▷ **Yes, but:** Now we do it for stochastic (i.e. non-deterministic), partially observable environments.

▷ **Recall:** We call the environment of an agent A

- ▷ fully observable, iff the A's sensors give it access to the complete state of the environment at any point in time, else partially observable.
- ▷ deterministic, iff the next state of the environment is completely determined by the current state and A's action, else stochastic.
- ▷ episodic, iff A's experience is divided into atomic episodes, where it perceives and then performes a single action. Crucially, the next episode does not depend on previous ones. Non-episodic environments are called sequential.

▷ **For now:** We restrict ourselves to episodic decision theory, which deals with choosing among actions based on the desirability of their *immediate* outcomes. (no need to treat time explicitly)

▷ **Later:** We will study sequential decision problems, where utility of a state depends on a sequence of decisions of the agent. (chapter 25)

SOME RIGHTS RESERVED

Overview

▷ We now know how to update our world model, represented as (a set of) random variables, given observations. Now we need to *act*.

▷ For that we need to answer two questions:

▷ **Questions:**

- ▷ Given a world model and a set of *actions*, what will the likely consequences of each action be?
- ▷ How "good" are these consequences?

▷ **Idea:**

- ▷ Represent actions as "special random variables":
  Given disjoint actions $a_1, \ldots, a_n$, introduce a random variable A with domain $\{a_1, \ldots, a_n\}$. Then we can model/query $\mathbb{P}(X|A = a_i)$.
- ▷ Assign *numerical values* to the possible outcomes of actions (i.e. a function $u\colon \mathrm{dom}(X) \to \mathbb{R}$) indicating their desirability.
- ▷ Choose the action that maximizes the *expected value* of u

Definition 23.1.3. Decision theory investigates decision problems, i.e. how a utility-based agent a deals with choosing among actions based on the desirability of their outcomes given by a real-valued utility function U on states $s \in S$: i.e. $U\colon S \to \mathbb{R}$.

Definition 23.1.4. Episodic decision theory deals with choosing among actions based on the desirability of their *immediate* outcomes it is simpler than sequential decision theory, which takes all outcomes into account and therefore needs a representation of time.

▷

SOME RIGHTS RESERVED

Decision Theory

▷ If our states are random variables, then we obtain a random variable for the utility function:

▷ **Observation:** Let $X_i\colon \Omega \to D_i$ random variables on a probability model $\langle \Omega, P \rangle$ and $f\colon D_1 \times \ldots \times D_n \to E$. Then $F(x) := f(X_0(x), \ldots, X_n(x))$ is a random variable $\Omega \to E$.

▷ **Definition 23.1.5.** Given a probability model $\langle \Omega, P \rangle$ and a random variable $X : \Omega \to D$ with $D \subseteq \mathbb{R}$, then $E(X) := \sum_{x \in D} P(X = x) \cdot x$ is called the expected value (or expectation) of X. (Assuming the sum/series is actually defined!)

Analogously, let $e_1, \ldots, e_n$ a sequence of events. Then the expected value of X given $e_1, \ldots, e_n$ is defined as $E(X|e_1, \ldots, e_n) := \sum_{x \in D} P(X = x \mid e_1, \ldots, e_n) \cdot x$.

▷ Putting things together:

▷ **Definition 23.1.6.** Let $A : \Omega \to D$ a random variable (where D is a set of actions) $X_i : \Omega \to D_i$ random variables (the state), and $U : D_1 \times \ldots \times D_n \to \mathbb{R}$ a utility function. Then the expected utility of the action $a \in D$ is the expected value of U (interpreted as a random variable) given $A = a$; i.e.

$$\mathrm{EU}(a) := \sum_{\langle x_1, \ldots, x_n \rangle \in D_1 \times \ldots \times D_n} P(X_1 = x_1, \ldots, X_n = x_n \mid A = a) \cdot U(x_1, \ldots, x_n)$$

SOME RIGHTS RESERVED

The next agent architecture generalizes goal-based agents (again). This time to get around the problems of how to find suitable goals (aka. the meaning of life) or how to deal with conflicting goals.

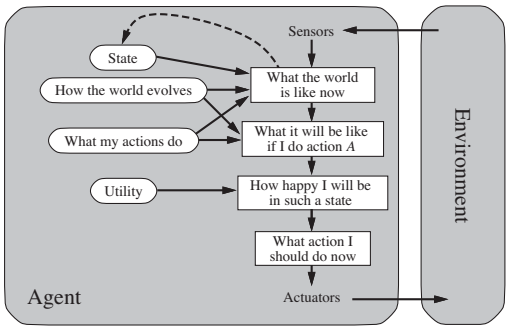
Utility-based Agents

▷ **Problem:** In goal-based agents, the goals are fixed in the design. What if the world changes or goals conflict?

▷ **Idea:** Introduce a general notion of desirability of states, use that to guide action selection.

▷ **Definition 23.1.7.** A utility-based agent uses a world model along with a utility function that models its preferences among the states of that world. It chooses the action that leads to the best expected utility.

▷ **Agent Schema:** We use the utility function on states to guide action selection:



SOME RIGHTS RESERVED

Maximizing Expected Utility (Ideas)

▷ **Definition 23.1.8 (MEU principle for Rationality).** We call an action rational if it maximizes expected utility (MEU). An utility-based agent is called rational, iff it always chooses a rational action.

▷ **Hooray:** This solves all of AI. (in principle)

▷ **Problem:** There is a long, long way towards an operationalization ;)

▷ **Note:** An agent can be entirely rational (consistent with MEU) without ever representing or manipulating utilities and probabilities.

▷ **Example 23.1.9.** A reflex agent for tic tac toe based on a perfect lookup table is rational if we take (the negative of) "winning/drawing in n steps" as the utility function.

▷ **Example 23.1.10 (AI1).** Heuristics in tree search (greedy search, A^*) and game-play (minimax, alpha-beta pruning) maximize "expected" utility.

⇝ In fully observable, deterministic environments, "expected utility" reduces to a specific determined utility value:

$\mathrm{EU}(a) = U(T(S(s,e),a))$, where e the most recent percept, s the current state, S the sensor model and T the transition model.

▷ Now let's figure out how to actually assign utilities!

SOME RIGHTS RESERVED

Recap: Episodic Decision Theory in AI-1

▷ **Observation:** In AI-1, the environment of an agent was

- ▷ fully observable, so the sensor model S and transition model T are functions.
- ▷ deterministic, so we know the result states of all actions.
- ▷ The "expected utility" of an action is just the utility of the result state.

So maximizing expected utility is easy: $\mathrm{EU}(a) = U(T(S(s,e),a))$, where e the most recent percept, and s the current state.

▷ **Intuition:** Utility functions can be handled like heuristics were in AI-1.

SOME RIGHTS RESERVED

Preview: Episodic Decision Theory in AI-1/2

▷ **Problem:** In AI-2, the environment may be

- ▷ partially observable, so we do not know the "current state".
- ▷ stochastic, so we do not know the result state of an action.

▷ **Idea:** Treat the result state of an action a as a random variable R_a.

- ▷ Study $P(R_a = s' \mid a, \mathbf{e})$ given evidence observations $\mathbf{e}$.

▷ The expected utility $\mathrm{EU}(a)$ of an action a is then

$$\mathrm{EU}(a|\mathbf{e}) = \sum_{s'} P(R_a = s' \mid a,\mathbf{e}) \cdot U(s')$$

▷ **Intuitively:** A formalization of what it means to "do the right thing".

▷ **Hooray:** This solves all of the AI problem. (in principle)

▷ **Problem:** There is a long long way towards an operationalization. (do that now)

SOME RIGHTS RESERVED

Outline of this Chapter

▷ Rational preferences

▷ Utilities and Money

▷ Multi attribute utility functions

▷ Decision networks

▷ Value of information

SOME RIGHTS RESERVED

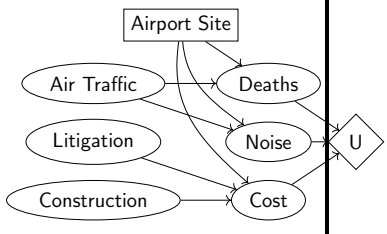
23.2 Decision Networks

As we already use Bayesian networks for the belief state of an utility-based agent, integrating utilities and possible actions into the network suggests itself naturally. This leads to the notion of a decision network.

Decision networks

Definition 23.2.1. A decision network is a Bayesian network with two additional kinds of nodes:

▷
- ▷ action nodes, representing a set of possible actions, and (rect. nodes)
- ▷ A single utility node (also called value node). (diamond node)



▷ **General Algorithm:** Given evidence $E_j = e_j$, and action nodes $A_1, \ldots, A_k$, compute the expected utility of each action, given the evidence, i.e. return the sequence of actions

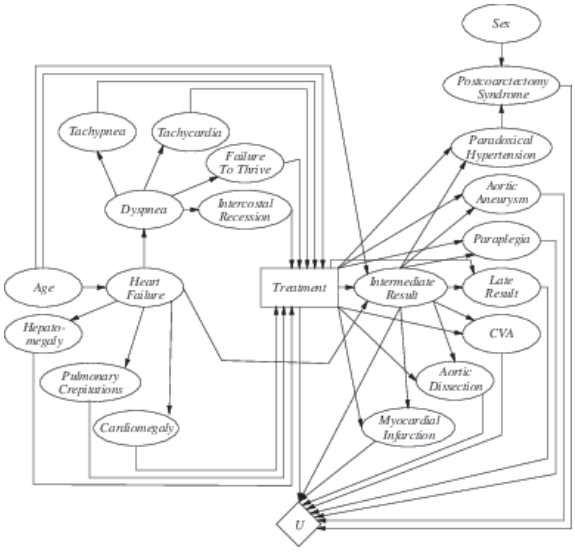
$$\operatorname{argmax}_{a_1,\ldots,a_k} \overbrace{\sum_{\langle x_1,\ldots,x_n\rangle} \underbrace{P(X_i = x_i \mid A_1 = a_1,\ldots,A_k = a_k, E_j = e_j)}_{\text{usual Bayesian Network inference}} \cdot U(X_i = x_i)}^{=\text{expected utility of } a_1,\ldots,a_k}$$

▷ **Note** the sheer number of summands in the sum above in the general case! (⇝ We will simplify where possible later)

SOME RIGHTS RESERVED

Decision Networks: Example

▷ **Example 23.2.2 (A Decision-Network for Aortic Coarctation).** from [Luc96]



SOME RIGHTS RESERVED

23.3 Preferences and Utilities

We will now provide the last of the missing pieces for utility-based agents by answering the question "where does the utility function come from in practice?". We cannot just ask humans – the only successful model of intelligence – because they do not know; neither can we just observe utility functions. What we can observe however is preferences, from which – courtesy of Ramsay's theorem – we can construct utility functions that induce these same preferences.

Preferences in Deterministic Environments

▷ **Problem:** How do we determine the utility of a state? (We cannot directly measure our satisfaction/happiness in a possibly future state...) (What unit would we even use?)

▷ **Example 23.3.1.** I have to decide whether to go to class today (or sleep in). What is the utility of this lecture? (obviously 42)

▷ **Idea:** We can let people/agents choose between two states (subjective preference) and derive a utility from these choices.

▷ **Example 23.3.2.** "*Give me your cell-phone or I will give you a bloody nose*". ⇝
To make a decision in a deterministic environment, the agent must determine whether it prefers a state without phone to one with a bloody nose?

▷ **Definition 23.3.3.** Given states A and B (we call them prizes) an agent can express preferences of the form

- ▷ $A \succ B$ A prefered over B
- ▷ $A \sim B$ indifference between A and B
- ▷ $A \succeq B$ B not prefered over A

i.e. Given a set $\mathcal{S}$ (of states), we define binary relations $\succ$ and $\sim$ on $\mathcal{S}$.

SOME RIGHTS RESERVED

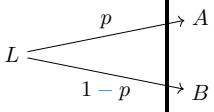
Preferences in Non-Deterministic Environments

▷ **Problem:** In nondeterministic environments we do not have full information about the states we choose between.

▷ **Example 23.3.4 (Airline Food).** "*Do you want chicken or pasta*" (but we cannot see through the tin foil)

▷ **Definition 23.3.5.**

Let $\mathcal{S}$ a set of states. We call a random variable X with domain $\{A_1, \ldots, A_n\} \subseteq \mathcal{S}$ a lottery and write $[p_1,A_1;\ldots;p_n,A_n]$, where $p_i = P(X = A_i)$.



▷ **Idea:** A lottery represents the result of a nondeterministic action that can have outcomes A_i with prior probability p_i. For the binary case, we use $[p,A\,;\,(1-p),B]$. We can then extend preferences to include lotteries, as a measure of how *strongly* we prefer one prize over another.

▷ **Convention:** We assume $\mathcal{S}$ to be *closed under lotteries*, i.e. lotteries themselves are also states. That allows us to consider lotteries such as $[p,A\,;\,(1-p),[q,B\,;\,(1-q),C]]$.

SOME RIGHTS RESERVED

Rational Preferences

▷ **Note:** Preferences of a rational agent must obey certain constraints – An agent with *rational* preferences can be described as an MEU-agent.

▷ **Definition 23.3.6.** We call a set $\succ$ of preferences rational, iff the following constraints hold:

Orderability	$A \succ B \vee B \succ A \vee A \sim B$
Transitivity	$A \succ B \wedge B \succ C \Rightarrow A \succ C$
Continuity	$A \succ B \succ C \Rightarrow (\exists p.[p,A\,;\,(1-p),C] \sim B)$
Substitutability	$A \sim B \Rightarrow [p,A\,;\,(1-p),C] \sim [p,B\,;\,(1-p),C]$
Monotonicity	$A \succ B \Rightarrow ((p > q) \Leftrightarrow [p,A\,;\,(1-p),B] \succ [q,A\,;\,(1-q),B])$
Decomposability	$[p,A\,;\,(1-p),[q,B\,;\,(1-q),C]] \sim [p,A\,;\,((1-p)q),B\,;\,((1-p)(1-q)),C]$

▷ From a set of rational preferences, we can obtain a meaningful utility function.

SOME RIGHTS RESERVED

The rationality constraints can be understood as follows:

Orderability: $A \succ B \vee B \succ A \vee A \sim B$ Given any two prizes or lotteries, a rational agent must either prefer one to the other or else rate the two as equally preferable. That is, the agent cannot avoid deciding. Refusing to bet is like refusing to allow time to pass.

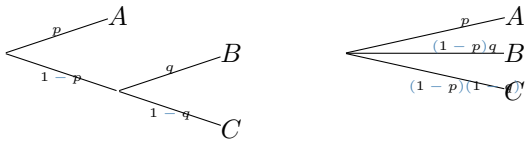
Transitivity: $A \succ B \wedge B \succ C \Rightarrow A \succ C$

Continuity: $A \succ B \succ C \Rightarrow (\exists p.[p,A\,;(1-p),C] \sim B)$ If some lottery B is between A and C in preference, then there is some probability p for which the rational agent will be indifferent between getting B for sure and the lottery that yields A with probability p and C with probability $1-p$.

Substitutability: $A \sim B \Rightarrow [p,A\,;(1-p),C] \sim [p,B\,;(1-p),C]$ If an agent is indifferent between two lotteries A and B, then the agent is indifferent between two more complex lotteries that are the same except that B is substituted for A in one of them. This holds regardless of the probabilities and the other outcome(s) in the lotteries.

Monotonicity: $A \succ B \Rightarrow ((p > q) \Leftrightarrow [p,A\,;(1-p),B] \succ [q,A\,;(1-q),B])$ Suppose two lotteries have the same two possible outcomes, A and B. If an agent prefers A to B, then the agent must prefer the lottery that has a higher probability for A (and vice versa).

Decomposability: $[p,A\,;(1-p),[q,B\,;(1-q),C]] \sim [p,A\,;((1-p)q),B\,;((1-p)(1-q)),C]$ Compound lotteries can be reduced to simpler ones using the laws of probability. This has been called the "no fun in gambling" rule because it says that two consecutive lotteries can be compressed into a single equivalent lottery: the following two are equivalent:

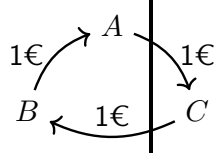


Rational Preferences contd.

▷ Violating the rationality constraints from Definition 23.3.6 leads to self-evident irrationality.

▷ **Example 23.3.7.** An agent with intransitive preferences can be induced to give away all its money:

- ▷ If $B \succ C$, then an agent who has C would pay (say) 1€to get B
- ▷ If $A \succ B$, then an agent who has B would pay (say) 1€to get A
- ▷ If $C \succ A$, then an agent who has A would pay (say) 1€to get C



SOME RIGHTS RESERVED

Ramseys Theorem and Value Functions

▷ **Theorem 23.3.8.** *(Ramsey, 1931; von Neumann and Morgenstern, 1944)*

Given rational preferences, there exists a real valued function U such that $U(A) \geq U(B)$, iff $A \succeq B$ and $U([p_1,S_1\,;\ldots;p_n,S_n]) = \sum_i p_i U(S_i)$

▷ This is an existence theorem, uniqueness not guaranteed.

▷ **Note:** Agent behavior is *invariant* w.r.t. positive linear transformations, i.e. an agent with utility function $U'(x) = k_1 U(x) + k_2$ where $k_1 > 0$ behaves exactly like one with U.

▷ **Observation:** With deterministic prizes only (no lottery choices), only a total ordering on prizes can be determined.

▷ **Definition 23.3.9.** We call a total ordering on states a value function or ordinal utility function. (If we don't need to care about *relative* utilities of states, e.g. to compute non-trivial expected utilities, that's all we need anyway!)

SOME RIGHTS RESERVED

23.4 Utilities in Practice

Now that we know where utilities come from in theory let us consider practicalities: how can we assess utilities in common applications of decision theory like medicine or insurance, is money a good utility function – after all, that is certainly what the "homo economicus" theory suggests.

Utilities

▷ **Intuition:** Utilities map states to real numbers.

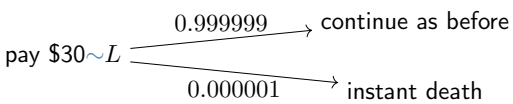
▷ **Question:** Which numbers exactly?

▷ **Definition 23.4.1 (Standard approach to assessment of human utilities).** Compare a given state A to a standard lottery L_p that has

- ▷ "best possible prize" $u_\top$ with probability p
- ▷ "worst possible catastrophe" $u_\bot$ with probability $1-p$

adjust lottery probability p until $A \sim L_p$. Then $U(A) = p$.

▷ **Example 23.4.2.** Choose $u_\top \mathrel{\widehat{=}}$ current state, $u_\bot \mathrel{\widehat{=}}$ instant death



SOME RIGHTS RESERVED

Popular Utility Functions

▷ **Definition 23.4.3.** Normalized utilities: $u_\top = 1$, $u_\bot = 0$.

(Not very meaningful, but at least it's independent of the specific problem...)

▷ **Example 23.4.4 (Obviously).** *Money* (Very intuitive, often easy to determine, but actually not well-suited as a utility function (see later))

▷ **Definition 23.4.5.** Micromorts: one millionth chance of instant death.

(useful for Russian roulette, paying to reduce product risks, etc.)

▷ **But:** Not necessarily a good measure of risk, if the risk is "merely" severe injury or illness...

▷ The following measure is better (more informative)

▷ **Definition 23.4.6.** QALYs: quality adjusted life years

QALYs are useful for medical decisions involving substantial risk.

SOME RIGHTS RESERVED

Comparing Utilities

▷ **Problem:** What is the monetary value of a micromort?

▷ **Just ask people:** What would you pay to avoid playing Russian roulette with a million-barrelled revolver? (Usually: quite a lot!)

▷ **But their behavior suggests a lower price:**

- ▷ Driving in a car for 370km incurs a risk of one micromort;
- ▷ Over the life of your car – say, $150,000\text{km}$ that's 400 micromorts.
- ▷ People appear to be willing to pay about $10,000€$ more for a safer car that halves the risk of death. ($\rightsquigarrow 25€$ per micromort)

This figure has been confirmed across many individuals and risk types.

▷ Of course, this argument holds only for small risks. Most people won't agree to kill themselves for $25\text{M}€$. (Also: People are pretty bad at estimating and comparing risks, especially if they are small.) (Various cognitive biases and heuristics are at work here!)

SOME RIGHTS RESERVED

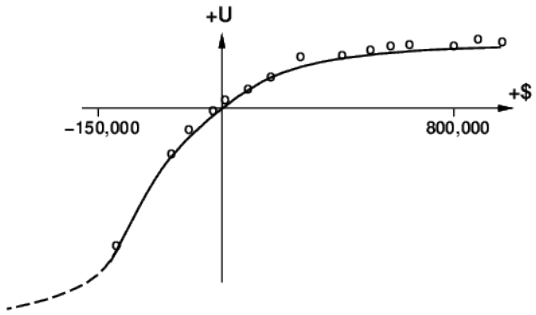
Money vs. Utility

▷ Money does *not* behave as a utility function should.

▷ Given a lottery L with expected monetary value $\text{EMV}(L)$, usually $U(L) < U(\text{EMV}(L))$, i.e., people are risk averse.

▷ **Utility Curve:** For what probability p am I indifferent between a prize x and a lottery $[p,M\$\,;(1-p),0\$]$ for large numbers M?

▷ Typical empirical data, extrapolated with risk prone behavior for debitors:



▷ **Empirically:** Comes close to the logarithm on the natural numbers.

SOME RIGHTS RESERVED

23.5 Multi-Attribute Utility

In this section we will make the ideas introduced above more practical. The discussion above conceived utility functions as functions on atomic states, which were good enough for introducing the theory. But when we build decision models for utility-based agent we want to characterize states by attributes that are already random variables in the Bayesian network we use to represent the belief state. For factored states, the utility function can be expressed as a multivariate function on attribute values.

Utility Functions on Attributes

▷ **Recap:** So far we understand how to obtain utility functions $u\colon S \to \mathbb{R}$ on states $s \in S$ from (rational) preferences.

▷ **But** in practice, our actions often impact *multiple* distinct "attributes" that need to be weighed against each other.

⇝ Lotteries become complex very quickly.

▷ **Definition 23.5.1.** Let $X_1, \ldots, X_n$ be random variables with domains $D_1, \ldots, D_n$. Then we call a function $u\colon D_1 \times \ldots \times D_n \to \mathbb{R}$ a (multi-attribute) utility function on attributes $X_1, \ldots, X_n$.

▷ **Intuition:** Given a probabilistic belief state that includes random variables $X_1, \ldots, X_n$, and a utility function on attributes $X_1, \ldots, X_n$, we can still maximize expected utility! (MEU principle)

▷ **Note:** In the general (worst) case, a multi-attribute utility function on n random variables with domain sizes k each requires k^n parameters to represent.

▷ **But:** A utility function on multiple attributes often has "internal structure" that we can exploit to simplify things.

▷ **Example 23.5.2.** The distinct attributes are often "independent" with respect to their utility. (a higher-quality product is better than a lower-quality one that costs the same, and a cheaper product is better than an expensive one of the same quality)

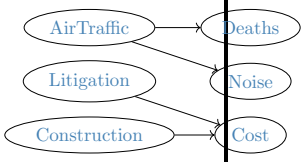
▷ **Preview:** Understand multi attribute utility functions and use Bayesian networks as representations of belief states.

Multi-Attribute Utility: Example

▷ **Example 23.5.3 (Assessing an Airport Site).**

▷ Attributes: Deaths, Noise, Cost.

▷ **Question**: What is $U(\text{Deaths}, \text{Noise}, \text{Cost})$ for a projected airport?



▷ How can complex utility function be assessed from preference behaviour?

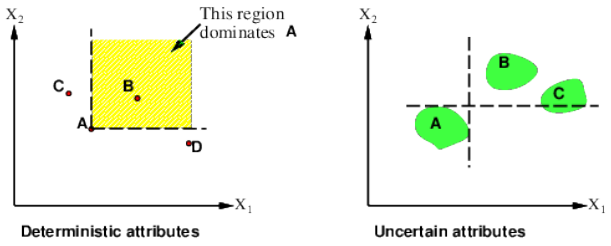
▷ **Idea 1:** Identify conditions under which decisions can be made without complete identification of $U(X_1, \ldots, X_n)$.

▷ **Idea 2:** Identify various types of *independence* in preferences and derive consequent canonical forms for $U(X_1, \ldots, X_n)$.

Strict Dominance

▷ **First Assumption:** U is often *monotone* in each argument. (wlog. growing)

▷ **Definition 23.5.4.** *(Informally)* An action B strictly dominates an action A, iff every possible outcome of B is at least as good as every possible outcome of A,



▷ If A strictly dominates B, we can just ignore B entirely.

▷ **Observation:** Strict dominance seldom holds in practice (life is difficult) but is useful for narrowing down the field of contenders.

Stochastic Dominance

▷ **Definition 23.5.5.** Let X_1, X_2 distributions with domains $\subseteq \mathbb{R}$.

X_1 stochastically dominates X_2 iff for all $t \in \mathbb{R}$, we have $P(X_1 \geq t) \geq P(X_2 \geq t)$, and for some t, we have $P(X_1 \geq t) > P(X_2 \geq t)$.

▷ **Observation 23.5.6.** *If U is monotone in X_1, and $\mathbb{P}(X_1|a)$ stochastically dominates $\mathbb{P}(X_1|b)$ for actions a, b, then a is always the better choice than b, with all other attributes X_i being equal.*

$\rightsquigarrow$ *If some action $\mathbb{P}(X_i|a)$ stochastically dominates $\mathbb{P}(X_i|b)$ for all attributes X_i, we can ignore b.*

▷ **Observation:** Stochastic dominance can often be determined without exact distributions using *qualitative* reasoning.

▷ **Example 23.5.7 (Construction cost increases with distance).** If airport location S_1 is closer to the city than $S_2 \rightsquigarrow S_1$ stochastically dominates S_2 on cost.q

SOME RIGHTS RESERVED

We have seen how we can do inference with attribute-based utility functions, let us consider the computational implications. We observe that we have just replaced one evil – exponentially many states (in terms of the attributes) – by another – exponentially many parameters of the utility functions.

Now we do what we always do in AI-2: we look for structure in the domain, do more theory to be able to turn such structures into computationally improved representations.

Preference structure: Deterministic

▷ **Recall:** In deterministic environments an agent has a value function.

▷ **Definition 23.5.8.** X_1 and X_2 preferentially independent of X_3 iff preference between $\langle x_1, x_2, z\rangle$ and $\langle x'_1, x'_2, z\rangle$ does not depend on z. (i.e. the tradeoff between x_1 and x_2 is independent of z)

▷ **Example 23.5.9.** E.g., $\langle$Noise, Cost, Safety$\rangle$: are preferentially independent
$\langle$20,000 suffer, 4.6 G$, 0.06 deaths/mpm$\rangle$ vs. $\langle$70,000 suffer, 4.2 G$, 0.06 deaths/mpm$\rangle$

▷ **Theorem 23.5.10 (Leontief, 1947).** *If every pair of attributes is preferentially independent of its complement, then every subset of attributes is preferentially independent of its complement: mutual preferential independence.*

▷ **Theorem 23.5.11 (Debreu, 1960).** *Mutual preferential independence implies that there is an additive value function: $V(S) = \sum_i V_i(X_i(S))$, where V_i is a value function referencing just one variable X_i.*

▷ Hence assess n single-attribute functions. (often a good approximation)

▷ **Example 23.5.12.** The value function for the airport decision might be

$$V(noise, cost, deaths) = -noise \cdot 10^4 - cost - deaths \cdot 10^{12}$$

SOME RIGHTS RESERVED

Preference structure: Stochastic

▷ **Definition 23.5.13.** $\mathbf{X}$ is utility independent of $\mathbf{Y}$ iff preferences over lotteries in $\mathbf{X}$ do not depend on particular values in $\mathbf{Y}$

▷ **Definition 23.5.14.** A set $\mathbf{X}$ is mutually utility independent (MUI), iff each subset is utility independent of its complement.

▷ **Theorem 23.5.15.** *For a MUI set of attributes $\mathcal{X}$, there is a multiplicative utility function of the form:* *[Kee74]*

$$U = \sum_{(\{X_0,\ldots,X_k\}\subseteq\mathcal{X})} \prod_{i=1}^{k} U_i(X_i = x_i)$$

⇝ *U can be represented using n single-attribute utility functions.*

▷ **System Support:** Routine procedures and software packages for generating preference tests to identify various canonical families of utility functions.

SOME RIGHTS RESERVED

Decision networks - Improvements

▷ There are multiple ways to improve inference in decision networks:

▷ Exploit "inner structure" of the utility function to simplify the computation,

▷ eliminate dominated actions,

▷ label pairs of nodes with *stochastic dominance*: If (the utility of) some attribute dominates (the utility of) another attribute, focus on the dominant one (e.g. if price is always more important than quality, ignore quality whenever the price between two choices differs)

▷ various techniques for variable elimination,

▷ policy iteration (more on that when we talk about Markov decision procedures)

SOME RIGHTS RESERVED

23.6 The Value of Information

So far we have tacitly been concentrating on actions that directly affect the environment. We will now come to a type of action we have hypothesized in the beginning of the course, but have completely ignored up to now: information gathering actions.

What if we do not have all information we need?

▷ We now know how to exploit the information we have to make decisions. But if we knew more, we might be able to make even better decisions in the long run - potentially at the cost

of gaining utility. (exploration vs. exploitation)

▷ **Example 23.6.1 (Medical Diagnosis).**

- ▷ We do not expect a doctor to already know the results of the diagnostic tests when the patient comes in.
- ▷ Tests are often expensive, and sometimes hazardous. (directly or by delaying treatment)
- ▷ **Therefore**: Only test, if
  - ▷ knowing the results lead to a significantly better treatment plan,
  - ▷ information from test results is not drowned out by a-priori likelihood.

▷ **Definition 23.6.2.** Information value theory is concerned with agent making decisions on information gathering rationally.

▷ **Intuition:** Simple form of sequential decision making. (action only impacts belief state).

▷ **Intuition:** With the new information, we can base the action choice to the *actual* information, rather than the average.

SOME RIGHTS RESERVED

Value of Information by Example

▷ **Idea:** Compute the expected *gain in utility* from acquiring information.

▷ **Example 23.6.3 (Buying Oil Drilling Rights).** There are n blocks of drilling rights available, exactly one block actually has oil worth k€, in particular:

- ▷ The prior probability of a block having oil is $\frac{1}{n}$ each. (mutually exclusive)
- ▷ The current price of each block is $\frac{k}{n}$€.
- ▷ A "consultant" offers an accurate survey of block (say) 3. How much should we be willing to pay for the survey?

▷ **Solution:** Compute the expected value of the best action given the information, minus the expected value of the best action without information.

▷ **Example 23.6.4 (Oil Drilling Rights contd.).**

- ▷ Survey may say "*oil in block 3 with probability* $\frac{1}{n}$" $\rightsquigarrow$ we buy block 3 for $\frac{k}{n}$€ and make a profit of $(k - \frac{k}{n})$€.
- ▷ Survey may say "*no oil in block 3 with probability* $\frac{n-1}{n}$" $\rightsquigarrow$ we buy another block, and make an expected profit of $\frac{k}{n-1} - \frac{k}{n}$€.
- ▷ Without the survey, the expected profit is 0.
- ▷ Expected profit is $\frac{1}{n} \cdot \frac{(n-1)k}{n} + \frac{n-1}{n} \cdot \frac{k}{n(n-1)} = \frac{k}{n}$.
- ▷ So, we should pay up to $\frac{k}{n}$€ for the information. (as much as block 3 is worth!)

SOME RIGHTS RESERVED

General formula (VPI)

▷ **Definition 23.6.5.** Let A the set of available actions and F a random variable. Given evidence $E_i = e_i$, let α be the action that maximizes expected utility a priori, and α_f the action that maximizes expected utility given $F = f$, i.e.: $\alpha = \operatorname{argmax}_{a \in A} \mathrm{EU}(a|E_i = e_i)$ and $\alpha_f = \operatorname{argmax}_{a \in A} \mathrm{EU}(a|E_i = e_i, F = f)$

The value of perfect information (VPI) on F given evidence $E_i = e_i$ is defined as

$$\mathrm{VPI}_{E_i=e_i}(F) := \Big(\sum_{f \in \mathrm{dom}(F)} P(F = f \mid E_i = e_i) \cdot \mathrm{EU}(\alpha_f|E_i = e_i, F = f)\Big) - \mathrm{EU}(\alpha|E_i = e_i)$$

▷ **Intuition:** The VPI is the expected gain from knowing the value of F relative to the current expected utility, and considering the relative probabilities of the possible outcomes of F.

SOME RIGHTS RESERVED

Properties of VPI

▷ **Observation 23.6.6 (VPI is Non-negative).**
$\mathrm{VPI}_E(F) \geq 0$ *for all* j *and* E *(in* expectation, *not* post hoc*)*

▷ **Observation 23.6.7 (VPI is Non-additive).**
$\mathrm{VPI}_E(F, G) \neq \mathrm{VPI}_E(F) + \mathrm{VPI}_E(G)$ *(consider, e.g., obtaining* F *twice)*

▷ **Observation 23.6.8 (VPI is Order-independent).**

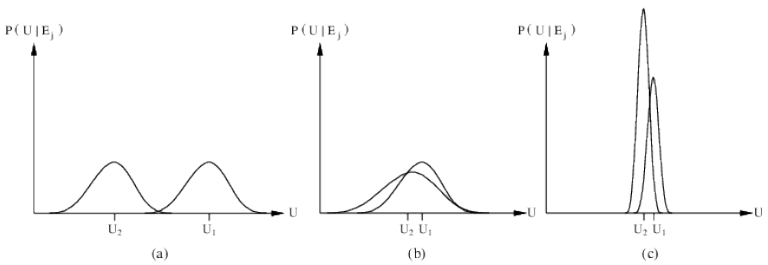
$$\mathrm{VPI}_E(F, G) = \mathrm{VPI}_E(F) + \mathrm{VPI}_{E,F}(G) = \mathrm{VPI}_E(G) + \mathrm{VPI}_{E,G}(F)$$

▷ **Note:** When more than one piece of evidence can be gathered, maximizing VPI for each to select one is not always optimal
⇝ evidence-gathering becomes a sequential decision problem.

SOME RIGHTS RESERVED

Qualitative behavior of VPI

▷ **Question:** Say we have three distributions for $P(U \mid E_j)$



Qualitatively: What is the value of information (VPI) in these three cases?

▷ **Answers:** reserved for the plenary sessions ⇝ be there!

▷ Note two things

  ▷ The difference between (b) and (c) is the distribution width, i.e. how close the possible outcomes are.

  ▷ The fact that U_2 has a high peak in (c) means that its expected value is known with higher certainty than U_1. (irrelevant to the argument)

SOME RIGHTS RESERVED

We will now use information value theory to specialize our utility-based agent from above.

A simple Information-Gathering Agent

▷ **Definition 23.6.9.** A simple information gathering agent. (gathers info before acting)

function Information–Gathering–Agent (percept) **returns** an action
  persistent: D, a decision network
  integrate percept into D
  $j := \operatorname{argmax}_k \mathrm{VPI}_E(E_k)/Cost(E_k)$
  if $\mathrm{VPI}_E(E_j) > Cost(E_j)$ **return** Request(E_j)
  else return the best action from D

The next percept after $\mathrm{Request}(E_j)$ provides a value for E_j.

▷ **Problem:** The information gathering implemented here is myopic, i.e. only acquires a single evidence variable, or acts immediately. (cf. greedy search)

▷ But it works relatively well in practice. (e.g. outperforms humans for selecting diagnostic tests)

▷ Strategies for non-myopic information gathering exist. (Not discussed in this course)

SOME RIGHTS RESERVED

Summary

▷ An MEU agent maximizes expected utility.

▷ Decision theory provides a framework for rational decision making.

▷ Decision networks augment Bayesian networks with action nodes and a utility node.

▷ Rational preferences allow us to obtain a utility function (orderability, transitivity, continuity, substitutability, monotonicity, decomposability)

▷ Multi-attribute utility functions can usually be "destructured" to allow for better inference and representation (can be monotone, attributes may dominate others, actions may dominate others, may be multiplicative,...)

▷ Information value theory tells us when to explore rather than exploit, using

▷ VPI (value of perfect information) to determine how much to "pay" for information.

SOME RIGHTS RESERVED

Chapter 24

Temporal Probability Models

Now that we have a full account for episodic decision theory – that is what agents in episodic (partially observable and/or stochastic) environments need – we go the first step towards a sequential decision theory: we study temporal models, i.e. probabilistic models that explicitly model time.

24.1 Modeling Time and Uncertainty with Markov Processes

In this section we will present the first temporal model: Markov processes: probability models that explicitly model some form of time, but make assumptions on the structure of the processes that make inference – which we will discuss in the next section– tractable.

Stochastic Processes

▷ **Observation:** The world changes in *stochastically predictable ways*.

▷ **Example 24.1.1.**

- ▷ The weather changes, but the weather tomorrow is somewhat predictable *given* today's weather and other factors, (which in turn (somewhat) depends on yesterday's weather, which in turn...)
- ▷ the stock market changes, but the stock price tomorrow is probably related to today's price,
- ▷ A patient's blood sugar changes, but their blood sugar is related to their blood sugar 10 minutes ago (in particular if they didn't eat anything in between)

▷ How do we model this?

▷ **Definition 24.1.2.** Let $\langle \Omega, P \rangle$ a probability space and $\langle S, \preceq \rangle$ a (not necessarily *totally*) ordered set.

A sequence of random variables $(X_t)_{t \in S}$ with $\mathrm{dom}(X_t) = D$ is called a stochastic process over the time structure S.

▷ **Note:** As the sample space can be a Cartesian product, this also covers the case where we have multiple random variables for different components that contribute to a stochastic process.

▷ **Intuition:** X_t models the outcome of the random variable X at time step t. The sample space Ω corresponds to the set of all possible sequences of outcomes.

▷ **Note:** We will almost exclusively use $\langle S, \preceq \rangle = \langle \mathbb{N}, \leq \rangle$.

▷ **Definition 24.1.3.** Given a stochastic process X_t over S and $a, b \in S$ with $a \preceq b$, we write $\mathbf{X}_{a:b}$ for the sequence $X_a, X_{a+1}, \ldots, X_{b-1}, X_b$ and $X_{a:b}^{=x}$ for $X_a = x_a, \ldots, X_b = x_b$.

SOME RIGHTS RESERVED

Stochastic Processes (Running Example)

▷ **Example 24.1.4 (Umbrellas).** You are a security guard in a secret underground facility, want to know it if is raining outside. Your only source of information is whether the director comes in with an umbrella.

- ▷ We have a stochastic process $\texttt{Rain}_0, \texttt{Rain}_1, \texttt{Rain}_2, \ldots$ of hidden variables, and
- ▷ a related stochastic process $\texttt{Umbrella}_0, \texttt{Umbrella}_1, \texttt{Umbrella}_2, \ldots$ of evidence variables.

...and a combined stochastic process $\langle \texttt{Rain}_0, \texttt{Umbrella}_0 \rangle, \langle \texttt{Rain}_1, \texttt{Umbrella}_1 \rangle, \ldots$

Note that $\texttt{Umbrella}_t$ only depends on $\texttt{Rain}_t$, not on e.g. $\texttt{Umbrella}_{t-1}$ (except indirectly through $\texttt{Rain}_t$ / $\texttt{Rain}_{t-1}$).

▷ In this example we actually have two stochastic processes: the weather and the umbrellas sequences.

▷ **Definition 24.1.5.** We call a stochastic process of *hidden* variables a state variable.

SOME RIGHTS RESERVED

Markov Processes

▷ **Idea:** Construct a Bayesian network from these variables (parents?)

...without everything exploding in size...?

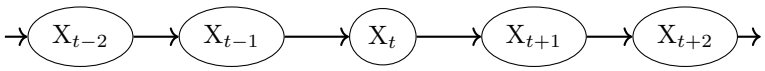
▷ **Definition 24.1.6.** Let $(X_t)_{t \in S}$ be a stochastic process. X has the (nth order) Markov property iff X_t only depends on a bounded subset of $\mathbf{X}_{0:t-1}$ – i.e. for all $t \in S$ we have $\mathbb{P}(X_t | X_0, \ldots X_{t-1}) = \mathbb{P}(X_t | X_{t-n}, \ldots X_{t-1})$ for some $n \in S$.

A stochastic process with the Markov property for some n is called a (nth order) Markov process.

▷ Important special cases:

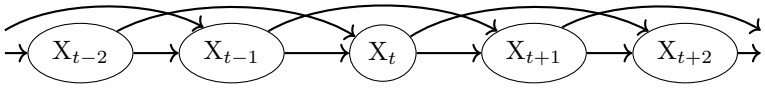
Definition 24.1.7.

- ▷ First-order Markov property: $\mathbb{P}(\mathbf{X}_t | \mathbf{X}_{0:t-1}) = \mathbb{P}(\mathbf{X}_t | \mathbf{X}_{t-1})$



A first order Markov process is called a Markov chain.

- ▷ Second-order Markov property: $\mathbb{P}(\mathbf{X}_t | \mathbf{X}_{0:t-1}) = \mathbb{P}(\mathbf{X}_t | \mathbf{X}_{t-2}, \mathbf{X}_{t-1})$



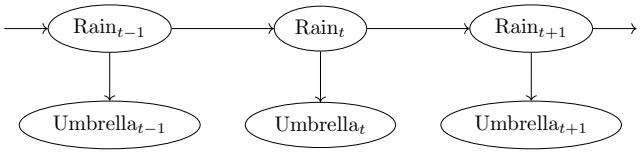
▷ **Intuition:** Increasing the order adds "memory" to the process, Markov chains have none.

▷ **Preview:** We will use Markov processess to model sequential environments.

SOME RIGHTS RESERVED

Markov Process Example: The Umbrella

▷ **Example 24.1.8 (Umbrellas continued).** We model the situation in a Bayesian network:



▷ **Problem:** This network does not actually have the first-order Markov property ...

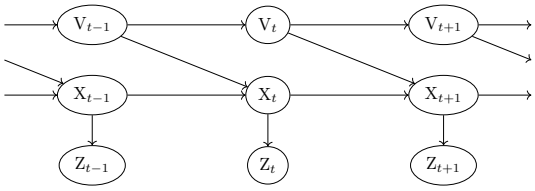
▷ **Possible fixes:** We have two ways to fix this:

1. Increase the order of the Markov process. (more dependencies $\rightsquigarrow$ more complex inference)
2. Add more state variables, e.g., Temp_t, Pressure_t. (more information sources)

SOME RIGHTS RESERVED

Markov Process Example: Robot Motion

▷ **Example 24.1.9 (Random Robot Motion).** Assume we want to track a robot wandering randomly on the X/Y plane, whose position we can only observe roughly (e.g. by approximate GPS coordinates:) Markov chain



- ▷ the velocity V_i may change unpredictably.
- ▷ the exact position X_i depends on previous position X_{i-1} and velocity V_{i-1}
- ▷ the position X_i influences the observed position Z_i.

▷ **Example 24.1.10 (Battery Powered Robot).** If the robot has a *battery*, the Markov property is violated!

- ▷ Battery exhaustion has a systematic effect on the change in velocity.

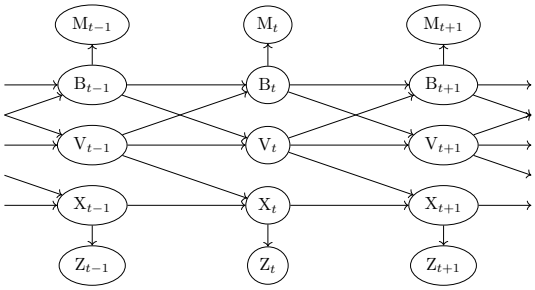
▷ This depends on how much power was used by all previous manoeuvres.

SOME RIGHTS RESERVED

Markov Process Example: Robot Motion

▷ **Idea:** We can restore the Markov property by including a state variable for the charge level B_t. (Better still: Battery level sensor)

▷ **Example 24.1.11 (Battery Powered Robot Motion).**



▷ Battery level B_i is influenced by previous level $B_{i-1}$and velocity V_{i-1}.

▷ Velocity V_i is influenced by previous level $B_{i-1}$and velocity V_{i-1}.

▷ Battery meter M_i is only influenced by Battery level B_i.

SOME RIGHTS RESERVED

Stationary Markov Processes as Transition Models

▷ *Remark 24.1.12.* Given a temporal probability model with state variables X_t and evidence variables E_t, then $\mathbb{P}(X_t|\mathbf{X}_{0:t})$ is a transition model and $\mathbb{P}(E_t|\mathbf{X}_{0:t}, \mathbf{E}_{1:t-1})$ a sensor model in the sense of a model-based agent.

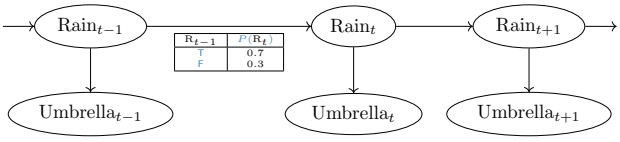
Note that we assume that the X_t do not depend on the E_t.

Also note that with the Markov property, the transition model simplifies to $\mathbb{P}(\mathbf{X}_t|\mathbf{X}_{t-n})$.

▷ **Problem:** Even with the Markov property the transition model is infinite. ($t \in \mathbb{N}$)

▷ **Definition 24.1.13.** A Markov chain is called stationary if the transition model is independent of time, i.e. $\mathbb{P}(X_t|X_{t-1})$ is the same for all t.

▷ **Example 24.1.14 (Umbrellas are stationary).** $\mathbb{P}(\texttt{Rain}_t|\texttt{Rain}_{t-1})$ does not depend on t. (need only one table)



▷ ⚠ Don't confuse "stationary" (Markov processes) with "static" (environments).

▷ We restrict ourselves to stationary Markov processes in AI-2.

SOME RIGHTS RESERVED

Markov Sensor Models

▷ **Recap:** The sensor model $\mathbb{P}(E_t|\mathbf{X}_{0:t}, \mathbf{E}_{1:t-1})$ allows us (using Bayes' rule et. al.) to update our belief state about X_t given the observations $\mathbf{E}_{0:t}$.

▷ **Problem:** The evidence variables E_t could depend on any of the variables $\mathbf{X}_{0:t}, \mathbf{E}_{1:t-1}$...

▷ **Definition 24.1.15.** We say that a sensor model has the sensor Markov property, iff $\mathbb{P}(E_t|\mathbf{X}_{0:t}, \mathbf{E}_{1:t-1}) = \mathbb{P}(E_t|X_t)$ – i.e., the sensor model depends only on the current state.

▷ **Assumptions on Sensor Models:** We usually assume the sensor Markov property and make it stationary as well: $\mathbb{P}(E_t|X_t)$ is fixed for all t.

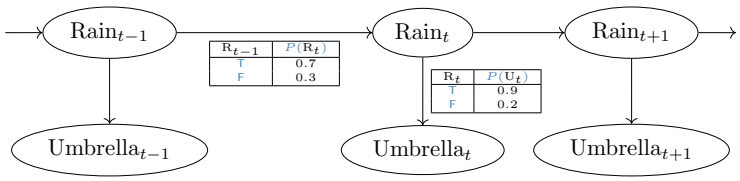
▷ **Definition 24.1.16 (Hidden Markov Models).**

- ▷ If a Markov chain X is stationary and finite domain, we can represent the transition model as a matrix $\mathbf{T}_{ij} := P(X_t = j \mid X_{t-1} = i)$.
- ▷ If a sensor model has the sensor Markov property, we can represent each observation $E_t = e_t$ at time t as the diagonal matrix $\mathbf{O}_t$ with $\mathbf{O}_{tii} := P(E_t = e_t \mid X_t = i)$.
- ▷ A pair $\langle X, E\rangle$ where X is a (stationary) Markov chain, E_i only depends on X_i, and E has the sensor Markov property is called a (stationary) hidden markov model (HMM). (X and E are single variables)

SOME RIGHTS RESERVED

Umbrellas, the full Story

▷ **Example 24.1.17 (Umbrellas, Transition & Sensor Models).**



This is a hidden Markov model

▷ **Note:** The influence goes from Rain_t to Umbrella_t. (causal dependency)

▷ **Observation 24.1.18.** *If we know the initial prior probabilities $\mathbb{P}(X_0)$ ($\widehat{=}$ time $t = 0$), then we can compute the full joint probability distribution as*

$$\mathbb{P}(\mathbf{X}_{0:t}, \mathbf{E}_{1:t}) = \mathbb{P}(X_0) \cdot (\prod_{i=1}^{t} \mathbb{P}(X_i|X_{i-1}) \cdot \mathbb{P}(E_i|X_i))$$

 SOME RIGHTS RESERVED

24.2 Inference: Filtering, Prediction, Smoothing, and Explanation

We now come to the inference tasks in temporal probability models – the computation an agent in in episodic (partially observable and/or stochastic) environments to perform to update its world model based on new evidence given by its percepts.

Inference tasks

▷ **Definition 24.2.1.** Given a Markov process with state variables X_t and evidence variables E_t, we are interested in the following Markov inference tasks:

- ▷ Filtering (or monitoring) $\mathbb{P}(X_t|E_{1:t}^{=e})$: Given the sequence $e = e_1, \ldots, e_t$ of observations up until time t, compute the likely state of the world at *current* time t.
- ▷ Prediction (or state estimation) $\mathbb{P}(X_{t+k}|E_{1:t}^{=e})$ for $k > 0$: Given the sequence $e = e_1, \ldots, e_t$ of observations up until time t, compute the likely *future* state of the world at time $t + k$.
- ▷ Smoothing (or hindsight) $\mathbb{P}(X_{t-k}|E_{1:t}^{=e})$ for $0 < k < t$: Given the sequence $e = e_1, \ldots, e_t$ of observations up until time t, compute the likely *past* state of the world at time $t - k$.
- ▷ Most likely explanation $\operatorname{argmax}_{\mathbf{x}_{1:t}} (P(X_{1:t}^{=x} \mid E_{1:t}^{=e}))$: Given the sequence $e = e_1, \ldots, e_t$ of observations up until time t, compute the most likely sequence of states that led to these observations.

  Note: The most likely sequence of states is *not* (necessarily) the sequence of most likely states ;-)

▷ In this section, we assume X and E to represent *multiple* variables, where X jointly forms a Markov chain and the E jointly have the sensor Markov property.

▷ In the case where X and E are stationary *single* variables, we have a stationary hidden Markov model and can use the matrix forms.

 SOME RIGHTS RESERVED

24.2.1 Filtering

The first temporal inference method is filtering, given a (temporal) chain of evidence, it updates the world model by filtering out the unlikely possible worlds.

Filtering (Computing the Belief State given Evidence)

▷ **Note:**

- ▷ Using the full joint probability distribution, we can compute any conditional probability we want, but not necessarily efficiently.
- ▷ We want to use filtering to update our "world model" $\mathbb{P}(X_t)$ based on a new observation

$E_t = e_t$ and our *previous* world model $\mathbb{P}(X_{t-1})$.

$\rightsquigarrow$ We want a function $\mathbb{P}(X_t|E_{1:t}^{=e}) = F(e_t, \underbrace{\mathbb{P}(X_{t-1}|E_{1:t-1}^{=e})}_{F(e_{t-1},\ldots)})$

▷ **Spoiler:**

$$F(e_t, \mathbb{P}(X_{t-1}|E_{1:t-1}^{=e})) = \alpha(\mathbf{O}_t \cdot \mathbf{T}^T \cdot \mathbb{P}(X_{t-1}|E_{1:t-1}^{=e}))$$

SOME RIGHTS RESERVED

Filtering Derivation

▷

$$\begin{aligned}
\mathbb{P}(X_t|E_{1:t}^{=e}) &= \mathbb{P}(X_t|E_t = e_t, E_{1:t-1}^{=e}) && \text{(dividing up evidence)}\\
&= \alpha(\mathbb{P}(E_t = e_t|X_t, E_{1:t-1}^{=e}) \cdot \mathbb{P}(X_t|E_{1:t-1}^{=e})) && \text{(using Bayes' rule)}\\
&= \alpha(\mathbb{P}(E_t = e_t|X_t) \cdot \mathbb{P}(X_t|E_{1:t-1}^{=e})) && \text{(sensor Markov property)}\\
&= \alpha(\mathbb{P}(E_t = e_t|X_t) \cdot (\sum_{x\in \mathrm{dom}(X)} \mathbb{P}(X_t|X_{t-1} = x, E_{1:t-1}^{=e}) \cdot P(X_{t-1} = x \mid E_{1:t-1}^{=e}))) && \text{(marginalization)}\\
&= \alpha(\underbrace{\mathbb{P}(E_t = e_t|X_t)}_{\text{sensor model}} \cdot (\sum_{x\in \mathrm{dom}(X)} \underbrace{\mathbb{P}(X_t|X_{t-1} = x)}_{\text{transition model}} \cdot \underbrace{P(X_{t-1} = x \mid E_{1:t-1}^{=e})}_{\text{recursive call}})) && \text{(conditional independence)}
\end{aligned}$$

▷ **Reminder:** In a stationary HMM, we have the matrices $\mathbf{T}_{ij} = P(X_t = j \mid X_{t-1} = i)$ and $\mathbf{O}_{tii} = P(E_t = e_t \mid X_t = i)$.

Then interpreting $\mathbb{P}(X_{t-1}|E_{1:t-1}^{=e})$ as a vector, the above corresponds exactly to the matrix multiplication $\alpha(\mathbf{O}_t \cdot \mathbf{T}^T \cdot \mathbb{P}(X_{t-1}|E_{1:t-1}^{=e}))$

▷ **Definition 24.2.2.** We call the inner part of the above expression the forward algorithm, i.e. $\mathbb{P}(X_t|E_{1:t}^{=e}) = \alpha(\mathrm{FORWARD}(e_t, \mathbb{P}(X_{t-1}|E_{1:t-1}^{=e}))) =: \mathbf{f}_{1:t}$.

SOME RIGHTS RESERVED

Filtering the Umbrellas

▷ **Example 24.2.3.** Let's assume:

- ▷ $\mathbb{P}(\mathtt{R}_0) = \langle 0.5, 0.5\rangle$, (Note that with growing t (and evidence), the impact of the prior at $t = 0$ vanishes anyway)
- ▷ $P(\mathtt{R}_{t+1} \mid \mathtt{R}_t) = 0.6$, $P(\neg\mathtt{R}_{t+1} \mid \neg\mathtt{R}_t) = 0.8$, $P(\mathtt{U}_t \mid \mathtt{R}_t) = 0.9$ and $P(\neg\mathtt{U}_t \mid \neg\mathtt{R}_t) = 0.85$

$\rightsquigarrow$ $\mathbf{T} = \begin{pmatrix} 0.6 & 0.4 \\ 0.2 & 0.8 \end{pmatrix}$

- ▷ The director carries an umbrella on days 1 and 2, and *not* on day 3.

$\rightsquigarrow$ $\mathbf{O}_1 = \mathbf{O}_2 = \begin{pmatrix} 0.9 & 0 \\ 0 & 0.15 \end{pmatrix}$ and $\mathbf{O}_3 = \begin{pmatrix} 0.1 & 0 \\ 0 & 0.85 \end{pmatrix}$.

Then:

▷ $\mathbf{f}_{1:1} := \mathbb{P}(\mathtt{R}_1|\mathtt{U}_1 = \mathsf{T}) = \alpha(\mathbb{P}(\mathtt{U}_1 = \mathsf{T}|\mathtt{R}_1) \cdot (\sum_{b\in\{\mathsf{T},\mathsf{F}\}} \mathbb{P}(\mathtt{R}_1|\mathtt{R}_0 = b) \cdot P(\mathtt{R}_0 = b)))$

$= \alpha(\langle 0.9, 0.15\rangle \cdot (\langle 0.6, 0.4\rangle \cdot 0.5 + \langle 0.2, 0.8\rangle \cdot 0.5)) = \alpha(\langle 0.36, 0.09\rangle) = \langle 0.8, 0.2\rangle$

▷ Using matrices: $\alpha(\mathbf{O}_1 \cdot \mathbf{T}^T \cdot \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix}) = \alpha(\begin{pmatrix} 0.9 & 0 \\ 0 & 0.15 \end{pmatrix} \cdot \begin{pmatrix} 0.6 & 0.2 \\ 0.4 & 0.8 \end{pmatrix} \cdot \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix})$

$= \alpha(\begin{pmatrix} 0.9 \cdot 0.6 & 0.9 \cdot 0.2 \\ 0.15 \cdot 0.4 & 0.15 \cdot 0.8 \end{pmatrix} \cdot \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix}) = \alpha(\begin{pmatrix} 0.9 \cdot 0.6 \cdot 0.5 + 0.9 \cdot 0.2 \cdot 0.5 \\ 0.15 \cdot 0.4 \cdot 0.5 + 0.15 \cdot 0.8 \cdot 0.5 \end{pmatrix}) =$

$\alpha(\begin{pmatrix} 0.36 \\ 0.09 \end{pmatrix})$

SOME RIGHTS RESERVED

Filtering the Umbrellas (Continued)

▷ **Example 24.2.4.** $\mathbf{f}_{1:1} := \mathbb{P}(\mathtt{R}_1|\mathtt{U}_1 = \mathsf{T}) = \langle 0.8, 0.2\rangle$

▷ $\mathbf{f}_{1:2} := \mathbb{P}(\mathtt{R}_2|\mathtt{U}_2 = \mathsf{T}, \mathtt{U}_1 = \mathsf{T}) = \alpha(\mathbf{O}_2 \cdot \mathbf{T}^T \cdot \mathbf{f}_{1:1}) = \alpha(\mathbb{P}(\mathtt{U}_2 = \mathsf{T}|\mathtt{R}_2) \cdot (\sum_{b\in\{\mathsf{T},\mathsf{F}\}} \mathbb{P}(\mathtt{R}_2|\mathtt{R}_1 = b) \cdot \mathbf{f}_{1:1}(b)))$

$= \alpha(\langle 0.9, 0.15\rangle \cdot (\langle 0.6, 0.4\rangle \cdot 0.8 + \langle 0.2, 0.8\rangle \cdot 0.2)) = \alpha(\langle 0.468, 0.072\rangle) = \langle 0.87, 0.13\rangle$

▷ $\mathbf{f}_{1:3} := \mathbb{P}(\mathtt{R}_3|\mathtt{U}_3 = \mathsf{F}, \mathtt{U}_2 = \mathsf{T}, \mathtt{U}_1 = \mathsf{T}) = \alpha(\mathbf{O}_3 \cdot \mathbf{T}^T \cdot \mathbf{f}_{1:2})$

$= \alpha(\mathbb{P}(\mathtt{U}_3 = \mathsf{F}|\mathtt{R}_3) \cdot (\sum_{b\in\{\mathsf{T},\mathsf{F}\}} \mathbb{P}(\mathtt{R}_3|\mathtt{R}_2 = b) \cdot \mathbf{f}_{1:2}(b)))$

$= \alpha(\langle 0.1, 0.85\rangle \cdot (\langle 0.6, 0.4\rangle \cdot 0.87 + \langle 0.2, 0.8\rangle \cdot 0.13)) = \alpha(\langle 0.0547, 0.3853\rangle) = \langle 0.12, 0.88\rangle$

SOME RIGHTS RESERVED

24.2.2 Prediction

The second temporal inference method is prediction: given a (temporal) chain of evidence up to now, it predicts the state of the world at some future time.

Prediction in Markov Chains

▷ Prediction: $\mathbb{P}(X_{t+k}|E_{1:t}^{=e})$ for $k > 0$.

▷ **Intuition:** Prediction is filtering without new evidence – i.e. we can use filtering until t, and then continue as follows:

▷ **Lemma 24.2.5.** *By the same reasoning as filtering:*

$$\mathbb{P}(X_{t+k+1}|E_{1:t}^{=e}) = \sum_{x\in\mathrm{dom}(X)} \underbrace{\mathbb{P}(X_{t+k+1}|X_{t+k} = x)}_{\text{transition model}} \cdot \underbrace{P(X_{t+k} = x \mid E_{1:t}^{=e})}_{\text{recursive call}} = \underbrace{\mathbf{T}^T \cdot \mathbb{P}(X_{t+k} = x|E_{1:t}^{=e})}_{\text{HMM}}$$

▷ **Observation 24.2.6.** *As $k \to \infty$, $\mathbb{P}(X_{t+k}|E_{1:t}^{=e})$ converges towards a fixed point called the stationary distribution of the Markov chain. (which we can compute from the equation $S = \mathbf{T}^T \cdot S$)*

⇝ *the impact of the evidence vanishes.*

⇝ *The stationary distribution only depends on the transition model.*

⇝ *There is a small window of time (depending on the transition model) where the evidence has enough impact to allow for prediction* beyond *the mere stationary distribution, called the mixing time of the Markov chain.*

⇝ *Predicting the future is difficult, and the further into the future, the more difficult it is (Who knew...)*

SOME RIGHTS RESERVED

24.2.3 Smoothing/Hindsight

The third temporal inference method is smoothing (also called hindsight), given a chain of evidence, it computes the most likely state of the world in at some time in the past, using the evidence between then and now to provide a better estimate than filtering with the past evidence could.

Smoothing

▷ Smoothing: $\mathbb{P}(X_{t-k}|E_{1:t}^{=e})$ for $k > 0$.

▷ **Intuition:** Use filtering to compute $\mathbb{P}(X_t|E_{1:t-k}^{=e})$, then recurse *backwards* from t until $t-k$.

$$\begin{aligned}
\mathbb{P}(X_{t-k}|E_{1:t}^{=e}) &= \mathbb{P}(X_{t-k}|E_{t-(k-1):t}^{=e}, E_{1:t-k}^{=e}) && \text{(Divide the evidence)}\\
&= \alpha(\mathbb{P}(E_{t-(k-1):t}^{=e}|X_{t-k}, E_{1:t-k}^{=e}) \cdot \mathbb{P}(X_{t-k}|E_{1:t-k}^{=e})) && \text{(Bayes Rule)}\\
&= \alpha(\underbrace{\mathbb{P}(E_{t-(k-1):t}^{=e}|X_{t-k})}_{=:\mathbf{b}_{t-(k-1):t}} \cdot \underbrace{\mathbb{P}(X_{t-k}|E_{1:t-k}^{=e})}_{=\mathbf{f}_{1:t-k}}) && \text{(cond. independence)}\\
&= \alpha(\mathbf{f}_{1:t-k} \times \mathbf{b}_{t-(k-1):t})
\end{aligned}$$

(where $\times$ denotes component-wise multiplication)

SOME RIGHTS RESERVED

Smoothing (continued)

▷ **Definition 24.2.7 (Backward message).** $\mathbf{b}_{t-k:t} = \mathbb{P}(E_{t-k:t}^{=e}|X_{t-(k+1)})$

$$\begin{aligned}
&= \sum_{x\in\mathrm{dom}(X)} \mathbb{P}(E_{t-k:t}^{=e}|X_{t-k}=x, X_{t-(k+1)}) \cdot \mathbb{P}(X_{t-k}=x|X_{t-(k+1)})\\
&= \sum_{x\in\mathrm{dom}(X)} P(E_{t-k:t}^{=e} \mid X_{t-k}=x) \cdot \mathbb{P}(X_{t-k}=x|X_{t-(k+1)})\\
&= \sum_{x\in\mathrm{dom}(X)} P(E_{t-k}=e_{t-k}, E_{t-(k-1):t}^{=e} \mid X_{t-k}=x) \cdot \mathbb{P}(X_{t-k}=x|X_{t-(k+1)})\\
&= \sum_{x\in\mathrm{dom}(X)} \underbrace{P(E_{t-k}=e_{t-k} \mid X_{t-k}=x)}_{\text{sensor model}} \cdot \underbrace{P(E_{t-(k-1):t}^{=e} \mid X_{t-k}=x)}_{=\mathbf{b}_{t-(k-1):t}} \cdot \underbrace{\mathbb{P}(X_{t-k}=x|X_{t-(k+1)})}_{\text{transition model}}
\end{aligned}$$

▷ **Note:** In a stationary hidden Markov model, we get the matrix formulation $\mathbf{b}_{t-k:t} = \mathbf{T} \cdot \mathbf{O}_{t-k} \cdot \mathbf{b}_{t-(k-1):t}$

▷ **Definition 24.2.8.** We call the associated algorithm the backward algorithm, i.e. $\mathbb{P}(X_{t-k}|E_{1:t}^{=e}) = \alpha(\underbrace{\text{FORWARD}(e_{t-k}, \mathbf{f}_{1:t-(k+1)})}_{\mathbf{f}_{1:t-k}} \times \underbrace{\text{BACKWARD}(e_{t-(k-1)}, \mathbf{b}_{t-(k-2):t})}_{\mathbf{b}_{t-(k-1):t}})$.

As a starting point for the recursion, we let $\mathbf{b}_{t+1:t}$ the uniform vector with 1 in every component.

SOME RIGHTS RESERVED

Smoothing example

▷ **Example 24.2.9 (Smoothing Umbrellas). Reminder:** We assumed $\mathbb{P}(\mathtt{R}_0) = \langle 0.5, 0.5\rangle$, $P(\mathtt{R}_{t+1} \mid \mathtt{R}_t) = 0.6$, $P(\neg\mathtt{R}_{t+1} \mid \neg\mathtt{R}_t) = 0.8$, $P(\mathtt{U}_t \mid \mathtt{R}_t) = 0.9$, $P(\neg\mathtt{U}_t \mid \neg\mathtt{R}_t) = 0.85$

$\rightsquigarrow \mathbf{T} = \begin{pmatrix} 0.6 & 0.4 \\ 0.2 & 0.8 \end{pmatrix}$, $\mathbf{O}_1 = \mathbf{O}_2 = \begin{pmatrix} 0.9 & 0 \\ 0 & 0.15 \end{pmatrix}$ and $\mathbf{O}_3 = \begin{pmatrix} 0.1 & 0 \\ 0 & 0.85 \end{pmatrix}$. (The director carries an umbrella on days 1 and 2, and *not* on day 3)

$\mathbf{f}_{1:1} = \langle 0.8, 0.2\rangle$, $\mathbf{f}_{1:2} = \langle 0.87, 0.13\rangle$ and $\mathbf{f}_{1:3} = \langle 0.12, 0.88\rangle$

Let's compute

$$\mathbb{P}(\mathtt{R}_1|\mathtt{U}_1 = \mathsf{T}, \mathtt{U}_2 = \mathsf{T}, \mathtt{U}_3 = \mathsf{F}) = \alpha(\mathbf{f}_{1:1} \times \mathbf{b}_{2:3})$$

▷ We need to compute $\mathbf{b}_{2:3}$ and $\mathbf{b}_{3:3}$:

▷ $\mathbf{b}_{3:3} = \mathbf{T} \cdot \mathbf{O}_3 \cdot \mathbf{b}_{4:3} = \begin{pmatrix} 0.6 & 0.4 \\ 0.2 & 0.8 \end{pmatrix} \cdot \begin{pmatrix} 0.1 & 0 \\ 0 & 0.85 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.4 \\ 0.7 \end{pmatrix}$

▷ $\mathbf{b}_{2:3} = \mathbf{T} \cdot \mathbf{O}_2 \cdot \mathbf{b}_{3:3} = \begin{pmatrix} 0.6 & 0.4 \\ 0.2 & 0.8 \end{pmatrix} \cdot \begin{pmatrix} 0.9 & 0 \\ 0 & 0.15 \end{pmatrix} \cdot \begin{pmatrix} 0.4 \\ 0.7 \end{pmatrix} = \begin{pmatrix} 0.258 \\ 0.156 \end{pmatrix}$

$\rightsquigarrow \alpha(\begin{pmatrix} 0.8 \\ 0.2 \end{pmatrix} \times \begin{pmatrix} 0.258 \\ 0.156 \end{pmatrix}) = \alpha(\begin{pmatrix} 0.2064 \\ 0.0312 \end{pmatrix}) = \begin{pmatrix} 0.87 \\ 0.13 \end{pmatrix}$

$\rightsquigarrow$ Given the evidence $\mathtt{U}_2, \neg\mathtt{U}_3$, the posterior probability for $\mathtt{R}_1$ went up from 0.8 to 0.87!

SOME RIGHTS RESERVED

Forward/Backward Algorithm for Smoothing

▷ **Definition 24.2.10.** Forward backward algorithm: returns the sequence of posterior distributions $\mathbb{P}(X_1)\ldots\mathbb{P}(X_t)$ given evidence $e_1, \ldots, e_t$:

function FORWARD-BACKWARD($\langle e_1, \ldots, e_t\rangle$,$\mathbb{P}(X_0)$)
- $f := \langle \mathbb{P}(X_0)\rangle$
- $b := \langle 1, 1, \ldots\rangle$
- $S := \langle \mathbb{P}(X_0)\rangle$
- **for** $i = 1, \ldots, t$ **do**
  - $f_i := \text{FORWARD}(f_{i-1}, e_i)$ */* filtering */*
- **for** $i = t, \ldots, 1$ **do**
  - $S_i := \alpha(f_i \times b)$ */* smoothing */*
  - $b := \text{BACKWARD}(b, e_i)$

return S

▷ Time complexity linear in t (polytree inference), Space complexity $\mathcal{O}(t \cdot |f|)$.

SOME RIGHTS RESERVED

24.2.4 Most Likely Explanation

The last temporal inference method is most likely explanation. In contrast to the other three, which compute a belief state, most likely explanation computes the sequence of events that best explain the evidence. Note that can be different from the sequence of most likely events.

Most Likely Explanation

▷ Smoothing allows us to compute the *sequence of most likely states* $X_1, \ldots, X_t$ given $E_{1:t}^{=e}$. What if we want the *most likely sequence* of states? i.e. $\max_{x_1,\ldots,x_t} (P(X_{1:t}^{=x} \mid E_{1:t}^{=e}))$?

▷ **Example 24.2.11.** Given the sequence $\mathtt{U}_1, \mathtt{U}_2, \neg\mathtt{U}_3, \mathtt{U}_4, \mathtt{U}_5$, the most likely state for $\mathtt{R}_3$ is F, but the most likely sequence *might* be that it rained throughout...

▷ **Prominent Application:** In speech recognition, we want to find the most likely word sequence, given what we have heard. (can be quite noisy)

▷ **Idea:**

- ▷ For every $x_t \in \mathrm{dom}(X)$ and $0 < i \leq t$, recursively compute the most likely path $X_1, \ldots, X_i$ ending in $X_i = x_i$ given the observed evidence.
- ▷ remember the x_{i-1} that most likely leads to x_i.
- ▷ Among the resulting paths, pick the one to *the* $X_t = x_t$ with the most likely path,
- ▷ and then recurse backwards.

⇝ we want to know $\max_{x_1,\ldots,x_{t-1}} \mathbb{P}(X_{1:t-1}^{=x}, X_t | E_{1:t}^{=e})$, and then pick the x_t with the maximal value.

SOME RIGHTS RESERVED

Most Likely Explanation (continued)

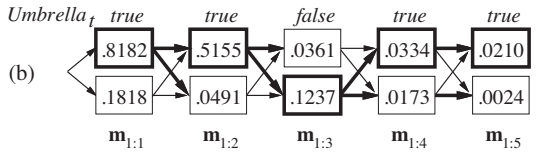
▷ By the same reasoning as for filtering:

$$\begin{aligned}&\max_{x_1,\ldots,x_{t-1}} \mathbb{P}(X_{1:t-1}^{=x}, X_t | E_{1:t}^{=e}) \\ &= \alpha(\underbrace{\mathbb{P}(E_t = e_t | X_t)}_{\text{sensor model}} \cdot \max_{x_{t-1}} (\underbrace{\mathbb{P}(X_t | X_{t-1} = x_{t-1})}_{\text{transition model}} \cdot \underbrace{\max_{x_1,\ldots,x_{t-2}} (P(X_{1:t-2}^{=x}, X_{t-1} = x_{t-1} \mid E_{1:t-1}^{=e}))}_{=:\mathbf{m}_{1:t-1}(x_{t-1})}))\end{aligned}$$

▷ $\mathbf{m}_{1:t}(i)$ gives the maximal probability that the most likely path up to t leads to state $X_t = i$.

▷ Note that we can leave out the α, since we're only interested in the maximum.

▷ **Example 24.2.12.** For the sequence [T, T, F, T, T]:



bold arrows: best predecessor measured by "best preceding sequence probability × transition probability"

SOME RIGHTS RESERVED

The Viterbi Algorithm

▷ **Definition 24.2.13.** The Viterbi algorithm now proceeds as follows:

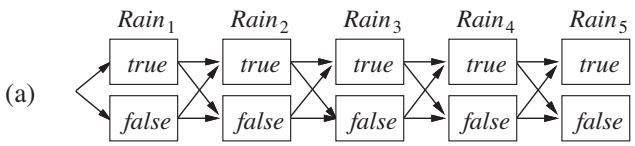
function VITERBI($\langle e_1, \ldots, e_t \rangle$,$\mathbb{P}(X_0)$)
- $m := \mathbb{P}(X_0)$ /* $\mathbf{m}_{1:i}$ */
- prev $:= \langle\rangle$ /* the most likely predecessor of each possible x_i */
- **for** $i = 1, \ldots, t$ **do**
  - $m' := \max_{x_{i-1}} (\mathbb{P}(E_i = e_i | X_i) \cdot \mathbb{P}(X_i | X_{i-1} = x_{i-1}) \cdot m_{x_{i-1}})$
  - $\text{prev}_{i-1} := \operatorname{argmax}_{x_{i-1}} (\mathbb{P}(E_i = e_i | X_i) \cdot \mathbb{P}(X_i | X_{i-1} = x_{i-1}) \cdot m_{x_{i-1}})$
  - $m \longleftarrow m'$
- $P := \langle 0, 0, ..., \operatorname{argmax}_{(x \in \text{dom}(X))} m_x \rangle$
- **for** $i = t - 1, \ldots, 0$ **do**
  - $P_i := \text{prev}_{i, P_{i+1}}$
- **return** P

▷ **Observation 24.2.14.** *Viterbi has linear time complexity and linear space complexity (needs to keep the most likely sequence leading to each state).*

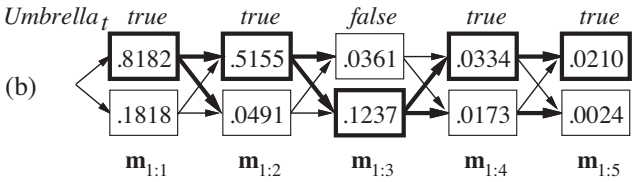
SOME RIGHTS RESERVED

Viterbi example

▷ **Example 24.2.15 (Viterbi for Umbrellas).** View the possible state sequences for Rain_t as paths through state graph.



Operation of the Viterbi algorithm for the sequence [T, T, F, T, T]:



▷ values are $\mathbf{m}_{1:t}$ (probability of best sequence reaching state at time t)

▷ bold arrows: best predecessor measured by "best preceding sequence probability × transition probability"

To find "most likely sequence", follow bold arrows back from "most likely state $\mathbf{m}_{1:5}$.

SOME RIGHTS RESERVED

24.3 Hidden Markov Models – Extended Example

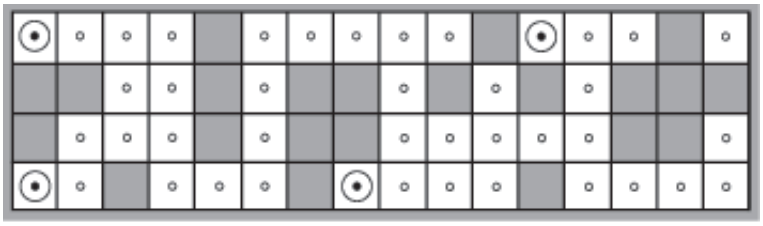
We will now collect all of the information on hidden Markov models and review it on a larger example: robot localization.

Example: Robot Localization using Common Sense

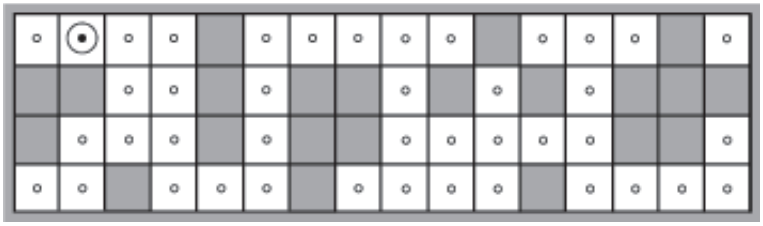
▷ **Example 24.3.1 (Robot Localization in a Maze).** A robot has four sonar sensors that tell it about obstacles in four directions: N, S, W, E.

We write the result where the sensor that detects obstacles in the north, south, and east as N S E.

We filter out the impossible states:



a) Possible robot locations after $e_1 = \text{N S W}$



b) Possible robot locations after $e_1 = \text{N S W}$ and $e_2 = \text{N S}$

▷ *Remark 24.3.2.* This only works for perfect sensors. (else no impossible states)

▷ What if our sensors are imperfect?

HMM Example: Robot Localization (Modeling)

▷ **Example 24.3.3 (HMM-based Robot Localization).** We have the following setup:

- ▷ A hidden Random variable X_t for robot location (domain: 42 empty squares)
- ▷ Let $N(i)$ be the set of neighboring fields of the field $X_i = x_i$
- ▷ The Transition matrix for the move action ($\mathbf{T}$ has $42^2 = 1764$ entries)

$$P(X_{t+1} = j \mid X_t = i) = \mathbf{T}_{ij} = \begin{cases} \frac{1}{|N(i)|} & \text{if } j \in N(i) \\ 0 & \text{else} \end{cases}$$

- ▷ We do not know where the robot starts: $P(X_0) = \frac{1}{n}$ (here $n = 42$)
- ▷ Evidence variable E_t: four bit presence/absence of obstacles in N, S, W, E. Let d_{it} be the number of wrong bits and ϵ the error rate of the sensor. Then

$$P(E_t = e_t \mid X_t = i) = \mathbf{O}_{tii} = (1-\epsilon)^{4-d_{it}} \cdot \epsilon^{d_{it}}$$

(We assume the sensors are independent)

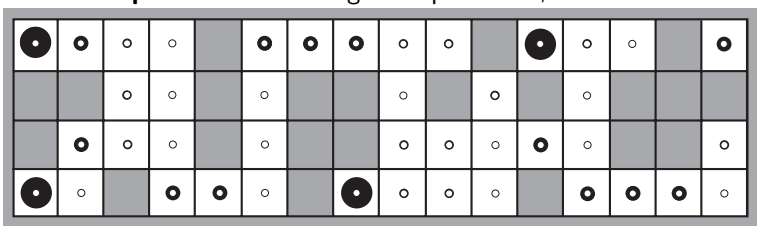
For example, the probability that the sensor on a square with obstacles in north and south would produce N S E is $(1-\epsilon)^3 \cdot \epsilon^1$.

▷ We can now use filtering for localization, smoothing to determine e.g. the starting location, and the Viterbi algorithm to find out how the robot got to where it is now.

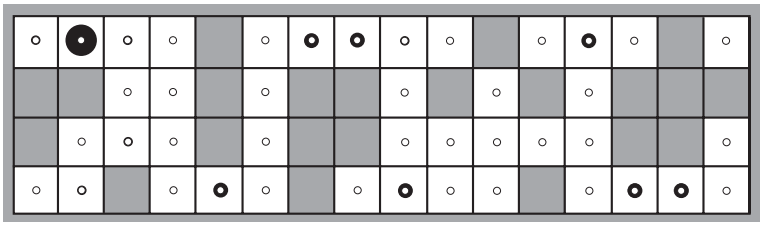
HMM Example: Robot Localization

▷ We use HMM filtering equation $\mathbf{f}_{1:t+1} = \alpha \cdot \mathbf{O}_{t+1}\mathbf{T}^T\mathbf{f}_{1:t}$ to compute posterior distribution over locations. (i.e. robot localization)

▷ **Example 24.3.4.** Redoing Example 24.3.1, with $\epsilon = 0.2$.



a) Posterior distribution over robot location after $\mathrm{E}_1 = \mathrm{N\ S\ W}$



b) Posterior distribution over robot location after $E_1 = N\ S\ W$ and $E_2 = N\ S$

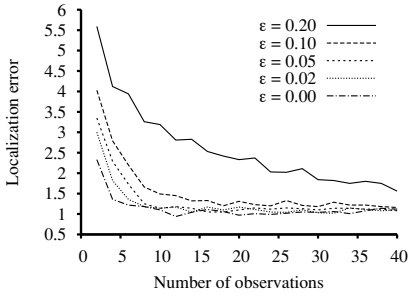
Still the same locations as in the "perfect sensing" case, but now other locations have non-zero probability.

SOME RIGHTS RESERVED

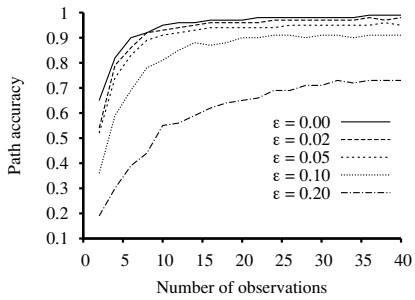
HMM Example: Further Inference Applications

▷ **Idea:** We can use smoothing: $\mathbf{b}_{k+1:t} = \mathbf{T}\mathbf{O}_{k+1}\mathbf{b}_{k+2:t}$ to find out where it started and the Viterbi algorithm to find the most likely path it took.

▷ **Example 24.3.5.** Performance of HMM localization vs. observation length (various error rates ϵ)



Localization error (Manhattan distance from true location)



Viterbi path accuracy (fraction of correct states on Viterbi path)

SOME RIGHTS RESERVED

24.4 Dynamic Bayesian Networks

Finally, we adapt the Bayesian networks, which we developed earlier to give use a space and computation-efficient representation of probability models (with some inner structure) to the dynamic case. This is relatively simple for stationary Markov processes where the dynamic Bayesian networks naturally divide into time slides – identical sub-networks.

Dynamic Bayesian networks

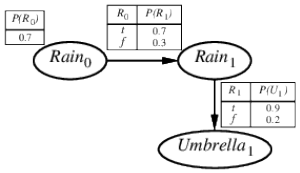
▷ **Definition 24.4.1.** A Bayesian network $\mathcal{D}$ is called dynamic (a DBN), iff its random variables are indexed by a time structure. We assume that $\mathcal{D}$ is

- ▷ time sliced, i.e. that the time slices $\mathcal{D}_t$ – the subgraphs of t-indexed random variables and the edges between them – are isomorphic.
- ▷ a stationary Markov chain, i.e. that variables X_t can only have parents in $\mathcal{D}_t$ and $\mathcal{D}_{t-1}$.

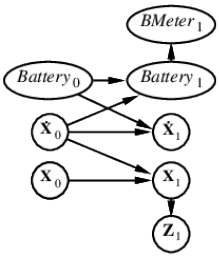
▷ $\mathbf{X}_t$, $\mathbf{E}_t$ contain arbitrarily many variables in a replicated Bayesian network.

▷ **Example 24.4.2.**

Umbrellas



Robot Motion

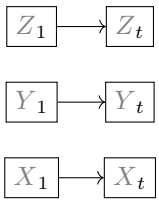


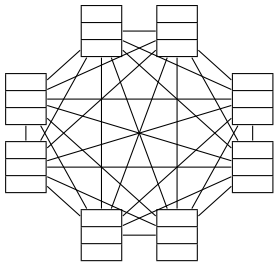
SOME RIGHTS RESERVED

DBNs vs. HMMs

▷ **Observation 24.4.3.**

- ▷ *Every HMM is a single-variable DBN.* *(trivially)*
- ▷ *Every DBN can be turned into an HMM.* *(combine variables into tuple ⇝ lose information about dependencies)*
- ▷ *DBNs have sparse dependencies ⇝ exponentially fewer parameters;*



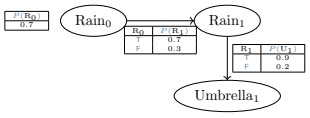


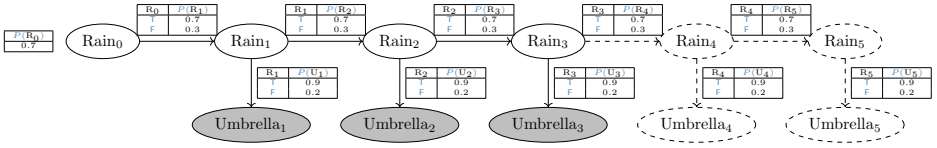
▷ **Example 24.4.4 (Sparse Dependencies).** With 20 Boolean state variables, three parents each, a DBN has $20 \cdot 2^3 = 160$ parameters, the corresponding HMM has $2^{20} \cdot 2^{20} \approx 10^{12}$.

SOME RIGHTS RESERVED

Exact inference in DBNs

▷ **Definition 24.4.5 (Naive method).** Unroll the network and run any exact algorithm.





▷ **Problem:** Inference cost for each update grows with t.

▷ **Definition 24.4.6.** Rollup filtering: add slice $t+1$, "sum out" slice t using variable elimination.

▷ **Observation:** Largest factor is $\mathcal{O}(d^{n+1})$, update cost $\mathcal{O}(d^{n+2})$, where d is the maximal domain size.

▷ **Note:** Much better than the HMM update cost of $\mathcal{O}(d^{2n})$

SOME RIGHTS RESERVED

Summary

▷ Temporal probability models use state and evidence variables replicated over time.

▷ Markov property and stationarity assumption, so we need both

  ▷ a transition model and $\mathbb{P}(\mathbf{X}_t|\mathbf{X}_{t-1})$

  ▷ a sensor model $\mathbb{P}(\mathbf{E}_t|\mathbf{X}_t)$.

▷ Tasks are filtering, prediction, smoothing, most likely sequence; (all done recursively with constant cost per time step)

▷ Hidden Markov models have a single discrete state variable; (used for speech recognition)

▷ DBNs subsume HMMs, exact update intractable.

SOME RIGHTS RESERVED

Chapter 25

Making Complex Decisions

We will now pick up the thread from chapter 23 but using temporal probabilistic models instead of simply probabilistic ones. We will first look at a sequential decision theory in the special case, where the environment is stochastic, but fully observable (Markov decision processes) and then lift that to obtain partially observable Markov decision processs and present an agent design based on that.

Outline

▷ We will now combine the ideas of stochastic process with that of acting based on maximizing expected utility:

▷ Markov decision processes (MDPs) for sequential environments.

▷ Value/policy iteration for computing utilities in MDPs.

▷ Partially observable MDP (POMDPs).

▷ Decision theoretic agents for POMDPs.

SOME RIGHTS RESERVED

25.1 Sequential Decision Problems

We will now set up the stage for sequential decision problems in AI by defining one simple instance: Markov Decision Problems, which will be the workhorse of this section. The main new feature we will discuss is that we will need to work with policies.

Sequential Decision Problems

▷ **Definition 25.1.1.** In sequential decision problems, the agent's utility depends on a sequence of decisions (or their result states).

▷ **Definition 25.1.2.** Utility functions on action sequences are often expressed in terms of immediate rewards that are incurred upon reaching a (single) state.

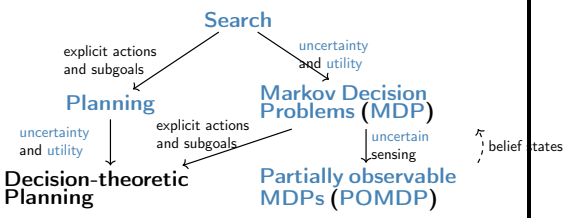
▷ **Methods:** depend on the environment:

▷ If it is fully observable ⇝ Markov decision process (MDP)

▷ else ⇝ partially observable MDP (POMDP).

▷ Sequential decision problems incorporate utilities, uncertainty, and sensing.

▷ **Preview:** Search problems and planning tasks are special cases.

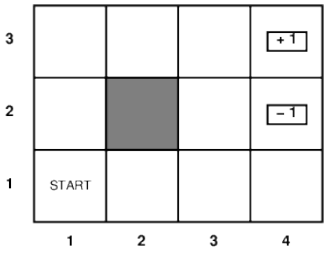


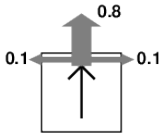
SOME RIGHTS RESERVED

We will fortify our intuition by an example. It is specifically chosen to be very simple, but to exhibit all the peculiarities of Markov decision problems, which we will generalize from this example.

Markov Decision Problem: Running Example

▷ **Example 25.1.3 (Running Example: The 4x3 World).** A (fully observable) 4×3 environment with non-deterministic actions:





▷ States $s \in \mathcal{S}$, actions $a \in \mathcal{A}$.

▷ Transition model: $P(s' \mid s,a) \widehat{=}$ probability that a in s leads to s'.

▷ reward function:

$$R(s) := \begin{cases} -0.04 & \text{if (small penalty) for nonterminal states} \\ \pm 1 & \text{if for terminal states} \end{cases}$$

SOME RIGHTS RESERVED

Perhaps what is more interesting than the components of an MDP is that is *not* a component: a belief state and/or sensor model. Recall that Markov decision processs are for fully observable environments.

Markov Decision Process

▷ **Motivation:** Let us (for now) consider sequential decision problems in a fully observable, stochastic environment with a Markovian transition model on a *finite* set of states and an additive reward function. (We will switch to partially observable ones later)

▷ **Definition 25.1.4.** A Markov decision process (MDP) $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, s_0, R \rangle$ consists of

- ▷ a set of $\mathcal{S}$ of states (with initial state $s_0 \in \mathcal{S}$),
- ▷ for every state s, a set of actions $\mathcal{A}s$.
- ▷ a transition model $\mathcal{T}(s,a) = \mathbb{P}(\mathcal{S}|s,a)$, and
- ▷ a reward function $R\colon \mathcal{S} \to \mathbb{R}$; we call $R(s)$ a reward.

▷ **Idea:** We use the rewards as a utility function: The goal is to choose actions such that the expected *cumulative* rewards for the "foreseeable future" is maximized.

⇝ need to take future actions and future states into account

SOME RIGHTS RESERVED

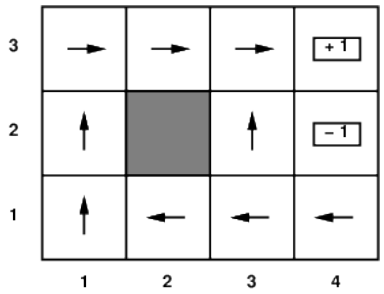
Solving MDPs

▷ In MDPs, the aim is to find an optimal policy $\pi(s)$, which tells us the best action for every possible state s. (because we can't predict where we might end up, we need to consider all states)

▷ **Definition 25.1.5.** A policy π for an MDP is a function mapping each state s to an action $a \in \mathcal{A}s$.

An optimal policy is a policy that maximizes the expected total rewards. (for some notion of "total"...)

▷ **Example 25.1.6.** Optimal policy when state penalty $R(s)$ is 0.04:

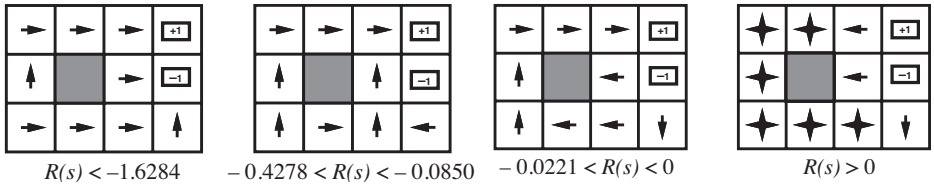


Note: When you run against a wall, you stay in your square.

SOME RIGHTS RESERVED

Risk and Reward

▷ **Example 25.1.7.** Optimal policy depends on the reward function $R(s)$.



▷ **Question:** Explain what you see in a qualitative manner!

▷ **Answer:** reserved for the plenary sessions ⇝ be there!

SOME RIGHTS RESERVED

25.2 Utilities over Time

In this section we address the problem that even if the transition models are stationary, the utilities may not be. In fact we generally have to take the utilities of state sequences into account in sequential decision problems. If we can derive a notion of the utility of a (single) state from that, we may be able to reuse the machinery we introduced above, so that is exactly what we will attempt.

Utility of state sequences

▷ Why rewards?

▷ **Recall:** We cannot observe/assess utility functions, only preferences ⇝ induce utility functions from rational preferences

▷ **Problem:** In MDPs we need to understand preferences between *sequences* of states.

▷ **Definition 25.2.1.** We call preferences on reward sequences stationary, iff

$$[r, r_0, r_0, r_1, r_2, \ldots] \succ [r, r'_0, r'_1, r'_2, \ldots] \Leftrightarrow [r_0, r_1, r_2, \ldots] \succ [r'_0, r'_1, r'_2, \ldots]$$

(i.e. rewards over time are "independent" of each other)

▷ **Good news:** the world is simpler than we may have thought]

▷ **Theorem 25.2.2.** *For stationary preferences, there are only two ways to combine rewards over time.*

- ▷ *additive rewards:* $U([s_0, s_1, s_2, \ldots]) = R(s_0) + R(s_1) + R(s_2) + \cdots$
- ▷ *discounted rewards:* $U([s_0, s_1, s_2, \ldots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots$ *where* $0 < \gamma < 1$ *is called discount factor.*

⇝ we can reduce utilities over time to rewards on individual states

SOME RIGHTS RESERVED

Utilities of State Sequences

▷ **Problem:** Infinite lifetimes ⇝ additive rewards may become infinite.

▷ **Possible Solutions:**

1. Finite horizon: terminate utility computation at a fixed time T

$$U([s_0, \ldots, s_\infty]) = R(s_0) + \cdots + R(s_T)$$

   ⇝ nonstationary policy: $\pi(s)$ depends on time left.

2. If there are absorbing states: for any policy π agent eventually "dies" with probability 1 ⇝ expected utility of every state is finite.

3. Discounting: assuming $\gamma < 1$, $R(s) \leq R_{\max}$,

$$U([s_0, s_1, \ldots]) = \sum_{t=0}^{\infty} \gamma^t R(s_t) \leq \sum_{t=0}^{\infty} \gamma^t R_{\max} = R_{\max}/(1-\gamma)$$

   Smaller γ ⇝ shorter horizon.

▷ We will only consider discounted rewards in this course

SOME RIGHTS RESERVED

Why discounted rewards?

▷ Discounted rewards are both convenient and (often) realistic:

▷ stationary preferences imply (additive rewards or) discounted rewards anyway,

▷ discounted rewards lead to finite utilities for (potentially) infinite sequences of states (we can compute expected utilities for the entire future),

▷ discounted rewards lead to stationary policies, which are easier to compute and often more adequate (unless we know that remaining time matters),

▷ discounted rewards mean we value *short-term gains* over *long-term gains* (all else being equal), which is often realistic (e.g. the same amount of money gained *early* gives more opportunity to spend/invest ⇝ potentially more utility in the long run)

▷ we can interpret the discount factor as a measure of *uncertainty about future rewards* ⇝ more robust measure in uncertain environments.

SOME RIGHTS RESERVED

Utility of States

▷ **Remember:** Given a sequence of states $S = s_0, s_1, s_2, \ldots$, and a discount factor $0 \leq \gamma < 1$, the utility of the sequence is

$$U(S) = \sum_{t=0}^{\infty} \gamma^t R(s_t)$$

▷ **Definition 25.2.3.** Given a policy π and a starting state s_0, let $S^\pi_{s_0}$ be the random variable

giving the sequence of states resulting from executing π at every state starting at s_0. (Since the environment is stochastic, we don't know the exact sequence.)

Then the policy utility is obtained as the expected utility of executing π starting in s_0. It is given by

$$U^\pi(s_0) := \mathrm{EU}(S^\pi_{s_0}).$$

We define the optimal policy $\pi^*_{s_0} := \operatorname{argmax}_\pi U^\pi(s_0)$.

▷ **Note:** This is perfectly well-defined, but almost always computationally infeasible.(requires considering *all possible (potentially infinite) sequences of states*)

SOME RIGHTS RESERVED

Utility of States (continued)

▷ **Observation 25.2.4.** $\pi^*_{s_0}$ *is independent of the state* s_0.

▷ *Proof sketch:* If π^*_a and π^*_b reach state c, then there is no reason to disagree from that point on – or with π^*_c, and we expect optimal policies to "meet at some state" sooner or later.

▷ ⚠ Observation 25.2.4 does not hold for finite horizon policies!

▷ **Definition 25.2.5.** We call $\pi^* := \pi^*_s$ for some s the optimal policy.

▷ **Definition 25.2.6.** The utility $U(s)$ of a state s is $U^{\pi^*}(s)$.

▷ **Remark:** $R(s) \mathrel{\widehat{=}}$ "immediate reward", whereas $U \mathrel{\widehat{=}}$ "long-term reward".

▷ Given the utilities of the states, choosing the best action is just MEU: maximize the expected utility of the immediate successor states

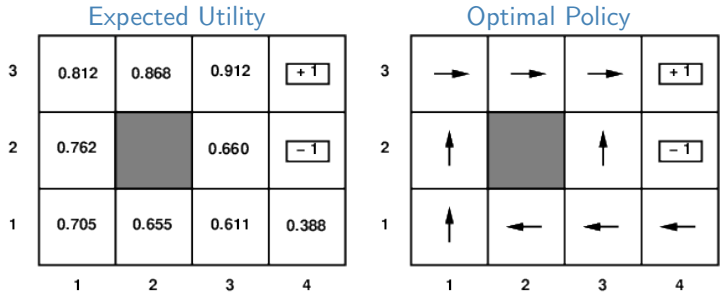
$$\pi^*(s) = \operatorname*{argmax}_{a \in A(s)} \Big(\sum_{s'} P(s' \mid s,a) \cdot U(s')\Big)$$

▷ $\rightsquigarrow$ given the "true" utilities, we can compute the optimal policy and vice versa.

SOME RIGHTS RESERVED

Utility of States (continued)

▷ **Example 25.2.7 (Running Example Continued).**



▷ **Question:** Why do we go left in $(3,1)$ and not up? (follow the utility)

SOME RIGHTS RESERVED

25.3 Value/Policy Iteration

In the last section we have rigorously *defined* the concept of an optimal policy and the utility of a state in a MDP, but these definitions are full of infinite sums and maxima, which are inherently difficult to compute. In this section we will introduce two algorithms that compute them. Both are based on the Bellman equation, the first one – value iteration – iterates Bellman updates in the space of state utilities and the second one – policy iteration – in policy space, which is generally finite, and may thus converge faster.

Dynamic programming: the Bellman equation

▷ **Problem:** We have defined $U(s)$ via the optimal policy: $U(s) := U^{\pi^*}(s)$, but how to compute it without knowing π^*?

▷ **Observation:** A simple relationship among utilities of neighboring states:

expected sum of rewards = current reward + $\gamma \cdot$ exp. reward sum after best action

▷ **Theorem 25.3.1 (Bellman equation (1957)).**

$$U(s) = R(s) + \gamma \cdot \max_{a \in A(s)} \sum_{s'} U(s') \cdot P(s' \mid s,a)$$

We call this equation the Bellman equation

▷ **Example 25.3.2.** $U(1,1) = -0.04$

$$+ \gamma \max\{\begin{array}{ll} 0.8U(1,2) + 0.1U(2,1) + 0.1U(1,1), & \textit{up} \\ 0.9U(1,1) + 0.1U(1,2) & \textit{left} \\ 0.9U(1,1) + 0.1U(2,1) & \textit{down} \\ 0.8U(2,1) + 0.1U(1,2) + 0.1U(1,1)\} & \textit{right} \end{array}$$

▷ **Problem:** One equation/state $\rightsquigarrow$ n nonlinear ($\max$ isn't) equations in n unknowns.

$\rightsquigarrow$ cannot use linear algebra techniques for solving them.

SOME RIGHTS RESERVED

Value Iteration Algorithm

▷ **Idea:** We use a simple iteration scheme to find a fixpoint:

1. start with arbitrary utility values,
2. update to make them locally consistent with the Bellman equation,
3. everywhere locally consistent $\rightsquigarrow$ global optimality.

▷ **Definition 25.3.3.** The value iteration algorithm for utility functions is given by the following algorithm with inputs

- ▹ mdp, an MDP with states S, actions $A(s)$, transition model $P(s' \mid s,a)$, rewards $R(s)$, and discount γ
- ▹ ϵ, the maximum error allowed in the utility of any state

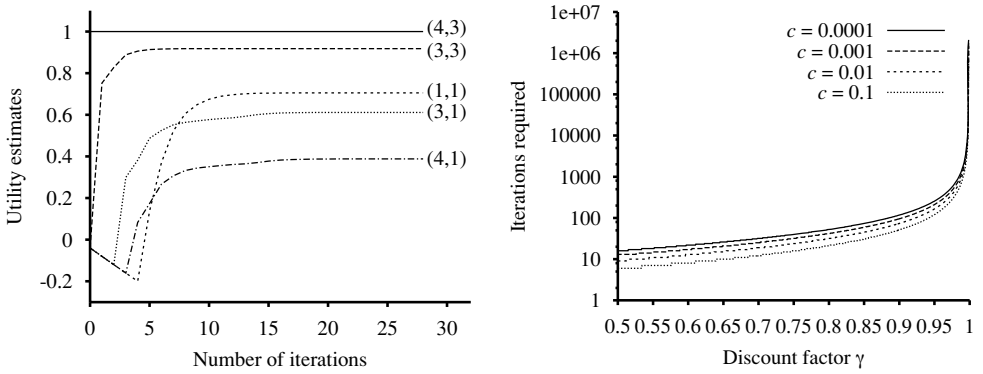
function VALUE−ITERATION (mdp,ϵ) **returns** a utility fn.
  inputs: mdp, an MDP with states S, actions $A(s)$, transition model $P(s' \mid s,a)$,
    rewards $R(s)$, and discount γ
    ϵ, the maximum error allowed **in** the utility of any state
  local variables: U, U', vectors of utilities **for** states **in** S, initially zero
    δ, the maximum change **in** the utility of any state **in** an iteration
  repeat
    $U := U'$; $\delta := 0$
    for each state s **in** S **do**
      $U'[s] := R(s) + \gamma \cdot \max_{a \in A(s)} (\sum_{s'} U[s'] \cdot P(s' \mid s,a))$
      if $|U'[s] - U[s]| > \delta$ **then** $\delta := |U'[s] - U[s]|$
    until $\delta < \epsilon(1-\gamma)/\gamma$
    return U

▷ **Remark:** Retrieve the optimal policy with $\pi[s] := \operatorname{argmax}_{a \in A(s)} (\sum_{s'} U[s'] \cdot P(s' \mid s,a))$

SOME RIGHTS RESERVED

Value Iteration Algorithm (Example)

▷ **Example 25.3.4 (Iteration on 4x3).**



(where $\varepsilon = c \cdot R_{max}$)

SOME RIGHTS RESERVED

Convergence

▷ **Definition 25.3.5.** The maximum norm is defined as $\|U\| = \max_s |U(s)|$, so $\|U - V\| =$ maximum difference between U and V.

▷ Let U^t and U^{t+1} be successive approximations to the true utility U during value iteration.

▷ **Theorem 25.3.6.** *For any two approximations U^t and V^t*

$$\left\|U^{t+1} - V^{t+1}\right\| \leq \gamma \left\|U^t - V^t\right\|$$

I.e., any distinct approximations get closer to each other over time
In particular, any approximation gets closer to the true U over time
⇝ value iteration converges to a unique, stable, optimal solution.

▷ **Theorem 25.3.7.** *If $\left\|U^{t+1} - U^t\right\| < \epsilon$, then $\left\|U^{t+1} - U\right\| < 2\epsilon\gamma/1 - \gamma$*
(once the change in U^t becomes small, we are almost done.)

▷ **Remark:** The policy resulting from U^t may be optimal long before the utilities converge!

SOME RIGHTS RESERVED

So we see that iteration with Bellman updates will always converge towards the utility of a state, even without knowing the optimal policy. That gives us a first way of dealing with sequential decision problems: we compute utility functions based on states and then use the standard MEU machinery. We have seen above that optimal policies and state utilities are essentially interchangeable: we can compute one from the other. This leads to another approach to computing state utilities: policy iteration, which we will discuss now.

Policy Iteration

▷ **Recap:** Value iteration computes utilities ⇝ optimal policy by MEU.

▷ This even works if the utility estimate is inaccurate. (⇜ policy loss small)

▷ **Idea:** Search for optimal policy and utility values simultaneously [How60]: Iterate

- ▷ policy evaluation: given policy π_i, calculate $U_i = U^{\pi_i}$, the utility of each state were π_i to be executed.
- ▷ policy improvement: calculate a new MEU policy π_{i+1} using 1 lookahead

Terminate if policy improvement yields no change in computed utilities.

▷ **Observation 25.3.8.** *Upon termination U_i is a fixpoint of Bellman update*
⇝ Solution to Bellman equation ⇝ π_i is an optimal policy.

▷ **Observation 25.3.9.** *Policy improvement improves policy and policy space is finite ⇝ termination.*

SOME RIGHTS RESERVED

Policy Iteration Algorithm

▷ **Definition 25.3.10.** The policy iteration algorithm is given by the following pseudocode: It takes as input an MDP with states S, actions $A(s)$, transition model $P(s' \mid s,a)$.

function POLICY–ITERATION(mdp) **returns** a policy
  inputs: mdp, and MDP with states S, actions $A(s)$, transition model $P(s' \mid s,a)$
  local variables: U a vector of utilities **for** states **in** S, initially zero
    π a policy indexed by state, initially random,
  repeat

U := POLICY−EVALUATION(π,U,mdp)
unchanged? := **true**
foreach state s **in** X **do**
  if $\max_{a \in A(s)} (\sum_{s'} P(s' \mid s,a) \cdot U(s')) > \sum_{s'} P(s' \mid s,\pi[s]) \cdot U(s')$ **then do**
    $\pi[s] := \operatorname{argmax}_{b \in A(s)} (\sum_{s'} P(s' \mid s,b) \cdot U(s'))$
    unchanged? := **false**
until unchanged?
return π

Policy Evaluation

▷ **Problem:** How to implement the POLICY−EVALUATION algorithm?

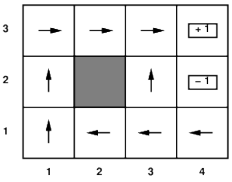
▷ **Solution:** To compute utilities given a fixed π: For all s we have

$$U(s) = R(s) + \gamma(\sum_{s'} U(s') \cdot P(s' \mid s,\pi(s)))$$

(i.e. Bellman equation with the maximum replaced by the current policy π)

▷ **Example 25.3.11 (Simplified Bellman Equations for π).**

$$\begin{aligned} U_i(1,1) &= -0.04 + 0.8U_i(1,2) + 0.1U_i(1,1) + 0.1U_i(2,1) \\ U_i(1,2) &= -0.04 + 0.8U_i(1,3) + 0.1U_i(1,2) \\ &\vdots \end{aligned}$$



▷ **Observation 25.3.12.** *n simultaneous linear equations in n unknowns, solve in $\mathcal{O}(n^3)$ with standard linear algebra methods.*

Modified Policy Iteration

▷ Value iteration requires many iterations, but each one is cheap.

▷ Policy iteration often converges in few iterations, but each is expensive.

▷ **Idea:** Use a few steps of value iteration (but with π fixed), starting from the value function produced the last time to produce an approximate value determination step.

▷ Often converges much faster than pure VI or PI.

▷ Leads to much more general algorithms where Bellman value updates and Howard policy updates can be performed locally in any order.

▷ **Remark:** Reinforcement learning algorithms operate by performing such updates based on the observed transitions made in an initially unknown environment.

25.4 Partially Observable MDPs

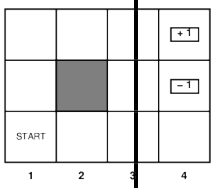
We will now lift the last restriction we made in the decision problems for our agents: in the definition of Markov decision processes we assumed that the environment was fully observable. As we have seen Observation 25.2.4 this entails that the optimal policy only depends on the current state.

Partial Observability

▷ **Definition 25.4.1.** A partially observable MDP (a POMDP for short) is a MDP together with an observation model O that has the sensor Markov property and is stationary: $O(s,e) = P(e \mid s)$.

▷ **Example 25.4.2 (Noisy 4x3 World).**

Add a partial and/or noisy sensor.
e.g. count number of adjacent walls ($1 \leq w \leq 2$)
with 0.1 error (noise)
If sensor reports 1, we are in $(3,?)$ (probably)



▷ **Problem:** Agent does not know which state it is in $\rightsquigarrow$ makes no sense to talk about policy $\pi(s)$!

▷ **Theorem 25.4.3 (Astrom 1965).** *The optimal policy in a POMDP is a function $\pi(b)$ where b is the belief state (probability distribution over states).*

▷ **Idea:** Convert a POMDP into an MDP in belief state space, where $\mathcal{T}(b,a,b')$ is the probability that the new belief state is b' given that the current belief state is b and the agent does a. I.e., essentially a filtering update step.

POMDP: Filtering at the Belief State Level

▷ **Recap:** Filtering updates the belief state for new evidence.

▷ For POMDPs, we also need to consider actions. (but the effect is the same)

▷ If b is the previous belief state and agent does action $A = a$ and then perceives $E = e$, then the new belief state is

$$b' = \alpha(\mathbb{P}(E = e|s') \cdot (\sum_s \mathbb{P}(s'|S = s, A = a) \cdot b(s)))$$

We write $b' = \text{FORWARD}(b,a,e)$ in analogy to recursive state estimation.

▷ **Fundamental Insight for POMDPs:** The optimal action only depends on the agent's current belief state. (good, it does not know the state!)

▷ **Consequence:** The optimal policy can be written as a function $\pi^*(b)$ from belief states to actions.

▷ **Definition 25.4.4.** The POMDP decision cycle is to iterate over

1. Given the current belief state b, execute the action $a = \pi^*(b)$
2. Receive percept e.
3. Set the current belief state to $\mathrm{FORWARD}(b, a, e)$ and repeat.

▷ **Intuition:** POMDP decision cycle is search in belief state space.

SOME RIGHTS RESERVED

Partial Observability contd.

▷ **Recap:** The POMDP decision cycle is search in belief state space.

▷ **Observation 25.4.5.** *Actions change the belief state, not just the (physical) state.*

▷ **Thus** POMDP solutions automatically include information gathering behavior.

▷ **Problem:** The belief state is continuous: If there are n states, b is an n-dimensional real-valued vector.

▷ **Example 25.4.6.** The belief state of the 4x3 world is a 11 dimensional continuous space. (11 states)

▷ **Theorem 25.4.7.** *Solving POMDPs is very hard!* *(actually, PSPACE hard)*

▷ **In particular,** none of the algorithms we have learned applies. (discreteness assumption)

▷ The real world is a POMDP (with initially unknown transition model T and sensor model O)

SOME RIGHTS RESERVED

25.5 Reducing POMDPs to belief-state-level MDPs

Reducing POMDPs to Belief-State MDPs

▷ **Idea:** Calculating the probability that an agent in belief state b reaches belief state b' after executing action a.

  ▷ if we knew the action and the *subsequent* percept e, then $b' = \mathrm{FORWARD}(b, a, e)$. (deterministic update to the belief state)

  ▷ but we don't, since b' depends on e. (let's calculate $P(e \mid a,b)$)

▷ **Idea:** To compute $P(e \mid a,b)$ — the probability that e is perceived after executing a in belief

state b — sum up over all actual states the agent might reach:

$$\begin{aligned} P(e \mid a,b) &= \sum_{s'} P(e \mid a,s',b) \cdot P(s' \mid a,b) \\ &= \sum_{s'} P(e \mid s') \cdot P(s' \mid a,b) \\ &= \sum_{s'} P(e \mid s') \cdot (\sum_{s} P(s' \mid s,a) \cdot b(s)) \end{aligned}$$

Write the probability of reaching b' from b, given action a, as $P(b' \mid b,a)$, then

$$\begin{aligned} P(b' \mid b,a) &= P(b' \mid a,b) = \sum_{e} P(b' \mid e,a,b) \cdot P(e \mid a,b) \\ &= \sum_{e} P(b' \mid e,a,b) \cdot (\sum_{s'} P(e \mid s') \cdot (\sum_{s} P(s' \mid s,a) \cdot b(s))) \end{aligned}$$

where $P(b' \mid e,a,b)$ is 1 if $b' = \mathrm{FORWARD}(b,a,e)$ and 0 otherwise.

▷ **Observation:** This equation defines a transition model for belief state space!

▷ **Idea:** We can also define a reward function for belief states:

$$\rho(b) := \sum_{s} b(s) \cdot R(s)$$

i.e., the expected reward for the actual states the agent might be in.

▷ Together, $P(b' \mid b,a)$ and $\rho(b)$ define an (observable) MDP on the space of belief states.

▷ **Theorem 25.5.1.** *An optimal policy $\pi^*(b)$ for this MDP, is also an optimal policy for the original POMDP.*

▷ **Upshot:** Solving a POMDP on a physical state space can be reduced to solving an MDP on the corresponding belief state space.

▷ **Remember:** The belief state is always observable to the agent, by definition.

SOME RIGHTS RESERVED

Ideas towards Value-Iteration on POMDPs

▷ **Recap:** The value iteration algorithm from ??? computes one utility value per state.

▷ **Problem:** We have infinitely many belief states $\rightsquigarrow$ be more creative!

▷ **Observation:** Consider an optimal policy π^*

- ▷ applied in a specific belief state b: π^* generates an action,
- ▷ for each subsequent percept, the belief state is updated and a new action is generated ...

For this specific b: $\pi^* \mathrel{\widehat{=}}$ a conditional plan!

▷ **Idea:** Think about conditional plans and how the expected utility of executing a fixed conditional plan varies with the initial belief state. (instead of optimal policies)

▷ **Definition 25.5.2.** Given a set of percepts E and a set of actions A, a conditional plan is either an action $a \in A$, or a tuple $\langle a, E', p_1, p_2 \rangle$ such that $a \in A, E' \subseteq E$, and p_1, p_2 are conditional plans.

It represents the strategy "First execute a, If we subsequently perceive $e \in E'$, continue with p_1, otherwise continue with p_2."

The depth of a conditional plan p is the maximum number of actions in any path from p before reaching a single action plan.

SOME RIGHTS RESERVED

Expected Utilities of Conditional Plans on Belief States

▷ **Observation 1:** Let p be a conditional plan and $\alpha_p(s)$ the utility of executing p in state s.

- ▷ the expected utility of p in belief state b is $\sum_s b(s) \cdot \alpha_p(s) \mathrel{\widehat{=}} b \cdot \alpha_p$ as vectors.
- ▷ the expected utility of a fixed conditional plan varies linearly with b
- ▷ $\rightsquigarrow$ the "*best* conditional plan to execute" corresponds to a hyperplane in belief state space.

▷ **Observation 2:** We can replace the *original* actions by conditional plans on those actions!

Let π^* be the subsequent optimal policy. At any given belief state b,

- ▷ π^* will choose to execute the conditional plan with highest expected utility
- ▷ the expected utility of b under the π^* is the utility of that plan:

$$U(b) = U^{\pi^*}(b) = \max_p (b \cdot \alpha_p)$$

- ▷ If the optimal policy π^* chooses to execute p starting at b, then it is reasonable to expect that it might choose to execute p in belief states that are very close to b;
- ▷ if we bound the depth of the conditional plans, then there are only finitely many such plans
- ▷ the continuous space of belief states will generally be divided into regions, each corresponding to a particular conditional plan that is optimal in that region.

▷ **Observation 3 (conbined):** The utility function $U(b)$ on belief states, being the maximum of a collection of hyperplanes, is piecewise linear and convex.

SOME RIGHTS RESERVED

A simple Illustrating Example

▷ **Example 25.5.3.** A world with states S_0 and S_1, where $R(S_0) = 0$ and $R(S_1) = 1$ and two actions:

- ▷ "Stay" stays put with probability 0.9
- ▷ "Go" switches to the other state with probability 0.9.

▷ The sensor reports the correct state with probability 0.6.

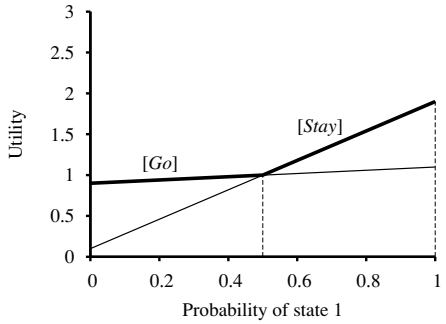
Obviously, the agent should "Stay" when it thinks it's in state S_1 and "Go" when it thinks it's in state S_0.

▷ The belief state has dimension 1. (the two probabilities sum up to 1)

▷ Consider the one-step plans $[Stay]$ and $[Go]$ and their direct utilities:

$$\begin{aligned}
\alpha_{([Stay])}(S_0) &= 0.9R(S_0) + 0.1R(S_1) = 0.1 \\
\alpha_{([stay])}(S_1) &= 0.9R(S_1) + 0.1R(S_0) = 0.9 \\
\alpha_{([go])}(S_0) &= 0.9R(S_1) + 0.1R(S_0) = 0.9 \\
\alpha_{([go])}(S_1) &= 0.9R(S_0) + 0.1R(S_1) = 0.1
\end{aligned}$$

▷ Let us visualize the hyperplanes $b{\cdot}\alpha_{([Stay])}$ and $b{\cdot}\alpha_{([Go])}$.



▷ The maximum represents the utility function for the finite-horizon problem that allows just one action

▷ in each "piece" the optimal action is the first action of the corresponding plan.

▷ Here the optimal one-step policy is to "Stay" when $b(1) > 0.5$ and "Go" otherwise.

▷ Compute the utilities for conditional plans of depth 2 by considering

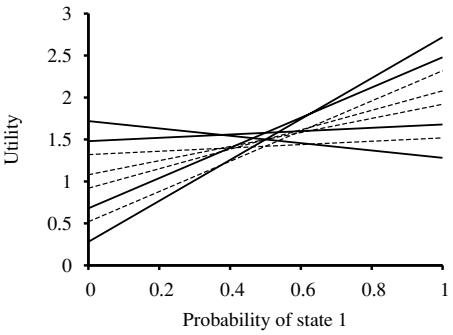
▷ each possible first action,

▷ each possible subsequent percept, and then

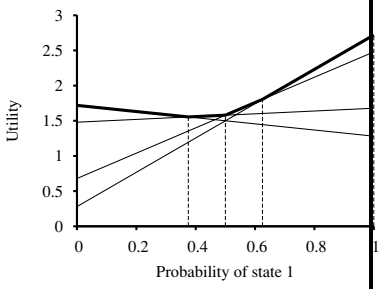
▷ each way of choosing a depth-1 plan to execute for each percept:

There are eight of depth 2:

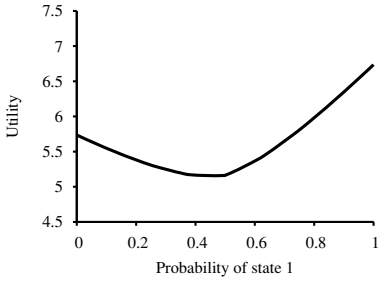
$$[Stay, \text{if } P = 0 \text{ then } Stay \text{ else } Stay \text{ fi}], [Stay, \text{if } P = 0 \text{ then } Stay \text{ else } Go \text{ fi}], \ldots$$



Four of them (dashed lines) are suboptimal for the whole belief space
We call them dominated (they can be ignored)



▷ There are four undominated plans, each optimal in their region



▷ **Idea:** Repeat for depth 3 and so on.

▷ **Theorem 25.5.4 (POMDP Plan Utility).** *Let p be a depth-d conditional plan whose initial action is a and whose depth-$d-1$-subplan for percept e is $p.e$, then*

$$\alpha_p(s) = R(s) + \gamma(\sum_{s'} P(s' \mid s,a)(\sum_e P(e \mid s') \cdot \alpha_{p.e}(s')))$$

▷ This recursion naturally gives us a value iteration algorithm,

SOME RIGHTS RESERVED

A Value Iteration Algorithm for POMDPs

▷ **Definition 25.5.5.** The POMDP value iteration algorithm for POMDPs is given by recursively updating

$$\alpha_p(s) = R(s) + \gamma(\sum_{s'} P(s' \mid s,a)(\sum_e P(e \mid s') \cdot \alpha_{p.e}(s')))$$

▷ **Observations:** The complexity depends primarily on the generated plans:

- ▷ Given $|A|$ actions and $|E|$ possible observations, there are are $|A|^{|E|^{d-1}}$ distinct depth-d plans.
- ▷ Even for the example with $d = 8$, we have 2255 (144 undominated)
- ▷ The elimination of dominated plans is essential for reducing this doubly exponential growth (but they are already constructed)

▷ Hopelessly inefficient in practice – even the 3x4 POMDP is too hard!

A Value Iteration Algorithm for POMDPs

▷ **Definition 25.5.6.** The POMDP value iteration algorithm for POMDPs takes the inputs

- ▷ $pomdp$, a POMDP with states S, actions $A(s)$, transition model $P(s' \mid s,a)$, sensor model $P(e \mid s)$, rewards $R(s)$, discount γ.
- ▷ ϵ the maximum error allowed in the utility of any state.

function POMDP−VALUE−ITERATION($pomdp$, ϵ) **returns** a utility **function**
  inputs: $pomdp$, a POMDP with states S, actions $A(s)$, transition model $P(s' \mid s,a)$,
    sensor model $P(e \mid s)$, rewards $R(s)$, discount γ
    ϵ the maximum error allowed **in** the utility of any state
  local variables: U , U', sets of plans p with associated utility vectors α_p
  U' := a set containing just the empty plan [], with $\alpha_{([])}(s) = R(s)$
  repeat
    $U := U'$
    U' := the set of all plans consisting of an action and, **for** each possible next percept,
      a plan **in** U with utility vectors computed via the POMDP Plan Utility Theorem
    U' := REMOVE−DOMPLANS(U')
  until MAX−DIFF(U,U') $< \epsilon(1-\gamma)/\gamma$
 return U

Where RemDomPlans and MaxDiff are implemented as linear programs.

A Value Iteration Algorithm for POMDPs

▷ **Observations:** The complexity depends primarily on the generated plans:

- ▷ Given $|A|$ actions and $|E|$ possible observations, there are are $\mathcal{O}(|A|^{|E|^{d-1}})$ distinct depth-d plans.
- ▷ Even for the example with $d = 8$, we have 2255 (144 undominated)
- ▷ The elimination of dominated plans is essential fo-r reducing this doubly exponential growth (but they are already constructed)

▷ Hopelessly inefficient in practice – even the 3x4 POMDP is too hard!

25.6 Online Agents with POMDPs

In the last section we have seen that even though we can in principle compute utilities of states – and thus use the MEU principle – to make decisions in sequential decision problems, all methods based on the "lifting idea" are hopelessly inefficient.

This section describes a different, approximate method for solving POMDPs, one based on look-ahead search.

DDN: Decision Networks for POMDPs

▷ **Idea:** Let's try to use the computationally efficient representations (dynamic Bayesian networks and decision networks) for POMDPs.

▷ **Definition 25.6.1.** A dynamic decision network (DDN) is a graph-based representation of a POMDP, where

- ▷ Transition and sensor model are represented as a DBN.
- ▷ Action nodes and utility nodes are added as in decision networks.

▷ In a DDN, a filtering algorithm is used to incorporate each new percept and action and to update the belief state representation.

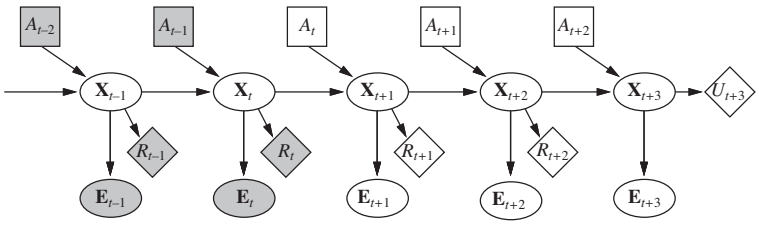
▷ Decisions are made in DDN by projecting forward possible action sequences and choosing the best one.

▷ DDNs – like the DBNs they are based on – are factored representations
⇝ typically exponential complexity advantages!

SOME RIGHTS RESERVED

Structure of DDNs for POMDPs

▷ **DDN for POMDPs:** The generic structure of a dymamic decision network at time t is



- ▷ POMDP state S_t becomes a set of random variables $\mathbf{X}_t$
- ▷ there may be multiple evidence variables $\mathbf{E}_t$
- ▷ Action at time t denoted by A_t. agent must choose a value for A_t.
- ▷ Transition model: $\mathbb{P}(\mathbf{X}_{t+1}|\mathbf{X}_t, A_t)$; sensor model: $\mathbb{P}(\mathbf{E}_t|\mathbf{X}_t)$.
- ▷ Reward functions R_t and utility U_t of state S_t.
- ▷ Variables with known values are gray, rewards for $t = 0, \ldots, t+2$, but utility for $t+3$($\widehat{=}$ discounted sum of rest)

▷ **Problem:** How do we compute with that?

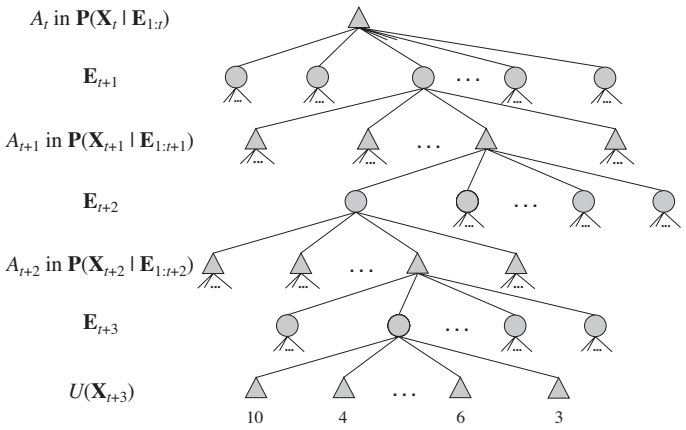
▷ **Answer:** All POMDP algorithms can be adapted to DDNs! (only need conditional probability tables)

SOME RIGHTS RESERVED

Lookahead: Searching over the Possible Action Sequences

▷ **Idea:** Search over the tree of possible action sequences (like in game-play)

▷ Part of the lookahead solution of the DDN above (three steps lookahead)

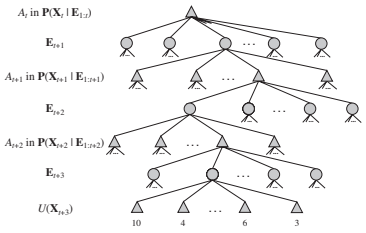


▷ circle $\widehat{=}$ chance nodes (the environment decides)

▷ triangle $\widehat{=}$ belief state (each action decision is taken there)

SOME RIGHTS RESERVED

Designing Online Agents for POMDPs



▷ Belief state at triangle computed by filtering with actions/percepts leading to it

▷ for decision A_{t+i} will use percepts $\mathbf{E}_{t+1:t+i}$ (even if values at time t unknown)

▷ thus a POMDP agent automatically takes into account the value of information and executes information gathering actions where appropriate.

▷ **Observation:** Time complexity for exhaustive search up to depth d is $\mathcal{O}(|A|^d \cdot |E|^d)$($|A| \widehat{=}$ number of actions, $|E| \widehat{=}$ number of percepts)

▷ **Upshot:** Much better than the POMDP value iteration algorithm with $\mathcal{O}(|A|^{|E|^{d-1}})$.

▷ **Empirically:** For problems in which the discount factor γ is not too close to 1, a shallow search is often good enough to give near-optimal decisions.

SOME RIGHTS RESERVED

Summary

- ▷ Decision theoretic agents for sequential environments.
- ▷ Building on temporal, probabilistic models/inference. (dynamic Bayesian networks)
- ▷ MDPs for fully observable case.
- ▷ Value/Policy Iteration for MDPs $\rightsquigarrow$ optimal policies.
- ▷ POMDPs for partially observable case.
- ▷ POMDP $\widehat{=}$ MDP on belief state space.
- ▷ The world is a POMDP with (initially) unknown transition and sensor models.

SOME RIGHTS RESERVED

Part II

Machine Learning

This part introduces the foundations of machine learning methods in AI. We discuss the problem of learning from observations in general, study inference-based techniques, and then go into elementary statistical methods for learning.

The current hype topics of deep learning, reinforcement learning, and large language models are only very superficially covered, leaving them to specialized courses.

Outline

- ▷ Learning agents
- ▷ Inductive learning
- ▷ Decision tree learning
- ▷ Measuring learning performance
- ▷ Computational Learning Theory
- ▷ Linear regression and classification
- ▷ Neural Networks
- ▷ Support Vector Machines

SOME RIGHTS RESERVED

Chapter 26

Learning from Observations

In this chapter we introduce the concepts, methods, and limitations of inductive learning, i.e. learning from a set of given examples.

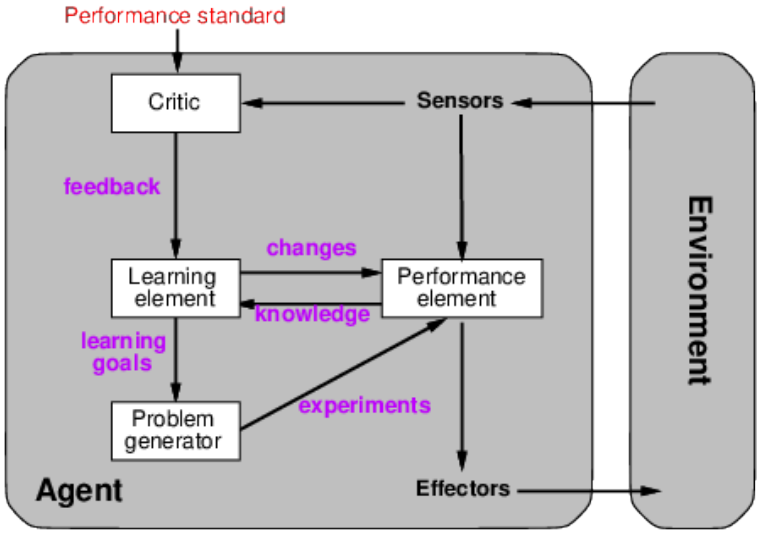
26.1 Forms of Learning

Learning (why is this a good idea)

- ▷ Learning is essential for unknown environments:
  - ▷ i.e., when agent designer lacks omniscience.
  - ▷ The world is a POMDP with (initially) unknown transition and sensor models.
- ▷ Learning is useful as an agent construction method.
  - ▷ i.e., expose the agent to reality rather than trying to write it down
- ▷ Learning modifies the agent's decision mechanisms to improve performance.

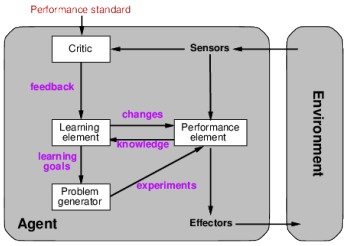
SOME RIGHTS RESERVED

Recap: Learning Agents



SOME RIGHTS RESERVED

Recap: Learning Agents (continued)



▷ **Definition 26.1.1.** Performance element is what we called "agent" up to now.

▷ **Definition 26.1.2.** Critic/learning element/problem generator do the "improving".

▷ **Definition 26.1.3.** Performance measure is fixed; (outside the environment)

- ▷ We can't adjust performance measure to flatter own behaviour!
- ▷ No standard *in the environment*: e.g. ordinary chess and suicide chess look identical.
- ▷ Essentially, certain kinds of percepts are "hardwired" as good/bad (e.g.,pain, hunger)

▷ **Definition 26.1.4.** Learning element may use knowledge already acquired in the performance element.

▷ **Definition 26.1.5.** Learning may require experimentation actions an agent might not normally consider such as dropping rocks from the Tower of Pisa.

SOME RIGHTS RESERVED

Ways of Learning

▷ Supervised learning: There's an unknown function $f\colon A \to B$ called the target function. We do know a set of pairs $T := \{\langle a_i, f(a_i)\rangle\}$ of examples. The goal is to find a hypothesis $h \in \mathcal{H} \subseteq A \to B$ based on T, that is "approximately" equal to f. (Most of the techniques we will consider)

▷ Unsupervised learning: Given a set of data A, find a *pattern* in the data; i.e. a function $f\colon A \to B$ for some predetermined B. (Primarily *clustering/dimensionality reduction*)

▷ Reinforcement learning: The agent receives a reward for each action performed. The goal is to iteratively adapt the action function to maximize the total reward. (Useful in e.g. game play)

SOME RIGHTS RESERVED

26.2 Inductive Learning Problems

Supervised learning a.k.a. inductive learning (a.k.a. Science)

▷ **Definition 26.2.1.** A supervised (or inductive) learning problem consists of the following data:

- ▷ A set of hypotheses $\mathcal{H}$ consisting of functions $A \to B$,
- ▷ a set of examples $T \subseteq A \times B$ called the training set, such that for every $a \in A$, there is at most one $b \in B$ with $\langle a, b\rangle \in T$, ($\rightsquigarrow$ is a function on some subset of A)

We assume there is an *unknown* function $f\colon A \to B$ called the target function with $T \subseteq f$.

▷ **Definition 26.2.2.** Inductive learning algorithms solve inductive learning problems by finding a hypothesis $h \in \mathcal{H}$ such that $h \sim f$ (for some notion of similarity).

▷ **Definition 26.2.3.** We call a supervised learning problem with target function $A \to B$ a classification problem if B is finite, and call the members of B classes.

We call it a regression problem if $B = \mathbb{R}$.

SOME RIGHTS RESERVED

Note: This is a highly simplified model of what a learning agent does: it

- ignores prior knowledge.
- assumes deterministic, observable environments.
- assumes examples are *given*.
- assumes that the agent *wants* to learn f. (why?)

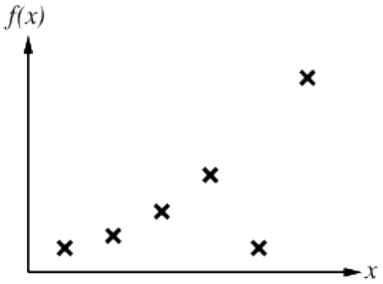
Inductive Learning Method

▷ **Idea:** Construct/adjust hypothesis $h \in \mathcal{H}$ to agree with a training set T.

▷ **Definition 26.2.4.** We call h consistent with f (on a set $T \subseteq \mathbf{dom}(f)$), if it agrees with f (on all examples in T).

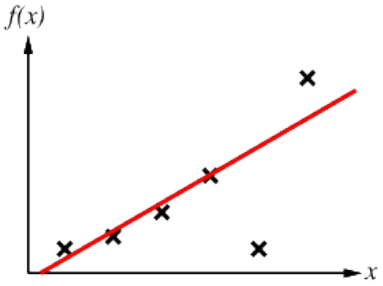
▷ **Example 26.2.5 (Curve Fitting).**

Training Set



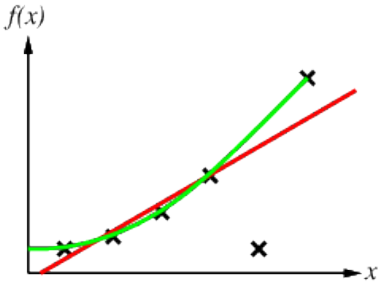
Linear Hypothesis

partially, approximately consistent



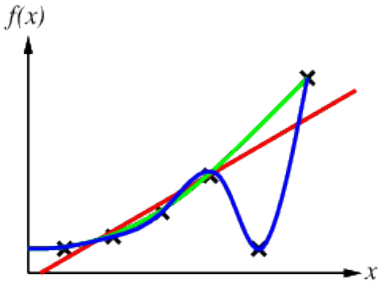
Quadratic Hypothesis

partially consistent



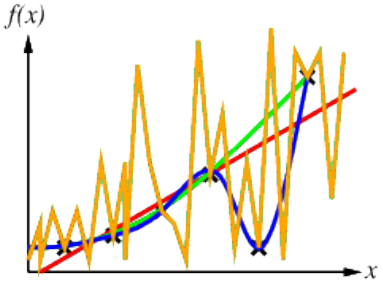
Degree-4 Hypothesis

consistent



High-degree Hypothesis

consistent



▷ **Ockham's-razor:** Maximize a combination of consistency and simplicity.

SOME RIGHTS RESERVED

Choosing the Hypothesis Space

▷ **Observation:** Whether we can find a consistent hypothesis for a given training set depends on the chosen hypothesis space.

▷ **Definition 26.2.6.** We say that an supervised learning problem is realizable, iff there is a hypothesis $h \in \mathcal{H}$ consistent with the training set T.

▷ **Problem:** We do not always know whether a given learning problem is realizable, unless we have prior knowledge. (depending on the hypothesis space)

▷ **Solution:** Make $\mathcal{H}$ large, e.g. the class of all Turing machines.

▷ **Tradeoff:** The computational complexity of the supervised learning problem is tied to the size of the hypothesis space. E.g. consistency is not even decidable for general Turing machines.

▷ Much of the research in machine learning has concentrated on simple hypothesis spaces.

▷ **Preview:** We will concentrate on propositional logic and related languages first.

SOME RIGHTS RESERVED

Independent and Identically Distributed

▷ **Problem:** We want to learn a hypothesis that fits the future data best.

▷ **Intuition:** This only works, if the training set is "representative" for the underlying process.

▷ **Idea:** We think of examples (seen and unseen) as a sequence, and express the "representativeness" as a *stationarity assumption* for the probability distribution.

▷ **Method:** Each example before we see it is a random variable E_j, the observed value $e_j = (x_j, y_j)$ samples its distribution.

▷ **Definition 26.2.7.** A sequence of $E_1, \ldots, E_n$ of random variables is independent and identically distributed (short IID), iff they are

- ▷ independent, i.e. $\mathbb{P}(E_j | E_{j-1}, E_{j-2}, \ldots) = \mathbb{P}(E_j)$ and
- ▷ identically distributed, i.e. $\mathbb{P}(E_i) = \mathbb{P}(E_j)$ for all i and j.

▷ **Example 26.2.8.** A sequence of die tosses is IID. (fair or loaded does not matter)

▷ **Stationarity Assumption:** We assume that the set $\mathcal{E}$ of examples is IID in the future.

SOME RIGHTS RESERVED

26.3 Learning Decision Trees

Attribute-based Representations

▷ **Definition 26.3.1.** In attribute-based representations, examples are described by

- ▷ attributes: (simple) functions on input samples, (think pre-classifiers on examples)
- ▷ their values, and (classify by attributes)
- ▷ classifications. (Boolean, discrete, continuous, etc.)

▷ **Example 26.3.2 (In a Restaurant).** Situations where I will/won't wait for a table:

Example	Attributes										Target
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	WillWait
X_1	T	F	F	T	Some	$$$	F	T	French	0–10	T
X_2	T	F	F	T	Full	$	F	F	Thai	30–60	F
X_3	F	T	F	F	Some	$	F	F	Burger	0–10	T
X_4	T	F	T	T	Full	$	F	F	Thai	10–30	T
X_5	T	F	T	F	Full	$$$	F	T	French	>60	F
X_6	F	T	F	T	Some	$$	T	T	Italian	0–10	T
X_7	F	T	F	F	None	$	T	F	Burger	0–10	F
X_8	F	F	F	T	Some	$$	T	T	Thai	0–10	T
X_9	F	T	T	F	Full	$	T	F	Burger	>60	F
X_{10}	T	T	T	T	Full	$$$	F	T	Italian	10–30	F
X_{11}	F	F	F	F	None	$	F	F	Thai	0–10	F
X_{12}	T	T	T	T	Full	$	F	F	Burger	30–60	T

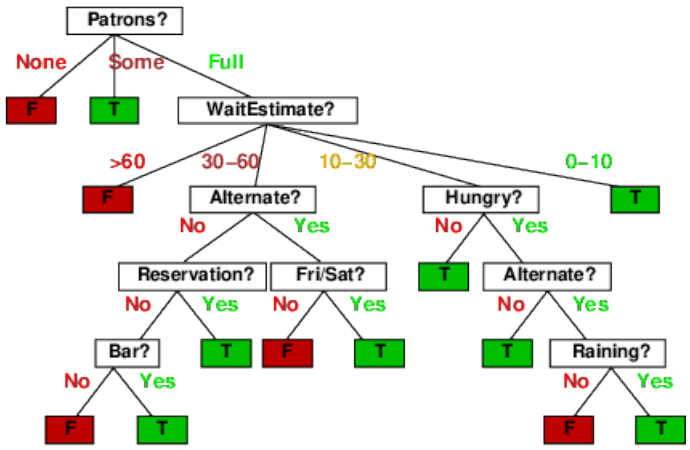
▷ **Definition 26.3.3.** For a Boolean classification we say that an example is positive (T) or negative (F) depending on its class.

SOME RIGHTS RESERVED

Decision Trees

▷ Decision trees are one possible representation for hypotheses.

▷ **Example 26.3.4 (Restaurant continued).** Here is the "real" tree for deciding whether to wait:



We evaluate the tree by going down the tree from the top, and always take the branch whose attribute matches the situation; we will eventually end up with a Boolean value; the result. Using the attribute values from X_3 in Example 26.3.2 to descend through the tree in Example 26.3.4 we indeed end up with the result "true". Note that

1. some of the original set of attributes X_3 are irrelevant.
2. the training set in Example 26.3.2 is realizable – i.e. the target is definable in hypothesis class of decision trees.

Decision Trees (Definition)

▷ **Definition 26.3.5.** A decision tree for a given attribute-based representation is a tree, where the non-leaf nodes are labeled by attributes, their outgoing edges by disjoint sets of attribute values, and the leaf nodes are labeled by the classifications.

▷ **Definition 26.3.6.** We call an attribute together with a set of attribute values (an inner node) with outgoing edge label an attribute test.

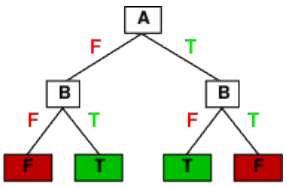
▷ The target function is a function $A_1 \times \ldots \times A_n \to C$, where A_i are the domains of the attributes and C is the set of classifications.

Expressivity

▷ Decision trees can express any function of the input attributes $\rightsquigarrow \mathcal{H} = A_1 \times \ldots \times A_n$

▷ **Example 26.3.7.** For Boolean functions, a path from the root to a leaf corresponds to a row in a truth table:

A	B	A xor B
F	F	F
F	T	T
T	F	T
T	T	F



$\rightsquigarrow$ a decision tree corresponds to a truth table (Formula in DNF)

▷ Trivially, for any training set there is a consistent hypothesis with one path to a leaf for each example, but it probably won't generalize to new examples.

▷ **Solution:** Prefer to find more *compact* decision trees.

Decision Tree Learning

▷ **Aim:** Find a small decision tree consistent with the training examples.

▷ **Idea:** (recursively) choose "most significant" attribute as root of (sub)tree.

▷ **Definition 26.3.8.** The following algorithm performs decision tree learning (DTL)

```
function DTL(examples, attributes, default) returns a decision tree
  if examples is empty then return default
   else if all examples have the same classification then return the classification
   else if attributes is empty then return MODE(examples)
   else
       best := Choose−Attribute(attributes, examples)
       tree := a new decision tree with root test best
       m := MODE(examples)
       for each value v_i of best do
           examples_i := {elements of examples with best = v_i}
           subtree := DTL(examples_i, attributes \ best, m)
           add a branch to tree with label v_i and subtree subtree
     return tree
```

Mode($examples$)= most frequent outcome in $example$.

SOME RIGHTS RESERVED

Note: We have three base cases:

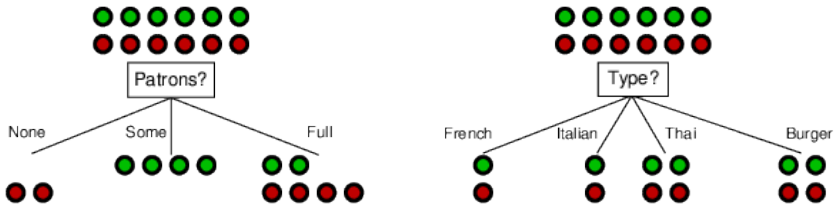
1. empty examples ⇜ arises for empty branches of non-Boolean parent attribute.
2. uniform example classifications ⇜ this is "normal" leaf.
3. *attributes* empty ⇜ target is not deterministic in input *attributes*.

The recursive step picks an attribute and then subdivides the examples.

Choosing an Attribute

▷ **Idea:** A good attribute splits the examples into subsets that are (ideally) "all positive" or "all negative".

▷ **Example 26.3.9.**



Attribute "Patrons?" is a better choice, it gives gives information about the classification.

▷ Can we make this more formal? ⇝ Use information theory! (up next)

SOME RIGHTS RESERVED

26.4 Using Information Theory

Information Entropy

▷ **Intuition:** Information answers questions.

▷ **Idea:** The more clueless I am about the answer initially, the more information is contained in the answer.

▷ **Scale:** $1\mathrm{b} \mathrel{\widehat{=}} 1$ bit $\mathrel{\widehat{=}}$ answer to Boolean question with prior probability $\langle 0.5, 0.5\rangle$.

▷ **Definition 26.4.1.** If the prior probability is $\langle P_1, \ldots, P_n\rangle$, then the information in an answer (also called entropy of the prior) is

$$I(\langle P_1, \ldots, P_n\rangle) := \sum_{i=1}^{n} -P_i \cdot \log_2(P_i)$$

Note: The case $P_i = 0$ requires special treatment. ($\log_2(0)$ is undefined)

▷ **Example 26.4.2 (Information of a Coin Toss).**

- ▹ For a fair coin toss we have $I(\langle \frac{1}{2}, \frac{1}{2}\rangle) = -\frac{1}{2}\log_2(\frac{1}{2}) - \frac{1}{2}\log_2(\frac{1}{2}) = 1\mathrm{b}$.
- ▹ With a loaded coin (99% heads) we have $I(\langle \frac{1}{100}, \frac{99}{100}\rangle) = 0.08\mathrm{b}$.
- ▹ Intuition: Information goes to 0 as head probability goes to 1.

SOME RIGHTS RESERVED

Information Gain in Decision Trees

▷ **Idea:** Suppose we have p examples classified as positive and n examples as negative. We can then estimate the probability distribution of the classification C with $\mathbb{P}(C) = \langle \frac{p}{p+n}, \frac{n}{p+n}\rangle$, and need $I(\mathbb{P}(C))$ bits to correctly classify a new example.

▷ **Example 26.4.3.** For 12 restaurant examples and $p = n = 6$, we need $I(\mathbb{P}(\mathrm{WillWait})) = I(\langle \frac{6}{12}, \frac{6}{12}\rangle) = 1\mathrm{b}$ of information. (i.e. exactly the information which of the two classes)

▷ Treating attributes also as random variables, we can compute how much information is needed *after* knowing the value for one attribute:

▷ **Example 26.4.4.** If we know $\mathrm{Pat} = \mathrm{Full}$, we only need $I(\mathbb{P}(\mathrm{WillWait}|\mathrm{Pat} = \mathrm{Full})) = I(\langle \frac{4}{6}, \frac{2}{6}\rangle) \approxeq 0.9$ bits of information.

▷ **Note:** The expected number of bits needed after an attribute test on A is

$$\sum_a P(A = a) \cdot I(\mathbb{P}(C|A = a))$$

▷ **Definition 26.4.5.** The information gain from an attribute test A is

$$\mathrm{Gain}(A) := I(\mathbb{P}(C)) - \sum_a P(A = a) \cdot I(\mathbb{P}(C|A = a))$$

SOME RIGHTS RESERVED

Information Gain (continued)

▷ **Definition 26.4.6.** Assume we know the results of some attribute tests $b := B_1 = b_1 \wedge \ldots \wedge B_n = b_n$. Then the conditional information gain from an attribute test A is

$$\mathrm{Gain}(A|b) := I(\mathbb{P}(C|b)) - \sum_a P(A = a \mid b) \cdot I(\mathbb{P}(C|a, b))$$

▷ **Example 26.4.7.** If the classification C is Boolean and we have p positive and n negative examples, the information gain is

$$\mathrm{Gain}(A) = I(\langle \frac{p}{p+n}, \frac{n}{p+n} \rangle) - \sum_a \frac{p_a + n_a}{p+n} I(\langle \frac{p_a}{p_a+n_a}, \frac{n_a}{p_a+n_a} \rangle)$$

where p_a and n_a are the numbers of positive and negative examples with $A = a$.

▷ **Example 26.4.8.**

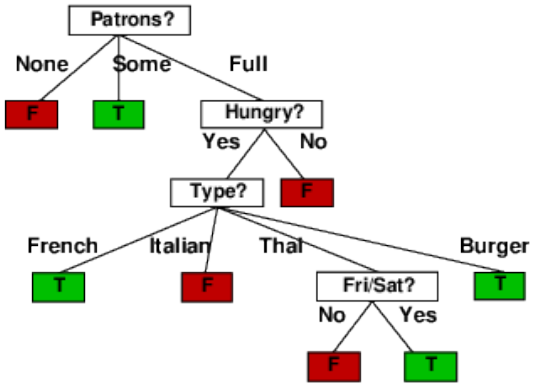
$$\begin{aligned}
\mathrm{Gain}(Patrons?) &= 1 - (\frac{2}{12} I(\langle 0, 1 \rangle) + \frac{4}{12} I(\langle 1, 0 \rangle) + \frac{6}{12} I(\langle \frac{2}{6}, \frac{4}{6} \rangle)) \\
&\approx 0.541\mathrm{b} \\
\mathrm{Gain}(Type) &= 1 - (\frac{2}{12} I(\langle \frac{1}{2}, \frac{1}{2} \rangle) + \frac{2}{12} I(\langle \frac{1}{2}, \frac{1}{2} \rangle) + \frac{4}{12} I(\langle \frac{2}{4}, \frac{2}{4} \rangle) + \frac{4}{12} I(\langle \frac{2}{4}, \frac{2}{4} \rangle)) \\
&\approx 0\mathrm{b}
\end{aligned}$$

▷ **Idea:** Choose the attribute that maximizes information gain.

SOME RIGHTS RESERVED

Restaurant Example contd.

▷ **Example 26.4.9.** Decision tree learned by DTL from the 12 examples using information gain maximization for Choose–Attribute:



▷ **Result:** Substantially simpler than "true" tree – a more complex hypothesis isn't justified by small amount of data.

26.5 Evaluating and Choosing the Best Hypothesis

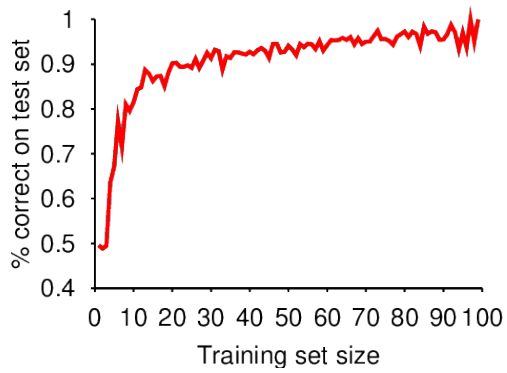
Performance measurement

▷ **Question:** How do we know that $h \approx f$? (Hume's *Problem of Induction*)

1. Use theorems of computational/statistical learning theory.
2. Try h on a new test set of examples. (use *same distribution over example space* as training set)

▷ **Definition 26.5.1.** The learning curve $\widehat{=}$ percentage correct on test set as a function of training set size.

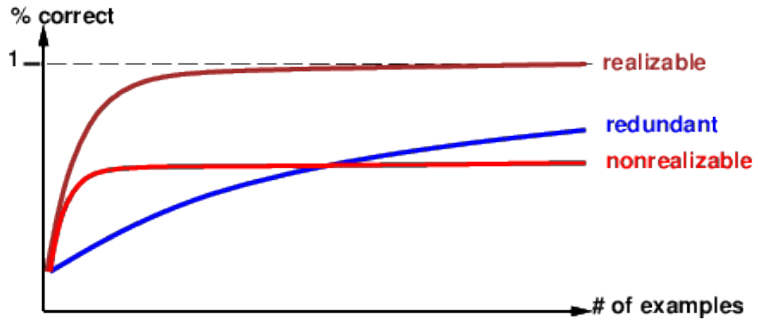
▷ **Example 26.5.2.** Restaurant data; graph averaged over 20 trials



Performance measurement contd.

▷ **Observation 26.5.3.** *The learning curve depends on*

- ▷ *realizable (can express target function) vs. non-realizable*
  non-realizability can be due to missing attributes or restricted hypothesis class (e.g., thresholded linear function)
- ▷ *redundant expressivity (e.g., lots of irrelevant attributes)*



Generalization and Overfitting

▷ **Observation:** Sometimes a learned hypothesis is more specific than the experiments warrant.

▷ **Definition 26.5.4.** We speak of overfitting, if a hypothesis h describes random error in the (limited) training set rather than the underlying relationship. Underfitting occurs when h cannot capture the underlying trend of the data.

▷ **Qualitatively:** Overfitting increases with the size of hypothesis space and the number of attributes, but decreases with number of examples.

▷ **Idea:** Combat overfitting by "generalizing" decision trees computed by DTL.

Decision Tree Pruning

▷ **Idea:** Combat overfitting by "generalizing" decision trees $\rightsquigarrow$ prune "irrelevant" nodes.

▷ **Definition 26.5.5.** For decision tree pruning repeat the following on a learned decision tree:

- ▷ Find a terminal test node n (only result leaves as children)
- ▷ If test is irrelevant, i.e. has low information gain, prune it by replacing n by with a leaf node.

▷ **Question:** How big should the information gain be to split ($\rightsquigarrow$ keep) a node?

▷ **Idea:** Use a statistical significance test.

▷ **Definition 26.5.6.** A result has statistical significance, if the probability they could arise from the null hypothesis (i.e. the assumption that there is no underlying pattern) is very low (usually 5%).

Determining Attribute Irrelevance

▷ For decision tree pruning, the null hypothesis is that the attribute is irrelevant.

▷ Compute the probability that the example distribution (p positive, n negative) for a terminal node deviates from the expected distribution under the null hypothesis.

▷ For an attribute A with d values, compare the actual numbers p_k and n_k in each subset s_k with the expected numbers (expected if A is irrelevant)
$\widehat{p}_k := p \cdot \frac{p_k+n_k}{p+n}$ and $\widehat{n}_k := n \cdot \frac{p_k+n_k}{p+n}$.

▷ A convenient measure of the total deviation is (sum of squared errors)

$$\Delta = \sum_{k=1}^{d} \frac{(p_k - \widehat{p}_k)^2}{\widehat{p}_k} + \frac{(n_k - \widehat{n}_k)^2}{\widehat{n}_k}$$

▷ **Lemma 26.5.7 (Neyman-Pearson).** *Under the null hypothesis, the value of Δ is distributed according to the χ^2 distribution with $d-1$ degrees of freedom. [JN33]*

▷ **Definition 26.5.8.** Decision tree pruning with Pearson's χ^2 with $d-1$ degrees of freedom for Δ is called χ^2 pruning. (χ^2 values from stats library.)

▷ **Example 26.5.9.** The *type* attribute has four values, so three degrees of freedom, so $\Delta = 7.82$ would reject the null hypothesis at the 5% level.

SOME RIGHTS RESERVED

Error Rates and Cross-Validation

▷ **Recall:** We want to learn a hypothesis that fits the future data best.

▷ **Definition 26.5.10.** Given an inductive learning problem with a set of examples $T \subseteq A \times B$, we define the error rate of a hypothesis $h \in \mathcal{H}$ as the fraction of errors:

$$\frac{|\{\langle x, y\rangle \in T \mid h(x) \neq y\}|}{|T|}$$

▷ **Caveat:** A low error rate on the training set does not mean that a hypothesis generalizes well.

▷ **Idea:** Do not use homework problems in the exam.

▷ **Definition 26.5.11.** The practice of splitting the data available for learning into

1. a training set from which the learning algorithm computes a hypothesis h and
2. a test set, which is used for evaluating h

is called cross-validation. (no peeking at test set allowed)

SOME RIGHTS RESERVED

Error Rates and Cross-Validation

▷ **Question:** What is a good ratio between training set and test set size?

- ▹ small training set $\rightsquigarrow$ poor hypothesis.
- ▹ small test set $\rightsquigarrow$ poor estimate of the accuracy.

▷ **Definition 26.5.12.** In k-fold cross-validation, we perform k rounds of learning, each with $1/k$ of the data as test set and average over the k error rates.

▷ **Intuition:** Each example does double duty: for training and testing.

▷ $k = 5$ and $k = 10$ are popular $\rightsquigarrow$ good accuracy at k times computation time.

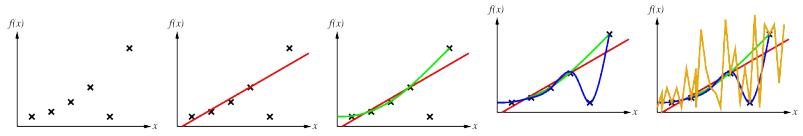
▷ **Definition 26.5.13.** If $k = |\mathbf{dom}(f)|$, then k-fold cross-validation is called leave one out cross validation (LOOCV).

SOME RIGHTS RESERVED

Model Selection

▷ **Definition 26.5.14.** The model selection problem is to determine – given data – a good hypothesis space.

▷ **Example 26.5.15.** What is the best polynomial degree to fit the data



▷ **Observation 26.5.16.** *We can solve the problem of "learning from observations f" in a two-part process:*

1. *model selection determines a hypothesis space $\mathcal{H}$,*
2. *optimization solves the induced inductive learning problem.*

▷ **Idea:** Solve the two parts together by iteration over "size". (they inform each other)

▷ **Problem:** Need a notion of "size" $\rightsquigarrow$ e.g. number of nodes in a decision tree.

▷ **Concrete Problem:** Find the "size" that best balances overfitting and underfitting to optimize test set accuracy.

SOME RIGHTS RESERVED

Model Selection Algorithm (Wrapper)

▷ **Definition 26.5.17.** The model selection algorithm (MSA) jointly optimizes model selection and optimization by partitioning and cross-validation:

function CROSS–VALIDATION–WRAPPER($Learner$,k,$examples$) **returns** a hypothesis

```
local variables: errT, an array, indexed by size, storing training–set error rates
                 errV, an array, indexed by size, storing validation–set error rates
for size = 1 to ∞ do
  errT[size], errV[size] := CROSS–VALIDATION(Learner,size,k,examples)
  if errT has converged then do
      best_size := the value of size with minimum errV[size]
      return Learner(best_size,examples)

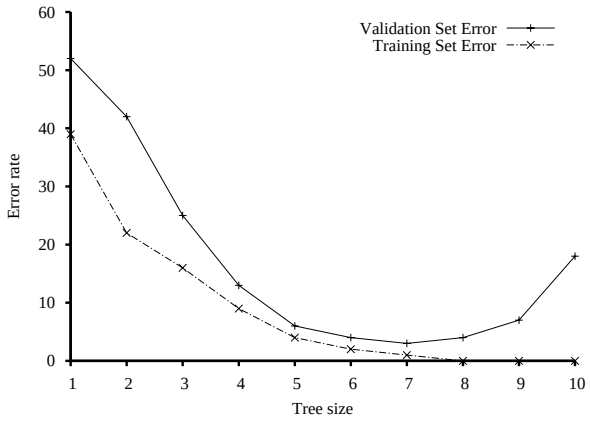
function CROSS–VALIDATION(Learner,size,k,examples) returns two values:
        average training set error rate, average validation set error rate
  fold_errT := 0; fold_errV := 0
  for fold = 1 to k do
      training_set, validation_set := PARTITION(examples,fold,k)
      h := Learner(size,training_set)
      fold_errT := fold_errT + ERROR–RATE(h,training_set)
      fold_errV := fold_errV + ERROR–RATE(h,validation_set)
  return fold_errT/k, fold_errV/k

function PARTITION(examples,fold,k) returns two sets:
  a validation set of size |examples|/k and the rest; the split is different for each fold value
```


SOME RIGHTS RESERVED

Error Rates on Training/Validation Data

▷ **Example 26.5.18 (An Error Curve for Restaurant Decision Trees).**
Modify DTL to be breadth-first, information gain sorted, stop after k nodes.



Stops when training set error rate converges, choose optimal tree for validation curve.(here a tree with 7 nodes)

SOME RIGHTS RESERVED

From Error Rates to Loss Functions

▷ **So far** we have been minimizing error rates. (better than maximizing ☺)

▷ **Example 26.5.19 (Classifying Spam).** It is much worse to classify ham (legitimate mails) as spam than vice versa. (message loss)

▷ **Recall Rationality:** Decision-makers should maximize expected utility (MEU).

▷ **So:** Machine learning should maximize "utility". (not only minimize error rates)

▷ machine learning traditionally deals with utilities in form of "loss functions".

▷ **Definition 26.5.20.** The loss function L is defined by setting $L(x, y, \widehat{y})$ to be the amount of utility lost by prediction $h(x) = \widehat{y}$ instead of $f(x) = y$. If L is independent of x, we often use $L(y, \widehat{y})$.

▷ **Example 26.5.21.** $L(spam, ham) = 1$, while $L(ham, spam) = 10$.

Generalization Loss

▷ **Note:** $L(y, y) = 0$. (no loss if you are exactly correct)

▷ **Definition 26.5.22 (Popular general loss functions).**

absolute value loss	$L_1(y, \widehat{y}) := \lvert y - \widehat{y} \rvert$	small errors are good
squared error loss	$L_2(y, \widehat{y}) := (y - \widehat{y})^2$	ditto, but differentiable
0/1 loss	$L_{0/1}(y, \widehat{y}) :=$ 0, if $y = \widehat{y}$, else 1	error rate

▷ **Idea:** Maximize expected utility by choosing hypothesis h that minimizes expected loss over all $(x,y) \in f$.

▷ **Definition 26.5.23.** Let $\mathcal{E}$ be the set of all possible examples and $\mathbb{P}(X, Y)$ the prior probability distribution over its components, then the expected generalization loss for a hypothesis h with respect to a loss function L is

$$\mathrm{GenLoss}_L(h) := \sum_{(x,y)\in\mathcal{E}} L(y, h(x)) \cdot P(x, y)$$

and the best hypothesis $h^* := \operatorname{argmin}_{h\in\mathcal{H}} \mathrm{GenLoss}_L(h)$.

Empirical Loss

▷ **Problem:** $\mathbf{P}(X, Y)$ is unknown $\rightsquigarrow$ learner can only estimate generalization loss:

▷ **Definition 26.5.24.** Let L be a loss function and E a set of examples with $|E| = N$, then we call

$$\mathrm{EmpLoss}_{L,E}(h) := \frac{1}{N} \Big(\sum_{(x,y)\in E} L(y, h(x)) \Big)$$

the empirical loss and $\widehat{h}^* := \operatorname{argmin}_{h\in\mathcal{H}} \mathrm{EmpLoss}_{L,E}(h)$ the estimated best hypothesis.

▷ There are four reasons why $\widehat{h}^*$ may differ from f:

1. Realizablility: then we have to settle for an approximation $\widehat{h}^*$ of f.

2. Variance: different subsets of f give different $\widehat{h}^* \rightsquigarrow$ more examples.
3. Noise: if f is non-deterministic, then we cannot expect perfect results.
4. Computational complexity: if $\mathcal{H}$ is too large to systematically explore, we make due with subset and get an approximation.

SOME RIGHTS RESERVED

Regularization

▷ **Idea:** Directly use empirical loss to solve model selection. (finding a good $\mathcal{H}$)
Minimize the weighted sum of empirical loss and hypothesis complexity. (to avoid overfitting).

▷ **Definition 26.5.25.** Let $\lambda \in \mathbb{R}$, $h \in \mathcal{H}$, and E a set of examples, then we call

$$\mathrm{Cost}_{L,E}(h) := \mathrm{EmpLoss}_{L,E}(h) + \lambda \cdot \mathrm{Complexity}(h)$$

the total cost of h on E.

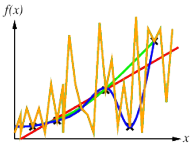
▷ **Definition 26.5.26.** The process of finding a total cost minimizing hypothesis

$$\widehat{h}^* := \operatorname*{argmin}_{h \in \mathcal{H}} \mathrm{Cost}_{L,E}(h)$$

is called regularization; Complexity is called the regularization function or hypothesis complexity.

▷ **Example 26.5.27 (Regularization for Polynomials).**

A good regularization function for polynomials is the sum of squares of exponents. $\rightsquigarrow$ keep away from wriggly curves!



SOME RIGHTS RESERVED

Minimal Description Length

▷ **Remark:** In regularization, empirical loss and hypothesis complexity are not measured in the same scale $\rightsquigarrow \lambda$ mediates between scales.

▷ **Idea:** Measure both in the same scale $\rightsquigarrow$ use information content, i.e. in bits.

▷ **Definition 26.5.28.** Let $h \in \mathcal{H}$ be a hypothesis and E a set of examples, then the description length of (h,E) is computed as follows:

1. encode the hypothesis as a Turing machine program, count bits.
2. count data bits:
   ▷ correctly predicted example $\rightsquigarrow 0\mathrm{b}$

- ▷ incorrectly predicted example $\rightsquigarrow$ according to size of error.

The minimum description length or MDL hypothesis minimizes the total countnumber of bits required.

- ▷ This works well in the limit, but for smaller problems there is a difficulty in that the choice of encoding for the program affects the outcome.
  - ▷ e.g., how best to encode a decision tree as a bit string?

SOME RIGHTS RESERVED

The Scale of Machine Learning

- ▷ Traditional methods in statistics and early machine learning concentrated on small-scale learning (50-5000 examples)
  - ▷ Generalization error mostly comes from
    - ▷ approximation error of not having the true f in the hypothesis space
    - ▷ estimation error of too few training examples to limit variance.
- ▷ In recent years there has been more emphasis on large-scale learning. (millions of examples)
  - ▷ generalization error is dominated by limits of computation
    - ▷ there is enough data and a rich enough model that we could find an h that is very close to the true f,
    - ▷ but the computation to find it is too complex, so we settle for a sub-optimal approximation.
  - ▷ Hardware advances (GPU farms, Amazon EC2, Google Data Centers, ...) help.

SOME RIGHTS RESERVED

26.6 Computational Learning Theory

A (General) Theory of Learning?

- ▷ **Main Question:** How can we be sure that our learning algorithm has produced a hypothesis that will predict the correct value for previously unseen inputs?
- ▷ **Formally:** How do we know that the hypothesis h is close to the target function f if we don't know what f is?
- ▷ **Other – more recent – Questions:**
  - ▷ How many examples do we need to get a good h?
  - ▷ What hypothesis space $\mathcal{H}$ should we use?
  - ▷ If the $\mathcal{H}$ is very complex, can we even find the best h, or do we have to settle for a local maximum in $\mathcal{H}$.

- ▷ How complex should h be?
- ▷ How do we avoid overfitting?

▷ "Computational Learning Theory" tries to answer these using concepts from AI, statistics, and theoretical CS.

SOME RIGHTS RESERVED

PAC Learning

▷ **Basic idea of Computational Learning Theory:**

- ▷ Any hypothesis h that is seriously wrong will almost certainly be "found out" with high probability after a small number of examples, because it will make an incorrect prediction.
- ▷ Thus, if h is consistent with a sufficiently large set of training examples is unlikely to be seriously wrong.
- ▷ $\rightsquigarrow$ h is probably approximately correct.

▷ **Definition 26.6.1.** Any learning algorithm that returns hypotheses that are probably approximately correct is called a PAC learning algorithm.

▷ Derive performance bounds for PAC learning algorithms in general, using the

▷ **Stationarity Assumption (again):** We assume that the set $\mathcal{E}$ of possible examples is IID $\rightsquigarrow$ we have a fixed distribution $\mathbf{P}(E) = \mathbf{P}(X, Y)$ on examples.

▷ **Simplifying Assumptions:** f is a function (deterministic) and $f \in \mathcal{H}$.

SOME RIGHTS RESERVED

PAC Learning

▷ Start with PAC theorems for Boolean functions, for which $L_{0/1}$ is appropriate.

▷ **Definition 26.6.2.** The error rate $\mathrm{error}(h)$ of a hypothesis h is the probability that h misclassifies a new example.

$$\mathrm{error}(h) := \mathrm{GenLoss}_{L_{0/1}}(h) = \sum_{(x,y)\in\mathcal{E}} L_{0/1}(y, h(x)) \cdot P(x, y)$$

▷ **Intuition:** $\mathrm{error}(h)$ is the probability that h misclassifies a new example.

▷ This is the same quantity as measured in the learning curves above.

▷ **Idea:** We can tolerate some error/loss $\rightsquigarrow$ we fix an acceptable $\epsilon > 0$ as an error threshold for "seriously bad" hypotheses. (the actual value of ϵ is irrelevant for the arguments below)

▷ **Definition 26.6.3.** A hypothesis h is called approximately correct, iff $\mathrm{error}(h) \leq \epsilon$.

We write $\mathcal{H}_b := \{h \in \mathcal{H} \,|\, \mathrm{error}(h) > \epsilon\}$ for the "seriously bad" hypotheses.

Sample Complexity

▷ Let's compute the probability that $h_b \in \mathcal{H}_b$ is consistent with the first N examples.

▷ We know $\text{error}(h_b) > \epsilon$
⇝ $P(h_b$ agrees with N examples$) \leq (1-\epsilon)^N$. (independence)
⇝ $P(\mathcal{H}_b$ contains consistent hyp.$) \leq |\mathcal{H}_b| \cdot (1-\epsilon)^N \leq |\mathcal{H}| \cdot (1-\epsilon)^N$. ($\mathcal{H}_b \subseteq \mathcal{H}$)
⇝ to bound this by a small δ, show the algorithm $N \geq \frac{1}{\epsilon} \cdot (\log_2(\frac{1}{\delta}) + \log_2(|\mathcal{H}|))$ examples.

▷ **Definition 26.6.4.** The number of required examples as a function of ϵ and δ is called the sample complexity of $\mathcal{H}$.

▷ **Example 26.6.5.** If $\mathcal{H}$ is the set of n-ary Boolean functions, then $|\mathcal{H}| = 2^{2^n}$.
⇝ sample complexity grows with $\mathcal{O}(\log_2(2^{2^n})) = \mathcal{O}(2^n)$.
There are 2^n possible examples,
⇝ PAC learning for Boolean functions needs to see (nearly) all examples.

Escaping Sample Complexity

▷ **Problem:** PAC learning for Boolean functions needs to see (nearly) all examples.
  ▷ $\mathcal{H}$ contains enough hypotheses to classify any given set of examples in all possible ways.
  ▷ In particular, for any set of N examples, the set of hypotheses consistent with those examples contains equal numbers of hypotheses that predict x_{N+1} to be positive and hypotheses that predict x_{N+1} to be negative.

▷ **Idea/Problem:** Restrict $\mathcal{H}$ in some way (but we may lose realizability)

▷ **Three Ways out of this Dilemma:**

1. Bring prior knowledge into the problem. (section 29.2)
2. Prefer simple hypotheses. (e.g. decision tree pruning)
3. Focus on "learnable subsets" of $\mathcal{H}$. (next)

PAC Learning: Decision Lists

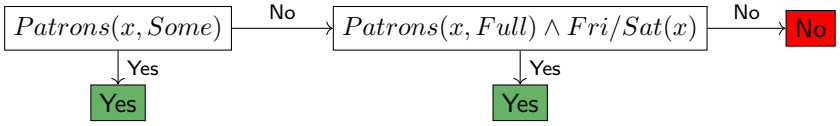
▷ **Idea:** Apply PAC learning to a "learnable hypothesis space".

▷ **Definition 26.6.6.** A decision list consists of a sequence of tests, each of which is a conjunction of literals. A test succeeds on an example E
  ▷ If a test succeeds when applied to an example description, the decision list specifies the value to be returned.

▷ If the test fails, processing continues with the next test in the list.

▷ **Remark:** Like decision trees, but restricted branching, but more complex tests.

▷ **Example 26.6.7 (A decision list for the Restaurant Problem).**



▷ **Lemma 26.6.8.** *Given arbitrary size conditions, decision lists can represent arbitrary Boolean functions.*

▷ This directly defeats our purpose of finding a "learnable subset" of $\mathcal{H}$.

SOME RIGHTS RESERVED

Decision Lists: Learnable Subsets (Size-Restricted Cases)

▷ **Definition 26.6.9.** The set of decision lists where tests are of conjunctions of at most k literals is denoted by $k\text{-}\mathbf{DL}$.

▷ **Example 26.6.10.** The decision list from Example 26.6.7 is in $2\text{-}\mathbf{DL}$.

▷ **Observation 26.6.11.** $k\text{-}\mathbf{DL}$ *contains* $k\text{-}\mathbf{DT}$*, the set of decision trees of depth at most* k*.*

▷ **Definition 26.6.12.** We denote the set of $k\text{-}\mathbf{DL}$ decision lists with at most n Boolean attributes with $k\text{-}\mathbf{DL}(n)$. The set of conjunctions of at most k literals over n attributes is written as $\mathrm{Conj}(k, n)$.

▷ Decision lists are constructed of optional yes/no tests, so there are at most $3^{|\mathrm{Conj}(k,n)|}$ distinct sets of component tests. Each of these sets of tests can be in any order, so $|k\text{-}\mathbf{DL}(n)| \leq 3^{|\mathrm{Conj}(k,n)|} \cdot |\mathrm{Conj}(k, n)|!$

SOME RIGHTS RESERVED

Decision Lists: Learnable Subsets (Sample Complexity)

▷ The number of conjunctions of k literals from n attributes is given by

$$|\mathrm{Conj}(k, n)| = \sum_{i=1}^{k} \binom{2n}{i}$$

thus $|\mathrm{Conj}(k, n)| = \mathcal{O}(n^k)$. Hence, we obtain (after some work)

$$|k\text{-}\mathbf{DL}(n)| = 2^{\mathcal{O}(n^k \log_2(n^k))}$$

▷ Plug this into the equation for the sample complexity: $N \geq \frac{1}{\epsilon} \cdot (\log_2(\frac{1}{\delta}) + \log_2(|\mathcal{H}|))$ to

obtain

$$N \geq \frac{1}{\epsilon} \cdot (\log_2(\frac{1}{\delta}) + \log_2(\mathcal{O}(n^k \log_2(n^k))))$$

▷ **Intuitively:** Any algorithm that returns a consistent decision list will PAC learn a $k-\mathbf{DL}$ function in a reasonable number of examples, for small k.

SOME RIGHTS RESERVED

Decision Lists Learning

▷ **Idea:** Use a greedy search algorithm that repeats

1. **find** test that agrees exactly with some subset E of the training set,
2. **add** it to the decision list under construction and remove E,
3. **construct** the remainder of the DL using just the remaining examples,

until there are no examples left.

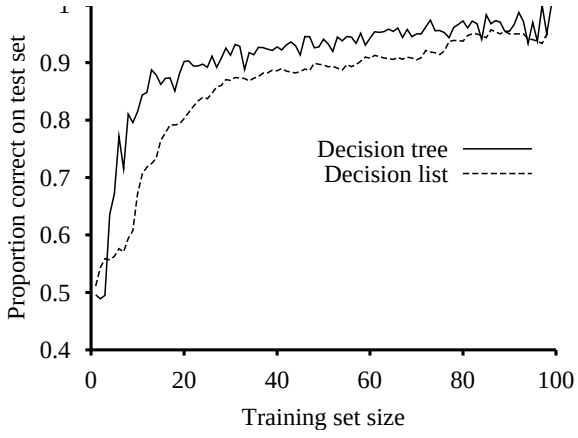
▷ **Definition 26.6.13.** The following algorithm performs decision list learning

function DLL(E) **returns** a decision list, or failure
  if E is empty **then return** (the trivial decision list) No
  t := a test that matches a nonempty subset E_t of E
    such that the members of E_t are all positive or all negative
  if there is no such t **then return** failure
  if the examples **in** E_t are positive **then** o := Yes **else** o := No
  return a decision list with initial test t and outcome o and remaining tests given by
    DLL($E \setminus E_t$)

SOME RIGHTS RESERVED

Decision Lists Learning in Comparison

▷ **Learning curves:** for DLL (and DTL for comparison)



▷ **Upshot:** The simpler DLL works quite well!

26.7 Regression and Classification with Linear Models

26.7.1 Univariate Linear Regression

Univariate Linear Regression

▷ **Definition 26.7.1.** A function with one argument is called univariate or unary.

▷ **Recall:** A mapping f between vector spaces is called linear, iff it preserves vector addition and scalar multiplication, i.e. $f(\alpha\cdot(v_1+v_2))=\alpha\cdot(f(v_1)+f(v_2))$.

▷ **Observation 26.7.2.** *A univariate, linear function $f:\mathbb{R}\to\mathbb{R}$ is of the form $f(x)=\mathrm{w}_1x+\mathrm{w}_0$ for some $\mathrm{w}_i\in\mathbb{R}$.*

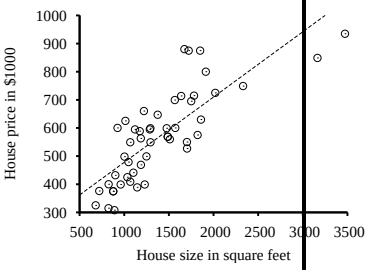
▷ We will use $\mathrm{w}:=\langle\mathrm{w}_1,\mathrm{w}_2\rangle$ to represent a linear function below.

▷ **Definition 26.7.3.** Given a vector w, we define $h_{\mathrm{w}}(x):=\mathrm{w}_1x+\mathrm{w}_0$.

▷ **Definition 26.7.4.** Given a set of examples $E\subseteq\mathbb{R}\times\mathbb{R}$, the task of finding h_{w} that best fits E is called linear regression.

▷ **Example 26.7.5.**

Examples of house price vs. square feet in houses sold in Berkeley in July 2009.
Also: A linear hypothesis that minimizes squared error loss $y=0.232x+246$.



Univariate Linear Regression by Loss Minimization

▷ **Idea:** Minimize squared error loss over $\{(x_i,y_i)\,|\,i\leq N\}$ (used already by Gauss)

$$\mathrm{Loss}(h_{\mathrm{w}_1,\mathrm{w}_2})=\sum_{j=1}^{N}L_2(y_j,h_{\mathrm{w}_1,\mathrm{w}_2}(x_j))=\sum_{j=1}^{N}(y_j-h_{\mathrm{w}_1,\mathrm{w}_2}(x_j))^2=\sum_{j=1}^{N}(y_j-(\mathrm{w}_1x_j+\mathrm{w}_0))^2$$

Task: find $\mathrm{w}^*:=\operatorname*{argmin}_{\mathrm{w}_1,\mathrm{w}_2}\mathrm{Loss}(h_{\mathrm{w}_1,\mathrm{w}_2})$.

▷ **Recall:** $\sum_{j=1}^{N}(y_j-(\mathrm{w}_1x_j+\mathrm{w}_0))^2$ is minimized, when the partial derivatives wrt. the w_i are zero, i.e. when

$$\frac{\partial}{\partial\mathrm{w}_0}(\sum_{j=1}^{N}(y_j-(\mathrm{w}_1x_j+\mathrm{w}_0))^2)=0\quad\text{and}\quad\frac{\partial}{\partial\mathrm{w}_1}(\sum_{j=1}^{N}(y_j-(\mathrm{w}_1x_j+\mathrm{w}_0))^2)=0$$

▷ **Observation:** These equations have a unique solution:

$$\mathrm{w}_1 = \frac{N(\sum_j x_j y_j) - (\sum_j x_j)(\sum_j y_j)}{N(\sum_j {x_j}^2) - (\sum_j x_j)^2} \qquad \mathrm{w}_0 = \frac{(\sum_j y_j) - \mathrm{w}_1(\sum_j x_j)}{N}$$

▷ **Remark:** Closed-form solutions only exist for linear regression, for other (differentiable) hypothesis spaces use gradient descent methods for adjusting/learning weights.

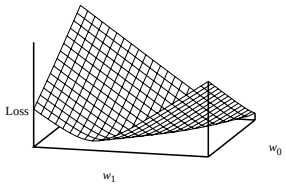
SOME RIGHTS RESERVED

A Picture of the Weight Space

▷ **Remark:** Many forms of learning involve adjusting weights to minimize loss.

▷ **Definition 26.7.6.** The weight space of a parametric model is the space of all possible combinations of parameters (called the weights). Loss minimization in a weight space is called weight fitting.

▷ The weight space of univariate linear regression is $\mathbb{R}^2$.
$\rightsquigarrow$ graph the loss function over $\mathbb{R}^2$.
Note: it is convex.



▷ **Observation 26.7.7.** *The squared error loss function is convex for any linear regression problem $\rightsquigarrow$ there are no local minima.*

SOME RIGHTS RESERVED

Gradient Descent Methods

▷ If we do not have closed form solutions for minimizing loss, we need to search.

▷ **Idea:** Use local search (hill climbing) methods.

▷ **Definition 26.7.8.** The gradient descent algorithm for finding a minimum of a continuous function F is hill climbing in the direction of the steepest descent, which can be computed by the partial derivatives of F.

function gradient−descent(F,w_1, w_2,α) **returns** a **local** minimum of F
  inputs: a differentiable **function** F and initial weights w.
  loop until w converges **do**
    for each w_i **do**
      $\mathrm{w}_i \longleftarrow \mathrm{w}_i - \alpha \frac{\partial}{\partial \mathrm{w}_i} F(\mathrm{w})$
    end for
  end loop

The parameter α is called the learning rate. It can be a fixed constant or it can decay as learning proceeds.

Gradient-Descent for Loss

▷ Let's try gradient descent for Loss.

▷ Work out the partial derivatives for one example (x,y):

$$\frac{\partial \text{Loss}(\mathbf{w})}{\partial \mathbf{w}_i} = \frac{\partial (y - h_{\mathbf{w}}(x))^2}{\partial \mathbf{w}_i} = 2(y - h_{\mathbf{w}}(x)) \frac{\partial (y - (\mathbf{w}_1 x + \mathbf{w}_0))}{\partial \mathbf{w}_i}$$

and thus

$$\frac{\partial \text{Loss}(\mathbf{w})}{\partial \mathbf{w}_0} = -2(y - h_{\mathbf{w}}(x)) \qquad \frac{\partial \text{Loss}(\mathbf{w})}{\partial \mathbf{w}_1} = -2(y - h_{\mathbf{w}}(x))x$$

Plug this into the gradient descent updates:

$$\mathbf{w}_0 \longleftarrow \mathbf{w}_0 - \alpha \cdot (-2(y - h_{\mathbf{w}}(x))) \qquad \mathbf{w}_1 \longleftarrow \mathbf{w}_1 - \alpha \cdot -2((y - h_{\mathbf{w}}(x))) \cdot x$$

Gradient-Descent for Loss (continued)

▷ Analogously for n training examples (x_j,y_j):

▷ **Definition 26.7.9.**

$$\mathbf{w}_0 \longleftarrow \mathbf{w}_0 - \alpha(\sum_j -2(y_j - h_{\mathbf{w}}(x_j))) \quad \mathbf{w}_1 \longleftarrow \mathbf{w}_1 - \alpha(\sum_j -2(y_j - h_{\mathbf{w}}(x_n))x_n)$$

These updates constitute the batch gradient descent learning rule for univariate linear regression.

▷ Convergence to the unique global loss minimum is guaranteed (as long as we pick α small enough) but may be very slow.

▷ Doing batch gradient descent on random subsets of the examples of fixed batch size n is called stochastic gradient descent (SGD). (More computationally efficient than updating for every example)

26.7.2 Multivariate Linear Regression

Multivariate Linear Regression

▷ **Definition 26.7.10.** A function with $n \geq 2$ arguments is called multivariate.

▷ **Observation 26.7.11.** *A linear, multivariate function f is of the form $f(x_1,\ldots,x_n) = \mathbf{w}_0 + \sum_{i=1}^n \mathbf{w}_i x_i$ for some weight vector $\mathbf{w} = \langle \mathbf{w}_0,\ldots,\mathbf{w}_n \rangle$.*

▷ x_0-**Trick:** Invent $x_0 := 1$, define $\mathbf{x} := \langle x_0, x_1, \ldots, x_n \rangle$, and use dot product or matrix multiplication:

$$f(x_1, \ldots, x_n) = \mathbf{w} \cdot \mathbf{x} = \mathbf{w}^t \mathbf{x} = \sum_{i=1}^{n} \mathbf{w}_i x_i$$

▷ We can use it for multivariate linear regression, the task of finding a linear function $h_{\mathbf{w}}$ that best fits a set $E \subseteq \mathbb{R}^n \times \mathbb{R}$ of examples.

▷ **Definition 26.7.12.** The best weight vector $\mathbf{w}^*$ minimizes squared-error loss over the examples in E: $\mathbf{w}^* := \operatorname{argmin}_{\mathbf{w}} \left(\sum_j L_2(y_j, \mathbf{w} \cdot E_j)\right)$, iff $E = \{E_1, \ldots, E_N\}$ and $E_j = \langle e_j^0, \ldots, e_j^n \rangle$

▷ **Observation 26.7.13.** *Gradient descent will reach the (unique) minimum of the loss function; the update equation for each weight $\mathbf{w}_i$ is*

$$\mathbf{w}_i \longleftarrow \mathbf{w}_i - \alpha \left(\sum_{j=1}^{N} e_j^i (y_j - h_{\mathbf{w}}(E_j))\right)$$

SOME RIGHTS RESERVED

Multivariate Linear Regression (Analytic Solutions)

▷ We can also solve analytically for the $\mathbf{w}^*$ that minimizes loss.

▷ Let $\vec{y}$ be the vector of outputs for the training examples, and $\mathbf{X}$ be the data matrix, i.e., the matrix of inputs with one n-dimensional example per row.

Then the solution $\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \vec{y}$ minimizes the squared error.

SOME RIGHTS RESERVED

Multivariate Linear Regression (Regularization)

▷ **Remark:** Univariate linear regression does not overfit, but in the multivariate case there might be "redundant dimensions" that result in overfitting.

▷ **Idea:** Use regularization with a complexity function based on weights.

▷ **Definition 26.7.14.** $\text{Complexity}(h_{\mathbf{w}}) := L_q(\mathbf{w}) = \sum_{i=1}^{n} |\mathbf{w}_i|^q$

▷ **Caveat:** Do not confuse this with the loss functions L_1 and L_2.

▷ **Problem:** Which q should we pick? (L_1 and L_2 minimize sum of absolute values/squares)

▷ **Answer:** It depends on the application.

▷ **Remark:** L_1-regularization tends to produce a sparse model, i.e. it sets many weights to 0, effectively declaring the corresponding attributes to be irrelevant.

Hypotheses that discard attributes can be easier for a human to understand, and may be less likely to overfit. (see [RN21, Section 18.6.2])

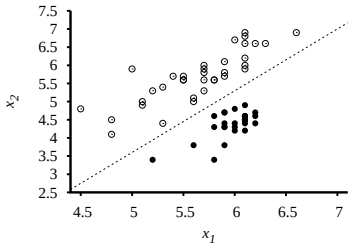
26.7.3 Linear Regression

Linear Classifiers with a hard Threshold

▷ **Idea:** The result of linear regression can be used for classification.

▷ **Example 26.7.15 (Nuclear Test Ban Verification).**

Plots of seismic data parameters: body wave magnitude x_1 vs. surface wave magnitude x_2. White: earthquakes, black: underground explosions
Also: $h_{\mathbf{w}^*}$ as a decision boundary $x_2 = 17x_1 - 4.9$.



▷ **Definition 26.7.16.** A decision boundary is a line (or a surface, in higher dimensions) that separates two classes of points. A linear decision boundary is called a linear separator and data that admits one are called linearly separable.

▷ **Example 26.7.17 (Nuclear Tests continued).** The linear separator for Example 26.7.15is defined by $-4.9 + 1.7x_1 - x_2 = 0$, explosions are characterized by $-4.9 + 1.7x_1 - x_2 > 0$, earthquakes by $-4.9 + 1.7x_1 - x_2 < 0$.

▷ **Useful Trick:** If we introduce dummy coordinate $x_0 = 1$, then we can write the classification hypothesis as $h_{\mathbf{w}}(\mathbf{x}) = 1$ if $\mathbf{w}\cdot\mathbf{x} > 0$ and 0 otherwise.

Linear Classifiers with a hard Threshold (Perceptron Rule)

▷ So $h_{\mathbf{w}_0,\ldots,\mathbf{w}_n}(\mathbf{x}) = 1$ if $\mathbf{w}_0, \ldots, \mathbf{w}_n\cdot\mathbf{x} > 0$ and 0 otherwise is well-defined, how to choose $\mathbf{w}_0, \ldots, \mathbf{w}_n$?

▷ Think of $h_{\mathbf{w}}(\mathbf{x}) = \mathcal{T}(\mathbf{w}\cdot\mathbf{x})$, where $\mathcal{T}(z) = 1$, if $z > 0$ and $\mathcal{T}(z) = 0$ otherwise. We call $\mathcal{T}$ a threshold function.

▷ **Problem:** $\mathcal{T}$ is not differentiable and $\frac{\partial\mathcal{T}}{\partial z} = 0$ where defined $\rightsquigarrow$

- ▷ No closed-form solutions by setting $\frac{\partial\mathcal{T}}{\partial z} = 0$ and solving.
- ▷ Gradient-descent methods in weight-space do not work either.

▷ We can learn weights by iterating over the following rule:

▷ **Definition 26.7.18.**Given an example $(\mathbf{x},y)$, the perceptron learning rule is

$$\mathbf{w}_i \longleftarrow \mathbf{w}_i + \alpha\cdot(y - h_{\mathbf{w}}(\mathbf{x}))\cdot x_i$$

▷ as we are considering 0/1 classification, there are three possibilities:

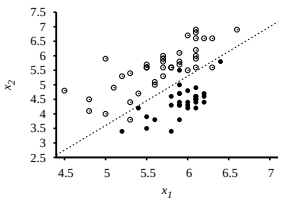
1. If $y = h_{\mathbf{w}}(\mathbf{x})$, then $\mathbf{w}_i$ remains unchanged.
2. If $y = 1$ and $h_{\mathbf{w}}(\mathbf{x}) = 0$, then $\mathbf{w}_i$ is in/decreased if x_i is positive/negative. (we want to make $\mathbf{w}\cdot\mathbf{x}$ bigger so that $\mathcal{T}(\mathbf{w}\cdot\mathbf{x}) = 1$)
3. If $y = 0$ and $h_{\mathbf{w}}(\mathbf{x}) = 1$, then $\mathbf{w}_i$ is de/increased if x_i is positive/negative. (we want to make $\mathbf{w}\cdot\mathbf{x}$ smaller so that $\mathcal{T}(\mathbf{w}\cdot\mathbf{x}) = 0$)

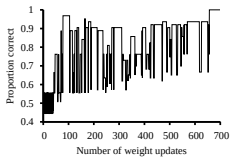
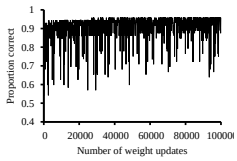
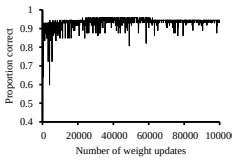
SOME RIGHTS RESERVED

Learning Curves for Linear Classifiers (Perceptron Rule)

▷ **Example 26.7.19.**

Learning curves (plots of total training set accuracy vs. number of iterations) for the perceptron rule on the earthquake/explosions data.



original data	noisy, non-separable data	learning rate decay $\alpha(t) = 1000/(1000+t)$
		
messy convergence 700 iterations	convergence failure 100,000 iterations	slow convergence 100,000 iterations

▷ **Theorem 26.7.20.** *Finding the minimal-error hypothesis is* **NP***-hard, but possible with learning rate decay.*

SOME RIGHTS RESERVED

26.7.4 Logistic Regression

Linear Classification with Logistic Regression

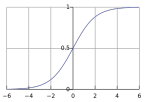
▷ **So far:** Passing the output of a linear function through a threshold function $\mathcal{T}$ yields a linear classifier.

▷ **Problem:** The hard nature of $\mathcal{T}$ brings problems:

- ▷ $\mathcal{T}$ is not differentiable nor continuous $\rightsquigarrow$ learning via perceptron rule becomes unpredictable.
- ▷ $\mathcal{T}$ is "overly precise" near the boundary $\leftsquigarrow$ need more graded judgments.

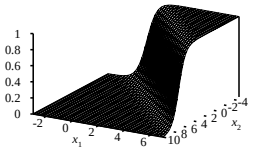
▷ **Idea:** Soften the threshold, approximate it with a differentiable function.

We use the standard logistic function $l(x) = \frac{1}{1+e^{-x}}$
So we have $h_{\mathbf{w}}(\mathbf{x}) = l(\mathbf{w}\cdot\mathbf{x}) = \frac{1}{1+e^{-(\mathbf{w}\cdot\mathbf{x})}}$



▷ **Example 26.7.21 (Logistic Regression Hypothesis in Weight Space).**

Plot of a logistic regression hypothesis for the earthquake/explosion data.
The value at $(\mathbf{w}_0,\mathbf{w}_1)$ is the probability of belonging to the class labeled 1.



We speak of the cliff in the classifier intuitively.

SOME RIGHTS RESERVED

Logistic Regression

▷ **Definition 26.7.22.** The process of weight fitting in $h_{\mathbf{w}}(\mathbf{x}) = \frac{1}{1+e^{-(\mathbf{w}\cdot\mathbf{x})}}$ is called logistic regression.

▷ There is no easy closed form solution, but gradient descent is straightforward,

▷ As our hypotheses have continuous output, use the squared error loss function L_2.

▷ For an example $(\mathbf{x},y)$ we compute the partial derivatives: (via chain rule)

$$\begin{aligned}\frac{\partial}{\partial \mathbf{w}_i} L_2(\mathbf{w}) &= \frac{\partial}{\partial \mathbf{w}_i}((y - h_{\mathbf{w}}(\mathbf{x}))^2)\\ &= 2\cdot h_{\mathbf{w}}(\mathbf{x})\cdot\frac{\partial}{\partial \mathbf{w}_i}(y - h_{\mathbf{w}}(\mathbf{x}))\\ &= -2\cdot h_{\mathbf{w}}(\mathbf{x})\cdot l'(\mathbf{w}\cdot\mathbf{x})\cdot\frac{\partial}{\partial \mathbf{w}_i}(\mathbf{w}\cdot\mathbf{x})\\ &= -2\cdot h_{\mathbf{w}}(\mathbf{x})\cdot l'(\mathbf{w}\cdot\mathbf{x})\cdot x_i\end{aligned}$$

SOME RIGHTS RESERVED

Logistic Regression (continued)

▷ The derivative of the logistic function satisfies $l'(z) = l(z)(1 - l(z))$, thus

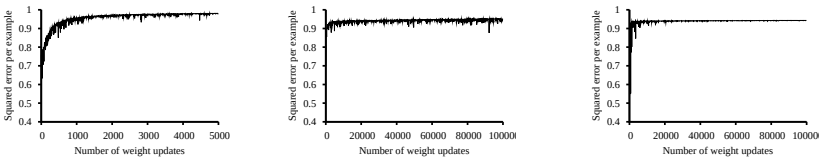
$$l'(\mathbf{w}\cdot\mathbf{x}) = l(\mathbf{w}\cdot\mathbf{x})(1 - l(\mathbf{w}\cdot\mathbf{x})) = h_{\mathbf{w}}(\mathbf{x})(1 - h_{\mathbf{w}}(\mathbf{x}))$$

▷ **Definition 26.7.23.** The rule for logistic update (weight update for minimizing the loss) is

$$\mathbf{w}_i \longleftarrow \mathbf{w}_i + \alpha\cdot(y - h_{\mathbf{w}}(\mathbf{x}))\cdot h_{\mathbf{w}}(\mathbf{x})\cdot(1 - h_{\mathbf{w}}(\mathbf{x}))\cdot x_i$$

▷ **Example 26.7.24 (Redoing the Learning Curves).**

original data | noisy, non-separable data | learning rate decay $\alpha(t) = 1000/(1000 + t)$



messy convergence 5000 iterations | convergence failure 100,000 iterations | slow convergence 100,000 iterations

▷ **Upshot:** Logistic update seems to perform better than perceptron update.

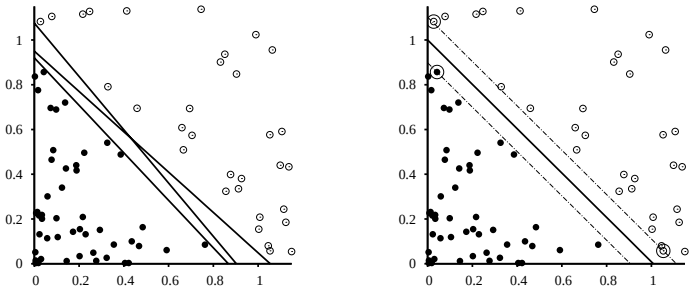
SOME RIGHTS RESERVED

26.8 Support Vector Machines

Support Vector Machines

▷ **Definition 26.8.1.** Given a linearly separable data set E the maximum margin separator is the linear separator s that maximizes the margin, i.e. the distance of the E from s.

▷ **Example 26.8.2.** All lines on the left are valid linear separators:



We expect the maximum margin separator on the right to generalize best

▷ **Note:** To find the maximum margin separator, we only need to consider the innermost points (circled above).

SOME RIGHTS RESERVED

Support Vector Machines (contd.)

▷ **Definition 26.8.3.** Support-vector machines (SVMs; also support-vector networks) are supervised learning models for classification and regression.

SVMs construct a maximum margin separator by prioritizing critical examples (support vectors).

▷ SVMs are still one of the most popular approaches for "off-the-shelf" supervised learning.

▷ **Setting:**

- ▹ We have a training set $E = \{\langle x_1, y_1\rangle, \ldots, \langle x_n, y_n\rangle\}$ where $x_i \in \mathbb{R}^p$ and $y_i \in \{-1, 1\}$ (instead of $\{1, 0\}$)
- ▹ The goal is to find a *hyperplane* in $\mathbb{R}^p$ that maximally separates the two classes (i.e. $y_i = -1$ from $y_i = 1$)

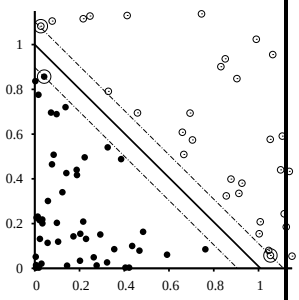
Remember A *hyperplane* can be represented as the set $\{x \,|\, (\mathbf{w}\cdot x) + b = 0\}$ for some vector $\mathbf{w}$ and scalar b. ($\mathbf{w}$ is orthogonal to the plane, b determines the offset from the origin)

SOME RIGHTS RESERVED

Finding the Maximum Margin Separator (Separable Case)

▷ **Idea:** The margin is bounded by the two hyperplanes described by $\{\mathbf{x} \,|\, (\mathbf{w}\cdot\mathbf{x}) + b + 1 = 0\}$ (lower boundary) and $\{\mathbf{x} \,|\, (\mathbf{w}\cdot\mathbf{x}) + b - 1 = 0\}$ (upper boundary).

⇝ The distance between them is $\frac{2}{\|\mathbf{w}\|_2}$.



▷ **Constraints:** To maximize the margin, minimize $\|\mathbf{w}\|_2$ while keeping x_i out of the margin:

$(\mathbf{w}\cdot x_i) + b \geq 1$ for $y_i = 1$ and $(\mathbf{w}\cdot x_i) + b \leq -1$ for $y_i = -1$

⇝ $y_i((\mathbf{w}\cdot x_i) - b) \geq 1$ for $1 \leq i \leq n$.

⇝ This is an optimization problem.

▷ **Theorem 26.8.4 (SVM equation).** *Let* $\alpha = \operatorname{argmax}_\alpha \left(\sum_j \alpha_j - \frac{1}{2}\left(\sum_{j,k} \alpha_j \alpha_k y_j y_k (x_j \cdot x_k)\right)\right)$ *under the constraints* $\alpha_j \geq 0$ *and* $\sum_j \alpha_j y_j = 0$.

The maximum margin separator is given by $\mathbf{w} = \sum_j \alpha_j x_j$ *and* $b = \mathbf{w}\cdot x_i - y_i$ *for any* x_i *where* $\alpha_i \neq 0$.

▷ *Proof sketch:* By the duality principle for optimization problems

SOME RIGHTS RESERVED

Finding the Maximum Margin Separator (Separable Case)

▷

$$\alpha = \operatorname{argmax}_\alpha \left(\sum_j \alpha_j - \frac{1}{2}\left(\sum_{j,k} \alpha_j \alpha_k y_j y_k (x_j \cdot x_k)\right)\right), \text{where } \alpha_j \geq 0, \quad \sum_j \alpha_j y_j = 0$$

▷ **Important Properties:**

- ▹ The weights α_j associated with each data point are zero except at the support vectors (the points closest to the separator),
- ▹ The expression is convex ⇝ the single global maximum can found efficiently,

▷ Data enter the expression only in the form of dot products of point pairs ⇝ once the optimal α_i have been calculated, we have $h(\mathbf{x}) = \mathrm{sign}(\sum_j \alpha_j y_j (\mathbf{x}\cdot\mathbf{x}_j) - b)$

▷ There are good software packages for solving such quadratic programming optimizations

SOME RIGHTS RESERVED

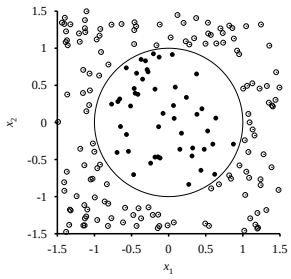
Support Vector Machines (Kernel Trick)

▷ What if the data is not linearly separable?

▷ **Idea:** Transform the data into a *feature space* where they are.

▷ **Definition 26.8.5.** A feature for data in $\mathbb{R}^p$ is a function $\mathbb{R}^p \to \mathbb{R}^q$.

▷ **Example 26.8.6 (Projecting Up a Non-Separable Data Set).**
The true decision boundary is ${x_1}^2 + {x_2}^2 \leq 1$.



⇝ use the feature "distance from center"

SOME RIGHTS RESERVED

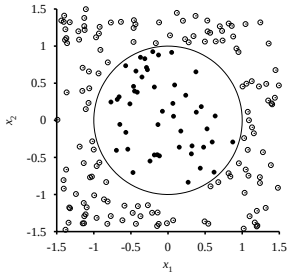
Support Vector Machines (Kernel Trick continued)

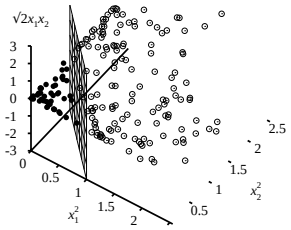
▷ **Idea:** Replace $x_i \cdot x_j$ by some other product on the feature space in the SVM equation

▷ **Definition 26.8.7.** A kernel function is a function $K : \mathbb{R}^p \times \mathbb{R}^p \to \mathbb{R}$ of the form $K(x_1, x_2) = \langle F(x_1), F(x_2) \rangle$ for some feature F and inner product $\langle \cdot, \cdot \rangle$ on the codomain of F.

▷ Smart choices for a kernel function often allow us to compute $K(x_i, x_j)$ without needing to compute F at all.

▷ **Example 26.8.8.** If we encode the distance from the center as the feature $F(\mathbf{x}) = \langle {x_1}^2, {x_2}^2, \sqrt{2} x_1 x_2 \rangle$ and define the kernel function as $K(x_i, x_j) = F(x_i) \cdot F(x_j)$, then this simplifies to $K(x_i, x_j) = (x_i \cdot x_j)^2$





SOME RIGHTS RESERVED

Support Vector Machines (Kernel Trick continued)

▷ **Generally:** We can learn non-linear separators by solving

$$\operatorname*{argmax}_{\alpha}\left(\sum_j \alpha_j - \frac{1}{2}\left(\sum_{j,k} \alpha_j \alpha_k y_j y_k K(\mathbf{x}_j, \mathbf{x}_k)\right)\right)$$

where K is a kernel function

▷ **Definition 26.8.9.** Let $X = \{x_1, \ldots, x_n\}$. A symmetric function $K : X \times X \to \mathbb{R}$ is called positive definite iff the matrix $K_{i,j} = K(x_i, x_j)$ is a positive definite matrix.

▷ **Theorem 26.8.10 (Mercer's Theorem).** *Every positive definite function K on X is a kernel function on X for* some *feature F.*

▷ **Definition 26.8.11.** The function $K(\mathbf{x}_j, \mathbf{x}_k) = (1 + (\mathbf{x}_j \cdot \mathbf{x}_j))^d$ is a kernel function corresponding to a feature space whose dimension is exponential in d. It is called the polynomial kernel.

SOME RIGHTS RESERVED

26.9 Artificial Neural Networks

Outline

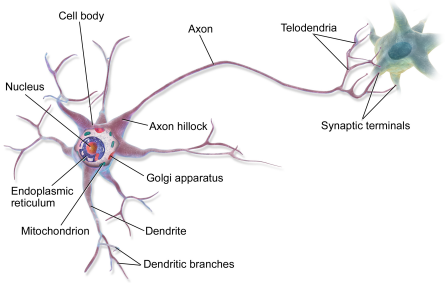
▷ Brains
▷ Neural networks
▷ Perceptrons
▷ Multilayer perceptrons
▷ Applications of neural networks

SOME RIGHTS RESERVED

Brains

▷ **Axiom 26.9.1 (Neuroscience Hypothesis).** *Mental activity consists primarily of electrochemical activity in networks of brain cells called neurons.*

▷ **Definition 26.9.2.** A neuron or nerve cell, is a cell that is excitable, firing electric signals called action potentials across a biological neural network and helps to receive and conduct impulses. Neurons communicate with other cells via synapse, which are specialized connections that commonly use minute amounts of chemical neurotransmitters to pass the electric signal from the presynaptic neuron to the target cell through the synaptic gap.



▷ **Definition 26.9.3.** The animal brain is a biological neural network

- ▷ with up to 10^{11} neurons of more than 20 types, 10^{14} synapses,
- ▷ a cycle time of $(1\text{ms}) - (10\text{ms})$
- ▷ Signals are noisy "spike trains" of electrical potential.

SOME RIGHTS RESERVED

Neural Networks as an approach to Artificial Intelligence

▷ One approach to artificial intelligence is to model and simulate brains. (and hope that AI comes along naturally)

▷ **Definition 26.9.4.** The AI subfield of neural networks (also called connectionism, parallel distributed processing, and neural computation) studies computing systems inspired by the biological neural networks that constitute brains.

▷ Neural networks are attractive computational devices, since they perform important AI tasks – most importantly learning and distributed, noise-tolerant computation – naturally and efficiently.

SOME RIGHTS RESERVED

Neural Networks – McCulloch-Pitts "unit"

▷ **Definition 26.9.5.** An artificial neural network is a directed graph such that every edge $a_i \to a_j$ is associated with a weight $w_{i,j} \in \mathbb{R}$, and each node a_j with parents $a_1, \ldots, a_n$ is associated with a function $f(w_{1,j}, \ldots, w_{n,j}, x_1, \ldots, x_n) \in \mathbb{R}$.

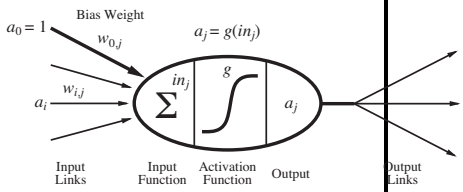
We call the output of a node's function its activation, the matrix $\mathbf{w}_{i,j}$ the weight matrix, the nodes units and the edges links.

▷ In 1943 McCulloch and Pitts proposed a simple model for a neuron/brain:

▷ **Definition 26.9.6.** A McCulloch-Pitts unit first computes a weighted sum of all inputs and

then applies an activation function g to it.

$$\begin{aligned} \text{in}_i &= \sum_j \mathbf{w}_{j,i} a_j \\ a_i &\leftarrow g(\text{in}_i) = g(+\sum_j \mathbf{w}_{j,i} a_j) \end{aligned}$$



If g is a threshold function, we call the unit a perceptron unit, if g is a logistic function a sigmoid perceptron unit.

A McCulloch-Pitts network is a neural network with McCulloch-Pitts units.

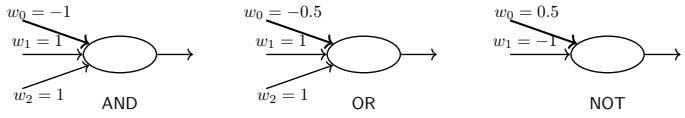
SOME RIGHTS RESERVED

Implementing Logical Functions as Units

▷ McCulloch-Pitts units are a gross oversimplification of real neurons, but its purpose is to develop understanding of what neural networks of simple units can do.

▷ **Theorem 26.9.7 (McCulloch and Pitts).** *Every Boolean function can be implemented as McCulloch-Pitts networks.*

▷ *Proof:* by construction

1. Recall that $a_i \longleftarrow g(\sum_j \mathbf{w}_{j,i} a_j)$. Let $g(r) = 1$ iff $r > 0$, else 0.
2. As for linear regression we use $a_0 = 1 \rightsquigarrow \mathbf{w}_{0,i}$ as a bias weight (or intercept) (determines the threshold)
3. 

4. Any Boolean function can be implemented as a DAG of McCulloch-Pitts units.

□

SOME RIGHTS RESERVED

Network Structures: Feed-Forward Networks

▷ We have models for neurons $\rightsquigarrow$ connect them to neural networks.

▷ **Definition 26.9.8.** A neural network is called a feed-forward network, if it is acyclic.

▷ **Intuition:** Feed-forward networks implement functions, they have no internal state.

▷ **Definition 26.9.9.** Feed-forward networks are usually organized in layers: a n layer network has a partition $\{L_0, \ldots, L_n\}$ of the nodes, such that edges only connect nodes from subsequent layer.

L_0 is called the input layer and its members input units, and L_n the output layer and its members output units. Any unit that is not in the input layer or the output layer is called

hidden.

SOME RIGHTS RESERVED

Network Structures: Recurrent Networks

▷ **Definition 26.9.10.** A neural network is called recurrent (a RNN), iff it has cycles.

- ▹ Hopfield networks have symmetric weights ($\mathbf{w}_{i,j} = \mathbf{w}_{j,i}$) $g(x) = \text{sign}(x)$, $a_i = \pm 1$; (holographic associative memory)
- ▹ Boltzmann machines use stochastic activation functions.

▷ Recurrent neural networks have cycles with delay $\rightsquigarrow$ have internal state (like flip-flops), can oscillate etc.

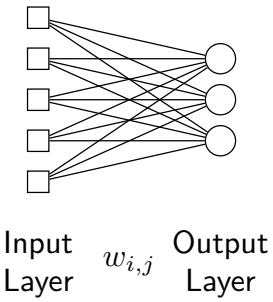
▷ Recurrent neural networks follow largely the same principles as feed-forward networks, so we will not go into details here.

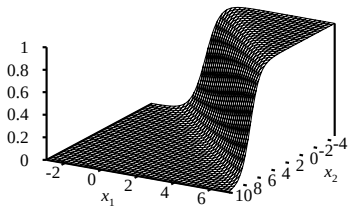
SOME RIGHTS RESERVED

Single-layer Perceptrons

▷ **Definition 26.9.11.** A perceptron network is a feed-forward network of perceptron units. A single layer perceptron network is called a perceptron.

▷ **Example 26.9.12.**





▷ All input units are directly connected to output units.

▷ Output units all operate separately, no shared weights $\rightsquigarrow$ treat as the combination of n perceptron units.

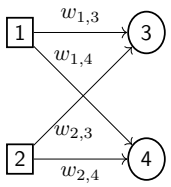
▷ Adjusting weights moves the location, orientation, and steepness of cliff.

SOME RIGHTS RESERVED

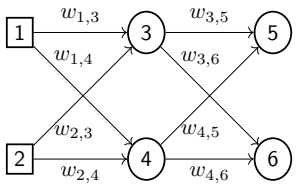
Feed-Forward Neural Networks (Example)

▷ Feed-forward network $\widehat{=}$ a parameterized family of nonlinear functions:

▷ **Example 26.9.13.** We show two feed-forward networks:



a) single layer (perceptron network)



b) 2 layer feed-forward network

$$\begin{aligned} a_5 &= g(\mathbf{w}_{3,5} \cdot a_3 + \mathbf{w}_{4,5} \cdot a_4) \\ &= g(\mathbf{w}_{3,5} \cdot g(\mathbf{w}_{1,3} \cdot a_1 + \mathbf{w}_{2,3} a_2) + \mathbf{w}_{4,5} \cdot g(\mathbf{w}_{1,4} \cdot a_1 + \mathbf{w}_{2,4} a_2)) \end{aligned}$$

▷ **Idea:** Adjusting weights changes the function: do learning this way!

SOME RIGHTS RESERVED

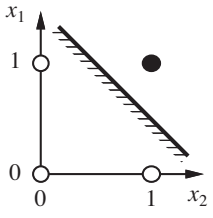
Expressivity of Perceptrons

▷ Consider a perceptron with g = step function (Rosenblatt, 1957, 1960)

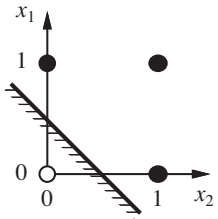
▷ Can represent AND, OR, NOT, majority, etc., but not XOR. (and thus no adders)

▷ Represents a linear separator in input space:

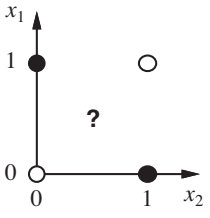
$$\sum_j \mathbf{w}_j x_j > 0 \quad \text{or} \quad \mathbf{w} \cdot \mathbf{x} > 0$$



(a) x_1 **and** x_2



(b) x_1 **or** x_2



(c) x_1 **xor** x_2

▷ Minsky & Papert (1969) pricked the first neural network balloon!

SOME RIGHTS RESERVED

Perceptron Learning

▷ **Idea:** For learning, we update the weights using gradient descent based on the generalization loss function.

▷ Let e.g. $L(\mathbf{w}) = (y - h_{\mathbf{w}}(x))^2$ (the squared error loss).

▷ We compute the gradient:

$$\begin{aligned}\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}_{j,k}} &= 2 \cdot (y_k - h_{\mathbf{w}}(x)_k) \cdot \frac{\partial (y - h_{\mathbf{w}}(x))}{\partial \mathbf{w}_{j,k}} = 2 \cdot (y_k - h_{\mathbf{w}}(x)_k) \cdot \frac{\partial}{\partial \mathbf{w}_{j,k}} (y - g(\sum_{j=0}^{n} \mathbf{w}_{j,k} x_j)) \\ &= -2 \cdot (y_k - h_{\mathbf{w}}(x)_k) \cdot g'(\mathrm{in}_k) \cdot x_j\end{aligned}$$

▷ $\rightsquigarrow$ Replacing the constant factor -2 by a learning rate parameter α we get the perceptron update rule:

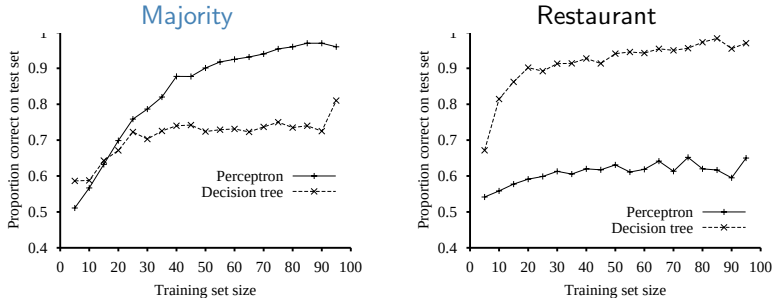
$$\mathbf{w}_{j,k} \leftarrow \mathbf{w}_{j,k} + \alpha \cdot (y_k - h_{\mathbf{w}}(x)_k) \cdot g'(\mathrm{in}_k) \cdot x_j$$

SOME RIGHTS RESERVED

Perceptron Learning contd.

▷ Iteration with the perceptron learning rule converges to a consistent function – *for any linearly separable data set*

▷ **Example 26.9.14 (Perceptron Learning).**



Perceptron learns the majority function easily, where DTL is hopeless.

▷ **Example 26.9.15 (Conversely).** DTL learns the restaurant function easily, where a perceptron is hopeless. (not representable)

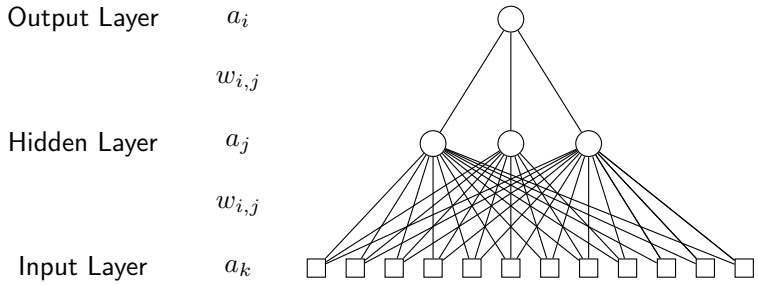
SOME RIGHTS RESERVED

Multilayer perceptrons

▷ **Definition 26.9.16.** In multi layer perceptrons (MLPs)

- ▷ layers are usually fully connected: every node is connected to all nodes in previous/next layers,
- ▷ numbers of hidden units typically chosen by hand.

▷ **Example 26.9.17.** A MLP with one hidden layer:

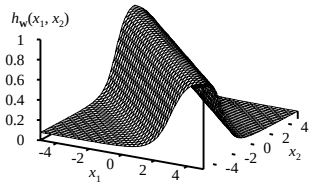


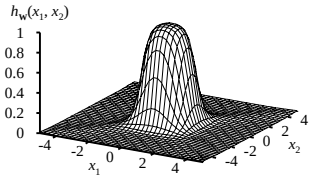
▷ **Definition 26.9.18.** Some MLPs have residual connections, i.e. connections that skip layers.

SOME RIGHTS RESERVED

Expressivity of MLPs

▷ All continuous functions w/ 2 layers, all functions w/ 3 layers.





▷ Combine two opposite-facing threshold functions to make a ridge.

▷ Combine two perpendicular ridges to make a bump.

▷ Add bumps of various sizes and locations to fit any surface.

▷ Proof requires exponentially many hidden units. (cf. DTL proof)

SOME RIGHTS RESERVED

Learning in Multilayer Networks

▷ **Note:** The output layer of a multilayer neural network is a single-layer perceptron whose input is the output of the last hidden layer.

⇝ We can use the perceptron learning rule to update the weights of the output layer; e.g. for a squared error loss function: $\mathbf{w}_{j,k} \leftarrow \mathbf{w}_{j,k} + \alpha \cdot (y_k - h_{\mathbf{w}}(\mathbf{x})_k) \cdot g'(\mathrm{in}_k) \cdot a_j$

▷ What about the hidden layers?

▷ **Idea:** The hidden node j is "responsible" for some fraction of the error proportional to the weight $\mathbf{w}_{j,k}$.

$\rightsquigarrow$ Back-propagate the error $\Delta_k = (y_k - h_{\mathbf{w}}(\mathbf{x})_k) \cdot g'(\mathrm{in}_j)$ from node k in the output layer to the hidden node j.

▷ Let's justify this:

$$\begin{aligned}\frac{\partial L(\mathbf{w})_k}{\partial \mathbf{w}_{i,j}} &= -2 \cdot \underbrace{(y_k - h_{\mathbf{w}}(\mathbf{x})_k) \cdot g'(\mathrm{in}_k)}_{=:\Delta_k} \cdot \frac{\partial \mathrm{in}_k}{\partial \mathbf{w}_{i,j}} \quad \text{(as before)} \\ &= -2 \cdot \Delta_k \cdot \frac{\partial(\sum_\ell \mathbf{w}_{\ell,k} a_\ell)}{\partial \mathbf{w}_{i,j}} = -2 \cdot \Delta_k \cdot \mathbf{w}_{j,k} \cdot \frac{\partial a_j}{\partial \mathbf{w}_{i,j}} = -2 \cdot \Delta_k \cdot \mathbf{w}_{j,k} \cdot \frac{\partial g(\mathrm{in}_j)}{\partial \mathbf{w}_{i,j}} \\ &= -2 \cdot \underbrace{\Delta_k \cdot \mathbf{w}_{j,k} \cdot g'(\mathrm{in}_j)}_{=:\Delta_{j,k}} \cdot a_i\end{aligned}$$

SOME RIGHTS RESERVED

Learning in Multilayer Networks (Hidden Layers)

▷

$$\frac{\partial L(\mathbf{w})_k}{\partial \mathbf{w}_{i,j}} = -2 \cdot \underbrace{\Delta_k \cdot \mathbf{w}_{j,k} \cdot g'(\mathrm{in}_j)}_{=:\Delta_{j,k}} \cdot a_i$$

▷ **Idea:** The total "error" of the hidden node j is the sum of all the connected nodes k in the next layer

▷ **Definition 26.9.19.** The back-propagation rule for hidden nodes of a multilayer perceptron is $\Delta_j \leftarrow g'(\mathrm{in}_j) \cdot (\sum_i \mathbf{w}_{j,i} \Delta_i)$.

The update rule for weights in a hidden layer is $\mathbf{w}_{k,j} \leftarrow \mathbf{w}_{k,j} + \alpha \cdot a_k \cdot \Delta_j$

▷ **Remark:** Most neuroscientists deny that back-propagation occurs in the brain.

▷ The back-propagation process can be summarized as follows:

1. Compute the Δ values for the output units, using the observed error.
2. Starting with output layer, repeat the following for each layer in the network, until the earliest hidden layer is reached:
   (a) Propagate the Δ values back to the previous (hidden) layer.
   (b) Update the weights between the two layers.

SOME RIGHTS RESERVED

Backprogagation Learning Algorithm

▷ **Definition 26.9.20.** The back-propagation learning algorithm is given the following pseudocode, where

1. *examples* is a set of examples, each with input vector $\mathbf{x}$ and output vector $\mathbf{y}$
2. *network* is a multilayer network with L layers, weights $\mathbf{w}_{i,j}$, activation function g

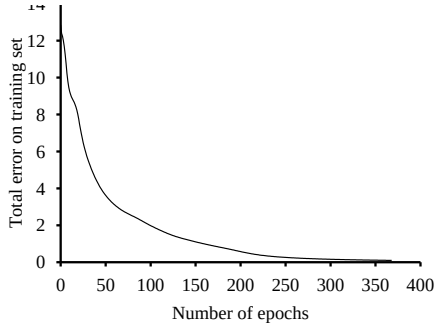
function BACK–PROP–LEARNING($examples$,$network$) **returns** a neural network
  inputs: $examples$, a set of examples, each with input vector $\mathbf{x}$ and output vector $\mathbf{y}$
    $network$, a multilayer network with L layers, weights $\mathbf{w}_{i,j}$, activation **function** g
  local variables: Δ, a vector of errors, indexed by network node
  foreach weight $\mathbf{w}_{i,j}$ **in** $network$ **do**
    $\mathbf{w}_{i,j}$:= a small random number
  repeat
    foreach example $(\mathbf{x}, \mathbf{y})$ **in** $examples$ **do**
      /* Propagate the inputs forward to compute the outputs */
      foreach node i **in** the input layer **do** $a_i := x_i$
      for $l = 2$ **to** L **do**
        foreach node j **in** layer l **do**
          $\text{in}_j := \sum_i \mathbf{w}_{i,j} a_i$
          $a_j := g(\text{in}_j)$
      /* Propagate deltas backward from output layer to input layer */
      foreach node j **in** the output layer **do** $\Delta[j] := g'(\text{in}_j) \cdot (y_j - a_j)$
      for $l = L - 1$ **to** 1 **do**
        foreach node i **in** layer l **do** $\Delta[i] := g'(\text{in}_i) \cdot (\sum_j \mathbf{w}_{i,j} \Delta[j])$
      /* Update every weight in network using deltas */
      foreach weight $\mathbf{w}_{i,j}$ **in** $network$ **do** $\mathbf{w}_{i,j} := \mathbf{w}_{i,j} + \alpha \cdot a_i \cdot \Delta[j]$
  until some stopping criterion is satisfied
  return $network$

SOME RIGHTS RESERVED

Back-Propagation – Properties

▷ Sum gradient updates for all examples in some "batch" and apply gradient descent.

▷ **Example 26.9.21.** Learning curve for 100 restaurant examples: finds exact fit.

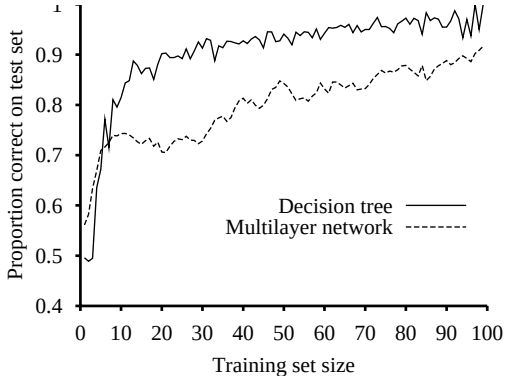


▷ **Typical Problems:** slow convergence, local minima.

SOME RIGHTS RESERVED

Back-Propagation – Properties (contd.)

▷ **Example 26.9.22.** Learning curve for MLPs with 4 hidden units:



▷ **Experience shows:** MLPs are quite good for complex pattern recognition tasks, but resulting hypotheses cannot be understood easily.

▷ This makes MLPs ineligible for some tasks, such as credit card and loan approvals, where law requires clear unbiased criteria.

SOME RIGHTS RESERVED

Handwritten Digit Recognition



▷ 400–300–10 unit MLP = 1.6% error

▷ LeNet: 768–192–30–10 unit MLP = 0.9% error

▷ Current best (kernel machines, vision algorithms) ≈ 0.6% error

SOME RIGHTS RESERVED

Summary

▷ Neural networks are a computing framework inspired by animal brains: large, complex networks of interconnected simple neurons which transmit signals when the weighted sum of input signals surpasses a given threshold.

▷ Neural networks can be extremely powerful (hypothesis space intractably complex)

▷ Perceptrons (one-layer networks) insufficiently expressive for most applications

▷ Multi-layer networks are sufficiently expressive; can be trained by gradient descent, i.e., error back-propagation

▷ Many applications: speech, driving, handwriting, fraud detection, etc.

▷ Engineering, cognitive modelling, and neural system modelling subfields have largely diverged

▷ Drawbacks: take long to converge, require large amounts of data, and are difficult to *interpret* (Why is the output what it is?)

SOME RIGHTS RESERVED

XKCD on Machine Learning

▷ **A Skepticists View:** see `https://xkcd.com/1838/`



SOME RIGHTS RESERVED

Summary of Inductive Learning

▷ Learning needed for unknown environments, lazy agent designers.

▷ Learning agent $\widehat{=}$ performance element + learning element.

▷ Learning method depends on type of performance element, available feedback, type of component to be improved, and its representation.

▷ For supervised learning, the aim is to find a simple hypothesis that is approximately consistent with training examples

▷ Decision tree learning using information gain.

▷ Learning performance $\widehat{=}$ prediction accuracy measured on test set

▷ PAC learning as a general theory of learning boundaries.

▷ Linear regression (hypothesis space of univariate linear functions).

▷ Linear classification by linear regression with hard and soft thresholds.

SOME RIGHTS RESERVED

Chapter 27

Statistical Learning

Above we learned how to reason in non-deterministic, partially observable environments by quantifying uncertainty and reasoning with it. The key resource there were probabilistic models and their efficient representations: Bayesian networks.

Above we assumed that these models were given, perhaps designed by the agent developer. We will now learn how these models can – at least partially – be learned from observing the environment.

Statistical Learning: Outline

▷ **Definition 27.0.1.** Statistical learning has the goal to learn the correct probability distribution of a random variable.

▷ **Example 27.0.2.**

- ▷ Bayesian learning, i.e. learning probabilistic models (e.g. the conditional probability tables in Bayesian networks) from observations.
- ▷ Maximum *a posteriori* and maximum likelihood learning
- ▷ Bayesian network learning
  - ▷ ML Parameter Learning with complete data
  - ▷ Naive Bayes Models/Learning

SOME RIGHTS RESERVED

27.1 Full Bayesian Learning

The Candy Flavors Example

▷ **Example 27.1.1.** Suppose there are five kinds of bags of candies:

1. 10% are h_1: 100% cherry candies
2. 20% are h_2: 75% cherry candies + 25% lime candies
3. 40% are h_3: 50% cherry candies + 50% lime candies
4. 20% are h_4: 25% cherry candies + 75% lime candies

5. 10% are h_5: 100% lime candies

Then we observe candies drawn from some bag:



What kind of bag is it? What flavour will the next candy be?

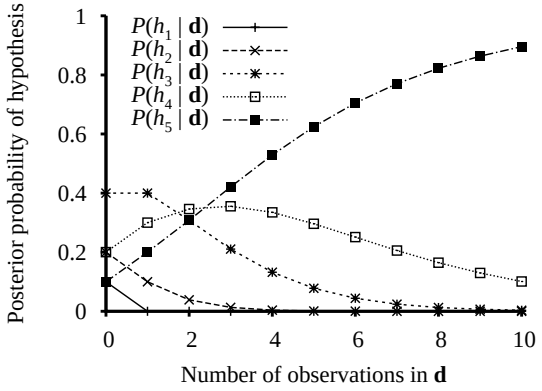
▷ **Note:** Every hypothesis is itself a probability distribution over the random variable "flavour".

SOME RIGHTS RESERVED

Candy Flavors: Posterior probability of hypotheses

▷ **Example 27.1.2.** Let d_i be the event that the ith drawn candy is green.

The probability of hypothesis h_i after n limes are observed ($\widehat{=}\ \mathbf{d}_{1:n} =: d$) is



if the observations are IID, i.e. $P(d \mid h_i) = \prod_j P(d_j \mid h_i)$ and the hypothesis prior is as advertised. (e.g. $P(d \mid h_3) = 0.5^{10} = 0.1\%$)

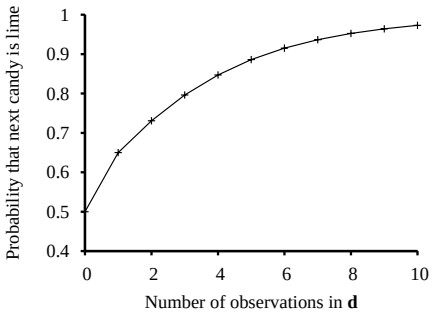
The posterior probabilities start with the hypothesis priors, change with data.

SOME RIGHTS RESERVED

Candy Flavors: Prediction Probability

▷ We calculate that the $n+1$-th candy is lime:

$$P(d_{n+1} = \text{lime} \mid d) = \sum_i P(d_{n+1} = \text{lime} \mid h_i) \cdot P(h_i \mid d)$$



⇝ we compute the expected value of *the probability of the next candy being lime* over all hypotheses (i.e. distributions).

⇝ "meta-distribution"

SOME RIGHTS RESERVED

Full Bayesian Learning

▷ **Idea:** View learning as Bayesian updating of a probability distribution over the hypothesis space:

- ▷ H is the hypothesis variable with values $h_1, h_2, \ldots$ and prior $\mathbb{P}(H)$.
- ▷ jth observation d_j gives the outcome of random variable D_j.
- ▷ $d := d_1, \ldots, d_N$ constitutes the training set of a inductive learning problem.

▷ **Definition 27.1.3.** Bayesian learning calculates the probability of each hypothesis and makes predictions based on this:

- ▷ Given the data so far, each hypothesis has a posterior probability:

$$P(h_i \mid d) = \alpha(P(d \mid h_i) \cdot P(h_i))$$

  where $P(d \mid h_i)$ is called the likelihood (of the data under each hypothesis) and $P(h_i)$ the hypothesis prior.
- ▷ Bayesian predictions use a likelihood-weighted average over the hypotheses:

$$\mathbb{P}(X|d) = \sum_i \mathbb{P}(X|d, h_i) \cdot P(h_i \mid d) = \sum_i \mathbb{P}(X|h_i) \cdot P(h_i \mid d)$$

▷ **Observation:** No need to pick one best-guess hypothesis for Bayesian predictions! (and that is all an agent cares about)

SOME RIGHTS RESERVED

The

Full Bayesian Learning: Properties

▷ **Observation:** Bayesian prediction eventually agrees with the true hypothesis.

- ▷ The probability of generating "uncharacteristic" data indefinitely is vanishingly small.

▷ *Proof sketch:* Argument analogous to PAC learning.

▷ **Problem:** Summing over the hypothesis space is often intractable.

▷ **Example 27.1.4.** There are $2^{2^6} = 18,446,744,073,709,551,616$ Boolean functions of 6 arguments.

▷ **Solution:** Approximate the learning methods to simplify.

SOME RIGHTS RESERVED

27.2 Approximations of Bayesian Learning

Maximum A Posteriori (MAP) Approximation

▷ **Goal:** Get rid of summation over the space of all hypotheses in predictions.

▷ **Idea:** Make predictions wrt. the "most probable hypothesis"!

▷ **Definition 27.2.1.** For maximum a posteriori learning (MAP learning) choose the MAP hypothesis h_{MAP} that maximizes $P(h_i \mid d)$.

I.e., maximize $P(d \mid h_i) \cdot P(h_i)$ or (even better) $\log_2(P(d \mid h_i)) + \log_2(P(h_i))$.

▷ Predictions made according to a MAP hypothesis h_{MAP} are approximately Bayesian to the extent that $\mathbb{P}(X|d) \approx \mathbb{P}(X|h_{\text{MAP}})$.

▷ **Example 27.2.2.** In our candy example, $h_{\text{MAP}} = h_5$ after three limes in a row

- ▷ a MAP learner then predicts that candy 4 is lime with probability 1.
- ▷ compare with Bayesian prediction of 0.8. (see prediction curves above)

▷ As more data arrive, the MAP and Bayesian predictions become closer, because the competitors to the MAP hypothesis become less and less probable.

▷ For deterministic hypotheses, $P(d \mid h_i)$ is 1 if consistent, 0 otherwise
⇝ MAP = simplest consistent hypothesis. (cf. science)

▷ **Remark:** Finding MAP hypotheses is often much easier than Bayesian learning, because it requires solving an optimization problem instead of a large summation (or integration) problem.

SOME RIGHTS RESERVED

Digression From MAP-learning to MDL-learning

▷ **Idea:** Reinterpret the log terms $\log_2(P(d \mid h_i)) + \log_2(P(h_i))$ in MAP learning:

- ▷ Maximizing $P(d \mid h_i) \cdot P(h_i) \mathrel{\widehat{=}}$ minimizing $-\log_2(P(d \mid h_i)) - \log_2(P(h_i))$.
- ▷ $-\log_2(P(d \mid h_i)) \mathrel{\widehat{=}}$ number of bits to encode data given hypothesis.
- ▷ $-\log_2(P(h_i)) \mathrel{\widehat{=}}$ additional bits to encode hypothesis. (section 26.4)

▷ **Indeed** if hypothesis predicts the data exactly – e.g. h_5 in candy example – then $\log_2(1) = 0$ ⇝ preferred hypothesis.

▷ This is more directly modeled by the following approximation to Bayesian learning:

▷ **Definition 27.2.3.** In minimum description length learning (MDL learning) the MDL hypothesis h_{MDL} minimizes the information entropy of the hypothesis likelihood.

SOME RIGHTS RESERVED

Maximum Likelihood (ML) approximation

▷ **Observation:** For large data sets, the prior becomes irrelevant. (we might not trust it anyways)

▷ **Idea:** Use this to simplify learning.

▷ **Definition 27.2.4.** Maximum likelihood learning (ML learning): choose the ML hypothesis h_{ML} maximizing $P(d \mid h_i)$. (simply get the best fit to the data)

▷ **Remark:** ML learning $\widehat{=}$ MAP learning for a uniform prior.(reasonable if all hypotheses are of the same complexity)

▷ ML learning is the "standard" (non Bayesian) statistical learning method.

SOME RIGHTS RESERVED

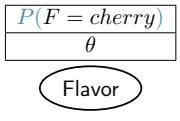
27.3 Parameter Learning for Bayesian Networks

ML Parameter Learning in Bayesian Nets

▷ Bayesian networks (with continuous random variables) often feature nodes with a particular *parametric* distribution $D(\theta)$ (e.g. normal, binomial, Poisson, etc.).

▷ How do we learn the parameters of these distributions from data?

▷ **Example 27.3.1.** We get a candy bag from a new manufacturer; what is the fraction θ of cherry candies? (Note: We use the probability itself as the parameter. This is somewhat boring, but simple.)



▷ **New Facet:** Any θ is possible: continuum of hypotheses h_θ
θ is a parameter for this simple (binomial) family of models; We call h_θ a MLP hypothesis and the process of learning θ MLP learning.

▷ **Example 27.3.2.** Suppose we unwrap N candies, c cherries and $\ell = N - c$ limes. These are IID observations, so the likelihood is $P(d \mid h_\theta) = \prod_{j=1}^{N} P(d_j \mid h_\theta) = \theta^c \cdot (1-\theta)^\ell$

SOME RIGHTS RESERVED

ML Parameter Learning in Bayes Nets

▷ **Trick:** When optimizing a product, optimize the logarithm instead! ($\log_2(!)$ is monotone and turns products into sums)

▷ **Definition 27.3.3.** The log likelihood is the binary logarithm of the likelihood. $L(d|h) := \log_2(P(d \mid h))$

▷ **Example 27.3.4.** Compute the log likelihood as (using Example 27.3.2)

$$L(d|h_\theta) = \log_2(P(d \mid h_\theta)) = \sum_{j=1}^{N} \log_2(P(d_j \mid h_\theta)) = c\log_2(\theta) + \ell\log_2(1-\theta)$$

Maximize this w.r.t. θ

$$\frac{\partial}{\partial\theta}(L(d|h_\theta)) = \frac{c}{\theta} - \frac{\ell}{1-\theta} = 0 \rightsquigarrow \theta = \frac{c}{c+\ell} = \frac{c}{N}$$

▷ **In English:** h_θ asserts that the actual proportion of cherries in the bag is equal to the observed proportion in the candies unwrapped so far! (. . . exactly what we should expect!) ($\rightsquigarrow$ Generalize to more interesting parametric models later)

▷ **Warning:** This causes problems with 0 counts!

SOME RIGHTS RESERVED

ML Learning for Multiple Parameters in Bayesian Networks

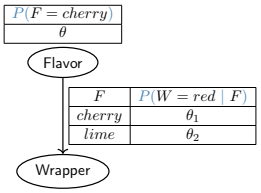
▷ **Cooking Recipe:**

1. Write down an expression for the likelihood of the data as a function of the parameter(s).
2. Write down the derivative of the log likelihood with respect to each parameter.
3. Find the parameter values such that the derivatives are zero

SOME RIGHTS RESERVED

Multiple Parameters Example

▷ **Example 27.3.5.** Red/green wrapper depends probabilistically on flavour:



▷ Likelihood for, e.g., cherry candy in green wrapper:

$$\begin{aligned} & P(F = cherry, W = green \mid h_{\theta,\theta_1,\theta_2}) \\ = \;& P(F = cherry \mid h_{\theta,\theta_1,\theta_2}) \cdot P(W = green \mid F = cherry, h_{\theta,\theta_1,\theta_2}) \\ = \;& \theta \cdot (1 - \theta_1) \end{aligned}$$

▷ **Ovservation:** For N candies, r_c red-wrapped cherry candies, etc. we have

$$P(d \mid h_{\theta,\theta_1,\theta_2}) = \theta^c \cdot (1-\theta)^\ell \cdot {\theta_1}^{r_c} \cdot (1-\theta_1)^{g_c} \cdot {\theta_2}^{r_\ell} \cdot (1-\theta_2)^{g_\ell}$$

SOME RIGHTS RESERVED

Multiple Parameters Example (contd.)

▷ Minimize the log likelihood:

$$\begin{aligned} L &= c\log_2(\theta) + \ell\log_2(1-\theta) \\ &+ r_c\log_2(\theta_1) + g_c\log_2(1-\theta_1) \\ &+ r_\ell\log_2(\theta_2) + g_\ell\log_2(1-\theta_2) \end{aligned}$$

▷ Derivatives of L contain only the relevant parameter:

$$\begin{aligned} \frac{\partial L}{\partial \theta} &= \frac{c}{\theta} - \frac{\ell}{1-\theta} = 0 \quad \rightsquigarrow \quad \theta = \frac{c}{c+\ell} \\ \frac{\partial L}{\partial \theta_1} &= \frac{r_c}{\theta_1} - \frac{g_c}{1-\theta_1} = 0 \quad \rightsquigarrow \quad \theta_1 = \frac{r_c}{r_c+g_c} \\ \frac{\partial L}{\partial \theta_2} &= \frac{r_\ell}{\theta_2} - \frac{g_\ell}{1-\theta_2} = 0 \quad \rightsquigarrow \quad \theta_2 = \frac{r_\ell}{r_\ell+g_\ell} \end{aligned}$$

▷ **Upshot:** With complete data, *parameters can be learned separately* in Bayesian networks.

▷ **Remaining Problem:** Have to be careful with zero values! (division by zero)

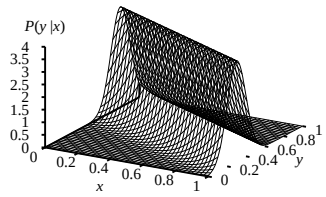
SOME RIGHTS RESERVED

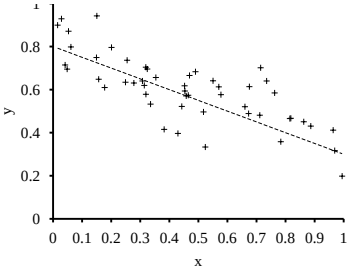
Example: Linear Gaussian Model

▷ A continuous random variable Y has the *linear-Gaussian distribution* with respect to a continuous random variable X, if the outcome of Y is determined by a linear function of the outcome of X *plus gaussian noise with a fixed variance* σ, i.e.

$$P(y_1 \leq Y \leq y_2 \mid X = x) = \int_{y_1}^{y_2} N(y; \theta_1 x + \theta_2, \sigma^2)\, \mathrm{d}y = \int_{y_1}^{y_2} \frac{1}{\sigma\sqrt{2\pi}} \cdot e^{-\frac{1}{2}\cdot\left(\frac{y-(\theta_1 x+\theta_2)}{\sigma}\right)^2}\, \mathrm{d}y$$

▷ ⇝ assuming σ given, we have two parameter θ_1 and θ_2 ⇝ Hypothesis space is $\mathbb{R} \times \mathbb{R}$





▷

SOME RIGHTS RESERVED

Example: Linear Gaussian Model

▷

$$P(y_1 \leq Y \leq y_2 \mid X = x) = \int_{y_1}^{y_2} \frac{1}{\sigma\sqrt{2\pi}} \cdot e^{-\frac{1}{2}\cdot\left(\frac{y-(\theta_1 x+\theta_2)}{\sigma}\right)^2} \, \mathrm{d}y$$

▷ ⇝ Given observations $X = x$, and $Y = y$, maximize $\prod_{i=1}^{N} \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(y_i-(\theta_1 x_i+\theta_2))^2}{2\sigma^2}}$ w.r.t. θ_1, θ_2. (we can ignore the integral for this)

▷ Using the log likelihood, this is equivalent to minimizing $\sum_{i=1}^{N}(y_i - (\theta_1 x_i + \theta_2))^2$.

▷ ⇝ minimizing The sum of squared errors gives the ML solution.

SOME RIGHTS RESERVED

Statistical Learning: Summary

▷ Full Bayesian learning gives best possible predictions but is intractable.

▷ MAP learning balances complexity with accuracy on training data.

▷ Maximum likelihood learning assumes uniform prior, OK for large data sets:

1. Choose a parameterized family of models to describe the data.
   ⇝ requires substantial insight and sometimes new models.
2. Write down the likelihood of the data as a function of the parameters.
   ⇝ may require summing over hidden variables, i.e., inference.
3. Write down the derivative of the log likelihood w.r.t. each parameter.
4. Find the parameter values such that the derivatives are zero.
   ⇝ may be hard/impossible; modern optimization techniques help.

▷ Naive Bayes models as a fall-back solution for machine learning:

▷ conditional independence of all attributes as simplifying assumption.

SOME RIGHTS RESERVED

Chapter 28

Reinforcement Learning

28.1 Reinforcement Learning: Introduction & Motivation

Unsupervised Learning

▷ **So far:** We have studied "learning from examples". (functions, logical theories, probability models)

▷ **Now:** How can agents learn "what to do" in the absence of labeled examples of "what to do". We call this problem unsupervised learning.

▷ **Example 28.1.1 (Playing Chess).** Learn transition models for own moves and maybe predict opponent's moves.

▷ **Problem:** The agent needs to have some feedback about what is good/bad
⇝ cannot decide "what to do" otherwise. (recall: external performance measure for learning agents)

▷ **Example 28.1.2.** The ultimate feedback in chess is whether you win, lose, or draw.

▷ **Definition 28.1.3.** We call a learning situation where there are no labeled examples unsupervised learning and the feedback involved a reward or reinforcement.

▷ **Example 28.1.4.** In soccer, there are intermediate reinforcements in the shape of goals, penalties, . . .

SOME RIGHTS RESERVED

Reinforcement Learning as Policy Learning

▷ **Definition 28.1.5.** Reinforcement learning is a type of unsupervised learning where an agent learns how to behave in an environment by performing actions and seeing the results.

▷ **Recap:** In section 25.1 we introduced rewards as parts of Markov decision processs (Markov decision processes) to define optimal policies.

▷ an optimal policy maximizes the expected total reward.

▷ **Idea:** The task of reinforcement learning is to use observed rewards to come up with an optimal policy.

▷ In Markov decision processs, the agent has total knowledge about the environment *and the reward function*, in reinforcement learning we do not assume this. (⇝ partially observable Markov decision processs+reward-learning)

▷ **Example 28.1.6.** You play a game without knowing the rules, and at some time the opponent shouts "*you lose!*"

SOME RIGHTS RESERVED

Scope and Forms of Reinforcement Learning

▷ **Reinforcement Learning solves all of AI:** An agent is placed in an environment and must learn to behave successfully therein.

▷ **KISS:** We will only look at simple environments and simple agent designs:

- ▷ A utility-based agent learns a utility function on states and uses it to select actions that maximize the expected outcome utility. (passive learning)
- ▷ A Q-learning agent learns an action-utility function, or Q-function, giving the expected utility of taking a given action in a given state. (active learning)
- ▷ A reflex agent learns a policy that maps directly from states to actions.

SOME RIGHTS RESERVED

28.2 Passive Learning

Passive Learning

▷ **Definition 28.2.1 (To keep things simple).** Agent uses a state-based representation in a fully observable environment:

- ▷ In passive learning, the agent's policy π is fixed: in state s, it always executes the action $\pi(s)$.
- ▷ Its goal is simply to learn how good the policy is – that is, to learn the utility function $U^{\pi}(s)$.

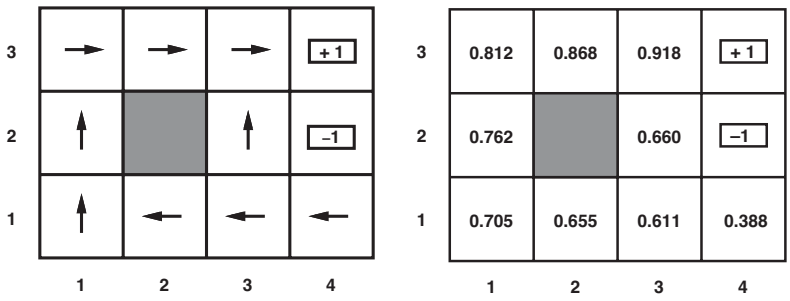
▷ The passive learning task is similar to the policy evaluation task (part of the policy iteration algorithm) but the agent does not know

- ▷ the transition model $P(s' \mid s,a)$, which specifies the probability of reaching state s' from state s after doing action a,
- ▷ the reward function $R(s)$, which specifies the reward for each state.

SOME RIGHTS RESERVED

Passive Learning by Example

▷ **Example 28.2.2 (Passive Learning).** We use the 4×3 world introduced above



Optimal Policy π Utilities, given π

▷ The agent executes a set of trials in the environment using its policy π.

▷ In each trial, the agent starts in state (1,1) and experiences a sequence of state transitions until it reaches one of the terminal states, (4,2) or (4,3).

▷ Its percepts supply both the current state and the reward received in that state.

SOME RIGHTS RESERVED

Passive Learning by Example

▷ **Example 28.2.3.** Typical trials might look like this:

1. $(1,1)_{-0.04} \rightsquigarrow (1,2)_{-0.04} \rightsquigarrow (1,3)_{-0.04} \rightsquigarrow (1,2)_{-0.04} \rightsquigarrow (1,3)_{-0.04} \rightsquigarrow (2,3)_{-0.04} \rightsquigarrow (3,3)_{-0.04} \rightsquigarrow (4,3)_{+1}$
2. $(1,1)_{-0.04} \rightsquigarrow (1,2)_{-0.04} \rightsquigarrow (1,3)_{-0.04} \rightsquigarrow (2,3)_{-0.04} \rightsquigarrow (3,3)_{-0.04} \rightsquigarrow (3,2)_{-0.04} \rightsquigarrow (3,3)_{-0.04} \rightsquigarrow (4,3)_{+1}$
3. $(1,1)_{-0.04} \rightsquigarrow (2,1)_{-0.04} \rightsquigarrow (3,1)_{-0.04} \rightsquigarrow (3,2)_{-0.04} \rightsquigarrow (4,2)_{-1}$.

▷ **Definition 28.2.4.** The utility is defined to be the expected sum of (discounted) rewards obtained if policy π is followed.

$$U^{\pi}(s) := E\left[\sum_{t=0}^{\infty} \gamma^t R(S_t)\right]$$

where $R(s)$ is the reward for a state, S_t (a random variable) is the state reached at time t when executing policy π, and $S_0 = s$. (for 4×3 we take the discount factor $\gamma = 1$)

SOME RIGHTS RESERVED

Direct Utility Estimation

▷ A simple method for direct utility estimation was invented in the late 1950s in the area of

adaptive control theory.

▷ **Definition 28.2.5.** The utility of a state is the expected total reward from that state onward (called the expected reward to go).

▷ **Idea:** Each trial provides a sample of the reward to go for each state visited.

▷ **Example 28.2.6.** The first trial in Example 28.2.3 provides a sample total reward of 0.72 for state (1,1), two samples of 0.76 and 0.84 for (1,2), two samples of 0.80 and 0.88 for (1,3), ...

▷ **Definition 28.2.7.** The direct utility estimation algorithm cycles over trials, calculates the reward to go for each state, and updates the estimated utility for that state by keeping the running average for that for each state in a table.

▷ **Observation 28.2.8.** *In the limit, the sample average will converge to the true expectation (utility) from Definition 28.2.4.*

▷ *Remark 28.2.9.* Direct utility estimation is just supervised learning, where each example has the state as input and the observed reward to go as output.

▷ **Upshot:** We have reduced reinforcement learning to an inductive learning problem.

SOME RIGHTS RESERVED

Adaptive Dynamic Programming

▷ **Problem:** The utilities of states are not independent in direct utility estimation!

▷ The utility of each state equals its own reward plus the expected utility of its successor states.

▷ **So:** The utility values obey a Bellman equation for a fixed policy π.

$$U^\pi(s) = R(s) + \gamma \cdot (\sum_{s'} P(s' \mid s,\pi(s)) \cdot U^\pi(s'))$$

▷ **Observation 28.2.10.** *By ignoring the connections between states, direct utility estimation misses opportunities for learning.*

▷ **Example 28.2.11.** Recall trial 2 in Example 28.2.3; state (3,3) is new.

2 $(1,1)_{-0.04} \rightsquigarrow (1,2)_{-0.04} \rightsquigarrow (1,3)_{-0.04} \rightsquigarrow (2,3)_{-0.04} \rightsquigarrow (3,3)_{-0.04} \rightsquigarrow (3,2)_{-0.04} \rightsquigarrow (3,3)_{-0.04} \rightsquigarrow (4,3)_{+1}$

- ▷ The next transition reaches (3,3), (known high utility from trial 1)
- ▷ Bellman equation: $\rightsquigarrow$ high $U^\pi(3,2)$ because $(3,2)_{-0.04} \rightsquigarrow (3,3)$
- ▷ But direct utility estimation learns nothing until the end of the trial.

▷ **Intuition:** Direct utility estimation searches for U in a hypothesis space that too large $\rightsquigarrow$ many functions that violate the Bellman equations.

▷ Thus the algorithm often converges very slowly.

SOME RIGHTS RESERVED

Adaptive Dynamic Programming

▷ **Idea:** Take advantage of the constraints among the utilities of states by

- ▷ learning the transition model that connects them,
- ▷ solving the corresponding Markov decision process using a dynamic programming method.

This means plugging the learned transition model $\mathbb{P}(s'|s,\pi(s))$ and the observed rewards $R(s)$ into the Bellman equations to calculate the utilities of the states.

▷ **As above:** These equations are linear (no maximization involved)(solve with any any linear algebra package).

▷ **Observation 28.2.12.** *Learning the model itself is easy, because the environment is fully observable.*

▷ **Corollary 28.2.13.** *We have a supervised learning task where the input is a state–action pair and the output is the resulting state.*

- ▷ *In the simplest case, we can represent the transition model as a table of probabilities.*
- ▷ *Count how often each action outcome occurs and estimate the transition probability $P(s' \mid s,a)$ from the frequency with which s' is reached by action a in s.*

▷ **Example 28.2.14.** In the 3 trials from Example 28.2.3, $Right$ is executed 3 times in $(1,3)$ and 2 times the result is $(2,3)$, so $P((2,3) \mid (1,3),Right)$ is estimated to be $2/3$.

SOME RIGHTS RESERVED

Passive ADP Learning Algorithm

▷ **Definition 28.2.15.** The passive ADP agent takes as input a percept indicating the current state s' and reward signal r'.

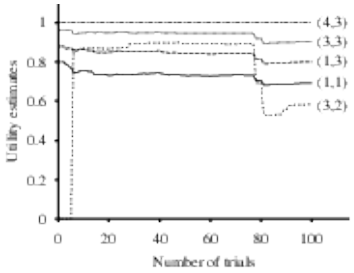
function PASSIVE–ADP–AGENT(percept) **returns** an action
  inputs: percept, a percept indicating the current state s' and reward signal r'
  persistent: π a fixed policy
    mdp, an MDP with model P, rewards R, discount γ
    U, a table of utilities, initially empty
    N_{sa}, a table of frequencies **for** state–action pairs, initially zero
    $N_{s'|sa}$, a table of outcome frequencies given state–action pairs, initially zero
    s, a, the previous state and action, initially null
  if s' is new **then** $U[s'] := r'$; $R[s'] := r'$
  if s is not null **then**
    increment $N_{sa}[s,a]$ and $N_{s'|sa}[s',s,a]$
    for each t such that $N_{s]|sa}[t,s,a]$ is nonzero **do**
      $P(t|s,a) := N_{s'|sa}[t,s,a]/N_{sa}[s,a]$
  U := POLICY–EVALUATION(π,mdp)
  if s'.TERMINAL? **then** s,a := null **else** $s,a := s',\pi[s']$
  return a

PolicyEval $U^\pi(s) := E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t)\right]$ in a MDP.

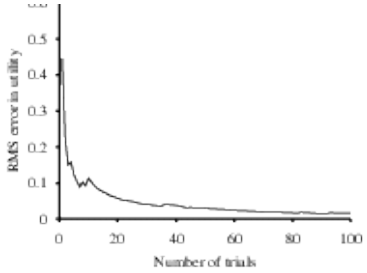
SOME RIGHTS RESERVED

Passive ADP Convergence

▷ **Example 28.2.16 (Passive ADP learning curves for the 4x3 world).** Given the optimal policy from Example 28.2.2



utility estimates/trials



error for $U(1,1)$: 20 runs of 100 trials

Note the large changes occurring around the 78th trial – this is the first time that the agent falls into the -1 terminal state at (4,2).

▷ **Observation 28.2.17.** *The ADP agent is limited only by its ability to learn the transition model.* *(intractable for large state spaces)*

▷ **Example 28.2.18.** In backgammon, roughly 10^{50} equations in 10^{50} unknowns.

▷ **Idea:** Use this as a baseline to compare passive learning algorithms

SOME RIGHTS RESERVED

28.3 Active Reinforcement Learning

Active Reinforcement Learning

▷ **Recap:** A passive learning agent has a fixed policy that determines its behavior.

▷ An active agent must also decide what actions to take.

▷ **Idea:** Adapt the passive ADP agent to handle this new freedom.

- ▷ learn a complete model with outcome probabilities for all actions, rather than just the model for the fixed policy. (use PASSIVE-ADP-AGENT)
- ▷ choose actions; the utilities to learn are defined by the optimal policy, they obey the Bellman equation:

$$U(s) = R(s) + \gamma \cdot \max_{a \in A(s)} (\sum_{s'} U(s') \cdot P(s' \mid s,a))$$

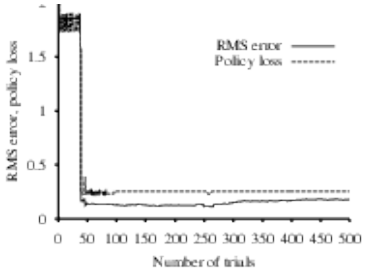
- ▷ solve with value/policy iteration techniques from section 25.3.
- ▷ choose a good action, e.g.
  - ▷ by one-step lookahead to maximize expected utility, or
  - ▷ if agent uses policy iteration and has optimal policy, execute that.

This agent/algorithm is greedy, since it only optimizes the next step!

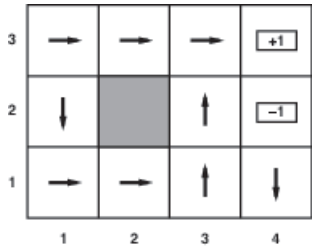
SOME RIGHTS RESERVED

Greedy ADP Learning (Evaluation)

▷ **Example 28.3.1 (Greedy ADP learning curves for the 4x3 world).**



average error/loss



suboptimal policy involved

The agent follows the optimal policy for the learned model at each step.

- ▷ It does not learn the true utilities or the true optimal policy!
- ▷ instead, in the 39th trial, it finds a policy that reaches the +1 reward along the lower route via (2,1), (3,1), (3,2), and (3,3).
- ▷ After experimenting with minor variations, from the 276th trial onward it sticks to that policy, never learning the utilities of the other states and never finding the optimal route via (1,2), (1,3), and (2,3).

SOME RIGHTS RESERVED

Exploration in Active Reinforcement Learning

▷ **Observation 28.3.2.** *Greedy active ADP learning agents very seldom converge against the optimal solution*

- ▷ *The learned model is not the same as the true environment,*
- ▷ *What is optimal in the learned model need not be in the true environment.*

▷ What can be done? The agent does not know the true environment.

▷ **Idea:** Actions do more than provide rewards according to the learned model:

- ▷ They also contribute to learning the true model by affecting the percepts received.
- ▷ By improving the model, the agent may reap greater rewards in the future.

▷ **Observation 28.3.3.** *An agent must make a tradeoff between*

- ▷ *exploitation to maximize its reward as reflected in its current utility estimates and*
- ▷ *exploration to maximize its long term well-being.*

Pure exploitation risks getting stuck in a rut. Pure exploration to improve one's knowledge is of no use if one never puts that knowledge into practice.

▷ Compare with the information gathering agent from section 23.6.

SOME RIGHTS RESERVED

Chapter 29

Knowledge in Learning

29.1 Logical Formulations of Learning

Knowledge in Learning: Motivation

▷ **Recap:** Learning from examples (or reinforcements). (last chapters)

▷ **Idea:** Construct a function with the input/output behavior observed in data.

▷ **Method:** Search for suitable functions in the hypothesis space. (e.g. decision trees)

▷ **Observation 29.1.1.** *Every learning task begins from zero.* *(except for the choice of hypothesis space)*

▷ **Problem:** We have to forget everything before we can learn something new.

▷ **Idea:** Utilize prior knowledge about the world! (represented e.g. in logic)

SOME RIGHTS RESERVED

A logical Formulation of Learning

▷ **Recall:** Examples are composed of descriptions (of the input sample) and classifications.

▷ **Idea:** Represent examples and hypotheses as logical formulae.

▷ **Example 29.1.2.** For attribute-based representations, we can use PL^1: we use predicate constants for Boolean attributes and classification and function constants for the other attributes.

▷ **Definition 29.1.3.** Logic based inductive learning tries to learn an hypothesis h that explains the classifications of the examples given their description, i.e. $h, \mathcal{D} \models \mathcal{C}$ (the explanation constraint), where

▷ $\mathcal{D}$ is the conjunction of the descriptions, and

▷ $\mathcal{C}$ the conjunction of their classifications.

▷ **Idea:** We solve the explanation constraint $h, \mathcal{D} \models \mathcal{C}$ for h where h ranges over some hypothesis space.

▷ **Refinement:** Use Occam's razor or additional constraints to avoid $h = \mathcal{C}$. (too easy otherwise/boring; see below)

SOME RIGHTS RESERVED

A logical Formulation of Learning (Restaurant Examples)

▷ **Example 29.1.4 (Restaurant Example again).** Descriptions are conjunctions of literals built up from

- ▷ predicates Alt, Bar, $\mathrm{Fri/Sat}$, Hun, Rain, and res
- ▷ equations about the functions Pat, Price, Type, and Est.

For instance the first example X_1 from Example 26.3.2, can be described as

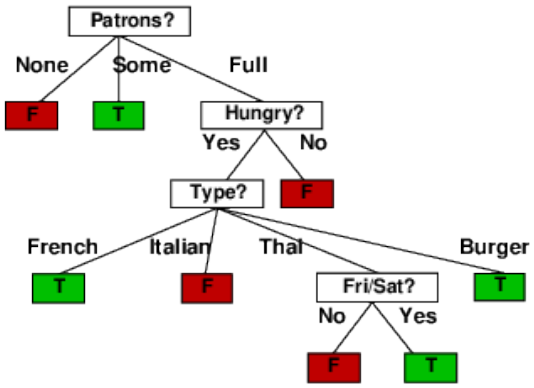
$$\mathrm{Alt}(X_1) \wedge \neg\mathrm{Bar}(X_1) \wedge \mathrm{Fri/Sat}(X_1) \wedge \mathrm{Hun}(X_1) \wedge \ldots$$

The classification is given by the goal predicate $\mathrm{WillWait}$, in this case $\mathrm{WillWait}(X_1)$ or $\neg\mathrm{WillWait}(X_1)$.

SOME RIGHTS RESERVED

A logical Formulation of Learning (Restaurant Tree)

▷ **Example 29.1.5 (Restaurant Example again; Tree).** The induced decision tree from Example 26.4.9



can be represented as

$$\begin{aligned}\forall r.\mathrm{WillWait}(r) \Leftrightarrow\ & \mathrm{Pat}(r,\mathrm{Some})\\ \vee\ & \mathrm{Pat}(r,\mathrm{Full}) \wedge \mathrm{Hun}(r) \wedge \mathrm{Type}(r,\mathrm{French})\\ \vee\ & \mathrm{Pat}(r,\mathrm{Full}) \wedge \mathrm{Hun}(r) \wedge \mathrm{Type}(r,\mathrm{Thai}) \wedge \mathrm{Fri/Sat}(r)\\ \vee\ & \mathrm{Pat}(r,\mathrm{Full}) \wedge \mathrm{Hun}(r) \wedge \mathrm{Type}(r,\mathrm{Burger})\end{aligned}$$

Method: Construct a disjunction of all the paths from the root to the positive leaves interpreted as conjunctions of the attributes on the path.

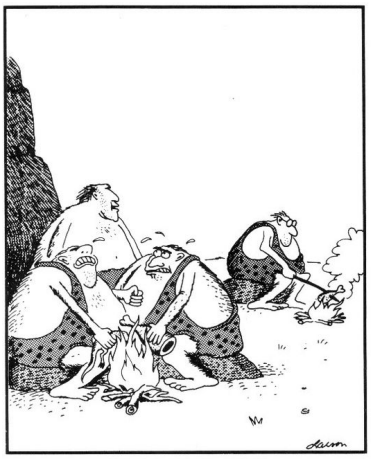
Note: The equivalence takes care of positive and negative examples.

SOME RIGHTS RESERVED

Cumulative Development

▷ **Example 29.1.6.** Learning from very few examples using background knowledge:

1. Caveman Zog and the fish on a stick:



"Hey! Look what Zog do!"

2. Generalizing from one Brazilian:
   Upon meeting her first Brazilian – Fernando – who speaks Portugese, Sarah
   - ▷ learns/generalizes that all Brazilians speak Portugese,
   - ▷ but not that all Brazilians are called Fernando.
3. General rules about effectiveness of antibiotics:
   When Johanna – gifted in diagnostics, but clueless in pharmacology – observes a doctor prescribing the antibiotic Proxadone for an inflamed foot, she learns/infers that Proxadone is effective against this ailment.

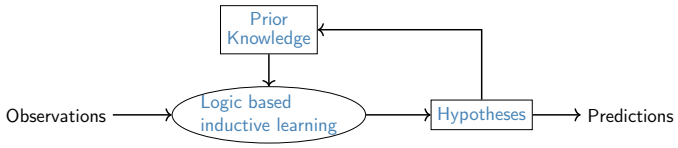
▷ **Observation:** The methods/algorithms from section 26.2 cannot replicate this. (why?)

▷ **Missing Piece:** The background knowledge!

▷ **Problem:** To use background knowledge, need a method to obtain it. (use learning)

▷ **Question:** How to use knowledge to learn more efficiently?

▷ **Answer:** Cumulative development: collect knowledge and use it in learning!



▷ **Definition 29.1.7.** We call the body of knowledge accumulated by (a group of) agents their background knowledge. It acts as prior knowledge in logic based learning processes.

SOME RIGHTS RESERVED

Adding Background Knowledge to Learning: Overview

▷ Explanation based learning (EBL)

▷ Relevance based learning (RBL)

▷ Knowledge based inductive learning (KBIL)

SOME RIGHTS RESERVED

Three Principal Modes of Inference

▷ **Definition 29.1.8.** Deduction $\widehat{=}$ knowledge extension

▷ **Example 29.1.9.** $\dfrac{rains \Rightarrow wet_street \;\; rains}{wet_street}\, D$

▷ **Definition 29.1.10.** Abduction $\widehat{=}$ explanation

▷ **Example 29.1.11.** $\dfrac{rains \Rightarrow wet_street \;\; wet_street}{rains}\, A$

▷ **Definition 29.1.12.** Induction $\widehat{=}$ learning general rules from examples

▷ **Example 29.1.13.** $\dfrac{wet_street \;\; rains}{rains \Rightarrow wet_street}\, I$

SOME RIGHTS RESERVED

29.2 Inductive Logic Programming

Knowledge-based Inductive Learning

▷ **Idea:** Background knowledge and new hypothesis combine to explain the examples.

▷ **Example 29.2.1.** Inferring disease D from the symptoms is not enough to explain the prescription of medicine M.

Need a new general rule: "M *is effective against* D" (induction from example)

▷ **Definition 29.2.2.** Knowledge based inductive learning (KBIL) replaces the explanation constraint by the KBIL constraint:

$$Background \wedge Hypothesis \wedge Descriptions \models Classifications$$

SOME RIGHTS RESERVED

Inductive Logic Programming

▷ **Definition 29.2.3.** Inductive logic programming (ILP) is logic based inductive learning method that uses logic programming as a uniform representation for examples, background knowledge and hypotheses.

Given an formalization of the background knowledge and a set of examples represented as a logical knowledge base of facts, an ILP system will derive a hypothesised logic program which entails all the positive and none of the negative examples.

▷ Main field of study for KBIL algorithms.

▷ Prior knowledge plays two key roles:

1. The effective hypothesis space is reduced to include only those theories that are consistent with what is already known.
2. Prior knowledge can be used to reduce the size of the hypothesis explaining the observations.
   ▷ Smaller hypotheses are easier to find.

▷ **Observation:** ILP systems can formulate hypotheses in first-order logic.

⇝ Can learn in environments not understood by simpler systems.

SOME RIGHTS RESERVED

Inductive Logic Programming

▷ Combines inductive methods with the power of first-order representations.

▷ Offers a rigorous approach to the general KBIL problem.

▷ Offers complete algorithms for inducing general, first-order theories from examples.

SOME RIGHTS RESERVED

29.2.1 An Example

ILP: An example

▷ General knowledge-based induction problem

$$Background \wedge Hypothesis \wedge Descriptions \models Classifications$$

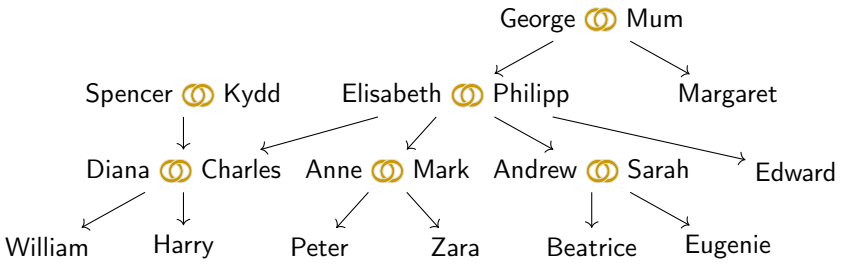
▷ **Example 29.2.4 (Learning family relations from examples).**

- ▷ Observations are an extended family tree
  - ▷ mom, dad and married relations
  - ▷ male and female properties
- ▷ Target predicates: grandparent, BrotherInLaw, Ancestor
- ⇝ The goal is to find a logical formula that serves as a *definition* of the target predicates
- ▷ equivalently: A Prolog program that *computes* the value of the target predicate
- ⇝ We obtain a perfectly comprehensible hypothesis

SOME RIGHTS RESERVED

British Royalty Family Tree (not quite not up to date)

▷ The facts about kinship and relations can be visualized as a family tree:



SOME RIGHTS RESERVED

Example

▷ Descriptions include facts like

- ▷ $\mathrm{dad}(Philip, Charles)$
- ▷ $\mathrm{mom}(Mum, Margaret)$
- ▷ $\mathrm{married}(Diana, Charles)$
- ▷ $\mathrm{male}(Philip)$
- ▷ $\mathrm{female}(Beatrice)$

▷ Sentences in classifications depend on the target concept being learned (in the example: 12 positive, 388 negative)

▷ $\text{grandparent}(Mum, Charles)$

▷ $\neg\text{grandparent}(Mum, Harry)$

▷ **Goal:** Find a set of sentences for hypothesis such that the entailment constraint is satisfied.

▷ **Example 29.2.5.** Without background knowledge, define grandparent in terms of mom and dad.

$\text{grandparent}(x,y) \Leftrightarrow (\exists z.\text{mom}(x,z) \wedge \text{mom}(z,y)) \vee (\exists z.\text{mom}(x,z) \wedge \text{dad}(z,y)) \vee \ldots \vee (\exists z.\text{dad}(x,z) \wedge \text{dad}(z,y))$

SOME RIGHTS RESERVED

Why Attribute-based Learning Fails

▷ **Observation:** Decision tree learning will get nowhere!

▷ To express the grandparent relationshiop as a (Boolean) attribute, pairs of people need to be objects $\text{grandparent}(\langle Mum, Charles\rangle)$.

▷ But then the example descriptions can not be represented

$$FirstElementIsMotherOfElizabeth(\langle Mum, Charles\rangle)$$

▷ A large disjunction of specific cases without any hope of generalization to new examples.

▷ **Generally:** Attribute-based learning algorithms are incapable of learning relational predicates.

SOME RIGHTS RESERVED

Background knowledge

▷ **Observation:** A little bit of background knowledge helps a lot.

▷ **Example 29.2.6.** If the background knowledge contains

$$\text{parent}(x,y) \Leftrightarrow \text{mom}(x,y) \vee \text{dad}(x,y)$$

then the grandparent predicate can be reduced to

$$\text{grandparent}(x,y) \Leftrightarrow (\exists z.\text{parent}(x,z) \wedge \text{parent}(z,y))$$

▷ **Definition 29.2.7.** A constructive induction algorithm creates new predicates to facilitate the expression of explanatory hypotheses.

▷ **Example 29.2.8.** Use constructive induction to introduce a predicate parent to simplify the definitions of the target predicates.

SOME RIGHTS RESERVED

29.2.2 Top-Down Inductive Learning: FOIL

Top-Down Inductive Learning

▷ **Definition 29.2.9.** Bottom-up learning methods start from the observations and work backwards.

▷ **Example 29.2.10.** In decision-tree learning, the tree is gradually grown until it is consistent with the observations.

▷ **Definition 29.2.11.** Top-down learning methods start from a general rule and specialize it on every example.

SOME RIGHTS RESERVED

Top-Down Inductive Learning: FOIL by Example

▷ Split positive and negative examples for the $\mathrm{grandpa}$ relationship:

- ▷ positive: $\langle George, Anne\rangle$, $\langle Philip, Peter\rangle$, $\langle Spencer, Harry\rangle$
- ▷ negative: $\langle George, Elizabeth\rangle$, $\langle Harry, Zara\rangle$, $\langle Charles, Philip\rangle$

▷ Construct a set of Horn clauses with head $\mathrm{grandpa}(x, y)$ such that the positive examples are instances of the $\mathrm{grandpa}$ relationship.

- ▷ Start with a clause with an empty body $\Rightarrow \mathrm{grandpa}(x, y)$.
- ▷ All examples are now classified as positive examples, so specialize to rule out the negative examples: Here are 3 potential additions:
  1. $\mathrm{dad}(x, y) \Rightarrow \mathrm{grandpa}(x, y)$
  2. $\mathrm{parent}(x, z) \Rightarrow \mathrm{grandpa}(x, y)$
  3. $\mathrm{dad}(x, z) \Rightarrow \mathrm{grandpa}(x, y)$
- ▷ The first one incorrectly classifies the 12 positive examples.
- ▷ The second one is incorrect on a larger part of the negative examples.
- ▷ Prefer the third clause and specialize to $\mathrm{dad}(x, z) \wedge \mathrm{parent}(z, y) \Rightarrow \mathrm{grandpa}(x, y)$.

SOME RIGHTS RESERVED

FOIL – The Algorithm

▷ **Definition 29.2.12.** The FOIL algorithm takes a set $examples$ of examples and a literal $target$, for the goal predicate.

function Foil($examples$,$target$) **returns** a set of Horn clauses
  inputs: $examples$, set of examples
    $target$, a literal **for** the goal predicate
  local variables: $clauses$, set of clauses, initially empty
  while $examples$ contains positive examples **do**
    $clause$:= NewClause($examples$,$target$)
    remove examples covered by clause from $examples$
    add $clause$ **to** $clauses$
  return $clauses$

▷ **Example 29.2.13.** Most impressive demonstration of FOIL:

▹ Learn the definitions of list-processing functions in Prolog from a small set of examples, using previously learned functions as background knowledge.

SOME RIGHTS RESERVED

FOIL – The Subroutines

▷ **Definition 29.2.14.**

function NewClause($examples$,$target$) **returns** a Horn clause
  local variables: $clause$, a clause with target as head and an empty body
    l, a literal **to** be added **to** the clause
    $extendedExamples$, a set of examples with values **for** new variables
  $extendedExamples := examples$
  while $extendedExamples$ contains negative examples **do**
    $l :=$ Choose−Literal(NewLits($clause$),$extendedExamples$)
    append l **to** the body of $clause$
    $extendedExamples :=$ map ExtendEx over $extendedExamples$
  return $clause$

function ExtendEx($example$,$literal$) **returns** a new example
  if $example$ satisfies $literal$
  then return the set of examples created by extending $example$ with each
    possible constant value **for** each new variable **in** $literal$
  else return the empty set

SOME RIGHTS RESERVED

FOIL: Choosing New Literals

▷ **Definition 29.2.15.**

function NewLits($clause$) **returns** a set of possibly "useful" literals

Two functions missing ⇝ see below. (next slides)

▷ $\mathrm{dad}(x,z) \Rightarrow \mathrm{grandpa}(x,y)$

▷ Add literals using predicates

▹ Negated or unnegated.

▹ Use any existing predicate (including the target)

▹ Arguments must be variables

▹ Each literal must include at least one variable from an earlier literal or from the head of the clause

▹ Valid: $Mother(z,u)$, $Married(z,z)$, $\mathrm{grandpa}(v,x)$

▹ Invalid: $Married(u,v)$

▷ Equality and negated equality literals

▹ E.g. $z \neq x$, empty list

▷ Arithmetic comparisons

▹ E.g. $x > y$, threshold values

▷ **Observation:** The way NewLits changes the clauses leads to a very large branching factor.

▷ **Idea:** Improve performance by using type information:

▹ E.g., $\text{parent}(x, n)$ where x is a person and n is a number is nonsensical

SOME RIGHTS RESERVED

FOIL: Choosing Literals

▷ **Definition 29.2.16.**
function ChooseLit($literals$) **returns** the "best" literal from $literals$

▷ ChooseLit uses a heuristic similar to information gain.

▷ Ockham's razor to eliminate hypotheses.

▹ If the clause becomes longer than the total length of the positive examples that the clause explains, this clause is not a valid hypothesis.

SOME RIGHTS RESERVED

29.2.3 Inverse Resolution

Inverse Resolution

▷ **Definition 29.2.17.** Inverse resolution in a nutshell

▹ Classifications follows from $Background \wedge Hypothesis \wedge Descriptions$.

▹ This can be proven by resolution.

▹ Run the proof backwards to find hypothesis.

▷ **Problem:** How to run the resolution proof backwards?

▷ **Recap:** In ordinary resolution we take two clauses $C_1 = L \vee R_1$ and $C_2 = \neg L \vee R_2$ and resolve them to produce the resolvent $C = R_1 \vee R_2$.

▷ **Idea:** Two possible variants of inverse resolution:

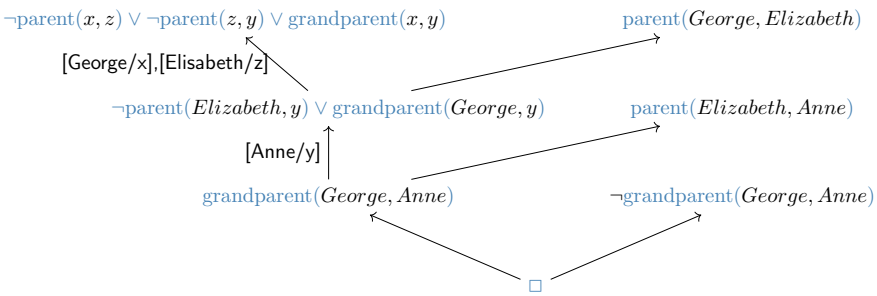
▹ Take resolvent C and produce two clauses C_1 and C_2.

▹ Take C and C_1 and produce C_2.

SOME RIGHTS RESERVED

Generating Inverse Proofs (Example)

1. Start with an example classified as both positive and negative (Need a contradiction)
2. Invent clauses that resolve with a fact in our knowledge base



$\neg\text{parent}(x,z)\vee\neg\text{parent}(z,y)\vee\text{grandparent}(x,y)$ is equivalent to $\text{parent}(x,z)\wedge\text{parent}(z,y)\Rightarrow\text{grandparent}(x,y)$

SOME RIGHTS RESERVED

Generating Inverse Proofs

▷ Inverse resolution is a search algorithm: For any C and C_1 there can be several or even an infinite number of clauses C_2.

▷ **Example 29.2.18.** There were alternatives to picking $\text{parent}(Elizabeth,y)\Rightarrow\text{grandparent}(George,y)$:

- ▷ $\text{parent}(Elizabeth,Anne)\Rightarrow\text{grandparent}(George,Anne)$
- ▷ $\text{parent}(z,Anne)\Rightarrow\text{grandparent}(George,Anne)$
- ▷ $\text{parent}(z,y)\Rightarrow\text{grandparent}(George,y)$

▷ The clauses C_1 that participate in each step can be chosen from Background, Descriptions, Classifications or from hypothesized clauses already generated.

▷ ILP needs restrictions to make the search manageable

- ▷ Eliminate function symbols
- ▷ Generate only the most specific hypotheses
- ▷ Use Horn clauses
- ▷ All hypothesized clauses must be consistent with each other
- ▷ Each hypothesized clause must agree with the observations

SOME RIGHTS RESERVED

New Predicates and New Knowledge

▷ An inverse resolution procedure is a complete algorithm for learning first-order theories:

- ▷ If some unknown hypothesis generates a set of examples, then an inverse resolution procedure can generate hypothesis from the examples.

▷ Can inverse resolution infer the law of gravity from examples of falling bodies?

- ▷ Yes, given suitable background mathematics!

▷ Monkey and typewriter problem: How to overcome the large branching factor and the lack of structure in the search space?

SOME RIGHTS RESERVED

New Predicates and New Knowledge

▷ Inverse resolution is capable of generating new predicates:

- ▷ Resolution of C_1 and C_2 into C eliminates a literal that C_1 and C_2 share.
- ▷ This literal might contain a predicate that does not appear in C.
- ▷ When working backwards, one possibility is to generate a new predicate from which to construct the missing literal.

SOME RIGHTS RESERVED

New Predicates and New Knowledge

▷ **Example 29.2.19.**

$$Father(George; y) \Rightarrow P(x, y) \qquad P(George; y) \Rightarrow Ancestor(George, y)$$
$$[George/x]$$
$$Father(George; y) \Rightarrow Ancestor(George, y)$$

P can be used in later inverse resolution steps.

▷ **Example 29.2.20.** $\mathrm{mom}(x, y) \Rightarrow P(x, y)$ or $\mathrm{dad}(x, y) \Rightarrow P(x, y)$ leading to the "Parent" relationship.

▷ Inventing new predicates is important to reduce the size of the definition of the goal predicate.

▷ Some of the deepest revolutions in science come from the invention of new predicates. (e.g. Galileo's invention of acceleration)

SOME RIGHTS RESERVED

Applications of ILP

▷ ILP systems have outperformed knowledge-free methods in a number of domains.

▷ Molecular biology: the GOLEM system has been able to generate high-quality predictions of

protein structures and the therapeutic efficacy of various drugs.

▷ GOLEM is a completely general-purpose program that is able to make use of background knowledge about any domain.

SOME RIGHTS RESERVED

Part III

Natural Language

This part introduces the basics of natural language processing and the use of natural language for communication with humans.

Fascination of (Natural) Language

▷ **Definition 29.2.21.** A natural language is any form of spoken or signed means of communication that has evolved naturally in humans through use and repetition without conscious planning or premeditation.

▷ **In other words:** The language you use all day long, e.g. English, German, ...

▷ **Why Should we care about natural language?:**

- ▷ Even more so than thinking, language is a skill that only humans have.
- ▷ It is a miracle that we can express complex thoughts in a sentence in a matter of seconds.
- ▷ It is no less miraculous that a child can learn tens of thousands of words and complex syntax in a matter of a few years.

SOME RIGHTS RESERVED

Natural Language and AI

▷ Without natural language capabilities (understanding and generation) no AI!

▷ Ca. 100.000 years ago, humans learned to speak, ca. 7.000 years ago, to write.

▷ Alan Turing based his test on natural language: (for good reason)

- ▷ We want AI agents to be able to communicate with humans.
- ▷ We want AI agents to be able to acquire knowledge from written documents.

▷ In this part, we analyze the problem with specific information-seeking tasks:

- ▷ Language models (Which strings are English/Spanish/etc.)
- ▷ Text classification (E.g. spam detection)
- ▷ Information retrieval (aka. Search Engines)
- ▷ Information extraction (finding objects and their relationships in texts)

SOME RIGHTS RESERVED

Chapter 30

Natural Language Processing

30.1 Introduction to NLP

The general context of AI-2 is natural language processing (NLP), and in particular natural language understanding (NLU). The dual side of NLU: natural language generation (NLG) requires similar foundations, but different techniques is less relevant for the purposes of this course.

What is Natural Language Processing?

- **Generally:** Studying of natural languages and development of systems that can use/generate these.
- **Definition 30.1.1.** Natural language processing (NLP) is an engineering field at the intersection of computer science, AI, and linguistics which is concerned with the interactions between computers and human (natural) languages. Most challenges in NLP involve:
  - Natural language understanding (NLU) that is, enabling computers to derive meaning (representations) from human or natural language input.
  - Natural language generation (NLG) which aims at generating natural language or speech from meaning representations.
- For communication with/among humans we need both NLU and NLG.

SOME RIGHTS RESERVED

Language Technology

- Language Assistance:
  - written language: Spell/grammar/style-checking,
  - spoken language: dictation systems and screen readers,
  - multilingual text: machine-supported text and dialog translation, eLearning.
- Information management:
  - search and classification of documents, (e.g. Google/Bing)
  - information extraction, question answering. (e.g. `http://ask.com`)

▷ Dialog Systems/Interfaces:

- ▷ information systems: at airport, tele-banking, e-commerce, call centers,
- ▷ dialog interfaces for computers, robots, cars. (e.g. Siri/Alexa)

▷ **Observation:** The earlier technologies largely rely on pattern recognition, the latter ones need to compute the meaning of the input utterances, e.g. for database lookups in information systems.

SOME RIGHTS RESERVED

30.2 Natural Language and its Meaning

Before we embark on the journey into understanding the meaning of natural language, let us get an overview over what the concept of "semantics" or "meaning" means in various disciplines.

What is (NL) Semantics? Answers from various Disciplines!

▷ **Observation:** Different (academic) disciplines specialize the notion of semantics (of natural language) in different ways.

▷ **Philosophy** has a long history of trying to answer it, e.g.

- ▷ Platon ⇝ cave allegory, Aristotle ⇝ syllogisms.
- ▷ Frege/Russell ⇝ sense vs. referent. ("*Michael Kohlhase*" vs. "*Odysseus*")

▷ **Linguistics/Language Philosophy:** We need semantics e.g. in translation
"*Der Geist ist willig aber das Fleisch ist schwach!*" vs.
"*Der Schnaps ist gut, aber der Braten ist verkocht!*" (meaning counts)

▷ **Psychology/Cognition:** Semantics ≙ "what is in our brains" (⇝ mental models)

▷ **Mathematics** has driven much of modern logic in the quest for foundations.

- ▷ Logic as "foundation of mathematics" solved as far as possible
- ▷ In daily practice syntax and semantics are not differentiated (much).

▷ **Logic@AI/computer science** tries to define meaning and compute with them. (applied semantics)

- ▷ makes syntax explicit in a formal language (formulae, sentences)
- ▷ defines truth/validity by mapping sentences into "world" (interpretation)
- ▷ gives rules of truth-preserving reasoning (inference)

SOME RIGHTS RESERVED

A good probe into the issues involved in natural language understanding is to look at translations between natural language utterances – a task that arguably involves understanding the utterances first.

Meaning of Natural Language; e.g. Machine Translation

- **Idea:** Machine translation is very simple! (we have good lexica)
- **Example 30.2.1.** "*Peter liebt Maria.*" $\rightsquigarrow$ "*Peter loves Mary.*"
- this only works for simple examples!
- **Example 30.2.2.** "*Wirf der Kuh das Heu über den Zaun.*" $\not\rightsquigarrow$ "*Throw the cow the hay over the fence.*" (differing grammar; Google Translate)
- **Example 30.2.3.** Grammar is not the only problem
  - "*Der Geist ist willig, aber das Fleisch ist schwach!*"
  - "*Der Schnaps ist gut, aber der Braten ist verkocht!*"
- **Observation 30.2.4.** *We have to understand the meaning for high-quality translation!*

SOME RIGHTS RESERVED

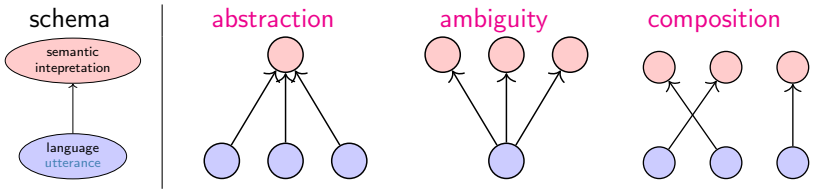
If it is indeed the meaning of natural language, we should look further into how the form of the utterances and their meaning interact.

Language and Information

- **Observation:** Humans use words (sentences, texts) in natural languages to represent and communicate information.
- **But:** What really counts is not the words themselves, but the meaning information they carry.
- **Example 30.2.5 (Word Meaning).**

"*Newspaper*" $\rightsquigarrow$   

- For questions/answers, it would be very useful to find out what words (sentences/texts) mean.
- **Definition 30.2.6.** Interpretation of natural language utterances: three problems

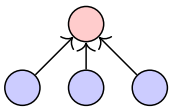


SOME RIGHTS RESERVED

Let us support the last claim a couple of initial examples. We will come back to these phenomena again and again over the course of the course and study them in detail.

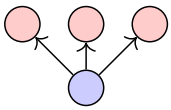
Language and Information (Examples)

▷ **Example 30.2.7 (Abstraction).**



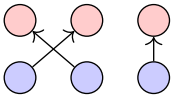
"*Car*" and "*automobile*" have the same meaning.

▷ **Example 30.2.8 (Ambiguity).**



A "*bank*" can be a financial institution or a geographical feature.

▷ **Example 30.2.9 (Composition).**



"*Every student sleeps*" $\rightsquigarrow \forall x.student(x) \Rightarrow sleep(x)$

SOME RIGHTS RESERVED

But there are other phenomena that we need to take into account when compute the meaning of NL utterances.

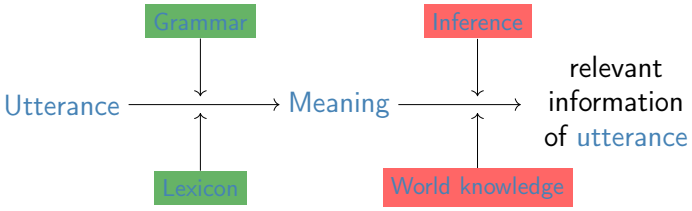
Context Contributes to the Meaning of NL Utterances

▷ **Observation:** Not all information conveyed is linguistically realized in an utterance.

▷ **Example 30.2.10.** "*The lecture begins at 11:00 am.*" What lecture? Today?

▷ **Definition 30.2.11.** We call a piece i of information linguistically realized in an utterance U, iff, we can trace i to a fragment of U.

▷ **Definition 30.2.12 (Possible Mechanism).** Inferring the missing pieces from the context and world knowledge:



We call this process semantic/pragmatic analysis.

SOME RIGHTS RESERVED

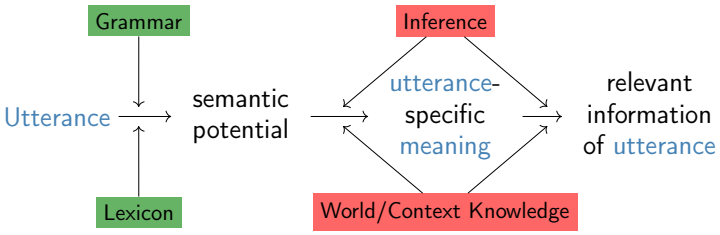
We will look at another example, that shows that the situation with semantic/pragmatic analysis is even more complex than we thought. Understanding this is one of the prime objectives of the AI-2 course.

Context Contributes to the Meaning of NL Utterances

▷ **Example 30.2.13.** "*It starts at eleven.*" What starts?

▷ Before we can resolve the time, we need to resolve the anaphor "*it*".

▷ **Possible Mechanism:** More Inference!



⇝ Semantic/pragmatic analysis is quite complex! (prime topic of AI-2)

SOME RIGHTS RESERVED

Example 30.2.13 is also a very good example for the claim Observation 30.2.4 that even for high-quality (machine) translation we need semantics.

30.3 Looking at Natural Language

The next step will be to make some observations about natural language and its meaning, so that we get an intuition of what problems we will have to overcome on the way to modeling natural language.

Fun with Diamonds (are they real?) [Dav67]

▷ **Example 30.3.1.** We study the truth conditions of adjectival complexes:

- ▷ "*This is a diamond.*" ($\models diamond$)
- ▷ "*This is a blue diamond.*" ($\models diamond$, $\models blue$)
- ▷ "*This is a big diamond.*" ($\models diamond$, $\not\models big$)
- ▷ "*This is a fake diamond.*" ($\models \neg diamond$)
- ▷ "*This is a fake blue diamond.*" ($\models blue$?, $\models diamond$?)
- ▷ "*Mary knows that this is a diamond.*" ($\models diamond$)
- ▷ "*Mary believes that this is a diamond.*" ($\not\models diamond$)

SOME RIGHTS RESERVED

Logical analysis vs. conceptual analysis: These examples — mostly borrowed from Davidson:tam67 — help us to see the difference between "logical analysis" and "conceptual analysis".

We observed that from "*This is a big diamond.*" we cannot conclude "*This is big*". Now consider the sentence "*Jane is a beautiful dancer*". Similarly, it does not follow from this that Jane is beautiful, but only that she dances beautifully. Now, what it is to be beautiful or to be a

beautiful dancer is a complicated matter. To say what these things are is a problem of conceptual analysis. The job of semantics is to uncover the logical form of these sentences. Semantics should tell us that the two sentences have the same logical forms; and ensure that these logical forms make the right predictions about the entailments and truth conditions of the sentences, specifically, that they don't entail that the object is big or that Jane is beautiful. But our semantics should provide a distinct logical form for sentences of the type: "*This is a fake diamond.*" From which it follows that the thing is fake, but not that it is a diamond.

Ambiguity: The dark side of Meaning

▷ **Definition 30.3.2.** We call an utterance ambiguous, iff it has multiple meanings, which we call readings.

▷ **Example 30.3.3.** All of the following sentences are ambiguous:

- ▷ "*John went to the bank.*" (river or financial?)
- ▷ "*You should have seen the bull we got from the pope.*" (three readings!)
- ▷ "*I saw her duck.*" (animal or action?)
- ▷ "*John chased the gangster in the red sports car.*" (three-way too!)

SOME RIGHTS RESERVED

One way to think about the examples of ambiguity on the previous slide is that they illustrate a certain kind of indeterminacy in sentence meaning. But really what is indeterminate here is what sentence is represented by the physical realization (the written sentence or the phonetic string). The symbol "*duck*" just happens to be associated with two different things, the noun and the verb. Figuring out how to interpret the sentence is a matter of deciding which item to select. Similarly for the syntactic ambiguity represented by PP attachment. Once you, as interpreter, have selected one of the options, the interpretation is actually fixed. (This doesn't mean, by the way, that as an interpreter you necessarily do select a particular one of the options, just that you can.) **A**

brief digression: Notice that this discussion is in part a discussion about compositionality, and gives us an idea of what a non-compositional account of meaning could look like. The radical pragmatic view is a non-compositional view: it allows the information content of a sentence to be fixed by something that has no linguistic reflex.

To help clarify what is meant by compositionality, let me just mention a couple of other ways in which a semantic account could fail to be compositional.

- Suppose your syntactic theory tells you that S has the structure $[a[bc]]$ but your semantics computes the meaning of S by first combining the meanings of a and b and then combining the result with the meaning of c. This is non-compositional.
- Recall the difference between:

  1. Jane knows that George was late.
  2. Jane believes that George was late.

  Sentence 1. entails that George was late; sentence 2. doesn't. We might try to account for this by saying that in the context of the verb "*believe*", a clause doesn't mean what it usually means, but something else instead. Then the clause "*that George was late*" is assumed to contribute different things to the informational content of different sentences. This is a non-compositional account.

Quantifiers, Scope and Context

▷ **Example 30.3.4.** "*Every man loves a woman.*" (Keira Knightley or his mother!)

▷ **Example 30.3.5.** "*Every car has a radio.*" (only one reading!)

▷ **Example 30.3.6.** "*Some student in every course sleeps in every class at least some of the time.*" (how many readings?)

▷ **Example 30.3.7.** "*The president of the US is having an affair with an intern.*" (2002 or 2000?)

▷ **Example 30.3.8.** "*Everyone is here.*" (who is everyone?)

SOME RIGHTS RESERVED

Observation: If we look at the first sentence, then we see that it has two readings:

1. there is one woman who is loved by every man.

2. for each man there is one woman whom that man loves.

These correspond to distinct situations (or possible worlds) that make the sentence true. We call this quantifier scope ambiguity

Observation: For the second example we only get one reading: the analogue of 2. The reason for this lies not in the logical structure of the sentence, but in concepts involved. We interpret the meaning of the word "*has*" as the relation "has as physical part", which in our world carries a certain uniqueness condition: If a is a physical part of b, then it cannot be a physical part of c, unless b is a physical part of c or vice versa. This makes the structurally possible analogue to 1. impossible in our world and we discard it.

Observation: In the examples above, we have seen that (in the worst case), we can have one reading for every ordering of the quantificational phrases in the sentence. So, in the third example, we have four of them, we would get $4! = 24$ readings. It should be clear from introspection that we (humans) do not entertain 12 readings when we understand and process this sentence. Our models should account for such effects as well.

Context and Interpretation: It appears that the last two sentences have different informational content on different occasions of use. Suppose I say "*Everyone is here.*" at the beginning of class. Then I mean that everyone who is enrolled is here. Suppose I say it later in the day at a meeting; then I mean that everyone who is meant to be at the meeting is here. What shall we say about this? Here are three different kinds of solution:

Definition 30.3.9. The radical semantic view on quantifier scope ambiguity holds that on every occasion of use, the sentence literally means that everyone in the world is here, and so is strictly speaking false. An interpreter recognizes that the speaker has said something false, and uses general principles to figure out what the speaker actually meant.

Definition 30.3.10. The radical pragmatic view on quantifier scope ambiguity holds that what the semantics provides is in some sense incomplete. What the sentence means is determined in part by the context of utterance and the speaker's intentions. The differences in meaning are entirely due to extra-linguistic facts which have no linguistic reflex.

Definition 30.3.11. The intermediate view on quantifier scope ambiguity holds that the logical form of sentences with the quantifier "*every*" contains a slot for information which is contributed by the context. So extra-linguistic information is required to fix the meaning; but the contribution of this information is mediated by linguistic form.

We now come to a phenomenon of natural language, that is a paradigmatic challenge for pragmatic analysis: anaphora – the practice of replacing a (complex) reference with a mere pronoun.

More Context: Anaphora – Challenge for Pragmatic Analysis

▷ **Example 30.3.12 (Anaphoric References).**

- ▷ "*John is a bachelor. His wife is very nice.*" (Uh, what?, who?)
- ▷ "*John likes his dog Spiff even though he bites him sometimes.*" (who bites?)
- ▷ "*John likes Spiff. Peter does too.*" (what to does Peter do?)
- ▷ "*John loves his wife. Peter does too.*" (whom does Peter love?)
- ▷ "*John loves golf, and Mary too.*" (who does what?)

▷ **Definition 30.3.13.** A word or phrase is called anaphoric (or an anaphor), if its interpretation depends upon another phrase in context. In a narrower sense, an anaphor refers to an earlier phrase (its antecedent), while a cataphor to a later one (its postcedent).

Definition 30.3.14. The process of determining the antecedent or postcedent of an anaphoric phrase is called anaphor resolution.

Definition 30.3.15. An anaphoric connection between anaphor and its antecedent or postcedent is called direct, iff it can be understood purely syntactically. An anaphoric connection is called indirect or a bridging reference if additional knowledge is needed.

▷ Anaphora are another example, where natural languages use the inferential capabilities of the hearer/reader to "shorten" utterances.

▷ Anaphora challenge pragmatic analysis, since they can only be resolved from the context using world knowledge.

SOME RIGHTS RESERVED

Anaphora are also interesting for pragmatic analysis, since they introduce (often initially massive amoungs of) ambiguity that needs to be taken care of in the language understanding process.

We now come to another challenge to pragmatic analysis: presuppositions. Instead of just being subject to the context of the readers/hearers like anaphora, they even have the potential to change the context itself or even affect their world knowledge.

Context is Personal and Keeps Changing

▷ **Example 30.3.16.** Consider the following sentences involving definite description:

1. "*The king of America is rich.*" (true or false?)
2. "*The king of America isn't rich.*" (false or true?)
3. "*If America had a king, the king of America would be rich.*" (true or false!)
4. "*The king of Buganda is rich.*" (Where is Buganda?)
5. "*. . . Joe Smith. . . The CEO of Westinghouse announced budget cuts.*" (CEO=J.S.!)

How do the phrase "king of America" interact with your context and world knowledge?

▷ The interpretation or whether they make sense at all depends on the context!

▷ **Note:** Last two examples feed back into the context or even world knowledge:

- ▷ If 4. is uttered by an Africa expert, we add "*Buganda exists and is a monarchy*" to our world knowledge
- ▷ We add "*Joe Smith is the CEO of Westinghouse to the context/world knowledge*" (happens all the time in newpaper articles)

SOME RIGHTS RESERVED

30.4 Language Models

Natural Languages vs. Formal Language

- ▷ **Recap:** A formal language is a set of strings.
- ▷ **Example 30.4.1.** Programming languages like Java or C++ are formal languages.
- ▷ *Remark 30.4.2.* Natural languages like English, German, or Spanish are not.
- ▷ **Example 30.4.3.** Let us look at concrete examples
  - ▷ "*Not to be invited is sad!*" (definitely English)
  - ▷ "*To not be invited is sad!*" (controversial)
- ▷ **Idea:** Let's be lenient, instead of a crisp set, use a probability distribution.
- ▷ **Definition 30.4.4.** A (statistical) language model is a probability distribution over sequences of characters or words.
- ▷ **Idea:** Try to learn/derive language models from text corpora.
- ▷ **Definition 30.4.5.** A text corpus (or simply corpus; plural corpora) is a large and structured collection of natural language documents.
- ▷ **Definition 30.4.6.** In corpus linguistics, corpora are used to do statistical analysis and hypothesis testing, checking occurrences or validating linguistic rules within a specific natural language.

SOME RIGHTS RESERVED

N-gram Character Models

- ▷ Written text is composed of characters: letters, digits, punctuation, and spaces.
- ▷ **Idea:** Let's study language models for sequences of characters.
- ▷ As for Markov processes, we write $P(\mathbf{c}_{1:N})$ for the probability of a character sequence $\mathtt{c}_1 \ldots \mathtt{c}_n$ of length N.
- ▷ **Definition 30.4.7.** We call an character sequence of length n an n gram (unigram, bigram, trigram for $n = 1, 2, 3$).

▷ **Definition 30.4.8.** An n gram model is a Markov process of order $n-1$.

▷ *Remark 30.4.9.* For a trigram model, $P(\mathrm{c}_i \mid \mathbf{c}_{1:i-1}) = P(\mathrm{c}_i \mid \mathrm{c}_{(i-2)},\mathrm{c}_{(i-1)})$. Factoring with the chain rule and then using the Markov property, we obtain

$$P(\mathbf{c}_{1:N}) = \prod_{i=1}^{N} P(\mathrm{c}_i \mid \mathbf{c}_{1:i-1}) = \prod_{i=1}^{N} P(\mathrm{c}_i \mid \mathrm{c}_{(i-2)},\mathrm{c}_{(i-1)})$$

▷ **Thus,** a trigram model for a language with 100 characters, $\mathbb{P}(\mathrm{c}_i|\mathbf{c}_{i-2:i-1})$ has 1.000.000 entries. It can be estimated from a corpus with 10^7 characters.

SOME RIGHTS RESERVED

Applications of N-Gram Models of Character Sequences

▷ What can we do with N gram models?

▷ **Definition 30.4.10.** The problem of language identification is given a text, determine the natural language it is written in.

▷ *Remark 30.4.11.* Current technology can classify even short texts like "*Hello, world*", or "*Wie geht es Dir*" correctly with more than 99% accuracy.

▷ **One approach:** Build a trigram language model $\mathbb{P}(\mathrm{c}_i|\mathbf{c}_{i-2:i-1},\ell)$ for each candidate language ℓ by counting trigrams in a ℓ-corpus.

Apply Bayes' rule and the Markov property to get the most likely language:

$$\begin{aligned}\ell^* &= \operatorname*{argmax}_{\ell}\left(P(\ell \mid \mathbf{c}_{1:N})\right)\\ &= \operatorname*{argmax}_{\ell}\left(P(\ell)\cdot P(\mathbf{c}_{1:N} \mid \ell)\right)\\ &= \operatorname*{argmax}_{\ell}\left(P(\ell)\cdot\left(\prod_{i=1}^{N} P(\mathrm{c}_i \mid \mathbf{c}_{i-2:i-1},\ell)\right)\right)\end{aligned}$$

The prior probability $P(\ell)$ can be estimated, it is not a critical factor, since the trigram language models are extremely sensitive.

SOME RIGHTS RESERVED

Other Applications of Character N-Gram Models

▷ Spelling correction is a direct application of a single-language language model: Estimate the probability of a word and all off-by-one variants.

▷ **Definition 30.4.12.** Genre classification is deciding whether a text is a news story, a legal document, a scientific article, etc.

▷ *Remark 30.4.13.* While many features help make this classification, counts of punctuation and other character n-gram features go a long way [KNS97].

▷ **Definition 30.4.14.** Named entity recognition (NER) is the task of finding names of things in a document and deciding what class they belong to.

▷ **Example 30.4.15.** In "*Mr. Sopersteen was prescribed aciphex.*" NER should recognize that "*Mr. Sopersteen*" is the name of a person and "*aciphex*" is the name of a drug.

▷ *Remark 30.4.16.* Character-level language models are good for this task because they can associate the character sequence "*ex* " with a drug name and "*steen*" with a person name, and thereby identify words that they have never seen before.

SOME RIGHTS RESERVED

N-Grams over Word Sequences

▷ **Idea:** n gram models apply to word sequences as well.

▷ **Problems:** The method works identically, but

1. There are many more words than characters. (100 vs. 10^5 in Englisch)
2. And what is a word anyways? (space/punctuation-delimited substrings?)
3. Data sparsity: we do not have enough data! For a language model for 10^5 words in English, we have 10^{15} trigrams.
4. Most training corpora do not have all words.

SOME RIGHTS RESERVED

Word N-Grams: Out-of-Vocab Words

▷ **Definition 30.4.17.** Out of vocabulary (OOV) words are unknown words that appear in the test corpus but not training corpus.

▷ *Remark 30.4.18.* OOV words are usually content words such as names and locations which contain information crucial to the success of NLP tasks.

▷ **Idea:** Model OOV words by

1. adding a new word token, e.g. <UNK> to the vocabulary,
2. in the training corpus, replacing the respective first occurrence of a previously unknown word by <UNK>,
3. counting n grams as usual, treating <UNK> as a regular word.

This trick can be refined if we have a word classifier, then use a new token per class, e.g. <EMAIL> or <NUM>.

SOME RIGHTS RESERVED

What can Word N-Gram Models do?

▷ **Example 30.4.19 (Test n-grams).** Build unigram, bigram, and trigram language models over the words [RN21], randomly sample sequences from the models.

1. Unigram: "*logical are as are confusion a may right tries agent goal the was ...*"
2. Bigram: "*systems are very similar computational approach would be represented ...*"
3. Trigram: "*planning and scheduling are integrated the success of naive bayes model ...*"

▷ **Clearly** there are differences, how can we measure them to evaluate the models?

▷ **Definition 30.4.20.** The perplexity of a sequence $\mathbf{c}_{1:N}$ is defined as

$$\text{Perplexity}(\mathbf{c}_{1:N}) := P(\mathbf{c}_{1:N})^{-1/N}$$

▷ **Intuition:** The reciprocal of probability, normalized by sequence length.

▷ **Example 30.4.21.** For a language with n characters or words and a language model that predicts that all are equally likely, the perplexity of any sequence is n.

If some characters or words are more likely than others, and the model reflects that, then the perplexity of correct sequences will be less than n.

▷ **Example 30.4.22.** In Example 30.4.19, the perplexity was 891 for the unigram model, 142 for the bigram model and 91 for the trigram model.

SOME RIGHTS RESERVED

30.5 Part of Speech Tagging

Language Models and Generalization

▷ **Recall:** n-grams can predict that a word sequence like "*a black cat*" is more likely than "*cat black a*". (as trigram 1. appears 0.000014% in a corpus and 2. never)

▷ **Native Speakers However:** Will tell you that "*a black cat*" matches a familiar pattern: article-adjective-noun, while "*cat black a*" does not!

▷ **Example 30.5.1.** Consider "*the fulvous kitten*" a native speaker reasons that it

- ▷ matches the determiner-adjective-noun pattern
- ▷ "*fulvous*" ($\widehat{=}$ brownish yellow) ends in "*ous*" $\rightsquigarrow$ adjective

So by generalization this is (probably) correct English.

▷ **Observation:** The order of syntactical categories of words plays a role in English!

▷ **Problem:** How can we compute them? (up next)

SOME RIGHTS RESERVED

Part-of-Speech Tagging

▷ **Definition 30.5.2.** Part-of-speech tagging (also POS tagging, POST, or grammatical tagging) is the process of marking up a word in corpus with tags (called POS tags) as corresponding to a particular part of speech (a category of words with similar syntactic properties) based on both its definition and its context.

▷ **Example 30.5.3.** A sentence tagged with POS tags from the Penn treebank: (see below)

From	the	start	,	it	took	a	person	with	great	qualities	to	succeed
IN	DT	NN	,	PRP	VBD	DT	NN	IN	JJ	NNS	TO	VB

1. "*From*" is tagged as a preposition (IN)
2. "*the*" as a determiner (DT)
3. . . .

▷ **Observation:** Even though POS tagging is uninteresting in its own right, it is useful as a first step in many NLP tasks.

▷ **Example 30.5.4.** In text-to-speech synthesis, a POS tag of "noun" for "*record*" helps determine the correct pronunciation (as opposed to the tag "verb")

SOME RIGHTS RESERVED

The Penn Treebank POS tags

▷ **Example 30.5.5.** The following 45 POS tags are used by the Penn treebank:

Tag	Word	Description	Tag	Word	Description
CC	*and*	Coordinating conjunction	PRP$	*your*	Possessive pronoun
CD	*three*	Cardinal number	RB	*quickly*	Adverb
DT	*the*	Determiner	RBR	*quicker*	Adverb, comparative
EX	*there*	Existential there	RBS	*quickest*	Adverb, superlative
FW	*per se*	Foreign word	RP	*off*	Particle
IN	*of*	Preposition	SYM	+	Symbol
JJ	*purple*	Adjective	TO	*to*	to
JJR	*better*	Adjective, comparative	UH	*eureka*	Interjection
JJS	*best*	Adjective, superlative	VB	*talk*	Verb, base form
LS	*1*	List item marker	VBD	*talked*	Verb, past tense
MD	*should*	Modal	VBG	*talking*	Verb, gerund
NN	*kitten*	Noun, singular or mass	VBN	*talked*	Verb, past participle
NNS	*kittens*	Noun, plural	VBP	*talk*	Verb, non-3rd-sing
NNP	*Ali*	Proper noun, singular	VBZ	*talks*	Verb, 3rd-sing
NNPS	*Fords*	Proper noun, plural	WDT	*which*	Wh-determiner
PDT	*all*	Predeterminer	WP	*who*	Wh-pronoun
POS	*'s*	Possessive ending	WP$	*whose*	Possessive wh-pronoun
PRP	*you*	Personal pronoun	WRB	*where*	Wh-adverb
$	$	Dollar sign	#	#	Pound sign
"	`	Left quote	"	'	Right quote
(	[	Left parenthesis	)	]	Right parenthesis
,	,	Comma	.	!	Sentence end
:	;	Mid-sentence punctuation			

SOME RIGHTS RESERVED

Computing Part of Speech Tags

▷ **Idea:** Treat the POS tags in a sentence as state variables $\mathbf{C}_{1:n}$ in a HMM: the words are the evidence variables $\mathbf{W}_{1:n}$, use prediction for POS tagging.

▷ The HMM is a generative model that

- ▷ starts in the tag predicted by the prior probability (usually IN) (problematic!)
- ▷ and then, for each step makes two choices:
  - ▷ what word – e.g. "*From*" – should be emitted
  - ▷ what state – e.g. DT – should come next

▷ **This works, but** there are problems:

- ▷ The HMM does not consider context other than the current state. (Markov property)
- ▷ It does not have any idea what the sentence is trying to convey.

▷ **Idea:** Use the Viterbi algorithm to find the most probable sequence of hidden states (POS tags).

▷ POS taggers based on the Viterbi algorithm can reach an F_1 score of up to 97%.

SOME RIGHTS RESERVED

The Viterbi algorithm for POS tagging – Details

▷ We need a transition model $P(\mathrm{C}_t \mid \mathrm{C}_{t-1})$: the probability of one POS tag following another.

▷ **Example 30.5.6.** $P(\mathrm{C}_t = VB \mid \mathrm{C}_{t-1} = MD) = 0.8$ means that given a modal verb (e.g. "*would*") the following word is a verb (e.g. "*think*") with probability 0.8.

▷ **Question:** Where does the number 0.8 come from?

▷ **Answer:** From counts in the corpus – with appropriate smoothing!
There are 13124 instances of MD in the Penn treebank and 10471 are followed by a VB.

▷ For the sensor model $P(\mathrm{W}_t = would \mid \mathrm{C}_t = MD) = 0.1$ means that if we choose a modal verb, we will choose "*would*" 10% of the time.

▷ These numbers also come from the corpus with appropriate smoothing.

▷ **Limitations:** HMM models only know about the transition and sensor models
In particular, we cannot take into account that e.g. words ending in "*ous*" are likely adjectives.

▷ We will see methods based on neural networks later.

SOME RIGHTS RESERVED

30.6 Text Classification

Text Classification as a NLP Task

▷ **Problem:** Often we want to (ideally) automatically see who can best deal with a given

document (e.g. e-mails in customer service)

▷ **Definition 30.6.1.** Given a set of categories the task of deciding which one a given document belongs to is called text classification or categorization.

▷ **Example 30.6.2.** Language identification and genre classification are examples of text classification.

▷ **Example 30.6.3.** Sentiment analysis – classifying a product review as positive or negative.

▷ **Example 30.6.4.** Spam detection – classifying an email message as spam or ham (i.e. non-spam).

SOME RIGHTS RESERVED

Spam Detection

▷ **Definition 30.6.5.** Spam detection – classifying an email message as spam or ham (i.e. non-spam)

▷ **General Idea:** Use NLP/machine learning techniques to learn the categories.

▷ **Example 30.6.6.** We have lots of examples of spam/ham, e.g.

Spam (from my spam folder)	Ham (in my inbox)
Wholesale Fashion Watches -57% today. Designer watches for cheap ...	The practical significance of hypertree width in identifying more ...
You can buy ViagraFr$1.85 All Medications at unbeatable prices! ...	Abstract: We will motivate the problem of social identity clustering: ...
WE CAN TREAT ANYTHING YOU SUFFER FROM JUST TRUST US ...	Good to see you my friend. Hey Peter, It was good to hear from you. ...
Sta.rt earn*ing the salary yo,u d-eserve by o'btaining the prope,r crede'ntials!	PDS implies convexity of the resulting optimization problem (Kernel Ridge ...

▷ **Specifically:** What are good features to classify e-mails by?

- ▷ n-grams like "*for cheap*" and "*You can buy*" indicate spam (but also occur in ham)
- ▷ character-level features: capitalization, punctuation (e.g. in "*yo,u d-eserve*")

▷ **Note:** We have two complementary ways of talking about classification: (up next)

- ▷ using language models
- ▷ using machine learning

SOME RIGHTS RESERVED

Spam Detection as Language Modeling

▷ **Idea:** Define two n-gram language models:

1. one for $\mathbf{P}(\text{Message}|\text{spam})$ by training on the spam folder
2. one for $\mathbf{P}(\text{Message}|\text{ham})$ by training on the inbox

Then we can classify a new message m with an application of Bayes' rule:

$$\underset{c\in\{\text{spam},\text{ham}\}}{\operatorname{argmax}}(P(c \mid m)) = \underset{c\in\{\text{spam},\text{ham}\}}{\operatorname{argmax}}(P(m \mid c)P(c))$$

where $P(c)$ is estimated just by counting the total number of spam and ham messages.

▷ This approach works well for spam detection, just as it did for language identification.

SOME RIGHTS RESERVED

Classifier Success Measures: Precision, Recall, and F_1 score

▷ We need a way to measure success in classification tasks.

▷ **Definition 30.6.7.** Let $f_C\colon S \to \mathbb{B}$ be a binary classifier for a class $C \subseteq S$, then we call $a \in S$ with $f_C(a) = \mathsf{T}$ a false positive, iff $a \notin C$ and $f_C(a) = \mathsf{F}$ a false negative, iff $a \in C$. False positives and negatives are erros of f_C. True positives and negatives occur when f_C correctly indicates actual membership in S.

▷ **Definition 30.6.8.** The precision of f_C is defined as $\frac{\#(TP)}{\#(TP)+\#(FP)}$ and the recall is $\frac{\#(TP)}{\#(TP)+\#(FN)}$, where TP is the set of true positives and $FN/FP$the sets of false negatives and false positives of f_C.

▷ **Intuitively** these measure the rates of:

- ▷ true positives in class C. (precision high, iff few false positives)
- ▷ true positives in ${f_C}^{-1}(\mathsf{T})$. (recall high, iff few true positives forgotten, i.e. few false negatives)

▷ **Definition 30.6.9.** The F_1 score combines precision and recall into a single number: (harmonic mean)

$$2\frac{\text{precision}\cdot\text{recall}}{(\text{precision}+\text{recall})}$$

▷ **Observation:** Classifiers try to reach precision and recall $\rightsquigarrow$ F_1 score of 1.

- ▷ if that is impossible, compromize on one $\rightsquigarrow$ F_β score . (application-dependent)
- ▷ The F_β score generalizes the F_1 score by weighing the precision β times as important as recall.

SOME RIGHTS RESERVED

30.7 Information Retrieval

Information Retrieval

▷ **Definition 30.7.1.** An information need is an individual or group's desire to locate and obtain information to satisfy a conscious or unconscious need.

▷ **Definition 30.7.2.** An information object is medium that is mainly used for its information

content.

▷ **Definition 30.7.3.** Information retrieval (IR) deals with the representation, organization, storage, and maintenance of information objects that provide users with easy access to the relevant information and satisfy their various information needs.

▷ **Observation (Hjørland 1997):** Information need is closely related to relevance: If something is relevant for a person in relation to a given task, we might say that the person needs the information for that task.

▷ **Definition 30.7.4.** Relevance denotes how well an information object meets the information need of the user. Relevance may include concerns such as timeliness, authority or novelty of the object.

▷ **Observation:** We normally come in contact with IR in the form of web search.

▷ **Definition 30.7.5.** Web search is a fully automatic process that responds to a user query by returning a sorted document list relevant to the user requirements expressed in the query.

▷ **Example 30.7.6.** Google and Bing are web search engines, their query is a bag of words and documents are web pages, PDFs, images, videos, shopping portals.

SOME RIGHTS RESERVED

Vector Space Models for IR

▷ **Idea:** For web search, we usually represent documents and queries as bags of words over a fixed vocabulary V. Given a query Q, we return all documents that are "similar".

▷ **Definition 30.7.7.** Given a vocabulary V, a word $w \in V$, and a document d, then we define the raw term frequency (often just called the term frequency) of w in d as the number of occurrences of w in d.

▷ **Definition 30.7.8.** A multiset of words in $V = \{t_1, \ldots, t_n\}$ is called a bag of words (BOW), and can be represented as a word frequency vectors in $\mathbb{N}^{|V|}$: the vector of raw word frequencies.

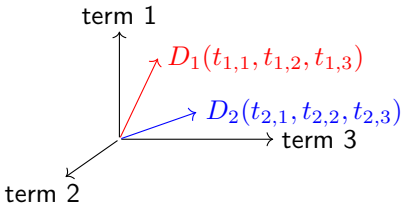
▷ **Example 30.7.9.** If we have two documents: $d_1 =$ "*Have a good day!*" and $d_2 =$ "*Have a great day!*", then we can use $V =$ "*Have*", "*a*", "*good*", "*great*", "*day*" and can represent "*good*" as $\langle 0, 0, 1, 0, 0\rangle$, "*great*" as $\langle 0, 0, 0, 1, 0\rangle$, and d_1 a $\langle 1, 1, 1, 0, 1\rangle$.

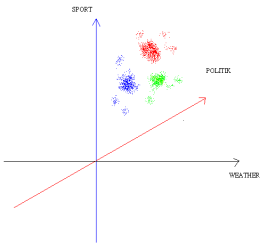
Words outside the vocabulary are ignored in the BOW approach. So the document $d_3 =$ "*What a day, a good day*" is represented as $\langle 0, 2, 1, 0, 2\rangle$.

SOME RIGHTS RESERVED

Vector Space Models for IR

▷ **Idea:** Query and document are similar, iff the angle between their word frequency vectors is small.





▷ **Lemma 30.7.10 (Euclidean Dot Product Formula).** $A \cdot B = \|A\|_2 \|B\|_2 \cos\theta$, *where* θ *is the angle between* A *and* B.

▷ **Definition 30.7.11.** The cosine similarity of A and B is $\cos\theta = \frac{A \cdot B}{\|A\|_2 \|B\|_2}$.

SOME RIGHTS RESERVED

TFIDF: Term Frequency/Inverse Document Frequency

▷ **Problem:** Word frequency vectors treat all the words equally.

▷ **Example 30.7.12.** In an query "*the brown cow*", the "*the*" is less important than "*brown cow*". (because "*the*" is less specific)

▷ **Idea:** Introduce a weighting factor for the word frequency vector that de-emphasizes the dimension of the more (globally) frequent words.

▷ **Technically:** We need to normalize the word frequency vectors first:

▷ **Definition 30.7.13.** Given a document d and a vocabulary word $t \in V$, the normalized term frequency (confusingly often called just term frequency) $\mathrm{tf}(t,d)$ is the raw term frequency divided by $|d|$.

▷ **Definition 30.7.14.** Given a document collection $D = \{d_1, \ldots, d_N\}$ and a word t the inverse document frequency is given by $\mathrm{idf}(t,D) := \log_{10}\left(\frac{N}{|\{d \in D \mid t \in d\}|}\right)$.

▷ **Definition 30.7.15.** We define $\mathrm{tfidf}(t,d,D) := \mathrm{tf}(t,d) \cdot \mathrm{idf}(t,D)$.

▷ **Idea:** Use the tfidf-vector with cosine similarity for information retrieval instead.

▷ **Definition 30.7.16.** Let D be a document collection with vocabulary $V = \{t_1, \ldots, t_{|V|}\}$, then the tfidf-vector $\overline{\mathrm{tfidf}}(d,D) \in \mathbb{N}^{|V|}$ is defined by $\overline{\mathrm{tfidf}}(d,D)_i := \mathrm{tfidf}(t_i,d,D)$.

▷ **Definition 30.7.17.** tfidf is one of the standard methods for information retrieval nowadays. (but more is needed)

SOME RIGHTS RESERVED

TF-IDF Example

▷ Let $D := \{d_1, d_2\}$ be a document corpus over the vocabulary

$$V = \{\text{“}this\text{”}, \text{“}is\text{”}, \text{“}a\text{”}, \text{“}sample\text{”}, \text{“}another\text{”}, \text{“}example\text{”}\}$$

with word frequency vectors $\langle 1, 1, 1, 2, 0, 0\rangle$ and $\langle 1, 1, 0, 0, 2, 3\rangle$.

▷ Then we compute for the word “*this*”

- ▹ $\mathrm{tf}(\text{“}this\text{”}, d_1) = \frac{1}{5} = 0.2$ and $\mathrm{tf}(\text{“}this\text{”}, d_2) = \frac{1}{7} \approxeq 0.14$,
- ▹ idf is constant over D, we have $\mathrm{idf}(\text{“}this\text{”}, D) = \log_{10}(\frac{2}{2}) = 0$,
- ▹ thus $\mathrm{tfidf}(\text{“}this\text{”}, d_1, D) = 0 = \mathrm{tfidf}(\text{“}this\text{”}, d_2, D)$. (“*this*” occurs in both)

▷ The word “*example*” is more interesting, since it occurs only in d_2 (thrice)

- ▹ $\mathrm{tf}(\text{“}example\text{”}, d_1) = \frac{0}{5} = 0$ and $\mathrm{tf}(\text{“}example\text{”}, d_2) = \frac{3}{7} \approxeq 0.429$.
- ▹ $\mathrm{idf}(\text{“}example\text{”}, D) = \log_{10}(\frac{2}{1}) \approxeq 0.301$,
- ▹ thus $\mathrm{tfidf}(\text{“}example\text{”}, d_1, D) = 0 \cdot 0.301 = 0$ and $\mathrm{tfidf}(\text{“}example\text{”}, d_2, D) \approxeq 0.429 \cdot 0.301 = 0.129$.

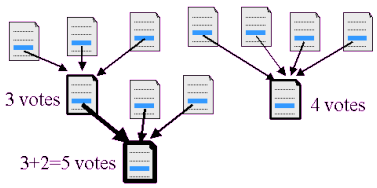
SOME RIGHTS RESERVED

Once an answer set has been determined, the results have to be sorted, so that they can be presented to the user. As the user has a limited attention span – users will look at most at three to eight results before refining a query, it is important to rank the results, so that the hits that contain information relevant to the user's information need early. This is a very difficult problem, as it involves guessing the intentions and information context of users, to which the search engine has no access.

Ranking Search Hits: e.g. Google's Page Rank

▷ **Problem:** There are many hits, need to sort them. (e.g. by importance)

▷ **Idea:** A web site is important, . . . if many other hyperlink to it.



▷ **Refinement:** . . . , if many important web pages hyperlink to it.

▷ **Definition 30.7.18.** Let A be a web page that is hyperlinked from web pages $S_1, \ldots, S_n$, then the page rank PR of A is defined as

$$\mathrm{PR}(A) = 1 - d + d\left(\frac{\mathrm{PR}(S_1)}{C(S_1)} + \cdots + \frac{\mathrm{PR}(S_n)}{C(S_n)}\right)$$

where $C(W)$ is the number of links in a page W and $d = 0.85$.

▷ *Remark 30.7.19.* PR(A) is the probability of reaching A by random browsing.

SOME RIGHTS RESERVED

Getting the ranking right is a determining factor for success of a search engine. In fact, the early of Google was based on the pagerank algorithm discussed above (and the fact that they figured out a revenue stream using text ads to monetize searches).

30.8 Information Extraction

Information Extraction

▷ **Definition 30.8.1.** Information extraction is the process of acquiring information by skimming a text and looking for occurrences of a particular class of object and for relationships among objects.

▷ **Example 30.8.2.** Extracting instances of addresses from web pages, with attributes for street, city, state, and zip code;

▷ **Example 30.8.3.** Extracting instances of storms from weather reports, with attributes for temperature, wind speed, and precipitation.

▷ **Observation:** In a limited domain, this can be done with high accuracy.

SOME RIGHTS RESERVED

Attribute-Based Information Extraction

▷ **Definition 30.8.4.** In attribute-based information extraction we assume that the text refers to a single object and the task is to extract a factored representation.

▷ **Example 30.8.5 (Computer Prices).** Extracting from the text "*IBM ThinkBook 970. Our price: $399.00*" the attribute-based representation
\{Manufacturer=IBM, Model=ThinkBook970,Price=$399.00\}.

▷ **Idea:** Try a template-based approach for each attribute.

▷ **Definition 30.8.6.** A template is a finite automaton that recognizes the information to be extracted. The template often consists of three sub-automata per attribute: the prefix pattern followed by the target pattern (it matches the attribute value) and the postfix pattern.

▷ **Example 30.8.7 (Extracing Prices with Regular Expressions).**
When we want to extract computer price information, we could use regular expressions for the automata, concretely, the

- ▷ prefix pattern: .∗price[:]?
- ▷ target pattern: [$][0−9]+([.][0−9][0−9])?
- ▷ postfix pattern: + shipping|

▷ **Alternative:** Take all the target matches and choose among them.

▷ **Example 30.8.8.** For "*List price $99.00, special sale price $78.00, shipping $3.00.*" take the lowest price that is within 50% of the highest price. ⇝ "*$78.00*"

SOME RIGHTS RESERVED

Relational Information Extraction

▷ **Question:** Can we also do structured representations?

▷ **Answer:** That is the next step up from attribute-based information extraction.

▷ **Definition 30.8.9.** The task of a relational extraction system is to extract multiple objects and the relationships among them from a text.

▷ **Example 30.8.10.** When these systems see the text "*$249.99,*" they need to determine not just that it is a price, but also which object has that price.

▷ **Example 30.8.11.** FASTUS is a typical relational extraction system, which handles news stories about corporate mergers and acquisitions. It can read the story

> Bridgestone Sports Co. said Friday it has set up a joint venture in Taiwan with a local concern and a Japanese trading house to produce golf clubs to be shipped to Japan.

and extract the relations:

$$e \in JointVentures \wedge Product(e,"golfclubs") \wedge Date(e,"Friday")$$

$$Member(e,"BridgestoneSportsCo") \wedge Member(e,"alocalconcern")$$

$$Member(e,"aJapanesetradinghouse")$$

SOME RIGHTS RESERVED

30.9 Grammar

Phrase Structure Grammars (Motivation)

▷ **Problem Recap:** We do not have enough text data to build word sequence language models ⇜ data sparsity.

▷ **Idea:** Categorize words into classes and then generalize "acceptable word sequences" into "acceptable word class sequences" ⇝ phrase structure grammars.

▷ **Advantage:** We can get by with much less information.

▷ **Example 30.9.1 (Generative Capacity).** 10^3 structural rules over a lexicon of 10^5 words generate most German sentences.

▷ Vervet monkeys, antelopes etc. use isolated symbols for sentences.
⇝ restricted set of communicable propositions, no generative capacity.

▷ **Disadvantage:** Grammars may over-generate or under-generate.

▷ The formal study of grammars was introduced by Noam Chomsky in 1957 [Cho65].

SOME RIGHTS RESERVED

We fortify our intuition about these – admittedly very abstract – constructions by an example and introduce some more vocabulary.

Phrase Structure Grammars (cont.)

▷ **Example 30.9.2.** A simple phrase structure grammar G:

S	::=	$NP\ Vi$	S	::=	$NP\ Vt\ NP$
NP	::=	$Det\ N$	NP	::=	$Name \mid Pron$
Det	::=	**the** | **a** | **each** | ...	$Name$	::=	**Peter** | **Mary** | ...
N	::=	**dog** | **teacher** | ...	Vt	::=	**loved** | **killed** | ...
Vi	::=	**slept** | **snored** | ...	$Pron$	::=	**he** | **she** | **it** | ...

Here S, is the start symbol, NP, Det, N, and Vi are nonterminals.

▷ **Definition 30.9.3.** A production rule whose head is a single non-terminal and whose body consists of a single terminal is called lexical or a lexical insertion rule.

Definition 30.9.4. The subset of lexical rules of a grammar G is called the lexicon of G and the set of body symbols the vocabulary (or alphabet). The nonterminals in their heads are called lexical categories of G.

▷ **Note:** Given that production rules whose right hand side is a BNF alternative of terminals can be expanded into lexical rules, we consider them as lexical as well. (the last three lines of G above)

▷ **Definition 30.9.5.** The non-lexicon production rules are called structural, and the nonterminals in the heads are called phrasal or syntactic categories.

SOME RIGHTS RESERVED

In the grammar G from Example 30.9.2 we only use the past tense forms of the (transitive and intransitive) verbs since it minimizes the need for inflection to keep the grammar very simple.

But not only are parse trees more informative to human readers, they also guide our intuition for parsing algorithms as we will see now.

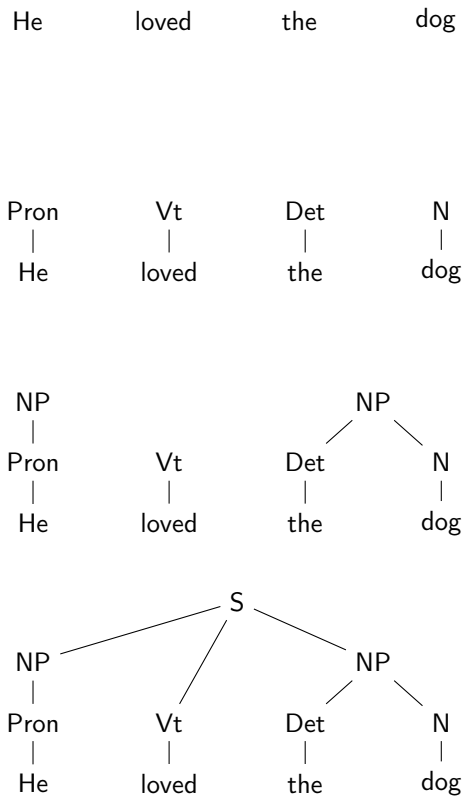
Context-Free Parsing

▷ **Recall:** The sentences accepted by a grammar are those the start symbol can be rewritten into.

▷ **Definition 30.9.6.** Top-down parsing algorithms work by systematically searching the space of derivation (or parse trees) from the start symbol (downwards).

▷ **Definition 30.9.7.** Bottom-up parsing algorithms work by replacing any substring that matches the body of a production rule with its head.

▷ **Example 30.9.8.** Using the grammar from Example 30.9.2, we get the following parse trees in bottom-up parsing:



(Traditional linear notation: [S [NP [Pron He]] [Vt loved] [NP [Det the] [N dog]]])

▷ Bottom-up parsing algorithms tend to be more efficient than top-down ones.

SOME RIGHTS RESERVED

Context-Free Parsing

▷ Bottom-up parsing algorithms tend to be more efficient than top-down ones.

▷ Efficient context-free parsing algorithms run in $\mathcal{O}(n^3)$, run at several thousand words/second for real grammars.

▷ **Theorem 30.9.9.** *Context-free parsing* $\widehat{=}$ *Boolean matrix multiplication!*

▷ $\rightsquigarrow$ unlikely to find faster practical algorithms. (details in [Lee02])

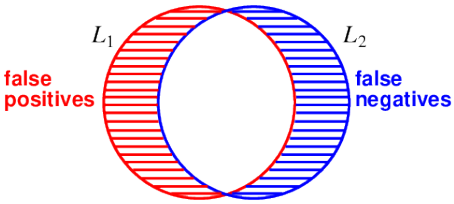
 SOME RIGHTS RESERVED

We now come to a problem that is common to all natural languages: grammaticality is not easily formalized by grammars – even though we know a lot about their syntactic structure, the set of sentences perceived as grammatical by native speakers is not sufficiently regular to be described by a small set of rules.

Grammaticality Judgments

▷ **Problem:** The formal language $\mathbf{L}(G)$ accepted by a grammar G may differ from the natural language L_n it supposedly models.

▷ **Definition 30.9.10.** We say that a grammar G over-generates, iff it accepts strings outside of L_n (false positives) and under-generates, iff there are L_n strings (false negatives) that $\mathbf{L}(G)$ does not accept.



▷ Adjusting $\mathbf{L}(G)$ to agree with L_n is an !

- ▷ * "*the gold grab the wumpus*"
- ▷ * "*I smell the wumpus the gold*"
- ▷ "*I give the wumpus the gold*"
- ▷ * "*I donate the wumpus the gold*"

▷ Intersubjective agreement somewhat reliable, independent of semantics!

▷ Real grammars (100–5000 rules) are insufficient even for "proper" English.

 SOME RIGHTS RESERVED

Probabilistic, Context-Free Grammars

▷ **Recall:** We introduced grammars as an efficient substitute for language models.

▷ **Problem (Poor Subsitute):** Grammars are deterministic language models.

▷ **Idea:** Add a probabilistic component to grammars.

▷ **Definition 30.9.11.** A probabilistic context-free grammar (PCFG) is a phrase structure grammar, where every production rule is associated with a probability. For a production rule $h \to b$ with probability p we write $\mathbf{h} \to \mathbf{b}[p]$.

▷ **Idea:** A PCFG induces a language model by assigning probabilities to its sentences.

▷ **Definition 30.9.12.** Let G be a PCFG, S a sentence of G, and D a G-derivation of S, then the probability of D is the product of the probabilites of the production rules in all steps of D. The probability of S is the sum of the probabilities of all its derivations.

SOME RIGHTS RESERVED

Example: The Wumpus Grammar (Lexicon)

▷ **Example 30.9.13 (Wumpus Grammar Lexicon).**

Noun	→	**stench**[.05] | **breeze**[.01] | **wumpus**[.15] | **pits**[.05] | ...
Verb	→	**is**[.1] | **feel**[.1] | **smells**[.05] | **stinks**[.05] | ...
TransVerb	→	**see**[.1] | **shoot**[.1] | ...
Adjective	→	**right**[.1] | **dead**[.05] | **smelly**[.02] | **breezy**[.02] | ...
Adverb	→	**here**[.05] | **ahead**[.05] | **nearby**[.02] | ...
Pronoun	→	**me**[.1] | **you**[.03] | **I**[.1] | **it**[.1] | ...
RelPron	→	**that**[.4] | **which**[.15] | **who**[.2] | **whom**[.02] | ...
Name	→	**John**[.01] | **Mary**[.01] | **Boston**[.01] | ...
Det	→	**the**[.4] | **a**[.3] | **an**[.1] | **every**[.05] | ...
Preposition	→	**to**[.2] | **in**[.1] | **on**[.05] | **near**[.1] | ...
Conjunction	→	**and**[.5] | **or**[.1] | **but**[.2] | **yet**[.2] | ...
Digit	→	**0**[.2] | **1**[.2] | **2**[.2] | **3**[.2] | **4**[.2] | **5**[.2] | ...

Divided into closed and open classes

SOME RIGHTS RESERVED

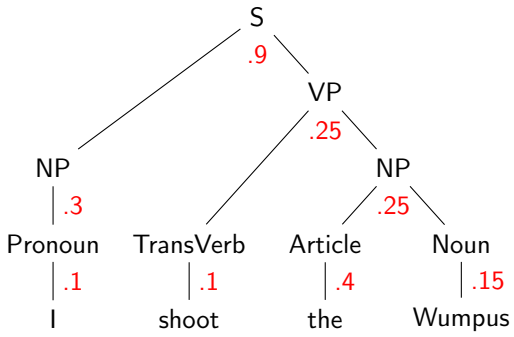
Wumpus Grammar

▷

S	→	*NP VP*	[.9]	I + feel a breeze
	|	*S Conjunction S*	[.1]	I feel a breeze + and + I smell a wumpus
NP	→	*Pronoun*	[.3]	I
	|	*Name*	[.1]	John
	|	*Noun*	[.1]	pits
	|	*Det Noun*	[.25]	the + wumpus
	|	*Det Adjs Noun*	[.05]	the + smelly dead + wumpus
	|	*Digit Digit*	[.05]	3 4
	|	*NP PP*	[.1]	the wumpus + in 1 3
	|	*NP RelClause*	[.05]	the wumpus + that is smelly
VP	→	*Verb*	[.25]	stinks
	|	*TransVerb NP*	[.25]	see + the Wumpus
	|	*VP NP*	[.25]	feel + a breeze
	|	*VP Adjective*	[.05]	is + smelly
	|	*VP PP*	[.1]	turn + to the east
	|	*VP Adverb*	[.1]	go + ahead
Adjs	→	*Adjective*	[.8]	smelly
	|	*Adjective Adjs*	[.2]	smelly + dead
PP	→	*Prep NP*	[1]	to + the east
RelClause	→	**that** *VP*	[1]	that + is smelly

PCFG Parsing

▷ **Example 30.9.14.** Reconsidering Example 30.9.8 with the Wumpus grammar above, we get the PCFG parse tree:



It has the probability $.9 \cdot .3 \cdot .1 \cdot .25 \cdot .1 \cdot .25 \cdot .4 \cdot .15 = 1.013 \times 10^{-5}$.

As this is the only derivation, this is also the probability of the sentence.

Learning PCFG Probabilities from Data

▷ **Recall:** A PCFG has many rules, each with a probability.

▷ **Problem:** Where do they come from?

▷ **Idea:** Learn/sample them from data. (but what data?)

▷ **Definition 30.9.15.** A treebank is a parsed text corpus that annotates syntactic or semantic sentence structure.

▷ **Idea:** To learn the probability for the rule $R := H \to B$, look for parse subtrees that match R (i.e. with root category H) in the treebank.

If has N subtrees and n match R, then annotate R with probability n/N.

▷ Treebanks have revolutionized practical linguistics.

▷ There are many treebanks: `https://en.wikipedia.org/wiki/Treebank`

- ▷ multiple languages (from Abaza to Yoruba)
- ▷ multiple language types (newswire to spontaneous speech)

▷ **Outlook:** Treebank less grammar learning (rules and probabilitiess) is possible, but much more difficult.

The Penn Treebank

▷ **Definition 30.9.16.** The Penn treebank [MMS93] is a treebank of newswire texts (nearly 5 million words) annotated with part of speech and parse tree structure, using human labor assisted by some automated tools.

▷ **Example 30.9.17.** A tree from the Penn treebank for the sentence "*Her eyes were glazed as if she didn't hear or even see him.*":

```
[ [S [NP−SBJ−2 Her eyes]
     [VP were
         [VP glazed
             [NP ∗−2]
             [SBAR−ADV as if
                 [S [NP−SBJ she]
                     [VP did n't
                         [VP [VP hear [NP ∗−1]]
                             or
                             [VP [ADVP even] see [NP ∗−1]]
                             [NP−1 him]]]]]]]
 .]
```

Note: two S-rooted subtrees, one with NP−SBJ−2 child and one with NP SBJ.

SOME RIGHTS RESERVED

Advertisement: Logic-Based Natural Language Semantics

▷ **Advanced Course:** "Logic-Based Natural Language Semantics" (next semester)

- ▷ 2 × 90 min lecture/week (expected: ≤ 10 (active) Students)

▷ **Contents:**

- ▷ Foundations of Natural Language Semantics (NLS)
- ▷ Montague's Method of Fragments (Grammar, semantics construction, Logic)
- ▷ Implementing Fragments in GLF (Grammatical Framework (GF) and MMT)
- ▷ Inference Systems for Natural Language Pragmatics (tableau machine)
- ▷ Advanced logical systems for NLS. (modal, higher-order, dynamic Logics)

▷ **Grading:** 60 min exam in early October.

▷ **Course Intent:** Groom students for bachelor/master theses and as KWARC research assistants.

SOME RIGHTS RESERVED

Chapter 31

Deep Learning for NLP

Deep Learning for NLP: Agenda

▷ **Observation:** Symbolic and statistical systems have demonstrated success on many NLP tasks, but their performance is limited by the endless complexity of natural language.

▷ **Idea:** Given the vast amount of text in machine-readable form, can data-driven machine-learning based approaches do better?

▷ In this chapter, we explore this idea, using – and extending – the methods from ???.

▷ Overview:

1. Word embeddings
2. Recurrent neural networks for NLP
3. Sequence-to-sequence models
4. Transformer Architecture
5. Pretraining and transfer learning.

SOME RIGHTS RESERVED

31.1 Word Embeddings

Word Embeddings

▷ **Problem:** For ML methods in NLP, we need numerical data. (not words)

▷ **Idea:** Embed words or word sequences into real vector spaces.

▷ **Definition 31.1.1.** A word embedding is a mapping from words in context into a real vector space $\mathbb{R}^n$ used for natural language processing.

▷ **Definition 31.1.2.** A vector is called one hot, iff all components are 0 except for one 1. We call a word embedding one hot, iff all of its vectors are.

One hot word embeddings are rarely used for actual tasks, but often used as a *starting point* for better word embeddings.

▷ **Example 31.1.3 (Vector Space Methods in Information Retrieval).**
Word frequency vectors are induced by adding up one hot word embeddings.

▷ **Example 31.1.4.** Given a corpus D – the context – the tfidf word embedding is given by $\text{tfidf}(t,d,D) := \text{tf}(t,d) \cdot \log_{10}(\frac{|D|}{|\{d \in D \mid t \in d\}|})$, where $\text{tf}(t,d)$ is the term frequency of word t in document d.

▷ **Intuition behind these two:** Words that occur in similar documents are similar.

SOME RIGHTS RESERVED

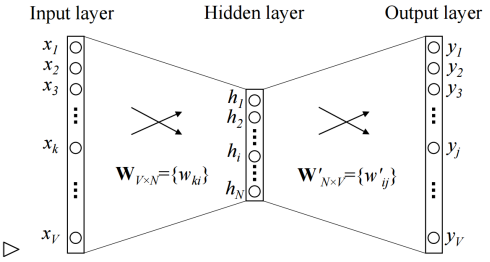
Word2Vec

▷ **Definition 31.1.5 (Idea).** Use *feature extraction* to map words to vectors in $\mathbb{R}^N$:

Train a neural network on a "dummy task", throw away the output layer, use the previous layer's output (of size N) as the word embedding

▷ **First Attempt: Dimensionality Reduction:** Train to predict the original one hot vector:

- ▷ For a vocabulary size V, train a network with a single hidden layer; i.e. three layers of sizes (V, N, V). The first two layers will compute our embeddings.
- ▷ Feed the one hot encoded input word into the network, and train it on the one hot vector itself, using a softmax activation function at the output layer.



▷

▷ **Definition 31.1.6.** The softmax function normalizes a vector $\langle z_1, \ldots, z_k \rangle$ of raw scores into a probability distribution: $\sigma(z)_i := e^{z_i} / (\sum_{j=1}^{k} e^{z_j})$ for $i \in \{z_1, \ldots, z_k\}$.

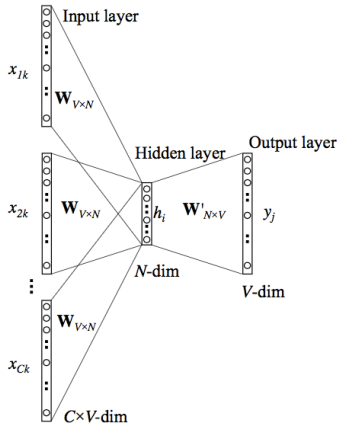
SOME RIGHTS RESERVED

Word2Vec: The Continuous Bag Of Words (CBOW) Algorithm

▷ **Distributional Semantics:** "a word is characterized by the company it keeps".

▷ **Better Idea:** Predict a word from its context:

- ▷ For a context window size n, take all sequences of $2n+1$ words in our corpus (e.g. "*the brown cow jumps over the moon*" for $n=3$) as training data. We call the word at the center ("*jumps*") the target word, and the remaining words the context words.
- ▷ For every such sentence, pass all context words (one hot encoded) through the first layer of the network, yielding $C := 2n$ vectors.
- ▷ Pass their average into the output layer (the average pooling layer) with a softmax activation function, and train the network to predict the target word. (sum pooling also works)
- ▷ The parameter N (the size of the hidden vector) can be chosen freely.



SOME RIGHTS RESERVED

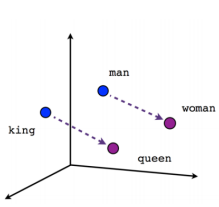
Properties

▷ Word embeddings like CBOW have interesting properties:

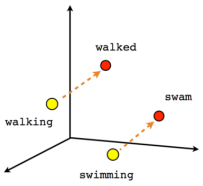
- ▷ *Similarity*: Using e.g. cosine similarity to compare vectors, we can find words with similar meanings.
- ▷ Semantic and syntactic relationships emerge as arithmetic relations:

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

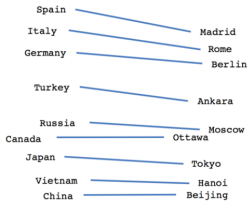
$$\text{germany} - \text{country} + \text{capital} \approx \text{berlin}$$



Male-Female



Verb tense



Country-Capital

SOME RIGHTS RESERVED

Common Word Embeddings

▷ **Observation:** Word embeddings are crucial as first steps in any NN-based NLP methods.

▷ In practice it is often sufficient to use generic, pretrained word embeddings

▷ **Definition 31.1.7.** Common pretrained – i.e. trained for generic NLP applications word

embeddings include

- ▷ Word2vec: the original system that established the concept. (see above)
- ▷ GloVe (Global Vectors)
- ▷ FASTTEXT (embeddings for 157 languages)

▷ But we can also train our own word embedding (together with main task). (up next)

SOME RIGHTS RESERVED

Learning POS tags and Word embeddings simultaneously

▷ Specific word embeddings are trained on a carefully selected corpus and tend to emphasize the characteristics of the task.

▷ **Example 31.1.8.** POS tagging – even though simple – is a good but non-trivial example.

Recall that many words can have multiple POS tags, e.g. "*cut*" can be

- ▷ a present tense verb (transitive or intransitive)
- ▷ a past tense verb
- ▷ a infinitive verb
- ▷ a past participle
- ▷ an adjective
- ▷ a noun.

If a nearby temporal adverb refers to the past $\rightsquigarrow$ this occurrence may be a past tense verb.

▷ **Note:** CBOW treats all context words identically regardless of *order*, but in POS tagging the exact *positions* of the words matter.

SOME RIGHTS RESERVED

The POS/Embedding: Setup

▷ We use the following process for learning a POS tagger including word embeddings from a POS tagged corpus:

▷ **Example 31.1.9 (Preprocessing/Setup).**

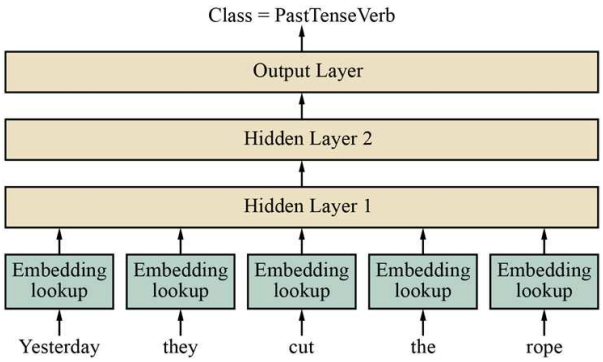
1. Choose a width w – an odd number of words for the prediction window.
   A choice of $w = 5$ means that the tag is predicted with a context of 2 words before/after the word.
2. Split every sentence in the corpus into prediction windows for each word; together with the tags these constitute the training examples.
3. Create a vocabulary V of the unique, sufficiently common word tokens in the corpus $v := |V|$.
4. Sort V in any order

5. Choose a value d for the size of the word embedding vector.
6. Create a new $v \times d$ matrix $\mathbf{E}$; the word embedding matrix has the word embedding of the ith word in V in row.
7. Initialize $\mathbf{E}$ randomly (or from pretrained vectors).

SOME RIGHTS RESERVED

The POS/Embedding: Network

▷ **Example 31.1.10.** The POS/Embedding network has the following form/layers:



1. The input to the network is a word sequence of length w. (here 5; but see below)
2. The first layer consists of w copies $\mathbf{E}$.
3. The second (hidden) layer z_1 with weight matrix $\mathbf{W}_1$ computes $z_1 = l(\mathbf{W}_1 x)$
4. The third (hidden) layer z_2 with weight matrix $\mathbf{W}_2$ computes $z_2 = l(\mathbf{W}_2 z_1)$
5. The output layer computes the probability distribution $y = \sigma(\mathbf{W}_{out} z_2)$ over the possible POS tags for the middle word with a weight matrix $\mathbf{W}_{out}$.

SOME RIGHTS RESERVED

The POS/Embedding: Computation

▷ To encode a word sequence $\mathbf{w}$ concatenate the encodings of each word $\rightsquigarrow$ input vector x of length wd.

▷ **Problem:** Every word in $\mathbf{w}$ will have the same encoding irrespective of its place in $\mathbf{w}$.

▷ **Answer:** But it will be treated differently in the first hidden layer (by a different part).

▷ Train the weights in $\mathbf{E}$, $\mathbf{W}_1$, $\mathbf{W}_2$, and $\mathbf{W}_{out}$ using gradient descent.

▷ **Example 31.1.11.** If all goes well in training, "*cut*" will be labeld as a past tense verb given the context which includes the temporal past word "*yesterday*".

▷ **Note:** Better *positional encoding* techniques exist (e.g. sinusoidal), but for fixed small context window sizes, this works well.

31.2 Recurrent Neural Networks

Recurrent Neural Networks in NLP

▷ Word embeddings give a good representation of words in isolation.

▷ But natural language consists of word sequences ⇜⇝ surrounding words provide context!

▷ For simple tasks like POS tagging, a fixed-size window of e.g. 5 words is sufficient.

▷ **Observation:** For advanced tasks like question answering we need more context!

▷ **Example 31.2.1.** In the sentence "*Eduardo told me that Miguel was very sick so I took him to the hospital*", the pronouns "*him*" refers to "*Miguel*" and not "*Eduardo*".(14 words of context)

▷ **Observation:** Language models with n-grams or n-word feed-forward networks have problems:
Either the context is too small or the model has too many parameters! (or both)

▷ **Observation:** Feed-forward networks N also have the problem of asymmetry: whatever N learns about a word w at position n, it has to relearn about w at position $m \neq n$.

▷ **Idea:** What about recurrent neural networks – nets with cycles? (up next)

RNNs for Time Series

▷ **Idea:** RNNs – neural networks with cycles – have memory

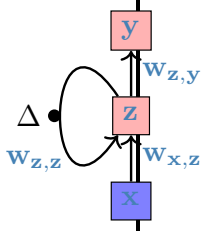
⇝ use that for more context in neural NLP.

▷ **Example 31.2.2 (A simple RNN).**

The RNN on the right has an input layer $\mathbf{x}$, a hidden layer $\mathbf{z}$ with recurrent connections and delay Δ, and an output layer $\mathbf{y}$ as shown on the right.
Defining equations for time step t:

$$\begin{aligned} \mathbf{z}_t &= \mathbf{g}_\mathbf{z}(\mathbf{w}_{\mathbf{z},\mathbf{z}}\mathbf{z}_{t-\Delta} + \mathbf{w}_{\mathbf{x},\mathbf{z}}\mathbf{x}_t) \\ \mathbf{y}_t &= \mathbf{g}_\mathbf{y}(\mathbf{w}_{\mathbf{z},\mathbf{y}}\mathbf{z}_t) \end{aligned}$$

where $\mathbf{g}_\mathbf{z}$ and $\mathbf{g}_\mathbf{y}$ are the activation functions for the hidden and output layers.



▷ **Intuition:** RNNs are a bit like HMMs and dynamic Bayesian Networks:

They make a Markov assumption:

▷ the hidden state $\mathbf{z}$ suffices to capture the input from all previous inputs.

▷ $\Delta \mathrel{\widehat{=}}$ the order of the HMM.

▷ **Side Benefit:** RNNs solve the asymmetry problem ↜ the $\mathbf{w}_{z,z}$ are the same at every step.

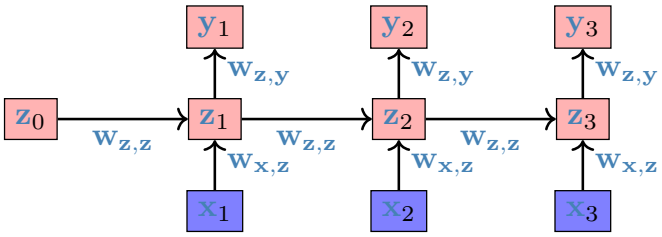
SOME RIGHTS RESERVED

Training RNNs for NLP

▷ **Idea:** For training, unroll an RNN into a feed-forward network ⇝ back-propagation.

▷ **Definition 31.2.3.** The back-propagation through time algorithm (BPTT) unrolls an RNN using the zero vector as initial value for training, while carefully maintaining the identity of $\mathbf{w}_{z,z}$ over all steps.

▷ **Example 31.2.4.** The RNN from Example 31.2.2 unrolled three times with $\Delta = 1$.



Problem: The weight matrices $\mathbf{w}_{x,z}$, $\mathbf{w}_{z,z}$, and $\mathbf{w}_{z,y}$ are shared over all time slices.

SOME RIGHTS RESERVED

Bidirectional RNN for more Context

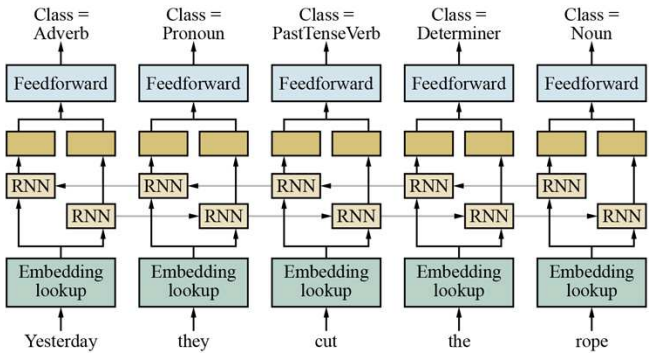
▷ **Observation:** RNNs only take left context – i.e. words before – into account, but we may also need right context – the words after.

▷ **Example 31.2.5.** For "*Eduardo told me that Miguel was very sick so I took him to the hospital*" the pronoun "*him*" resolves to "*Miguel*" with high probability.

If the sentence ended with "*to see Miguel*", then it should be "*Eduardo*".

▷ **Definition 31.2.6.** A bidirectional RNN concatenates a right-to-left model onto a left-to-right model.

▷ **Example 31.2.7.** Bidirectional RNNs can be used for POS tagging, extending the network from Example 31.1.10



Long Short-Term Memory RNNs

▷ **Problem:** When training a vanilla RNN using back-propagation through time, the long-term gradients which are back-propagated can "vanish" – tend to zero – or "explode" – tend to infinity.

▷ **Definition 31.2.8.** LSTMs provide a short-term memory for RNN that can last thousands of time steps, thus the name "long short-term memory". A LSTM can learn when to remember and when to forget pertinent information,

▷ **Example 31.2.9.** In NLP LSTMs can learn grammatical dependencies.

An LSTM might process the sentence "*Dave, as a result of his controversial claims, is now a pariah*" by

- ▷ remembering the (statistically likely) grammatical gender and number of the subject "*Dave*",
- ▷ note that this information is pertinent for the pronoun "*his*" and
- ▷ note that this information is no longer important after the verb "*is*".

LSTM: Idea

▷ **Idea:** Introduce a memory vector c in addition to the recurrent (short-term memory) vector z

- ▷ c is essentially copied from the previous time step, but can be modified by the forget gate f, the input gate i, and the output gate o.
- ▷ the forget gate f decides which components of c to retain or discard
- ▷ the input gate i decides which components of the *current* input to *add* to c (additive, not multiplicative $\rightsquigarrow$ no vanishing gradients)
- ▷ the output gate o decides which components of c to *output* as z

31.3 Sequence-to-Sequence Models

Neural Machine Translation

▷ **Question:** Machine translation (MT) is an important task in NLP, can we do it with neural networks?

▷ **Observation:** If there were a one-to-one correspondence between source words and target words MT would be a simple tagging task. But

- ▷ the three Spanish words "*caballo de mar*" translate to the English "*seahorse*" and
- ▷ the two Spanish words "*perro grande*" translate to English as "*big dog*".
- ▷ in English, the subject is usually first and in Fijian last.

▷ **Idea:** For MT, generate one word at a time, but keep track of the context, so that

- ▷ we can remember parts of the source we have not translated yet
- ▷ we remember what we already translated so we do not repeat ourselves.

We may have to process the whole source sentence before generating the target!

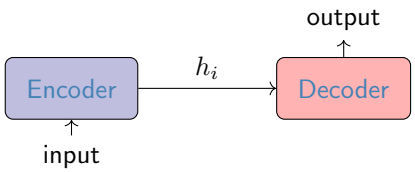
▷ **Remark:** This smells like we need LSTMs.

SOME RIGHTS RESERVED

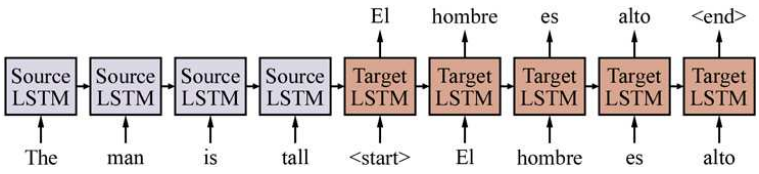
Sequence-To-Sequence Models

▷ **Idea:** Use two coupled RNNs, one for the source, and one for the target. The input for the target is the output of the last hidden layer of the source RNN.

▷ **Definition 31.3.1.** A sequence-to-sequence (seq2seq) model is a neural model for translating an input sequence x into an output sequence y by an encoder followed by a decoder generates y (the output sequence).



▷ **Example 31.3.2.** A simple seq2seq model (without embedding and output layers)



Each block represents one LSTM time step; inputs are fed successively followed by the token `<start>` to start the decoder.

Seq2Seq Evaluation

▷ **Remark:** Seq2seq models were a major breakthrough in NLP and MT. But they have three major shortcomings:

- ▷ nearby context bias: RNNs remember with their hidden layer, which has more information about a word in – say – step 56 than in step 5. BUT long-distance context can also be important.
- ▷ **fixed context size**: the entire information about the source sentence must be compressed into the fixed-dimensional – typically 1024 – vector. Larger vectors $\rightsquigarrow$ slow training and overfitting.

▷ **Idea:** Concatenate all source RNN hidden vectors to use all of them to mitigate the nearby context bias.

▷ **Problem:** Huge increase of weights $\rightsquigarrow$ slow training and overfitting.

Attention

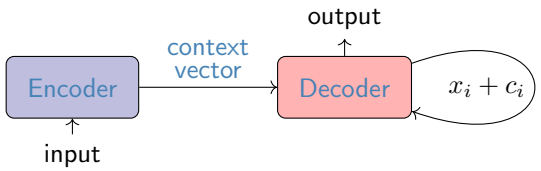
▷ **Bad Idea:** Concatenate all source RNN hidden vectors to use all of them to mitigate the nearby context bias.

▷ **Better Idea:** The decoder generates the target sequence one word at a time. $\rightsquigarrow$ Only a small part of the source is actually relevant.
The decoder must focus on different parts of the source for every word.

▷ **Idea:** We need a neural component that does context-free summarization.

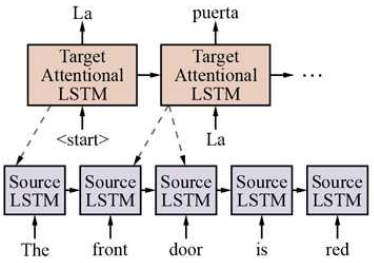
▷ **Definition 31.3.3.** An attentional seq2seq model is a seq2seq that passes along a context vector c_i in the decoder. If $h_i = RNN(h_{i-1}, x_i)$ is the standard decoder, then the decoder with attention is given by $h_i = RNN(h_{i-1}, x_i + c_i)$, where $x_i + c_i$ is the concatenation of the input x_i and context vectors c_i with

$$\begin{array}{rcll} r_{ij} & = & h_{i-1} \cdot s_j & \text{raw attention score} \\ a_{ij} & = & e^{r_{ij}}/(\sum_k e^{r_{ij}}) & \text{attention probability matrix} \\ c_i & = & \sum_j a_{ij} \cdot s_j & \text{context vector} \end{array}$$

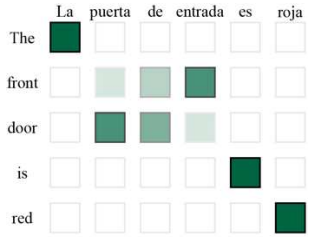


Attention: English to Spanish Translation

▷ **Example 31.3.4.** An attentional seq2seq model for English-to-Spanish translation



dashed lines represent attention



attention probablity matrix
darker colors ⇝ higher probabilities

▷ **Remarks:** The attention

- ▷ component learns no weights and supports variable-length sequences.
- ▷ is entirely latent – the developer does not influence it.

SOME RIGHTS RESERVED

Attention: Greedy Decoding

▷ During training, a seq2seq model tries to maximize the probability of each word in the training sequence, conditioned on the source and the previous target words.

▷ **Definition 31.3.5.** The procedure that generates the target one word at a time and feeds it back at the next time step is called decoding.

▷ **Definition 31.3.6.** Always selecting the highest probability word is called greedy decoding.

▷ **Problem:** This may not always maximize the probability of the whole sequence

▷ **Example 31.3.7.** Let's use a greedy decoder on "*The front door is red*".

- ▷ The correct translation is "*La puerta de entrada es roja*".
- ▷ Suppose we have generated the first word "*La*" for "*The*".
- ▷ A greedy decoder might propose "*entrada*" for "*front*".

▷ Greedy decoding is fast, but has no mechanism for correcting mistakes.

▷ **Solution:** Use an optimizing search algorithm! (e.g. local beam search)

SOME RIGHTS RESERVED

Decoding with Beam Search

▷ **Recall:** Greedy decoding is not optimal!

▷ **Idea:** Search for an optimal decoding (or at least a good one) using one of the search algorithms from Chapter 5 (Problem Solving and Search) in the AI lecture notes.

▷ Local beam search is a common choice in machine translation. Concretely:

- ▷ keep the top k hypotheses at each stage,
- ▷ extending each by one word using the top k choices of words,
- ▷ then chooses the best k of the resulting k^2 new hypotheses.

When all hypotheses in the beam generate the special <end> token, the algorithm outputs the highest scoring hypothesis.

▷ **Observation:** The better the seq2seq models get, the smaller we can keep beam size

Today beams of $b = 4$ are sufficient after $b = 100$ a decade ago.

SOME RIGHTS RESERVED

Decoding with Beam Search

▷ **Example 31.3.8.** A local beam search with beam size $b = 2$



▷ Word scores are log-probabilities generated by the decoder softmax.

▷ The hypothesis score is the sum of the word scores.

At time step 3, the highest scoring hypothesis "*La entrada*" can only generate low-probability continuations, so it "falls off the beam". (as intended)

SOME RIGHTS RESERVED

31.4 The Transformer Architecture

Self-Attention

▷ **Idea:** "Attention is all you need!" (see [Vas+17])

▷ So far, attention was used from the encoder to the decoder.

▷ Self-attention extends this so that each hidden states sequence also attends to itself.(*coder to *coder)

▷ **Idea:** Just use the dot product of the input vectors

▷ **Problem:** Always high, so each hidden state will be biased towards attending to itself.

▷ Self-attention solves this by first projecting the input into three different representations using three different weight matrices:

- ▷ the query vector $\mathbf{q}_i = \mathbf{W}_q \mathbf{x}_i \mathrel{\widehat{=}}$ standard attention
- ▷ key vector $\mathbf{k}_i = \mathbf{W}_k \mathbf{x}_i \mathrel{\widehat{=}}$ the source in seq2seq
- ▷ value vector $\mathbf{v}_i = \mathbf{W}_v \mathbf{x}_i$ is the context being generated

$$\begin{aligned} r_{ij} &= (\mathbf{q}_i \cdot \mathbf{k}_i)/\sqrt{d} \\ a_{ij} &= e^{r_{ij}}/(\sum_k e^{r_{ij}}) \\ c_i &= \sum_j a_{ij} \cdot \mathbf{v}_j \end{aligned}$$

where d is the dimension of $\mathbf{k}$ and $\mathbf{q}$.

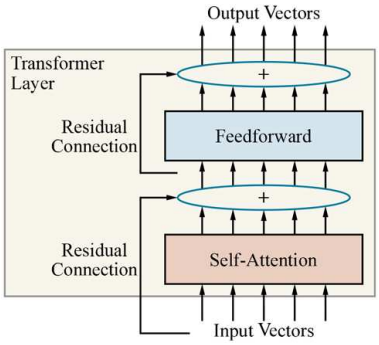
SOME RIGHTS RESERVED

The Transformer Architecture

▷ **Definition 31.4.1.** The transformer architecture uses neural blocks called transformers, which are built up from multiple transformer layers.

▷ **Remark:** The context modeled in self-attention is agnostic to word order ⇝ transformers use positional embeddings to cope with that.

▷ **Definition 31.4.2.** A transformer layer consists of self-attention, a feed-forward network, and residual connections to cope with the vanishing gradient problem.

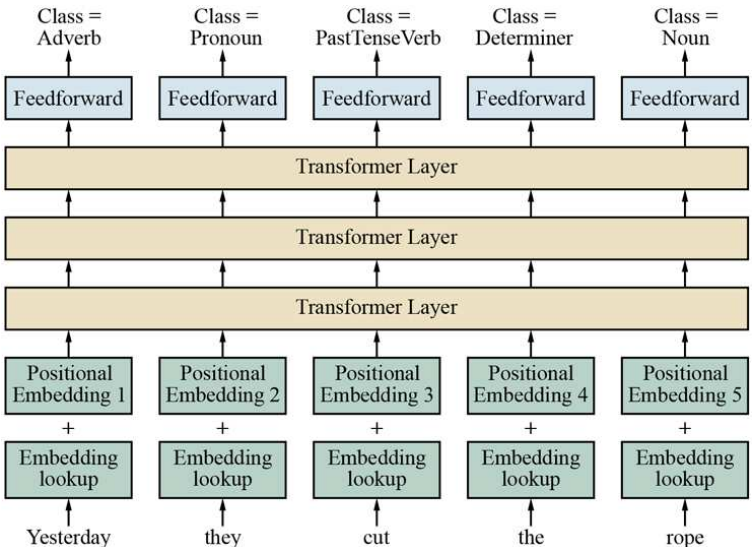


▷ In practice transformers consist of 6-7 transformer layers.

SOME RIGHTS RESERVED

A Transformer for POS tagging

▷ **Example 31.4.3.** A transformers for POS tagging:



SOME RIGHTS RESERVED

31.5 Large Language Models

Pretraining and Transfer Learning

- Getting enough data to build a robust model can be a challenge.
- In NLP we often work with unlabeled data
  - syntactic/semantic labeling is much more difficult $\rightsquigarrow$ costly than image labeling.
  - the Internet has lots of texts (adds $\sim 10^{11}$ words/day)
- **Idea:** Why not let other's do this work and re-use their training efforts.
- **Definition 31.5.1.** In pretraining we use
  - a large amount of shared general-domain language data to train an initial version of an NLP model.
  - a smaller amount of domain-specific data (perhaps labeled) to finetune it to the vocabulary, idioms, syntactic structures, and other linguistic phenomena that are specific to the new domain.
- Pretraining is a form of transfer learning:
- **Definition 31.5.2.** In Transfer learning (TL), knowledge learned from a task is re-used to boost performance on a related task.
- **Idea:** Take a pretrained neural network, replace the last layer(s), and then train those on your own corpus.
- **Observation:** Simple but surprisingly efficient!

 SOME RIGHTS RESERVED

Large Language Models

▷ **Definition 31.5.3.** A Large Language Model (LLM) is a generic pretrained neural network, providing embeddings for sentences or entire documents for NLP tasks. In practice, they (usually) combine the following components:

- ▷ Tokenization: Splitting text into tokens (characters, words, punctuation,...)
- ▷ embeddings for these tokens, (e.g., Word2vec – or we let the transformer learn them)
- ▷ positional embeddings of tokens (encodes where in a sentence a token is)
- ▷ a transformer architecture, trained on
- ▷ a masked token prediction task.

▷ LLMs can be used for a variety of tasks.

- ▷ *classification* (e.g., sentiment analysis, POS-tagging),
- ▷ *translation* (between languages, styles, etc.),
- ▷ *generation* (e.g., text completion, summarization, chatbots),
- ▷ ...

 SOME RIGHTS RESERVED

Tokenization - Byte Pair Encodings

▷ **So far:** we have encoded text either as sequences of characters (non-semantic) or as sequences of words (semantic, but virtually unlimited vocabulary, OOV-problems).

▷ **Idea:** Find a middle ground: Learn an optimal vocabulary of tokens from data and split text into a sequence of tokens.

▷ **Definition 31.5.4.** The Byte Pair Encoding (BPE) algorithm learns a vocabulary of tokens of given size $N > 256$ from a corpus $\mathcal{C}$, by doing the following:

- ▷ Let $\ell = 256$ and set $\mathrm{BPE}(\langle b \rangle) = b$ for every byte $0 \leq b \leq 255$.
- ▷ While $\ell < N$, find the most common pair of tokens (a, b) and let $\mathrm{BPE}(\langle a, b \rangle) = \ell + 1$ (and increase ℓ by 1).
- ▷ Repeat until $\ell = N$.

$\rightsquigarrow$ we obtain a one-hot encoding of tokens of size N, where the most common sequences of bytes are represented by a single token. By retaining $\mathrm{BPE}(\langle b \rangle) = b$, we avoid OOV problems.

▷ $\rightsquigarrow$ We can then train a word embedding on the resulting tokens

▷ Alternative techniques include *WordPiece* and *SentencePiece*.

 SOME RIGHTS RESERVED

Tokenization - Example

https://huggingface.co/spaces/Xenova/the-tokenizer-playground



SOME RIGHTS RESERVED

Positional encodings

▷ **Definition 31.5.5.** Let $\langle w_1, \ldots, w_n\rangle$ be a sequence of tokens. A positional encoding $\mathrm{PE}_i(w_i)$ is a vector that retains the position of w_i in the sequence *alongside* the word embedding of w_i.

We want positional encodings to satisfy the following properties:

1. $\mathrm{PE}_i(w) \neq \mathrm{PE}_j(w)$ for $i \neq j$,
2. PE should retain *distances*: if $i_1 - i_2 = j_1 - j_2$, then given the embeddings for w_1, w_2, we should be able to linearly transform $\langle \mathrm{PE}_{i_1}(w_1), \mathrm{PE}_{i_2}(w_2)\rangle$ into $\langle \mathrm{PE}_{j_1}(w_1), \mathrm{PE}_{j_2}(w_2)\rangle$.

⇝ no entirely separate embeddings for w_1, w_2 depending on positions

⇝ learning from short sentences generalizes (ideally) to longer ones

SOME RIGHTS RESERVED

Sinusoidal positional encoding

▷ **Idea:** Let $\mathrm{PE}_t(w) = \mathrm{E}(w) + p_t$, for some suitable p_t (where $\mathrm{E}(w)$ is the word embedding for token w).

⇝ p_t has the same dimensionality as our embedding E.

▷ **Idea:** Use a combination of sine and cosine functions with different frequencies for each dimension of the embedding.

▷ **Attention is all you need:** For a vocabulary size d, we define

$$(p_t)_i := \begin{cases} \sin\left(\frac{t}{c^{2k/d}}\right) & \text{if } i = 2k \\ \cos\left(\frac{t}{c^{2k/d}}\right) & \text{if } i = 2k+1 \end{cases}$$

for some constant c. (10000 in the paper)

▷ $\rightsquigarrow$ works for arbitrary sequence lengths and vocabulary sizes.

SOME RIGHTS RESERVED

Training Large Language Models

▷ Three strategies for training LLMs:

▷ *Masked Token Prediction:* Given a sentence (e.g. "The river rose five feet"), randomly replace tokens by a special *mask* token (e.g. "The river `[MASK]` five feet"). The LLM should predict the masked tokens (e.g. "rose"). (BERT et al; well suited for *generic* tasks)

▷ *Discrimination:* Train a small masked token prediction model M. Given a masked sentence, let M generated possible completions. Train the actual model to distinguish between tokens generated by M and the *original* tokens. (Google Electra et al; well suited for *generic* tasks)

▷ *Next Token Prediction:* Given the (beginning of) a sentence, predict the next token in the sequence. (GPT et al; well suited for *generative* tasks)

▷ $\rightsquigarrow$ All techniques turn an unlabelled corpus into a *supervised learning* task.

SOME RIGHTS RESERVED

Deep Learning for NLP: Evaluation

▷ Deep learning methods are currently dominant in NLP! (think ChatGPT)

- ▷ Data-driven methods are easier to develop and maintain than symbolic ones.
- ▷ They also perform better models crafted by humans. (with reasonable effort)

▷ But problems remain;

- ▷ DL methods work best on immense amounts of data. (small languages?)
- ▷ LLM contain knowledge, but integration with symbolic methods elusive.

▷ DL4NLP methods do very well, but only after processing orders of magnitude more data than humans do for learning language.

▷ This suggests that there is of scope for new insigths from all areas.

SOME RIGHTS RESERVED

Chapter 32

What did we learn in AI 1/2?

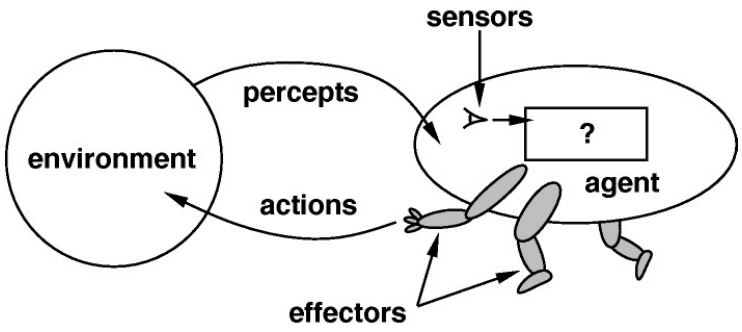
Topics of AI-1 (Winter Semester)

- ▷ Getting Started
  - ▷ What is artificial intelligence? (situating ourselves)
  - ▷ Logic programming in Prolog (An influential paradigm)
  - ▷ Intelligent Agents. (a unifying framework)
- ▷ Problem solving
  - ▷ Problem solving and search (Black Box World States and Actions)
  - ▷ Adversarial search (game playing) (A nice application of search)
  - ▷ constraint satisfaction problems (factored world states)
- ▷ Knowledge and Reasoning
  - ▷ Formal Logic as the mathematics of Meaning
  - ▷ Propositional logic and satisfiability (Atomic Propositions)
  - ▷ First-order logic and theorem proving (Quantification)
  - ▷ Logic programming (Logic + Search$\rightsquigarrow$ Programming)
  - ▷ Description logics and semantic web
- ▷ Planning
  - ▷ Planning Frameworks
  - ▷ Planning Algorithms
  - ▷ Planning and acting in the real world

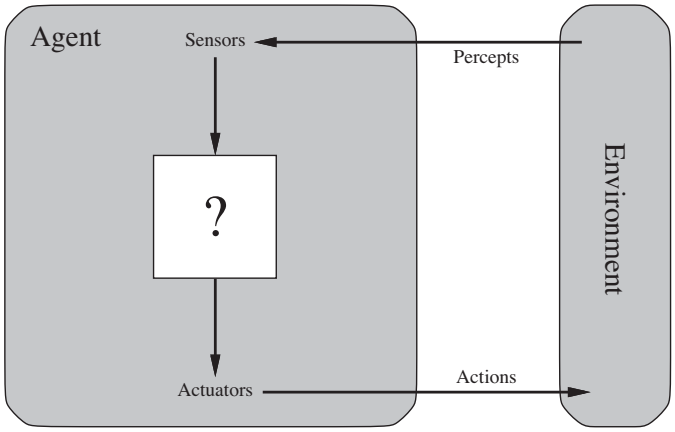
SOME RIGHTS RESERVED

Rational Agents as an Evaluation Framework for AI

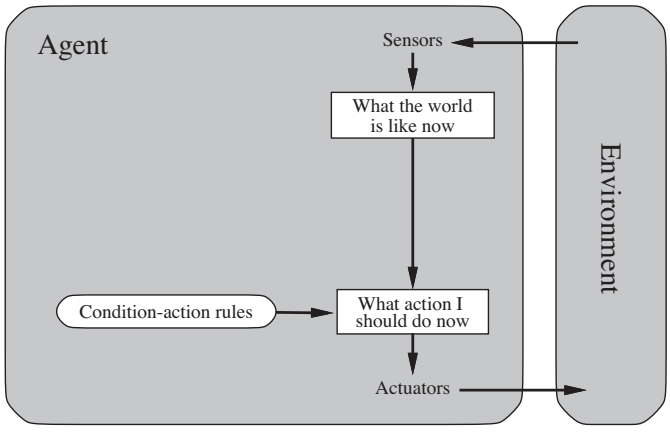
- ▷ Agents interact with the environment



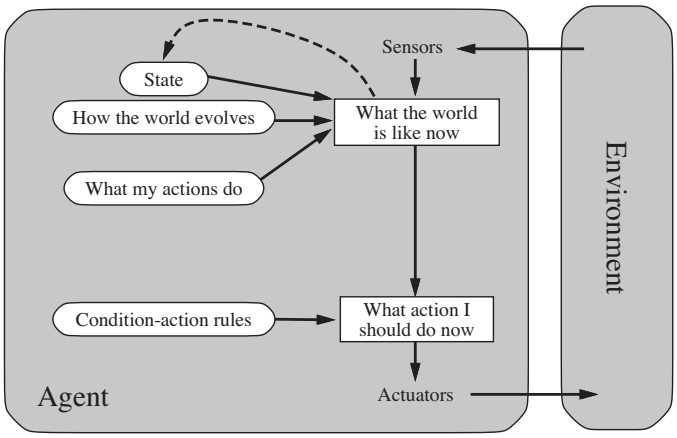
General agent schema



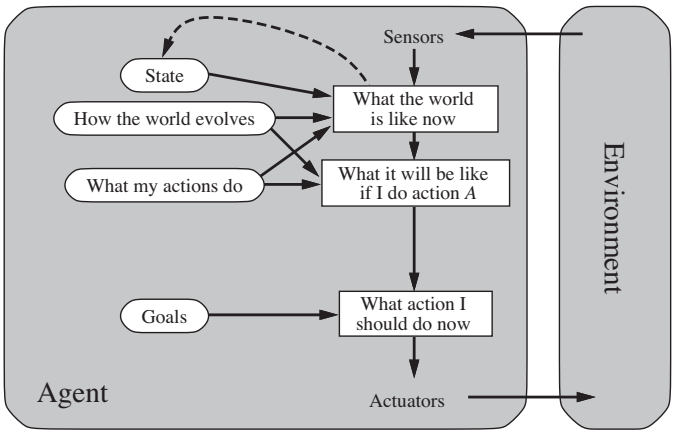
Reflex Agents



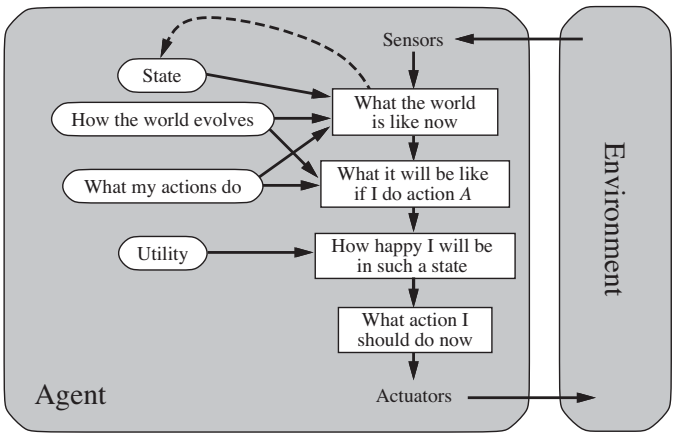
Reflex Agents with State



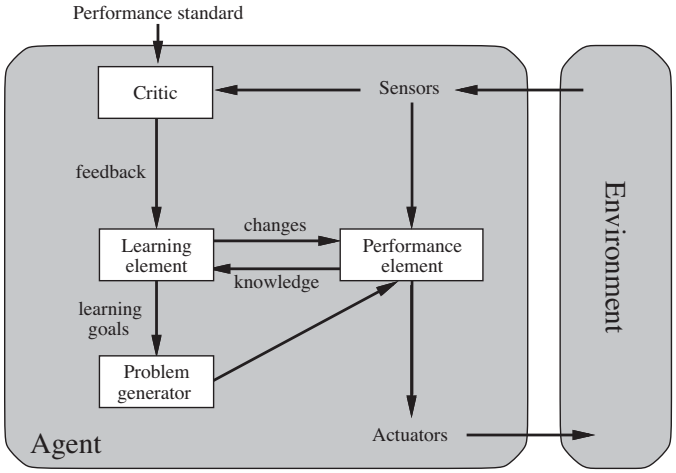
Goal-Based Agents



Utility-Based Agent



Learning Agents



SOME RIGHTS RESERVED

Rational Agent

- ▷ **Idea:** Try to design agents that are successful (do the right thing)
- ▷ **Definition 32.0.1.** An agent is called rational, if it chooses whichever action maximizes the expected value of the performance measure given the percept sequence to date. This is called the MEU principle.
- ▷ **Note:** A rational agent need not be perfect
  - ▷ only needs to maximize expected value (rational $\neq$ omniscient)
    - ▷ need not predict e.g. very unlikely but catastrophic events in the future
  - ▷ percepts may not supply all relevant information (Rational $\neq$ clairvoyant)
    - ▷ if we cannot perceive things we do not need to react to them.
    - ▷ but we may need to try to find out about hidden dangers (exploration)
  - ▷ action outcomes may not be as expected (rational $\neq$ successful)
    - ▷ but we may need to take action to ensure that they do (more often) (learning)
- ▷ Rational $\rightsquigarrow$ exploration, learning, autonomy

SOME RIGHTS RESERVED

Symbolic AI: Adding Knowledge to Algorithms

- ▷ Problem solving (Black Box States, Transitions, Heuristics)
  - ▷ **Framework**: Problem solving and Search (basic tree/graph walking)
  - ▷ **Variant**: Game playing (Adversarial search) (minimax + $\alpha\beta$-Pruning)
- ▷ Constraint Satisfaction Problems (heuristic search over partial assignments)

- ▷ States as partial variable assignments, transitions as assignment
- ▷ Heuristics informed by current restrictions, constraint graph
- ▷ Inference as constraint propagation (transferring possible values across arcs)

▷ Describing world states by formal language (and drawing inferences)

- ▷ Propositional logic and DPLL (deciding entailment efficiently)
- ▷ First-order logic and ATP (reasoning about infinite domains)
- ▷ **Digression**: Logic programming (logic + search)
- ▷ Description logics as moderately expressive, but decidable logics

▷ Planning: Problem solving using white-box world/action descriptions

- ▷ **Framework**: describing world states in logic as sets of propositions and actions by pre-conditions and add/delete lists
- ▷ **Algorithms**: e.g heuristic search by problem relaxations

SOME RIGHTS RESERVED

Topics of AI-2 (Summer Semester)

▷ Uncertain knowledge and reasoning

- ▷ Uncertainty
- ▷ Probabilistic reasoning
- ▷ Making decisions in episodic environments
- ▷ Problem solving in sequential environments

▷ Foundations of machine learning

- ▷ Learning from observations
- ▷ Knowledge in learning
- ▷ Statistical learning methods

▷ Natural language and communication (If there is time)

- ▷ Natural language processing
- ▷ Natural Language for communication

SOME RIGHTS RESERVED

Statistical AI: Adding uncertainty and Learning

▷ Problem solving under uncertainty (non-observable environment, stochastic states)

- ▷ **Framework**: Probabilistic Inference: Conditional Probabilities/Independence
- ▷ **Intuition**: Reasoning in Belief Space instead of State Space!

- ▷ **Implementation**: Bayesian Networks (exploit conditional independence)
- ▷ **Extension**: Utilities and Decision Theory (for static/episodic environments)

▷ Problem solving in Sequential Worlds:

- ▷ **Framework**: Markov Processes, transition models
- ▷ **Extension**: Markov decision processs, partially observable Markov decision processs (+ utilities/decisions)
- ▷ **Implementation**: Dynamic Bayesian Networks

▷ Machine learning: adding optimization in changing environments (unsupervised)

- ▷ **Framework**: Learning from Observations (positive/negative examples)
- ▷ **Intuitions**: finding consistent/optimal hypotheses in a hypothesis space
- ▷ **Problems**: consistency, expressivity, under/overfitting, computational/data resources.
- ▷ Extensions
  - ▷ knowledge in learning (based on logical methods)
  - ▷ statistical learning (optimizing the probability distribution over hypspace, learning BNs)
- ▷ Communication
  - ▷ Phenomena of natural language (NL is interesting/complex)
  - ▷ symbolic/statistical NLP (historic/as a backup)
  - ▷ Deep Learning for NLP (the current hype/solution)

SOME RIGHTS RESERVED

Topics of AI-3 – A Course not taught at FAU ☹

▷ Machine Learning

- ▷ Theory and Practice of Deep Learning
- ▷ More Reinforcement Learning

▷ Communicating, Perceiving, and Acting

- ▷ More NLP, dialogue, speech acts, ...
- ▷ Natural Language Semantics/Pragmatics
- ▷ Perception
- ▷ Robotics
- ▷ Emotions, Sentiment Analysis

▷ **The Good News:** All is not lost

- ▷ There are tons of specialized courses at FAU (more as we speak)
- ▷ Russell/Norvig's AIMA [RN21] cover some of them as well!

SOME RIGHTS RESERVED

Bibliography

[Cho65] Noam Chomsky. *Syntactic structures*. Den Haag: Mouton, 1965.

[Dav67] Donald Davidson. "Truth and Meaning". In: *Synthese* 17 (1967).

[DF31] B. De Finetti. "Sul significato soggettivo della probabilita". In: *Fundamenta Mathematicae* 17 (1931), pp. 298–329.

[Glo] *Grundlagen der Logik in der Informatik*. Course notes at `https://www8.cs.fau.de/_media/ws16:gloin:skript.pdf`. URL: `https://www8.cs.fau.de/_media/ws16:gloin:skript.pdf` (visited on 10/13/2017).

[How60] R. A. Howard. *Dynamic Programming and Markov Processes*. MIT Press, 1960.

[JN33] E. S. Pearson J. Neyman. "IX. On the problem of the most efficient tests of statistical hypotheses". In: *Philosophical Transactions of the Royal Society of London A: Mathematical, Physical and Engineering Sciences* 231.694-706 (1933), pp. 289–337. DOI: `10.1098/rsta.1933.0009`.

[Kee74] R. L. Keeney. "Multiplicative utility functions". In: *Operations Research* 22 (1974), pp. 22–34.

[KNS97] B. Kessler, G. Nunberg, and H. Schütze. "Automatic detection of text genre". In: *CoRR* cmp-lg/9707002 (1997).

[Koh08] Michael Kohlhase. "Using LaTeX as a Semantic Markup Format". In: *Mathematics in Computer Science* 2.2 (2008), pp. 279–304. URL: `https://kwarc.info/kohlhase/papers/mcs08-stex.pdf`.

[Lee02] Lillian Lee. "Fast context-free grammar parsing requires fast Boolean matrix multiplication". In: *Journal of the ACM* 49.1 (2002), pp. 1–15.

[Luc96] Peter Lucas. "Knowledge Acquisition for Decision-theoretic Expert Systems". In: *AISB Quarterly* 94 (1996), pp. 23–33. URL: `https://www.researchgate.net/publication/2460438_Knowledge_Acquisition_for_Decision-theoretic_Expert_Systems`.

[MMS93] Mitchell P. Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. "Building a large annotated corpus of English: the penn treebank". In: *Computational Linguistics* 19.2 (1993), pp. 313–330.

[Nor+18a] Emily Nordmann et al. *Lecture capture: Practical recommendations for students and lecturers*. 2018. URL: `https://osf.io/huydx/download`.

[Nor+18b] Emily Nordmann et al. *Vorlesungsaufzeichnungen nutzen: Eine Anleitung für Studierende*. 2018. URL: `https://osf.io/e6r7a/download`.

[Pra+94] Malcolm Pradhan et al. "Knowledge Engineering for Large Belief Networks". In: *Proceedings of the Tenth International Conference on Uncertainty in Artificial Intelligence*. UAI'94. Seattle, WA: Morgan Kaufmann Publishers Inc., 1994, pp. 484–490. ISBN: 1-55860-332-8. URL: `http://dl.acm.org/citation.cfm?id=2074394.2074456`.

[Ran17] Aarne Ranta. *Automatic Translation for Consumers and Producers*. Presentation given at the Chalmers Initiative Seminar. 2017. URL: https://www.grammaticalframework.org/~aarne/mt-digitalization-2017.pdf.

[RN21] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th. Pearson, 2021. ISBN: 978-1-292-40113-3, 978-1-292-40117-1.

[RN95] Stuart J. Russell and Peter Norvig. *Artificial Intelligence — A Modern Approach*. Upper Saddle River, NJ: Prentice Hall, 1995.

[She24] Esther Shein. 2024. URL: https://cacm.acm.org/news/the-impact-of-ai-on-computer-science-education/.

[sTeX] *sTeX: A semantic Extension of TeX/LaTeX*. URL: https://github.com/sLaTeX/sTeX (visited on 04/11/2026).

[Vas+17] Ashish Vaswani et al. "Attention is All you Need". In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

Part IV

Excursions

As this course is predominantly an overview over the topics of artificial intelligence, and not about the theoretical underpinnings, we give the discussion about these as a "suggested readings" part here.

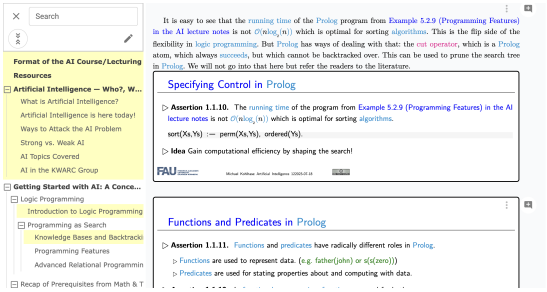
Appendix A

ALeA – AI-Supported Learning

In this chapter we introduce the ALeA (Adaptive Learning Assistant) system, a learning support system we will use to support students in AI-2.

ALeA: Adaptive Learning Assistant

- ▷ **Idea:** Use AI methods to help teach/learn AI (AI4AI)
- ▷ **Concretely:** Provide HTML versions of the AI-2 slides/lecture notes and embed learning support services into them. (for pre/postparation of lectures)
- ▷ **Definition A.0.1.** Call a document active, iff it is interactive and adapts to specific information needs of the readers. (lecture notes on steroids)
- ▷ **Intuition:** ALeA serves active course materials. (PDF mostly inactive)
- ▷ **Goal:** Make ALeA more like a instructor + study group than like a book!
- ▷ **Example A.0.2 (Course Notes).** ≙ Slides + Comments



⇝ yellow parts in table of contents (left) already covered in lectures.

SOME RIGHTS RESERVED

The central idea in the AI4AI approach – using AI to support learning AI – and thus the ALeA system is that we want to make course materials – i.e. what we give to students for preparing and postparing lectures – more like teachers and study groups (only available 24/7) than like static books.

VoLL-KI Portal at https://courses.voll-ki.fau.de

▷ **Portal for ALeA Courses:** https://courses.voll-ki.fau.de



▷ **AI-2 in ALeA:** https://alea.education/FAU/ai-2/SS26

- ▷ All details for the course.
- ▷ recorded syllabus (keep track of material covered in course)
- ▷ syllabus of the last semesters (for over/preview)

▷ **ALeA Status:** The ALeA system is deployed at FAU for over 1000 students taking eight courses

- ▷ (some) students use the system actively (our logs tell us)
- ▷ reviews are mostly positive/enthusiastic (error reports pour in)

SOME RIGHTS RESERVED

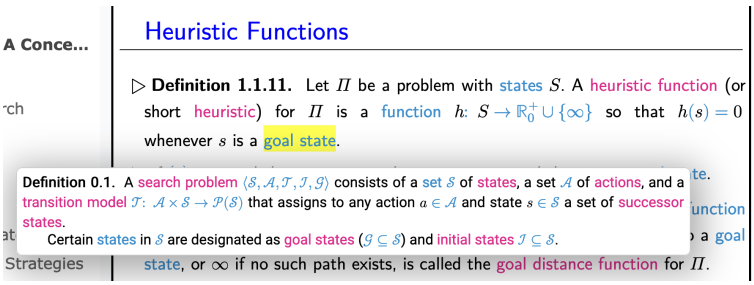
The ALeA AI-2 page is the central entry point for working with the ALeA system. You can get to all the components of the system, including two presentations of the course contents (notes- and slides-centric ones), the flashcards, the localized forum, and the quiz dashboard.

We now come to the heart of the ALeA system: its learning support services, which we will now briefly introduce. Note that this presentation is not really sufficient to understand what you may be getting out of them, you will have to try them, and interact with them sufficiently that the learner model can get a good estimate of your competencies to adapt the results to you.

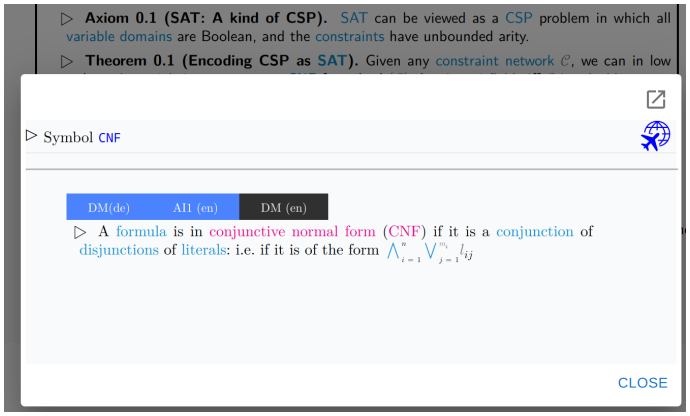
Learning Support Services in ALeA

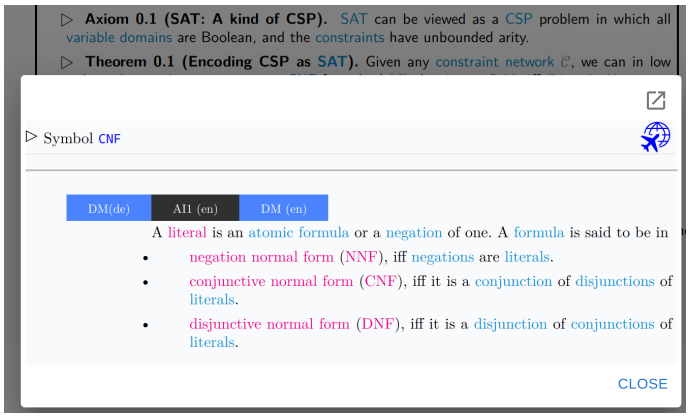
▷ **Idea:** Embed learning support services into active course materials.

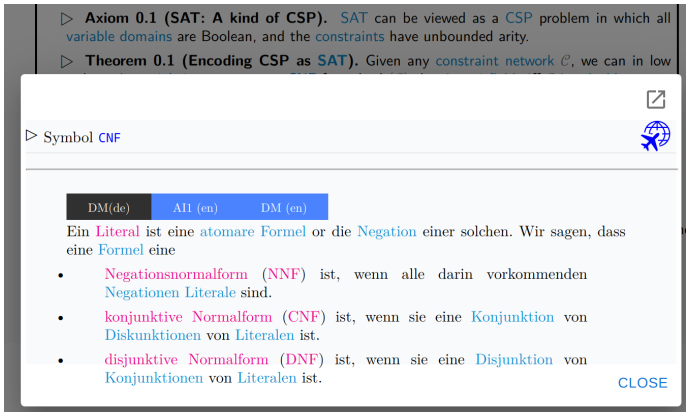
▷ **Example A.0.3 (Definition on Hover).** Hovering on a (cyan) term reference reminds us of its definition. (even works recursively)



▷ **Example A.0.4 (More Definitions on Click).** Clicking on a (cyan) term reference shows us more definitions from other contexts.







▷ **Example A.0.5 (Guided Tour).** A guided tour for a concept c assembles definitions/etc.

into a self-contained mini-course culminating at c.

c = countable ⇝



▷ **Example A.0.6 (Problems Everywhere).** We can deploy personalized practice problems

- ▷ at the end of a secion S, based on the concepts introduced in S (review knowledge)
- ▷ before a new section S, based on the concepts used in it. (traffic light)
- ▷ . . . in all generated materials.

▷ . . . your idea here . . . If the semantics supports it, we can build it! (the sky is the limit)

SOME RIGHTS RESERVED

Note that this is only an initial collection of learning support services, we are constantly working on additional ones. Look out for feature notifications (

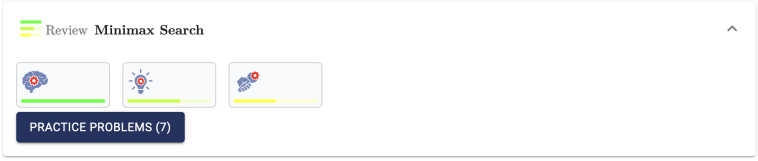
) on the upper right hand of the ALeA page.

(Practice/Remedial) Problems Everywhere

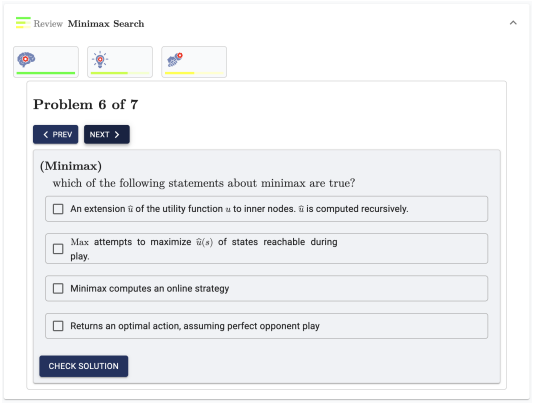
- ▷ **Problem:** Learning requires a mix of understanding and test-driven practice.
- ▷ **Idea:** ALeA supplies targeted practice problems everywhere.
- ▷ **Concretely:** Revision markers at the end of sections.
  - ▷ A relatively non-intrusive overview over competency



  - ▷ Click to extend it for details.



  - ▷ Practice problems as usual. (targeted to your specific competency)



SOME RIGHTS RESERVED

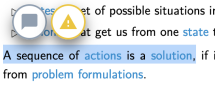
While the learning support services up to now have been adressed to individual learners, we now turn to services addressed to communities of learners, ranging from study groups with three learners, to whole courses, and even – eventually – all the alumni of a course, if they have not de-registered from ALeA.

Currently, the community aspect of ALeA only consists in localized interactions with the course materials. The ALeA system uses the semantic structure of the course materials to localize some interactions that are otherwise often from separate applications. Here we see two:

1. one for reporting content errors – and thus making the material better for all learners – and‘’
2. a localized course forum, where forum threads can be attached to learning objects.

Localized Interactions with the Community

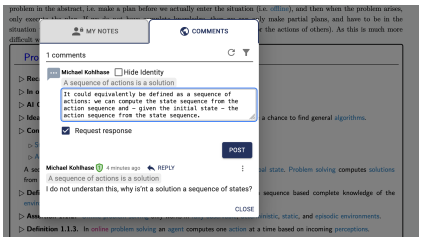
- ▷ Selecting text brings up localized – i.e. anchored on the selection – interactions:



  - ▷ post a (public) comment or take (private) note
  - ▷ report an error to the course authors/instructors

- ▷ Localized comments induce a thread in the ALeA forum (like the StudOn Forum, but targeted towards specific learning objects.)





▷ Answering questions gives karma $\widehat{=}$ a public measure of user helpfulness.

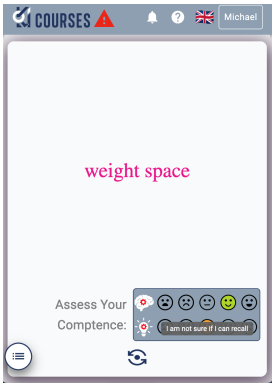
▷ Notes can be anonymous (⇝ generate no karma)

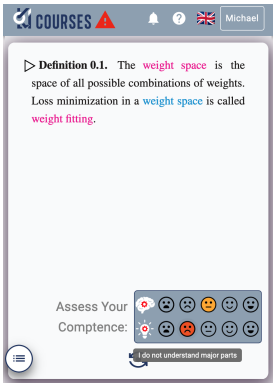
SOME RIGHTS RESERVED

We can use the same four models discussed in the space of guided tours to deploy additional learning support services, which we now discuss.

New Feature: Drilling with Flashcards

▷ Flashcards challenge you with a task (term/problem) on the front...





...and the definition/answer is on the back.

▷ Self-assessment updates the learner model (before/after)

▷ **Idea:** Challenge yourself to a card stack, keep drilling/assessing flashcards until the learner model eliminates all.

▷ **Bonus:** Flashcards can be generated from existing semantic markup (educational equivalent to free beer)

SOME RIGHTS RESERVED

We have already seen above how the learner model can drive the drilling with flashcards. It can also be used for the configuration of card stacks by configuring a domain e.g. a section in the course materials and a competency threshold. We now come to a very important issue that we always face when we do AI systems that interface with humans. Most web technology companies that take one the approach "the user pays for the services with their personal data, which is sold on" or integrate advertising for renumeration. Both are not acceptable in university setting.

But abstaining from monetizing personal data still leaves the problem how to protect it from intentional or accidental misuse. Even though the GDPR has quite extensive exceptions for research, the ALeA system – a research prototype – adheres to the principles and mandates of the GDPR. In particular it makes sure that personal data of the learners is only used in learning support services directly or indirectly initiated by the learners themselves.

Learner Data and Privacy in ALeA

- ▷ **Observation:** Learning support services in ALeA use the learner model; they
  - ▷ need the learner model data to adapt to the invidivual learner!
  - ▷ collect learner interaction data (to update the learner model)
- ▷ **Consequence:** You need to be logged in (via your FAU IDM credentials) for useful learning support services!
- ▷ **Problem:** Learner model data is highly sensitive personal data!
- ▷ **ALeA Promise:** The ALeA team does the utmost to keep your personal data safe. (SSO via FAU IDM/eduGAIN, ALeA trust zone)
- ▷ **ALeA Privacy Axioms:**
  1. ALeA only collects learner models data about logged in users.
  2. Personally identifiable learner model data is only accessible to its subject (delegation possible)
  3. Learners can always query the learner model about its data.
  4. All learner model data can be purged without negative consequences. (except usability degradation)
  5. Logging into ALeA is completely optional.
- ▷ **Observation:** Authentication for bonus quizzes are somewhat less optional, but you can always purge the learner model later.

SOME RIGHTS RESERVED

So, now that you have an overview over what the ALeA system can do for you, let us see what you have to concretely do to be able to use it.

Concrete Todos for ALeA

- ▷ **Recall:** You will use ALeA for the prepquizzes (or lose bonus points)
  All other use is optional. (but AI-supported pre/postparation can be helpful)
- ▷ To use the ALeA system, you will have to log in via SSO: (do it now)
  - ▷ go to `https://alea.education/FAU/ai-2/SS26`,
  - ▷ in the upper right hand corner you see 

,
  - ▷ log in via your FAU IDM credentials. (you should have them by now)

▷ You get access to your personal ALeA profile via 

(plus feature notifications, manual, and language chooser)

▷ **Problem:** Most ALeA services depend on the learner model. (to adapt to you)

▷ **Solution:** Initialize your learner model with your educational history!

▷ **Concretely**: enter taken CS courses (FAU equivalents) and grades.

▷ ALeA uses that to estimate your CS/AI competencies. (for your benefit)

▷ then ALeA knows about you; I don't! (ALeA trust zone)

SOME RIGHTS RESERVED

Even if you did not understand some of the AI jargon or the underlying methods (yet), you should be good to go for using the ALEA system in your day-to-day work.